

Herkes İçin Yapay Zekâ

Kuraldan derin öğrenmeye

Onur Önder

Herkes İçin Yapay Zekâ

Onur Önder

© 2026 Onur Önder. Tüm hakları saklıdır. Pazarlama ve satış hakları Fittechs Yazılım Anonim Şirketi'ne aittir. Bu kitabın hiçbir bölümü yazarın yazılı izni olmadan çoğaltılamaz, yayımlanamaz ya da elektronik ortama aktarılamaz (5846 sayılı Fikir ve Sanat Eserleri Kanunu).

Yazarın kendi yayınıdır.

Basılı sürüm ISBN: 978-625-00-5211-2

İnteraktif dijital sürüm ISBN: 978-625-00-5299-0

Birinci basım: Eylül 2026

Kapak ve sayfa tasarımı: Onur Önder

Redaksiyon: Onur Önder

Baskı ve cilt: [matbaa adı, adres, sertifika no]

Satış ve iletişim: Fittechs Yazılım Anonim Şirketi, Gayrettepe Mah. Yıldız Posta Cad. No: 8/34, İstanbul. Mecidiyeköy V.D. 3880798863. E-posta: support@fittechs.com. Web: onuronder.com

Bu kitabın interaktif dijital sürümü, 45 canlı demoyla, book.onuronder.com adresindedir.

Teşekkür

Hiç aklımda yokken birden beliren bir fikir oldu bu kitap. Neden diye sordum kendime: herkes biraz fikir sahibi olmasın mı, herkes ne kullandığını biraz anlamasın mı, nasıl bir döneme giriyoruz? Sonra dedim ki yapay zekâyı anlamak için önce bilgisayar ve çalışma prensiplerini anlamak lazım; hikâye böyle başladı.

Kitabın dayandığı fikirler bana ait değil. Turing'den Hinton'a, kural yazanlardan olasılık hesaplayanlara, farklı hikâyeleri yazan araştırmacılara ve öğrendiklerini açıkça paylaşan herkese ait. Belirtebildiğim kadarını atlamadan kaynakça bölümünde paylaştım; hepsine teşekkür borçluyum.

Son olarak zor zamanlarda, her ne olursa olsun yanımda, yamacımda olan; akli, merhameti ve sevgisiyle en büyük destekçim olan ve devam etme enerjimi sağlayan eşim Öykü'ye, evlatlarım Kuzey ve Poyraz'a, şefkatini ve sıcaklığını hiçbir zaman esirgemeyen, en ince ayrıntıları düşünen anneme ve ışıklar içinde uyuyan, gururunu ve sevgisini her zaman hissettiğim babama...

Metni satır satır okuyup düzelten, "burası anlaşılıyor" demekten çekinmeyen gözlere: Seda ve Gözde.

Hepinize teşekkür ederim; geri kalan tüm hatalar bana aittir.

Onur Önder

Önsöz

Bu kitap bir soruyla başlıyor: Bir makine düşünebilir mi? Soru yeni değil; dört yüz yıldır filozofları meşgul ediyor. Yeni olan, bu soruya cevap vermeyi deneyebilecek makinelerin artık masanın üstünde durması.

İlk hesap çarklarından bugünün sohbet robotlarına uzanan iki yüzyıllık bir hikâye anlatacağım. Hikâyenin kahramanları fikirler: kural, arama, olasılık, öğrenme, nöron, dikkat. Bunların çoğu bir öncekinin yetmediği yerde doğdu, bazıları da yan yana gelişti; bölümler bu fikirleri o sırayla ele alıyor.

Bu kitabın dijital sürümünde her kavramı kendi elinle deniyorsun: bir Turing makinesini adım adım çalıştırıyor, bir nöronun girdilerini çeviriyor, gürültüden bir görsel çıkarıyorsun. Basılı kitapta bu yok; onun yerine her deneyi senin için sayılarla ve çizimlerle adım adım yürüttüm. Sonra da sırayı sana bıraktım. "Kendin dene" alıştırımları kalem ve kâğıtla çözülüyor; canlı demoların bağlantıları da her şeklin altında.

Kitabı iki derinlikte yazdım. Ana akış, hiç ön bilgisi olmayan biri için. "Teknik derinlik" kutuları ise formülü, terimi ve matematiği merak edenler için. Kutuları atlayarak da okuyabilirsin; hiçbir şey eksik kalmaz. Merak ettiğinde geri dönersin.

Yapay zekâ hakkında konuşan herkesin bilmesi gerekeni, hiç kimseyi dışarıda bırakmadan anlatmayı denedim. Umarım bittiğinde başlangıçtaki soruya kendi cevabını vermiş olursun.

Onur Önder

İstanbul, Eylül 2026

Bu Kitabı Nasıl Okumalı

Kitap sekiz bölüm. Her bölüm bir dönemi ve o dönemin ana fikrini anlatıyor; sırayla okunmak üzere yazıldı. Yine de her bölüm kendi başına ayakta durur; ilgini çeken yerden de başlayabilirsin.

İki derinlik. Ana metin sade bir dille yazıldı ve tek başına eksiksizdir. Bazı alt bölümlerin sonunda **Teknik derinlik** başlıklı kutular var. Bunlar aynı fikri formülüyle, terimiyle ve matematiğiyle yeniden anlatır. Kutular atlanabilir; ana metin onlara dayanmaz. Kapaktaki ızgara ve ağ da bunu anlatır: aynı fikir, iki derinlik.

Kenar notları. “Kenar notu” diye başlayan kısa notlar sayfa kenarında değil, metnin akışı içinde durur: sol kenarı renkli bir çizgiyle ayrılmış, eğik yazılı küçük kutular. Konunun bugünle bağını ya da ilk bakışta görünmeyen bir ayrıntıyı verirler.

Şekiller. Dijital sürümdeki 45 canlı demonun her biri burada bir şekil bloğuna dönüştü. Her blok aynı düzende ilerler:

- *Kurulum:* şekilde ne gördüğün.
- *Adım adım* (sınama şekillerinde *Kendini sına*): deneyin kâğıt üstünde, gerçek sayılarla yürütülmüş hâli.
- *Ne oluyor?:* olanların bir paragraflık açıklaması.
- *Kendin dene:* kalem ve kâğıtla çözülecek bir ila üç soru. Cevaplar kitabın sonunda.

Süreç anlatan şekiller film şeridi gibidir: kareleri soldan sağa, satır satır izle.

Canlı demolar. Her şeklin altındaki QR kodu, o deneyin canlı hâlini telefonunda açar; giriş ya da satın alma gerekmez, yalnız o demo açılır. Kitabın tamamı, bütün demolarıyla birlikte dijital sürümdedir. Kitabın sonunda tüm demoların listesi ve bağlantıları var.

Kendini test et. Her bölüm bir kısa sınavla biter. Cevap anahtarı kitabın sonunda. Bazı şekiller de birer sınamadır (“Kendini sına”); onların cevapları ve gerekçeleri aynı yerde.

Bu bölümden kalanlar. Bölüm sonundaki madde listesi, o bölümden aklında kalması gerekenleri sıralar. Kitabı bitirdikten sonra yalnız bu listeleri okumak iyi bir tekrar olur.

İçindekiler

Bölüm 1 · Zekâ ve Makineler	13
1.1 Bir makine düşünebilir mi?	14
1.2 Zekâ nedir?	15
1.3 Düşünmek = hesaplamak mı?	18
1.4 Babbage'tan Turing'e	21
1.5 ENIAC ve modern bilgisayarın doğuşu	25
1.6 Dar YZ mi, Genel YZ mi?	28
1.7 Neden tam şimdi? Moore yasası	31
1.8 Kendini test et	35
 Bölüm 2 · Kuralların Çağı	 37
2.1 Kuralların çağı	38
2.2 Bilgiyi sembollerle temsil etmek	39
2.3 Eğer... ise...: kurallar ve uzman sistemler	42
2.4 Arama ve sezgisel kısayollar	45
2.5 Belirsizlikle başa çıkmak: olasılık ve Markov	49
2.6 Düzenliler ve dağınıklar (neats ve scruffies)	52
2.7 Kendini test et	55
 Bölüm 3 · Makineler Nasıl Öğrenir	 57
3.1 Makineler nasıl öğrenir?	58
3.2 Özellik ve etiket	59
3.3 Üç öğrenme türü	62
3.4 Sınıflandırma ve regresyon	65
3.5 Kümeleme ve anomali	68
3.6 Model nasıl iyileşir: kayıp ve gradyan inişi	72
3.7 Aşırı uyum ve topluluk öğrenmesi	75
3.8 Kendini test et	79
 Bölüm 4 · Yapay Beyin	 81
4.1 Yapay beyin: derin öğrenme	82
4.2 Tek bir yapay nöron	83
4.3 Katmanlar ve ileri besleme	86
4.4 Geri yayılım: hatadan öğrenmek	89

4.5 Görüntüyü görmek: evrişimli ağlar (CNN)	93
4.6 Diziyi anlamak: yinelemeli sinir ağları (RNN)	97
4.7 Sahteciliğin sanatı: GAN	101
4.8 Kendini test et	105
Bölüm 5 · Bugünün Yapay Zekâsı	107
5.1 Üretken çağ: tanımaktan üretmeye	108
5.2 Makine kelimeleri nasıl görür: token'lar	109
5.3 Anlamı sayıya çevirmek: gömüler	113
5.4 Dikkat: Transformer'ın kalbi	116
5.5 Dil modeli: bir sonraki kelimeyi tahmin et	120
5.6 Bir model nasıl yetişir: eğitim hattı	124
5.7 Görsel üretimi: difüzyon	128
5.8 Sınırlar: halüsinasyon, bağlam ve maliyet	132
5.9 Kendini test et	136
Bölüm 6 · YZ'yi Kullanmak ve İnşa Etmek	139
6.1 YZ'yi kullanmak ve inşa etmek	140
6.2 İstem mühendisliği: doğru soruyu sormak	141
6.3 RAG: modele kendi verini ver	146
6.4 Ajanlar: düşün, araç kullan, gözlemle	151
6.5 Bir YZ uygulamasının mimarisi	155
6.6 Gerçek dünyada yapay zekâ	159
6.7 Kendini test et	163
Bölüm 7 · Yapay Zekâ ve Toplum	167
7.1 Yapay zekâ ve toplum	168
7.2 Önyargı: veriden karara	169
7.3 Kara kutu mu, beyaz kutu mu?	172
7.4 Deepfake ve dezenformasyon	176
7.5 Düzenleme: riski sınıflandırmak	179
7.6 Hizalama: dediğin mi, demek istediğin mi?	185
7.7 Kendini test et	188
Bölüm 8 · Felsefe ve Gelecek	191
8.1 Felsefe ve gelecek	192
8.2 Turing testi: ayırt edebilir misin?	193
8.3 Çince Oda: anlamak mı, işlemek mi?	196

8.4 Dar YZ'den süper zekâya	199
8.5 Tekillik: mit mi, ciddi bir olasılık mı?	202
8.6 Sorumluluk ve haklar	205
8.7 Kendini test et	209
 Cevap Anahtarı	 211
 Sözlük	 235
 Kaynakça ve İleri Okuma	 247
 Canlı Demolar	 251
 Dizin	 253

1

BÖLÜM 1

Zekâ ve Makineler

*Bir makine düşünebilir mi?***1.1 Bir makine düşünebilir mi?**

Bu kitap “bir varmış bir yokmuş” diye başlamıyor; çünkü bu hikâye bitmedi, sen de içindesin. Başlangıcıysa çok eskiye, bilgisayarlardan çok öncesine uzanıyor. O günden beri sorulan soru aynı: düşünmek dediğimiz şey ne? Kafamızın içinde bir tür çark dönüyorsa, aynı çarkı bir makine de döndüremez mi? İnsanlar yüzyıllardır bunu tartışıyor.

Bu ilk bölümde yolun en başına gidiyoruz. Zekâ denen şey ne? Düşünmek gerçekten adım adım bir tarife dökülebilir mi? Babbage’ın dişli çarklarından modern bilgisayarın doğuşuna yürüyeceğiz. Yol bittiğinde, yapay zekânın (kısaca YZ) neden mümkün olduğu kendiliğinden görünecek.

***Kenar notu.** YZ yeni değil. “Makineler düşünebilir mi?” sorusu 1600’lerden beri filozofları meşgul ediyor. Yeni olan, buna cevap verebilecek donanım.*

TEKNİK DERİNLİK

Yapay zekâ, hesaplama kuramı ile zihin felsefesinin kesişiminde doğdu. Temel iddia şudur: eğer zihinsel süreçler biçimsel (formel) kurallarla tanımlanabiliyorsa, prensipte bir makine tarafından da yürütülebilir.

Bu bölüm zekânın tanımından başlar; hesaplama kuramının taşlarını (ikili gösterim, algoritma, Turing makinesi, depolanmış-program mimarisi) döşer ve “zayıf/güçlü YZ” ayrımıyla modern tartışmaya bağlanır. Sonraki bölümlerin öğrenme ve sinir ağı kavramları bu zemine oturur.

İlk soru en baştaki: makineye vermek istediğimiz zekâ ne?

1.2 Zekâ nedir?

Kolay bir soru gibi görünür ama “zekâ nedir?” sorusunun tek bir cevabı yok. Bilmece çözen mi zekidir, yeni taşındığı şehirde yolunu hemen bulan mı, yoksa üzgün arkadaşını tek bakışta anlayan mı? Belki hepsi; ama her biri bambaşka bir beceri.

Psikolog Howard Gardner da böyle düşünmüş: zekâ tek bir sayıya (IQ) sığmaz; birden çok türü vardır. 1983'te yedi zekâ alanı saydı; doğacı zekâyı daha sonra ekledi. Kuram tartışmalıdır ama zekânın tek parça olmadığını göstermesi bakımından öğretici. Aşağıda bu türlere bak; bugünkü YZ kimisinde usta, kimisinde daha emekleme çağında.

***Kenar notu.** Bir hesap makinesi bazı hesaplarda senden kat kat hızlı ama bir şakaya gülemez. Zeki olmak tek bir eksen değildir.*

Kurulum. Şekilde sekiz kart var; her kart, Gardner'ın kuramındaki bir zekâ türü. Yedisi 1983'teki ilk listeden; doğacı alan sonradan eklendi. Kartın altındaki çubuk, bugünkü yapay zekânın o alanda kabaca nerede durduğunu gösterir. Çubuklar ölçüm değil, anlatım için seçilmiş temsili düzeyler: dolu çubuk güçlü, yarım çubuk orta, kısacık çubuk zayıf demek.

Adım adım. Sekiz kartın tamamı, kaynaktaki sırayla:

Zekâ türü	Ne demek	Bugünkü YZ	Çubuk (temsili)
Dilsel	Dili, yazıyı ve anlatıyı kullanma becerisi.	Güçlü	%90
Mantıksal-Matematiksel	Sayılar, mantık ve sistematik akıl yürütme.	Güçlü	%90
Uzamsal	Mekânı, biçimleri ve görselleri kavrama.	Orta	%55

Zekâ türü	Ne demek	Bugünkü YZ	Çubuk (temsili)
Müziksel	Ritim, melodi ve sesin örüntülerini sezme.	Orta	%55
Bedensel-Kinestetik	Bedeni ve el becerisini ustaca kullanma.	Zayıf	%22
Kişilerarası	Başkalarını anlama, empati ve iletişim.	Zayıf	%22
İçsel	Kendi duygularını ve amaçlarını tanıma.	Zayıf	%22
Doğacı	Doğadaki canlı ve örüntüleri ayırt etme.	Orta	%55

Yüzdelere ölçülmüş başarı değil; üç düzeyin şekildeki çubuk uzunluğu. Tabloyu sayarsan iki güçlü, üç orta, üç zayıf çıkar. Güçlü ikisinin ortak yanı, sembollerle iş görmeleri: harfler, sayılar, kurallar. Bilgisayar da bu malzemeyle çalışır; sıradaki bölümdeki ikili kod bir sembol dizisi.

Zayıf üçü ise başka bir malzeme ister: bir beden, karşıdaki insanın yüzü, kendi iç dünyası. Bunların hiçbirisi sembole kolay dökülmez. Ortadaki üçü (uzam, müzik, doğa) ikisinin arasında durur. Sesin ve görüntünün bir kısmı sayıya çevrilebiliyor; bir kısmı hâlâ direniyor.

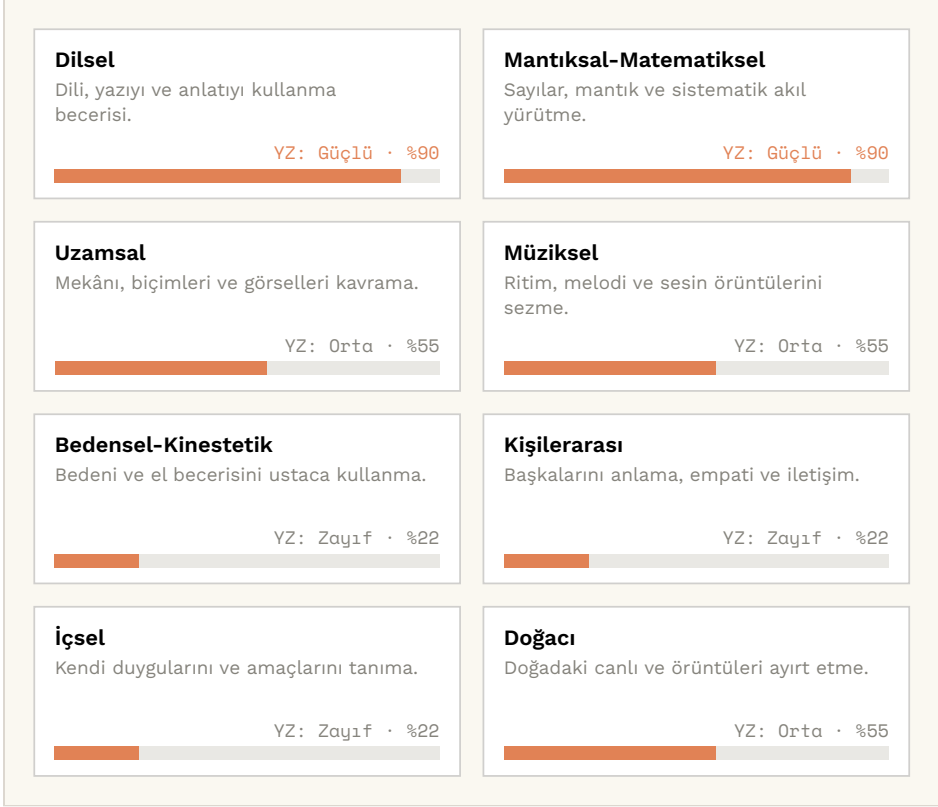
Listenin sırası bir başarı sıralaması değil. Kartlar ayrı becerileri sayıyor; hiçbirisi ötekenden daha zeki sayılmaz. YZ için de öyle: dilde güçlü olmak, öbür yedi alanda güçlü olmak anlamına gelmiyor.

Ne oluyor? Gardner'a göre zekâ tek bir sayı değil, sekiz ayrı yetenek ailesidir; şekildeki her kart bunlardan biri. Bugünkü yapay zekâ dilde ve mantıkta çok iyi; beden ve duygu işlerinde ise sonuç göreve ve sisteme bağlı, çoğunda hâlâ çok geride. Çubuklar temsili düzeydir; ölçülmüş başarı değildir.

Kendin dene. 1) Kitabı kapat ve sekiz zekâ türünü ezberden say. Kaçını hatırladın; hangileri unutuluyor? 2) Dünkü gününü düşün: Sabahtan akşama en çok hangi üç türü kullandın? Tabloya göre

bugünkü YZ bunların kaçında güçlü? 3) Tabloda zayıf olan üç türün ortak yanı ne? Bir cümleyle yaz.

Şekil 1.1 · Çoklu zekâyı keşfet



Şekil 1.1

Canlı demo: book.onuronder.com/d/1dbc74d595



TEKNİK DERİNLİK

Zekânın üzerinde uzlaşılmış tek bir tanımı yoktur; çalışan tanımlar genelde “hedefe yönelik uyum, soyutlama, öğrenme ve problem çözme kapasitesi” etrafında toplanır. YZ alanı

TEKNİK DERİNLİK · DEVAM

pragmatik bir tanım kullanır: insan zekâsı gerektiren görevleri yerine getirebilen sistemler.

Howard Gardner'ın Çoklu Zekâ Kuramı (Frames of Mind, 1983) zekâyı görece bağımsız alanlara ayırır. 1983'teki ilk liste yedi alandı: dilsel, mantıksal-matematiksel, uzamsal, müziksel, bedensel-kinestetik, kişilerarası ve içsel. Doğacı alanı Gardner daha sonra ekledi; buradaki sekizli liste ilk listenin aynısı değildir. Kuram psikometride tartışmalıdır ancak zekânın çok-boyutluluğunu göstermesi açısından öğreticidir: mevcut YZ bu boyutlarda son derece dengesizdir.

Asıl ders, zekânın tek boyutlu olmaması: YZ dil ve mantıkta güçlü, bedensel ve sosyal/içsel alanlarda zayıftır. Şekildeki üç düzey ölçülmüş başarı değil, anlatım için seçilmiş temsili etiketlerdir; görev ve sistem değişince sonuç da değişir.

Zekâ tek parça değilse, parçalarından en az biri adım adım bir tarife dökülebilir mi? Bir kek tarifi bu soruya iyi bir başlangıç.

1.3 Düşünmek = hesaplamak mı?

Şimdi mutfağa girelim. Elinde bir kek tarifi var: yumurtayı kır, çırp, unu ekle, fırına ver. Tarifi harfi harfine izleyen biri, kek yapmayı hiç anlamasa bile ortaya kek çıkarır. Yapay zekânın temelindeki fikir de bu: belki düşünmek de küçük, mekanik adımları sırayla uygulamaktır.

Eğer öyleyse, bu adımları bir makineye de yaptırabiliriz. Bugünün sayısal bilgisayarları bunun için iki şey kullanır:

- Makinenin kullanabileceği basit bir alfabe (çoğunlukla ikili kod: 0 ve 1)
- Net bir adım listesi (tarifin kendisi: algoritma)

Aşağıda ikili kodun nasıl çalıştığını kendin gör.

***Kenar notu.** “Algoritma” kelimesi 9. yüzyıl Pers matematikçisi el-Harezmi’nin adından gelir. İkili sistemi ise 1703’te Leibniz biçimselleştirdi.*

Kurulum. Şekilde yan yana sekiz kutu var. Her kutunun üstünde bir ağırlık yazıyor; soldan sağa 128, 64, 32, 16, 8, 4, 2 ve 1. Yanık kutu 1, sönük kutu 0 demek. Şekilde gösterimin açılış hâli var: 01001001. Altındaki sayı, yanık kutuların ağırlıklarının toplamı. Onun altında üç satır daha var: 5, 73 ve 255 aynı yolla yazılmış.

Adım adım. Önce ağırlık şeması:

Kutu	1	2	3	4	5	6	7	8
Ağırlık	128	64	32	16	8	4	2	1
Açılış	0	1	0	0	1	0	0	1

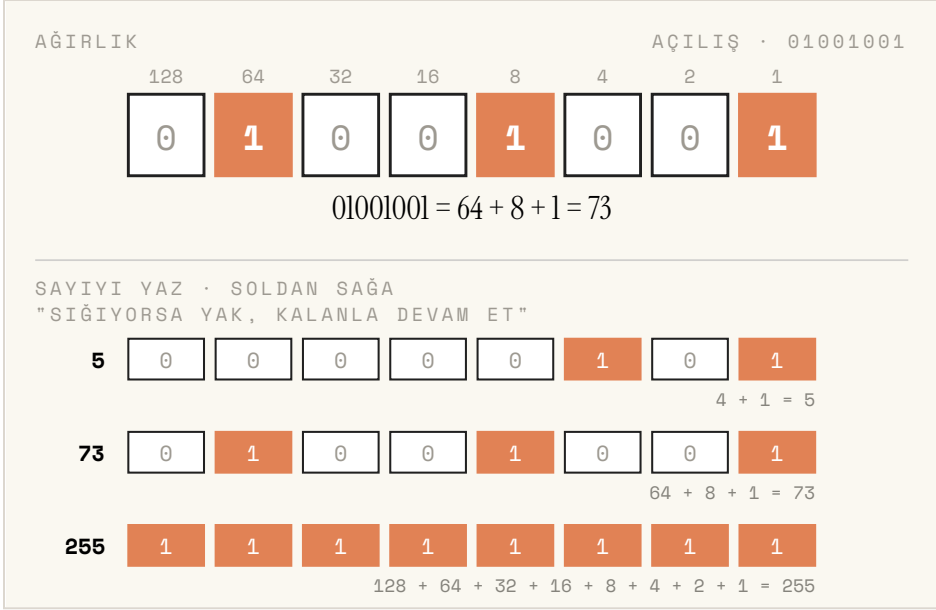
1. Açılıştaki sayıyı çöz. Yanık kutular 2, 5 ve 8; ağırlıkları 64, 8 ve 1. Topla: $64 + 8 + 1 = 73$. Demek ki 01001001 = 73.
2. Tersini de yap: 5 sayısını kutulara yaz. Soldan başla, her kutuya sor: “Ağırlığın sayıya sığıyor mu?” 128, 64, 32, 16 ve 8 sığmaz; hepsi sönük kalır. 4 sığar: yak, geriye 1 kalır. 2 sığmaz. 1 sığar: yak, geriye 0. Sonuç 00000101.
3. Aynı yolla 73’ü yaz. 128 sığmaz. 64 sığar, kalan 9. 32 ve 16 sığmaz. 8 sığar, kalan 1. 4 ve 2 sığmaz. 1 sığar, kalan 0. Sonuç 01001001; açılıştaki desenle aynı.
4. Son olarak 255: bütün kutuları yak. $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$; desen 11111111. Bu, sekiz kutunun söyleyebileceği en büyük sayı. Bir fazlası olan 256 için dokuzuncu bir kutu gerekir.

Kural tek: soldan sağa git, sığıyorsa yak, kalanla devam et. Her sayının tek bir deseni var; iki farklı desen aynı sayıyı vermez. Sekiz kutu 0’dan 255’e kadar 256 ayrı sayı tutar; bilgisayarlılar buna bir bayt der.

Ne oluyor? Sekiz kutu, sekiz ampul: sönük 0, yanık 1 demek. Değerler neden böyle? En sağdaki kutu en küçüğü, 1’i tutar. Sola

doğru her kutu, sağ komşusunun iki katını yüklenir: 1, 2, 4, 8, 16, 32, 64 ve en solda 128. Her yeni kutu, sağındaki kutuların toplamından bir fazlasını söyleyebilmeli; yoksa arada söylenemeyen sayılar kalırdı. Yanık kutuların değerlerini topla, sayın çıkar. Bilgisayarın bütün dünyası bu aç-kapa oyunu.

Şekil 1.2 · İkili kodu çöz



Şekil 1.2

Canlı demo: book.onuronder.com/d/7c96c14a25

Kendin dene. 1) 10 sayısını sekiz kutuyla yaz. 2) 10110000 deseni hangi sayı? 3) Yalnız en soldaki kutu yanıkrsa sayı kaç? Dokuzuncu bir kutu eklesen en büyük sayı ne olurdu?

TEKNİK DERİNLİK

Bu görüşe hesaplamacılık (computationalism) denir: zihin bir bilgi-işleme sistemidir ve biliş, sembol manipülasyonu biçiminde bir hesaplama. Kökleri Hobbess'a ("akıl yürütmek

TEKNİK DERİNLİK · DEVAM

hesaplamaktır”) uzanır; modern formu Putnam ve Fodor ile gelişti.

Pratikte iki taş kullanılır. İkili gösterim: modern sayısal bilgisayarlar bilgiyi çoğunlukla taban-2’de (bit’ler) kodlar; ikili gösterim hesaplamanın zorunlu koşulu değildir, ondalık ve analog makineler de hesap yapar (ENIAC ondalıktı). Boole cebiri (Boole, 1854) ve Shannon’ın 1937 tezi, mantığın elektrik anahtarlarıyla gerçekleştirilebileceğini gösterdi. Algoritma: sonlu, kesin tanımlı adımlar dizisi (terim, 9. yy matematikçisi el-Harezmi’den gelir). Bu ikisi algoritmaları mekanik olarak yürütülebilir kılar; düşünmenin bütünüyle hesaplamaya indirgenip indirgenemeyeceği ayrı bir sorudur.

İkili 01001001 = basamak değerlerinin toplamı = $64 + 8 + 1 = 73$. Her bit, soldan sağa 128, 64, 32, ... değerlerini temsil eder. 8 bit (1 bayt) 0–255 arası her sayıyı kodlar.

Genel kural: değer = $\sum b_i \cdot 2^i$, $i = 0 \dots 7$ sağdan sola sayılır; b_i ilgili bitin 0 ya da 1 değeridir. n bit 2^n ayrı sayı kodlar; 8 bit için $2^8 = 256$.

Alfabe ve tarif hazır; tarifi yürütecek makinenin hikâyesi 1800’lerde bir dişli çark hayaliyle başlıyor.

1.4 Babbage’tan Turing’e

İlk “bilgisayar” hayali 1800’lerde kuruldu. Charles Babbage dişlilerden ve kollardan oluşan dev bir hesap makinesi çizdi: Analitik Makine. Parası ve ömrü yetmedi; makinesini hiç göremeden öldü. Ama arkadaşı Ada Lovelace, o hayaldeki makine için ilk “tarifi” yazdı. Bu yüzden tarihin ilk programcısı sayılır.

Yaklaşık yüz yıl sonra Alan Turing hayali bir adım öteye taşıdı: Bandı okuyan, yazan ve iki yana kayan tek bir kutucuk, doğru kurallar ve yeterli bant verilirse, bir algoritmaya dökülebilen her hesabı

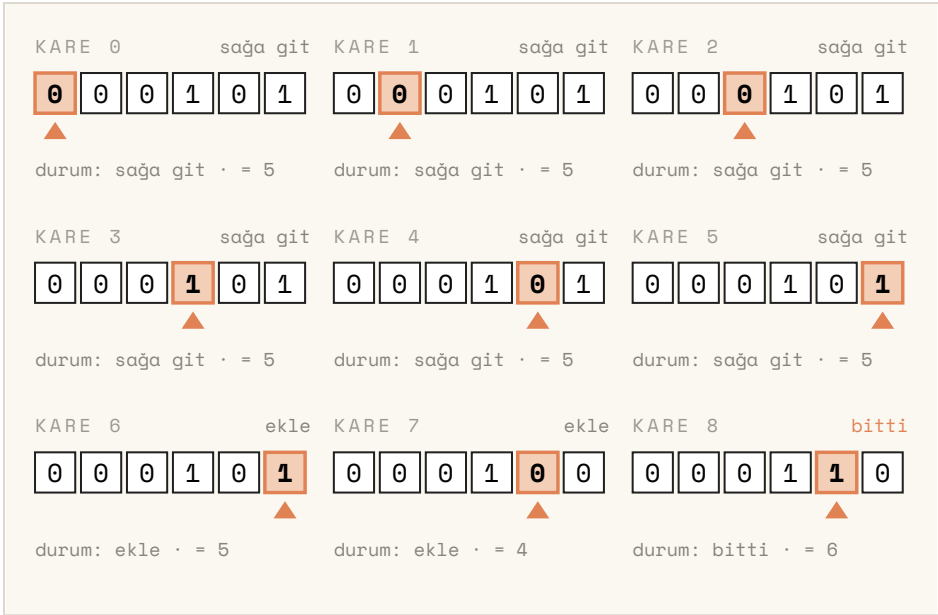
yapabilir. Buna Turing makinesi denir. Şekil 1.3'te, yalnızca sayıya 1 ekleyen küçük bir tanesinin nasıl düşündüğünü kare kare gör.

Kenar notu. Turing, 1948'de "Turochamp" adlı bir satranç algoritması tasarladı. O dönemin makineleri çalıştıramadığı için Turing hamleleri kâğıt üzerinde elle hesaplıyordu.

Kurulum. Şekilde altı gözlü bir bant var; üstünde gezen ok (▲) makinenin kafası. Bantta 000101 yazıyor, ikili sayı olarak 5. Makinenin üç durumu var: "sağa git", "ekle" (elde ile 1 ekle) ve "bitti". Film şeridindeki dokuz kare, 5'e 1 eklenirken atılan her adımı gösteriyor.

Adım adım. Kareleri sırayla oku; her karede bant, kafanın yeri ve durum var. Bant gözlerini soldan sağa sayarsan 0'dan 5'e.

0. Başlangıç. Kafa en soldaki gözde (0), durum "sağa git". Bant 0 0 0 1 0 1, sayı olarak 5.
1. Kafa bir kare sağa kayar (1). Bant değişmez. Kafa (▲), sayının en sağındaki basamağı bulmak için adım adım sağa gidiyor. Tıpkı elle toplama yapar gibi, işlem en sağdaki basamaktan başlar.
2. Kafa 2. gözde. Hâlâ yalnızca okuyor, yazmıyor.
3. Kafa 3. gözde; burada 1 var ama makine bu durumda 1'e aldırılmaz, geçer.
4. Kafa 4. gözde.
5. Kafa son gözde (5). Bant hâlâ 0 0 0 1 0 1.
6. Sağda göz kalmadı. Makine durum değiştirir: "ekle" (elde ile 1 ekle). Kafa yerinde, bant aynı.
7. Kafa son gözde 1 görür. 1'i 0 yapar, eldeyi bir sola taşır; kafa 4. göze geçer. Bant 0 0 0 1 0 0, bir anlığına 4. Kafa şimdi geriye, sola doğru 1 ekliyor: gördüğü her 1'i 0 yapıp eldeyi bir sola taşıyor; ilk 0'a rastladığında oraya 1 yazıp duruyor. Okulda öğrendiğimiz "elde var 1" kuralının aynısı.
8. Kafa 4. gözde 0 görür. Oraya 1 yazar ve durur; durum "bitti". Bant 0 0 0 1 1 0, sayı olarak 6. İşlem bitti.

Şekil 1.3 · Çalışan bir Turing makinesi (+1)

Şekil 1.3

Canlı demo: book.onuronder.com/d/ebaea4754b

Toplam sekiz hamle: beş kez sağa yürümek, bir durum değişimi, bir sıfırlama, bir yazma. Makine oku, yaz, sola ya da sağa git gibi bir avuç basit kuralla ikili bir sayıya 1 ekledi; beş artı bir altı etti, bant bunu 000110 diye söylüyor. Bu makinenin bütün marifeti 1 eklemek; başka bir hesap için başka bir kural tablosu gerekir. Bir Turing makinesinin düşünmesi bu: anlamı olmayan küçük mekanik adımlar, art arda.

Ne oluyor? Makine bandın üzerinde bir karınca gibi yürüyor: önce sağa gidip sayının ucunu buluyor, sonra dönüp 1 ekliyor. Eldeyi taşıyışı, senin kâğıtta toplama yapışından farksız. Bu makine yalnızca 1 ekliyor; başka kurallarla başka hesaplar yapılır. Bir tarife dökülebilen her hesap, sonunda bu minicik hamlelere iner.

Kendin dene. 1) Bandı 001111 (15) ile başlat ve kareleri kâğıtta sen çiz. Kaç tane 1 sıfırlanır, makine kaç hamlede biter, bantta hangi sayı kalır? 2) Bandı 010110 (22) ile başlat. Kafa sona varıp geri döndüğünde ilk gördüğü rakam ne? Kaç hamlede biter? 3) Bant 11111 (63) olsaydı ne olurdu? Makine en sol göze vardığında da 1 görüyor; sola gidemeyince durur.

TEKNİK DERİNLİK

Babbage'ın Analitik Makinesi (1837) bir “mill” (işlem birimi) ve “store” (bellek) içeren, delikli kartlarla programlanabilen genel amaçlı bir tasarımdı. Ada Lovelace'ın 1843 notları, Bernoulli sayılarını hesaplayan ve genelde ilk bilgisayar programı kabul edilen algoritmayı içerir.

Alan Turing'in 1936 tarihli “On Computable Numbers” makalesi, soyut Turing makinesini ve evrensel Turing makinesi fikrini tanıttı; bu, hesaplanabilirliğin biçimsel temelidir. Evrensel bir Turing makinesi, yeterli zaman ve bant verildiğinde, algoritmik olarak hesaplanabilir her işlemi uygun programla yürütebilir; aynı makale hesaplanamayan problemlerin de var olduğunu gösterir. Şekil 1.3'teki simülasyon ise evrensel değil, özel bir makinedir: bir ikili sayıya 1 ekler ve yalnızca oku/yaz ve durum geçişleriyle aritmetik yapar.

Makine sağa kayarak bandın sonuna gider, sonra sola dönerek ikili sayıya 1 ekler (elde mantığı): gördüğü 1'ler 0 olur, ilk 0 ise 1 olur.

Şekil 1.3'teki makinenin geçiş tablosu:

Durum	Okunan	Yazılan	Hareket	Yeni durum
sağa git	0 ya da 1	aynı	sağ (bant sonunda yerinde)	sağa git; bant sonundaysa “ekle”
ekle	1	0	sol	ekle; bant başındaysa “bitti”
ekle	0	1	dur	bitti

TEKNİK DERİNLİK · DEVAM

Üç satır, altı gözlü bant ve tek bir kafa: 0–62 arası her sayıya 1 eklemeye yeter. 63 için bant taşar ve 0 kalır.

Turing’in makinesi kâğıt üstünde bir düşünceydi. Onu gerçek lambalara ve kablolarla dökmek başka bir hikâye.

1.5 ENIAC ve modern bilgisayarın doğuşu

1945’te dev bir makine çalışmaya başladı: ENIAC; 1946’da kamuya tanıtıldı. Tonlarca ağırlığında, on binlerce lambayla çalışan bu makine, ilk genel amaçlı elektronik sayısal bilgisayarlardan biriydi. Huysuz bir yanı vardı: ona yeni bir iş öğretmek için kablolarını günlerce elle söküp takmak gerekiyordu.

Çözüm zarif bir fikirle geldi: tarifi de malzemelerin yanına, belleğe koy. Böylece makineyi yeniden kablolamak yerine yalnızca yeni talimat yüklersin. Cebindeki telefon dahil bugün neredeyse her bilgisayar bu düzeni kullanır. Aşağıdaki şekilde önce döngünün dönüşüne, sonra parçalarının ne yaptığına bak.

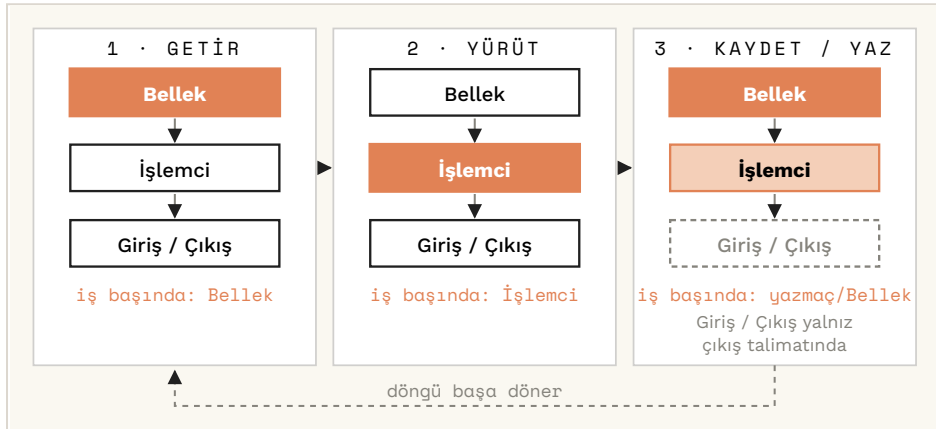
***Kenar notu.** ENIAC’ı yeniden programlamak günler sürerdi; kabloları elle bağlamak gerekirdi. Programı belleğe koyma fikri bu yüzden büyük bir adımdı.*

Kurulum. Şekilde üç kutu var: Bellek, İşlemci ve Giriş / Çıkış. Altlarında üç kare yan yana: “1 · Getir”, “2 · Yürüt”, “3 · Kaydet / Yaz”. Her karede o evrede iş başında olan parça koyu. Üçüncü kareden sonra döngü başa döner. Şekil temsili bir döngü; gerçek işlemcide her talimat üç eş süreli evreye bölünmez ve her talimat dış dünyaya çıkmaz.

Adım adım. Önce parçalar:

Evre	İş başındaki parça	Ne yapar
1 · Getir	Bellek	Hem programı (talimatları) hem veriyi birlikte saklar. Asıl yenilik burada: talimatlar da veri gibi tutulur.
2 · Yürüt	İşlemci	ALU hesaplar (toplama, karşılaştırma); Kontrol birimi bellekteki talimatları sırayla getirip yürütür.
3 · Kaydet / Yaz	Yazmaç ya da bellek; çıkış talimatında Giriş / Çıkış	Sonucu kaydet: sonuç bir yazmaca ya da belleğe yazılır. Yalnız bir çıkış talimatında sonuç Giriş / Çıkış'a gider; o parça dış dünyayla bağlantıdır: klavye, ekran, sensörler. Veri buradan girer, sonuç buradan çıkar.

Döngüyü küçük bir programla döndürelim. Bellekte iki satır olsun: birinci satırda “5 ile 3’ü topla”, ikinci satırda “sonucu ekrana yaz”. 5 ve 3 sayıları da bellekte dursun.

Şekil 1.4 · Getir – Yürüt – Yaz döngüsü

Şekil 1.4

Canlı demo: book.onuronder.com/d/093bb7d970

1. Getir. Kontrol birimi belleğe uzanır, birinci satırdaki talimatı alır: “5 ile 3’ü topla”. Talimat da 5 ve 3 gibi bellekten geldi; kablo değil, veri.
2. Yürüt. ALU toplamayı yapar: $5 + 3 = 8$. Sonuç şimdilik işlemcinin içinde.
3. Yaz. 8 belleğe geri konur; Giriş / Çıkış’ın bu turda işi yok, ekrana bir şey çıkmaz.
4. Getir. Döngü başa döndü; kontrol birimi ikinci satırı alır: “sonucu ekrana yaz”.
5. Yürüt. İşlemci 8’i bellekten okur ve çıkışa hazırlar.
6. Yaz. Ekranda 8 belirir. İki satırlık program bitti; sıradaki talimat varsa döngü sürer.

İki turda altı evre. ENIAC’ta birinci satırı değiştirmek günlerce kablo sökmek demektir. Burada tek yapılacak şey belleğin birinci satırına başka bir talimat yazmak, mesela “5 ile 3’ü çarp”. Döngü aynı kalır; yalnızca aldığı talimat değişir.

Ne oluyor? Bilgisayar hiç durmadan üç adımlık bir dans yapar: sıradaki talimatı hafızadan alır (getir), gereğini yapar (yürüt), sonucu bir yazmaca ya da belleğe kaydeder, çıkış talimatıysa ekrana verir (yaz). Telefonunun da bilgisayarının da kalbinde dönen döngü bu.

Kendin dene. 1) Telefonundaki hesap makinesine 7×6 yazdın. Üç evreyi kendi cümlelerinle sırala; her evrede hangi parça iş başında? 2) Yukarıdaki programa üçüncü bir satır ekle: “sonucu 2 ile çarp”. Kaç evre daha gerekir, belleğe hangi sayı yazılır? 3) Saniyede 3 milyar tur dönen bir işlemci düşün. Her tur üç evre. Bir saniyede kaç evre?

TEKNİK DERİNLİK

ENIAC (Mauchly & Eckert) ilk genel amaçlı elektronik dijital bilgisayarlardan biriydi; 1945’te çalışır duruma geldi, 1946’da kamuya tanıtıldı. ~17.500 vakum tüpü kullanıyor ve fişli panolarla yeniden kablolanarak programlanıyordu (ilk

TEKNİK DERİNLİK · DEVAM

programcıları altı kadın matematikçiydi). İç yapısında ikili değil, ondalık sayı düzeni kullanıyordu.

John von Neumann'ın 1945 EDVAC raporu, depolanmış-program ilkesini popülerleştirdi: program ve veri aynı bellekte tutulur. Mimari bir işlemci (ALU + kontrol birimi), bellek ve giriş/çıkıştan oluşur; bu parçalar bir veriyolu üzerinden haberleşir. İşlemci sürekli bir “getir-çöz-yürüt” döngüsü çevirir. “Von Neumann darboğazı” bu tasarımın bilinen sınırıdır.

“Getir” evresinde kontrol birimi bir sonraki talimatı bellekten okur ve çözer; “Yürüt”te işlemci işi yapar; “Yaz”da sonuç bir yazmaca ya da belleğe kaydedilir. Ekranı gönderme ayrı bir giriş/çıkış talimatıdır; her komut giriş/çıkış aygıtı kullanmaz.

Şekil 1.4'ün üç karesi, getir-çöz-yürüt döngüsünün temsili bir sadeleştirmesidir: çözme adımı “Getir”in içinde, kaydetme adımı ise yürütmenin sonunda ayrı bir kare olarak gösterildi.

Makine artık her tarifi yürütebiliyor. Bugün ona yapay zekâ derken bunun ne kadarını kastediyoruz?

1.6 Dar YZ mi, Genel YZ mi?

Filmlerdeki gibi her şeyi anlayan bir yapay zekâ henüz yok. Bugünkü YZ sistemleri “dar” (narrow) sayılır: eğitildiği işlerde ustadır; o işlerden uzaklaştıkça çabucak zorlanır. Satranç motoru seni yener ama ondan bir çorba tarifi isteyemezsin.

Alanlar arasında geniş ve aktarılabilir yetenekleri olan, insan gibi her alanda öğrenebilen varsayımsal sisteme “genel yapay zekâ” (AGI) denir; öylesi daha yapılmadı. Çok iş yapmak tek başına genel olmak demek değildir; bilinç ise apayrı bir soru. Aşağıdaki örnekleri sen ayır: hangisi bugün var, hangisi hâlâ bilim kurgu?

***Kenar notu.** Bir sistemin alanlar arasında geniş, aktarılabilir yetenekleri olması (genel) ile gerçekten “anlaması/bilinçli olması” (güçlü) farklı sorulardır. Bunları karıştırmamak modern YZ tartışmasının anahtarıdır.*

Kurulum. Şekilde üç sütun var: “Bugün kullanılan sistem”, “Varsayımsal sistem” ve “Bilinç sorusu”. Ortada beş kart duruyor. Kartların yanındaki kutuları kalemle sen dolduracaksın. Puan yok. Üçüncü sütun ayrı bir soru içindir: bilinç, genel yetenekle (AGI) aynı şey değildir.

Kendini sına. Her kartı bir sütuna koy. İlk iki sütunun ölçütü tek: sistem bugün gerçekten kullanılıyor mu, yoksa henüz yapılmamış bir varsayım mı? Karar verirken kartın ne kadar etkileyici olduğuna değil, bugün var olup olmadığına bak. Kart yetenekten çok bilinci soruyorsa üçüncü sütuna gider. Dördüncü kart genel yetenekle (AGI) ilgili. Beşinci kart bilinçle ilgili; bilinç sorusu ayrıdır, AGI tanımına girmez. Cevaplar ve gerekçeler kitabın sonunda.

#	Örnek	Bugün kullanılan sistem	Varsayımsal sistem	Bilinç sorusu
1	Satranç motoru	☐	☐	☐
2	Yüz tanıma sistemi	☐	☐	☐
3	Sohbet botu (dil modeli)	☐	☐	☐
4	Her mesleği insan gibi öğrenip yapan, kendi amaçları olan makine	☐	☐	☐
5	Kendini fark eden, bilinçli bir YZ	☐	☐	☐

Şekil 1.5 • Bugün var mı, yoksa bilim kurgu mu?

# Örnek	Bugün kullanılan sistem	Varsayımsal sistem	Bilinç sorusu
1 Satranç motoru	☐	☐	☐
2 Yüz tanıma sistemi	☐	☐	☐
3 Sohbet botu (dil modeli)	☐	☐	☐
4 Her mesleği insan gibi öğrenip yapan, kendi amaçları olan makine	☐	☐	☐
5 Kendini fark eden, bilinçli bir YZ	☐	☐	☐

Şekil 1.5

Canlı demo: book.onuronder.com/d/30661431ad

Beşini de işaretledikten sonra say: her sütunda kaç kart var? Dördüncü ve beşinci kartın ikisi de bugün yok, ama aynı soruyu sormuyor. Farkı bir cümleyle yazabiliyorsan bölümün ana fikri sende.

Ne oluyor? Belirli görevlerde ya da sınırlı bir görev kümesinde iyi olup insan düzeyinde alanlar arası genel öğrenme ve aktarım göstermeyen her sistem dar sınıfına girer; şiir de kod da yazan, istemdeki örneklerle yeni bir işe bir ölçüde uyum gösteren sohbet botları bile. Genel olan, alanlar arasında aktarılabilir yetenekle insan gibi her alanda öğrenebilen makinedir ve öylesi henüz yok. Bilinçli makine ise apayrı bir soru; genel olmak bilinç gerektirmez.

Kendin dene. 1) Kendi günlük hayatından üç YZ örneği yaz: navigasyon, çeviri, film önerisi gibi. Her birine bir sütun ver. 2) Dördüncü ve beşinci kartın ikisi de bugün yok; aynı soruyu mu soruyorlar? İkisine ayrı ayrı sor: her alanı öğrenebiliyor mu (genel), gerçekten anlıyor mu (güçlü)? Bir makine genel olup bilinçsiz olabilir

mi? 3) Satranç motoruna “bana bir çorba tarifi ver” desen ne olur? Cevabını, dar YZ’nin tanımını bir cümleyle yazarak ver.

TEKNİK DERİNLİK

Burada iki ayrı ayrım vardır. Dar/genel YZ bir yetenek ayrımıdır: dar YZ belirli görevlerde çalışır, genel YZ alanlar arasında geniş ve aktarılabilir yetenek demektir; bugünkü sistemler, çok görevli olanlar dahil, dar sınıftadır. Zayıf/güçlü YZ ise felsefi bir ayrımdır; terimleri felsefeci John Searle (1980, Çince Oda argümanı) ortaya attı. Searle’e göre zayıf YZ, zihni modelleyen ama gerçek anlama ya da bilince sahip olmayan bir programdır; bu, dar yetenek sınıfıyla aynı şey değildir.

Searle’ün “güçlü YZ” terimi makinenin gerçekten anlayıp anlamadığı (bir zihne sahip olup olmadığı) sorusuna dairdir; günlük dilde ise çoğu zaman “genel yapay zekâ” (AGI) ile karıştırılır. AGI, alanlar arası genelleme yapabilen varsayımsal bir yetenek düzeyidir; “güçlü YZ” ise felsefi bir iddiadır. İkisi aynı şey değildir.

AGI tanımında bile bilinç şartı yoktur: genel olmak, bilinçli olmak demek değildir.

Fikirler yüz yıldır ortadaydı. Öyleyse dar YZ bile neden ancak son yıllarda hayatımıza girdi?

1.7 Neden tam şimdi? Moore yasası

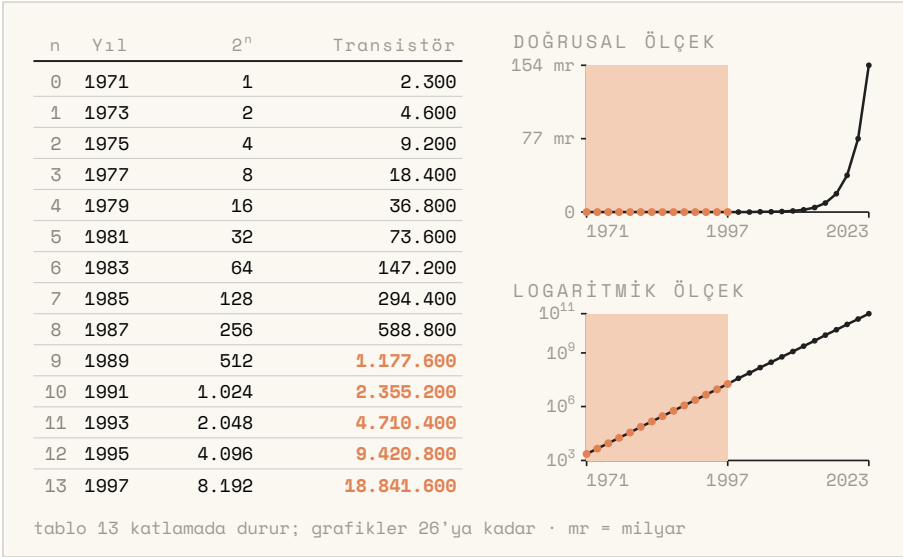
Fikirler 1900’lerin ortasında hazırды ama makineler cılızды. Şu eski piriñ masalını bilir misin? Satranç tahtasının ilk karesine bir piriñ, sonraki her kareye öncekinin iki katı... Tahtanın yarısında, 32 karede biriken piriñ $2^{32} - 1$ tane; tanesi 25 miligramsa yaklaşık 107 ton. Sonlara doğru miktar olağanüstü büyür. 1965’te Gordon Moore, çiplerdeki transistör sayısının tıpkı böyle düzenli aralıklarla, o sırada

yaklaşık her yıl, ikiye katlandığını fark etti; 1975'te ileriki dönem için bu aralığı kabaca iki yıl olarak revize etti.

İkiye katlanmak masum görünür ama tekrar tekrar olunca patlar. Aşağıda kendin dene: Şekil 1.6'daki tabloda sayıyı her iki yılda bir ikiye katla, nasıl fırladığını gör. Modern YZ'yi taşıyan da bu katlana katlana biriken işlem gücü.

***Kenar notu.** Kalınlığı 0.1 mm olan bir kâğıdı 42 kez ikiye katlayabilseydin kalınlığı yaklaşık 440 bin km olurdu; Ay'a olan uzaklıktan fazla. Üstel büyümenin gücü bu; çipler onlarca yıl bu hızla büyüdü.*

Şekil 1.6 • Üstel büyümeyi hisset



Şekil 1.6

Canlı demo: book.onuronder.com/d/d7182b2b78



Kurulum. Şekil 1971'den başlıyor. O yıl piyasaya çıkan ilk mikroişlemci Intel 4004'te 2.300 transistör vardı. Şekildeki her basamak “iki yıl geçti, sayı ikiye katlandı” demek; tablo 13

katlamada, 1997’de durur. Sağdaki iki küçük grafik aynı sayıları 2023’e, 26 katlamaya kadar taşır: doğrusal ölçekte eğri önce yere yapışır, sonra fırlar; logaritmik ölçekte düz bir çizgidir.

Adım adım. Tabloyu satır satır oku. Üçüncü sütun 2^n , dördüncü sütun 2.300×2^n . Her satır bir öncekinin tam iki katı; başka hiçbir kural yok. Bu, tarihsel Moore eğrisinin kendisi değil, üstel büyümeyi hissettirmek için kurulmuş temsili bir tablo.

Katlama (n)	Yıl	2^n	Transistör
0	1971	1	2.300
1	1973	2	4.600
2	1975	4	9.200
3	1977	8	18.400
4	1979	16	36.800
5	1981	32	73.600
6	1983	64	147.200
7	1985	128	294.400
8	1987	256	588.800
9	1989	512	1.177.600
10	1991	1.024	2.355.200
11	1993	2.048	4.710.400
12	1995	4.096	9.420.800
13	1997	8.192	18.841.600

İlk satırlar uslu: 2.300, 4.600, 9.200. On yılda sayı 32 katına çıkıyor ama hâlâ on binlerde. 1989’da bir milyon eşiği aşıyor: 1.177.600, yuvarlarsan 1.2 milyon. 1997’de 18.8 milyon. Tablo 13 katlamada, 26 yılda duruyor; şekildeki basamaklar 26 katlamaya kadar gidiyor.

Devam etsen ne olur? 20 katlamada, 2011’de, $2^n = 1.048.576$ ve sayı 2.411.724.800, kabaca 2.4 milyar. 26 katlamada, 2023’te, $2^n = 67.108.864$ ve sayı 154.350.387.200, kabaca 154.4 milyar. 1971 ile 2023

arasında, 26 katlamada, 67 milyon kat fark var. Pirinç masasında 32. karede 107 ton birikmişti; çipler 26. kareye 2023'te vardı. Sayı artışı hız artışı demek değil: daha çok transistör, her iş yükünde aynı oranda hızlanma getirmez.

Ne oluyor? Her satır “iki yıl geçti, transistör sayısı ikiye katlandı” demek. Birkaç satırda sayı kontrolden çıkıyor; üstel büyüme böyle bir şey. Transistör sayısı onlarca yıl bu hızla arttı; hız her iş yükünde aynı oranda artmadı ama bugünkü yapay zekâyı mümkün kılan birikim bu.

Kendin dene. 1) Tabloyu iki satır uzat: 1999 ve 2001 için 2ⁿ ve transistör sayısı. 2) 2.300'den başlayıp bir milyonu ilk geçen satır hangisi; kaç yıl sürdü? 3) Katlama süresi 2 değil 3 yıl olsaydı 1995'te kaç transistör olurdu? Tablodaki 1995 değeriyle karşılaştır.

TEKNİK DERİNLİK

Moore yasası bir doğa yasası değil, ampirik bir gözlem ve ekonomik eğilimdir. Moore'un 1965 öngörüsü, tümleşik devredeki bileşen sayısının yaklaşık her yıl ikiye katlanmasıydı; 1975'te ileriki dönem için bunu yaklaşık iki yılda bir katlanma olarak revize etti. Bu, hesaplama gücünün zorunlu olarak aynı hızda arttığını söyleyen bir fizik yasası değildir; daha çok transistör, her iş yükünde aynı oranda hız artışı demek değildir. Yine de bu üstel büyüme, derin öğrenmenin gerektirdiği yoğun matris hesaplamalarını ekonomik kıldı.

Üstel eğilim ~50 yıl sürdü (1971 Intel 4004: ~2.300 transistör → 2020'ler: on milyarlarca). Transistörlerin bazı yapıları atomik ölçeğe yaklaştıkça üretim ve fizik sınırları (ısı dahil) önem kazanıyor ve eğilim son yıllarda yavaşlıyor; sektör artık paralelleştirme ve GPU/TPU gibi özel YZ donanımına yöneliyor.

Üstel büyüme: yaklaşık her 2 yılda $\times 2$; n katlamada 2^n kat artış.

Şekil 1.6'nın formülü: $N(t) = N_0 \cdot 2^{(t/2)}$, $t = 1971$ 'den bu yana geçen yıl, $N_0 = 2.300$. $t = 52$ için $N = 2.300 \cdot 2^{26} \approx 1.5 \times 10^{11}$.

TEKNİK DERİNLİK · DEVAM

Gerçek çipler bu eğrinin biraz altında kalır; katlama süresi 2000'lerden sonra uzamıştır.

Zekâ, ikili kod, algoritma, Turing makinesi, depolanmış program, dar ile genel ayrımı, üstel büyüme. Aşağıdaki altı soru bunları yokluyor.

1.8 Kendini test et

Cevaplar kitabın sonunda.

1. Howard Gardner ne öne sürdü?
 - a) Beynin bir bilgisayar olduğunu
 - b) Zekânın tek değil, birçok türü olduğunu
 - c) Zekânın IQ ile tam ölçüldüğünü
 - d) Makinelerin asla düşünemeyeceğini
2. İkili kod hangi sembollerden oluşur?
 - a) A'dan Z'ye
 - b) Noktalar ve çizgiler
 - c) 0–9 arası
 - d) 0 ve 1
3. Evrensel bir Turing makinesi temelde ne yapabilir?
 - a) Yalnızca satranç oynar
 - b) Yeterli zaman ve bantla, basit kurallarla hesaplanabilir her işlemi
 - c) Sadece toplama yapar
 - d) Yalnızca metin saklar
4. Von Neumann mimarisinin temel fikri?
 - a) İnterneti icat etmek
 - b) Program ve veriyi aynı bellekte tutmak
 - c) Her iş için kabloları yeniden bağlamak
 - d) Ondalık yerine ikili kullanmak

5. Bugünkü yapay zekâlar hangi türdedir?

- a) Güçlü YZ
- b) Dar (narrow) YZ
- c) Genel YZ (AGI)
- d) Bilinçli YZ

6. Moore yasası ne der?

- a) İnternet hızı sabittir
- b) YZ insanı geçecek
- c) Transistör sayısı ~her 2 yılda ikiye katlanır
- d) Bilgisayarlar her yıl ucuzlar

Bu bölümden kalanlar

- Zekâ tek bir sayı değil; Gardner'ın sekiz türünde bugünkü YZ dilde ve mantıkta güçlü, bedende ve duyguda zayıf.
- Bugünün bilgisayarları iki taşla çalışır: ikili kod (0 ve 1) ve algoritma (net adım listesi). İkili kod yaygın bir seçimdir, hesaplamanın şartı değil; düşünmenin tümüyle hesaplama olup olmadığı ayrı bir soru.
- Sekiz kutu 0'dan 255'e her sayıyı tutar; yanık kutuların ağırlıklarını toplamak yeter.
- Turing makinesi yalnızca oku, yaz ve bir kare kay hamleleriyle çalışır; evrensel olanı, yeterli bant ve doğru programla, hesaplanabilir her işi yapar. Şekildeki özel makine yalnızca 1 ekler; 5'e 1 eklemek sekiz hamle sürdü.
- Depolanmış program: talimat da veri gibi bellekte durur; işlemci talimatı getirir, çözer, yürütür ve sonucu yazmaca ya da belleğe kaydeder. Ekrana gönderme ayrı bir giriş/çıkış talimatıdır.
- Bugünkü YZ sistemleri dardır, çok iş yapanlar bile; genel YZ (AGI) henüz yok; bilinç, genellikle ayrı bir soru.
- Moore yasası: transistör sayısı 1965 öngörüsünde her yıl, 1975 revizyonunda kabaca her iki yılda ikiye katlanır; sayı artışı hız artışıyla aynı şey değildir. Tablodaki 2.300, 26 katlamada 154 milyara ulaşır.

2

BÖLÜM 2

Kuralların Çağı

Öğrenmeden önce: elle yazılan zekâ

2.1 Kuralların çağı

Bugün yapay zekâ deyince verilerden öğrenen sistemleri düşünüyoruz. Oysa hikâyenin ilk büyük bölümü tam tersiydi: o çağın ustaları makineye dünyayı ezberletmeye çalıştı. Akıllı davranması için ne gerekiyorsa (bütün bilgileri, bütün kuralları) tek tek elle yazdılar.

Bu yaklaşıma “klasik” ya da “sembolik” YZ denir. Mantık, kurallar, arama ve uzman sistemler o çağın alet çantasıydı. Şimdi o çağa konuk oluyoruz: nasıl çalışıyordu, neleri başardı, neden bir gün tosladı? Hepsini kendin deneyeceksin.

***Kenar notu.** İlk YZ’ciler şöyle düşünüyordu: “Yeterince kural yazarsak, makine akıllı olur.” Bu fikir bazı işlerde çok iyi çalıştı, bazılarında ise hiç.*

TEKNİK DERİNLİK

Klasik YZ (sembolik YZ ya da GOFAl, açılımıyla “Good Old-Fashioned AI”), zekâyı biçimsel sembollerin kural-tabanlı manipülasyonu olarak ele alır. Temel varsayım: dünya hakkındaki bilgi açıkça temsil edilebilir ve akıl yürütme, bu temsiller üzerinde mantıksal işlemlerle yürütülebilir.

Bu paradigma 1950’lerden 1980’lere kadar baskındı ve mantık programlama, arama algoritmaları ve uzman sistemler gibi güçlü araçlar üretti. Bu bölüm bu araçları kurar; sonunda “bilgi edinme darboğazı” ve kırılganlık sorunlarının, zaten araştırılmakta olan öğrenmeye dayalı yaklaşımlara (Bölüm 3) ilgiyi neden artırdığını gösterir.

İlk soru en temeli: bir makine Tekir’in kedi olduğunu nereden bilir?

2.2 Bilgiyi sembollerle temsil etmek

Bir makineye Tekir'i nasıl öğretirsin? Makine onu göremez, okşayamaz; ancak senin yazdığın cümlelerden tanır: “Tekir bir kedidir”, “kedi bir memelidir”, “memeli bir hayvandır”. Klasik YZ bilgiyi böyle saklar: açık semboller ve aralarındaki bağlar.

İşin güzel yanı, makinenin bu bağları izleyip kimsenin söylemediği bilgiye kendisinin ulaşması. “Tekir bir hayvandır” cümlesini hiç duymadı; ama zinciri halka halka izleyince bunu kendisi bulur. Şekil 2.1'de dört soru var; makinenin düşünüşünü orada adım adım oku.

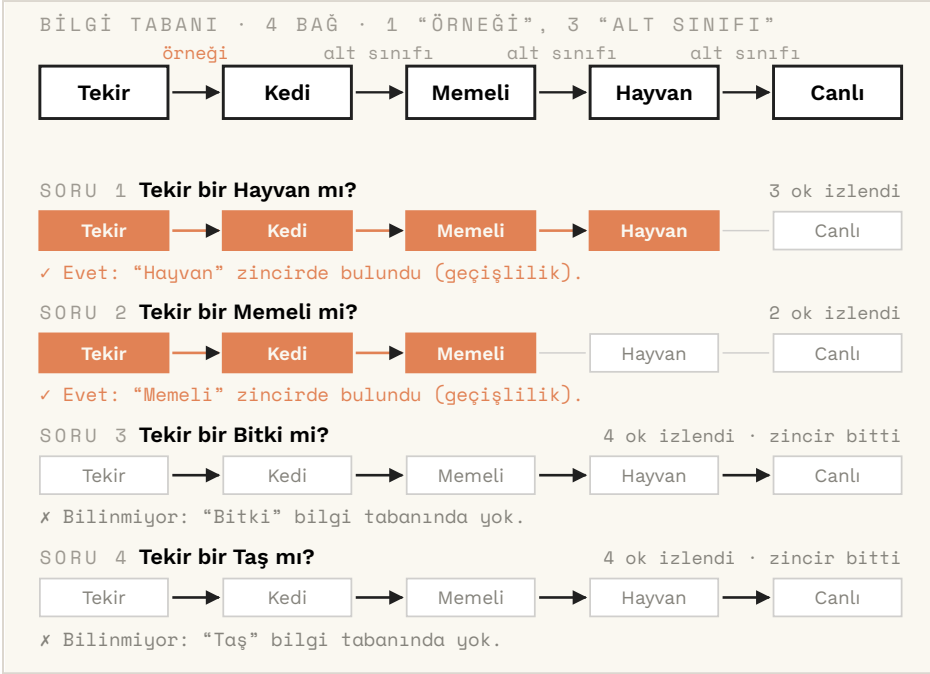
***Kenar notu.** Sembolik YZ'nin gücü: az sayıda kuraldan çok sayıda yeni bilgi türetebilir. Zayıflığı: önce o kuralları birinin yazması gerekir.*

Kurulum. Şekilde beş kutu tek sıra hâlinde dizili: Tekir, Kedi, Memeli, Hayvan, Canlı. Kutuları iki tür ok bağlar. İlk ok “örneği” diye etiketli ve bir bireyi sınıfına bağlar: Tekir, Kedi sınıfının bir örneğidir (instance-of). Öbür üç ok “alt sınıfı” diye etiketli ve bir sınıfı üst sınıfına bağlar: Kedi, Memeli'nin alt sınıfıdır (subclass-of); Memeli de Hayvan'ın, Hayvan da Canlı'nın. Makinenin bütün bilgisi bu dört ok; başka hiçbir şey bilmiyor. Altta dört soru var; her soru için makine zinciri baştan yürür ve kararını söyler.

Adım adım. Önce “Tekir bir Memeli mi?” sorusu.

1. Makine Tekir'den başlar. Aradığı sınıfın adı “Memeli”. Tekir bir sınıf değil, bir birey; ondan çıkan “örneği” oku, Tekir'in Kedi sınıfına ait olduğunu söyler.
2. Kedi kutusuna geçer. Bu kutunun adı “Memeli” değil; Kedi'den çıkan “alt sınıfı” oku Memeli'ye gider.
3. Memeli kutusuna varır. Ad eşleştirdi: Tekir, Kedi'nin örneği; Kedi de Memeli'nin alt sınıfı; öyleyse Tekir bir memelidir. Şekilde ilk üç kutu turuncuya boyanır ve karar yazılır: ✓ Evet: “Memeli” zincirde bulundu (geçişlilik).

Şekil 2.1 • Bilgi zinciriyle çıkarım



Şekil 2.1

Canlı demo: book.onuronder.com/d/4ea6179a6f

İki ok izledi. “Tekir bir Memeli’dir” cümlesini kimse yazmadı; makine onu türetti.

Sonra “Tekir bir Bitki mi?” sorusu.

1. Tekir: birey; “örneği” oku Kedi’ye.
2. Kedi: kutunun adı “Bitki” değil; “alt sınıfı” oku Memeli’ye.
3. Memeli: ad eşleşmedi; ok Hayvan’a.
4. Hayvan: ad eşleşmedi; ok Canlı’ya.
5. Canlı: ad eşleşmedi. Canlı’dan çıkan ok yok, zincir bitti. Karar: **x** Bilinmiyor: “Bitki” bilgi tabanında yok.

Dört sorunun tamamı:

Soru	İzlenen ok	Karar
Tekir bir Hayvan mı?	3	Evet
Tekir bir Memeli mi?	2	Evet
Tekir bir Bitki mi?	4, zincir bitti	Bilinmiyor
Tekir bir Taş mı?	4, zincir bitti	Bilinmiyor

Karar “Hayır” değil, “Bilinmiyor”. Makine Tekir’in bitki olmadığını iddia etmiyor; yalnızca elindeki oklarla bu bilgiye ulaşamadığını söylüyor. Bilmediği konuda susmak sembolik sistemlerin iyi huyudur.

Ne oluyor? Makine yalnızca “kim kimin türü” bilgisini tutuyor: Tekir bir kedi, kedi bir memeli... Sorunca zinciri halka halka izliyor ve kimsenin ona söylemediği bir bilgiye kendisi ulaşıyor. Zincirde yoksa da “bilinmiyor” diyor.

Kendin dene. 1) Zincirin sonuna bir ok daha ekle: Canlı bir Varlık’tır. “Tekir bir Varlık mı?” sorusuna makine ne der, kaç ok izler? 2) Soruyu ters çevir: “Kedi bir Tekir mi?” Oklar tek yönlü; makine ne cevaplar? Bu cevap sence doğru mu? 3) Makineye “Tekir bir Bitki değildir” dedirtmek için bilgi tabanına ne eklemek gerekirdi?

TEKNİK DERİNLİK

Sembolik YZ’de bilgi, bilgi temsili (knowledge representation) ile kodlanır: semantik ağlar, çerçeveler (frames), ontolojiler ya da mantık önermeleri. Varlıklar ve aralarındaki ilişkiler (ör. is-a, has-a) açıkça tanımlanır.

Çıkarım (inference), bu temsiller üzerinde kuralların uygulanmasıdır. Şekil 2.1’deki gösterim bir is-a hiyerarşisinde geçişliliği (transitivity) kullanır. Ontolojide iki bağ ayrılır: Tekir, Kedi sınıfının bir örneğidir (instance-of); Kedi, Memeli sınıfının alt sınıfıdır (subclass-of). “Tekir instance-of Kedi” ve “Kedi subclass-of Memeli” ise “Tekir instance-of Memeli” türetilir; bu nedenle Tekir bir memelidir. Şekil 2.1 iki bağı ayrı etiketle

TEKNİK DERİNLİK · DEVAM

gösterir: ilk ok “örneği”, öbürleri “alt sınıfı”. Sembolik akıl yürütmenin özü budur.

Bilgi tabanında bir instance-of (örneği) bağı ve onu izleyen subclass-of (alt sınıfı) bağları var. Hedef zincirde varsa “Evet”, yoksa “Bilinmiyor”.

Geçişlilik kuralı sınıflar arasında biçimsel olarak şöyle yazılır: $\text{subclass-of}(A, B) \wedge \text{subclass-of}(B, C) \rightarrow \text{subclass-of}(A, C)$. Örnek için: $\text{instance-of}(a, B) \wedge \text{subclass-of}(B, C) \rightarrow \text{instance-of}(a, C)$. Motor bu kuralı zincir boyunca tekrar tekrar uygular; beş düğümlük zincirde en fazla dört adımda ya hedefe ulaşır ya da zincirin sonuna gelir.

Zincir yalnızca “bir ...dır” bağlarını izledi. Makine bildiğinden eyleme nasıl geçer, yağmur yağıyorsa ne yapmalı? Bunun için “eğer ... ise” kuralları gerekiyor.

2.3 Eğer... ise...: kurallar ve uzman sistemler

Klasik YZ’nin kalbi küçücük bir cümleyle atar: “EĞER şu doğruysa, O ZAMAN şunu yap.” Buna kural denir. Kuralları üst üste koyunca, bir alanda usta gibi davranan sistemler doğar. Bunlara uzman sistemler denir; bir doktorun bilgisini soru-cevap kurallarına döken çıraklar gibi.

1980’lerde bu çıraklar tıptan mühendisliğe her yerde çalıştı. Aşağıda minik bir tanesi var: koşulları sen aç kapa; hangi kuralın ateşlendiğini, önerinin nasıl doğduğunu Şekil 2.2’de gör.

***Kenar notu.** Bir kuralın çıktısı başka bir kuralı tetikleyebilir; buna “zincirleme” denir. Sistem böylece tek bir olgudan uzun bir akıl yürütme zinciri kurabilir.*

Kurulum. Şekilde üstte üç olgu var: “Yağmur yağıyor” açık (✓), “Hava soğuk” ve “Rüzgârlı” kapalı (○). Altında beş kural kartı, R1’den R5’e; her kartın solunda EĞER koşulu, sağında O ZAMAN önerisi. Ateşlenen kart koyulaşır; R5’in yanındaki “zincir” işareti, onun başka bir kurala bağlı olduğunu söyler. En altta ateşlenen kuralların önerileri yan yana dizilir; şeklin alt tablosu üç senaryoyu listeler, aşağıda tek tek açılıyor. Başlangıçta yalnız R1 yanıyor: Şemsiye al.

Adım adım. Sistem her seferinde aynı üç işi yapar: açık olgulara bakar, her kuralın EĞER kısmını bu olgularla karşılaştırır, tutanları ateşler. Üç senaryo:

Senaryo	Açık olgular	Ateşlenen kurallar	Öneri satırı
A (başlangıç)	Yağmur	R1	Şemsiye al
B	Yağmur, Rüzgâr	R1, R5	Şemsiye al · Dikkat: şemsiye ters dönebilir!
C	Soğuk, Rüzgâr	R2, R3	Mont giy · Atkı tak

Senaryo A’da yalnız yağmur açık. R1’in koşulu “Yağmur yağıyor”: tutuyor, ateşler. R2 ve R3 soğuk ister, R5 rüzgâr ister; üçü de sessiz. R4 “yağmur YOK” der; yağmur var, o da sessiz.

Senaryo B’de rüzgârı da açtık. R5’in EĞER kısmında bir olgu değil, başka bir kuralın sonucu var: “Şemsiye al” tetiklendi. Bu yüzden motor iki tur döner. Birinci turda R1 ateşler ve çalışma belleğine “şemsiye” notunu düşer. İkinci turda R5 bu notu görür, rüzgârla birleştirir ve ateşler. İki olgudan iki öneri çıktı; ikincisi ilkinin üstüne kurulu, zincirleme dedikleri bu. Yağmuru kapatırsan R1 söner, not düşülmez ve R5 de susar: rüzgâr tek başına yetmez.

Şekil 2.2 • Küçük bir uzman sistem

OLGULAR • SENARYO A

✓ Yağmur yağıyor

○ Hava soğuk

○ Rüzgârlı

EĞER

O ZAMAN

R1	Yağmur yağıyor	→	Şemsiye al
R2	Hava soğuk	→	Mont giy
R3	Soğuk VE rüzgârlı	→	Atkı tak
R4	Yağmur YOK VE soğuk DEĞİL	→	Hafif giyinebilirsin
R5 zincir	“Şemsiye al” tetiklendi VE rüzgârlı	→	Dikkat: şemsiye ters dönebilir!

SİSTEM ÖNERİSİ

Şemsiye al

ÜÇ SENARYO

Senaryo	Açık olgular	Ateşlenen Öneri satırı
A (başlangıç)	Yağmur	R1 Şemsiye al
B	Yağmur, Rüzgâr	R1, R5 Şemsiye al • Dikkat: şemsiye ters dönebilir!
C	Soğuk, Rüzgâr	R2, R3 Mont giy • Atkı tak

Şekil 2.2

Canlı demo: book.onuronder.com/d/045489d51c

Senaryo C’de yağmur kapalı, soğuk ve rüzgâr açık. R1 ve R5 sessiz. Soğuk R2’yi ateşler; soğuk VE rüzgâr R3’ü. R4 yağmur yok diye umutlanır ama “soğuk DEĞİL” koşulu tutmaz; sessiz kalır.

Ne oluyor? Sistem, açtığın her koşula bakıp elindeki kurallarla karşılaştırıyor: hangisinin “EĞER” kısmı tutuyorsa o kural ateşleniyor ve önerisini söylüyor. R5’e dikkat: o, başka bir kuralın sonucunu bekliyor. Kurallar birbirine böyle zincirleniyor.

Kendin dene. 1) Üç olgu da açık: hangi kurallar ateşler, öneri satırında kaç öneri olur? 2) Yalnız “Rüzgârlı” açık. Sistem ne önerir? Bu öneri sana mantıklı geliyor mu; hangi kural eksik? 3) R5’i yalnızca “Rüzgârlı” koşuluna bağlasaydık ne kaybederdik?

TEKNİK DERİNLİK

Üretim kuralları (production rules) IF-THEN biçimindedir; bir çıkarım motoru (inference engine) çalışma belleğindeki olgularla eşleşen kuralları tetikler. İleri zincirleme (forward chaining) olgulardan sonuçlara, geri zincirleme (backward chaining) hedeften kanıtı doğru ilerler.

Uzman sistemler (ör. MYCIN, 1970'ler) bu mimariyi kullandı: bir bilgi tabanı (kurallar) + çıkarım motoru. Şekil 2.2'deki gösterimde bir kuralın ürettiği olgunun başka bir kuralı tetiklemesini (zincirleme) de göreceksin. Sınır: kuralların elle yazılması ve istisnaların patlaması.

R5 zincirlemedir: yalnızca R1 ateşlediyse (şemsiye) ve rüzgâr varsa tetiklenir.

Gösterimin motoru iki geçişlidir. Birinci geçiş yalnızca olgulara bağlı kuralları değerlendirir ve ürettikleri türetilmiş olguları (R1 → şemsiye) çalışma belleğine ekler. İkinci geçiş bütün kuralları bu genişletilmiş bellekle yeniden değerlendirir. Daha uzun zincirler için aynı döngü, yeni olgu üretilmeyene kadar tekrarlanır.

Kurallar ne yapılacağını söylüyor; ama seçenek binlerceyse hangisini önce denemeli? Klasik YZ'nin ikinci aracı bunun için var: arama.

2.4 Arama ve sezgisel kısayollar

Labirentte çıkışı bulmak, satrançta hamle seçmek, şehirde rota çizmek... Hepsi aynı oyunun çeşitleri: Önünde bir sürü olasılık var ve içlerinden hedefe götüreni arıyorsun.

En sabırlı yöntem her ihtimali tek tek denemektir ama bu çok yavaş olabilir. Sezgisel yöntem (heuristic) kestirmeden gider: "Hangi yön daha umut verici?" diye tahmin yürütür. Şekil 2.3'te ikisini yarıştırdık: sezgisiz olan her yeri tarar, sezgisel olan burnunu hedefe çevirir.

***Kenar notu.** Ağgözlü sezgisel yöntemler hız kazandırır ama bedeli vardır: bazen en iyi çözümü kaçırabilirler. “Yeterince iyi”yi “mükemmel”e tercih ederler. A* gibi daha dikkatli yöntemler, uygun bir sezgiyle garantiyi geri alır.*

Kurulum. Şekilde 8 sütun, 6 satırlık bir ızgara var; 48 karenin 11’i siyah duvar. Sol üst köşede S (başlangıç), sağ alt köşede H (hedef). Duvarlar üç dikey engel oluşturuyor: 3. sütunda üstten dört kare, 5. sütunda alttan dört kare, 7. sütunda ortada üç kare. Bu yüzden düz gitmek imkânsız; yol birinci engeli alttan, ikincisini üstten, üçüncüsünü yine alttan dolaşmak zorunda. İki panel aynı ızgarayı gösteriyor: solda sezgisiz arama, sağda sezgisel arama. Açık turuncu kareler taranmış, koyu turuncu kareler bulunan yol. Her karedeki sayı tarama sırası; S 1’dir.

Adım adım. Sezgisiz arama (BFS):

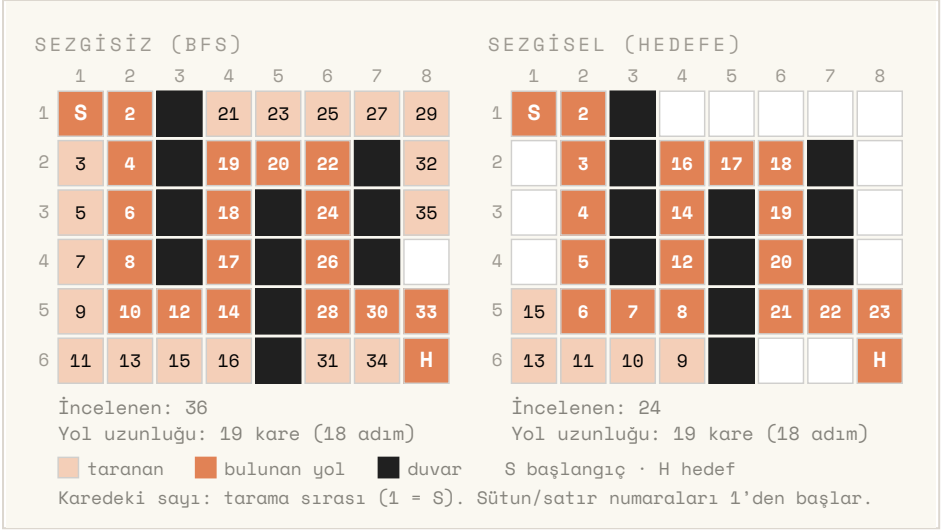
1. S’den başlar, komşularını bir sıraya alır; sıraya ilk giren ilk çıkar.
2. Halka halka yayılır: önce S’ye bir adım uzaktaki kareler, sonra iki adım, sonra üç. Yön bilgisi yok; sol koridoru, alt satırı, orta koridoru aynı sabırla tarar.
3. H sıradan çıktığında durur. Şekildeki sayaç “İncelenen: 36” diyor. 37 boş karenin 36’sı; yalnız 8. sütun, 4. satırdaki kareye el değmedi.
4. Yol geriye doğru okunur: 19 kare (S ve H dâhil), 18 adım. Bu, olabilecek en az adımlı yol; ızgarada her adım aynı bedelde olduğu için en ucuz yol da bu.

Ağgözlü sezgisel arama (hedefe):

1. Her kareye bir puan verir: hedefe sokak mesafesi, sütun farkı artı satır farkı. S için $7 + 5 = 12$.
2. Sıradan hep en düşük puanlı kareyi çeker. Sol koridordan aşağı iner (puan her adımda düşer), birinci engelin altından geçip 4. sütuna varır.
3. Burada aldanır: alt satırdaki kareler H ile aynı satırda olduğundan puanca yakın görünür. 5. sütun, 6. satırdaki duvar sağa giden yolu

- kestiği için alt satırı sola doğru dener: 4, 3, 2, 1. sütun, sonra 1. sütun, 5. satır. Beş kare boşa gitti.
4. Puan sırasına göre yukarı döner, 4. sütundan 2. satıra çıkar, 6. sütundan iner, H'ye ulaşır. Şekildeki sayaç “İncelenen: 24” diyor.
5. Yol: yine 19 kare, 18 adım.

Şekil 2.3 · Yol bulma: sezgisiz ile sezgili



Şekil 2.3

Canlı demo: book.onuronder.com/d/b8544b96e9

Yöntem	Taranan kare	Yol (kare)
Sezgisiz (BFS)	36	19
Açgözlü sezgisel (hedef)	24	19

Bu ızgarada açgözlü arama üçte bir daha az kare gezdi ve yine en kısa yolu buldu. Ama bu bir garanti değil, şans: puanı düşük görünen bir çıkmaz onu beş kare oyaladı. Daha sinsi bir labirentte aynı huy onu uzun bir dolambaca da sokabilirdi. BFS'nin 36 karesi, garantinin bedeli. Gidilen yolu da puana katan A*, aynı sezgiyle hem garantiyi hem tasarrufu korur.

Ne oluyor? İki arayıcıyı yarıştıyorsun. Sezgisiz olan her yönü sabırla tarar; her adım aynı bedeldeyse sonunda en az adımlı yolu bulur ama çok kare gezer. Ağgözlü sezgisel olan hep hedefe doğru koşar; az kare gezer ama bazen en kısa yolu kaçıır. Hız ile garanti arasındaki takas bu; A^* gibi yöntemler gidilen yolu da hesaba katarak garantiyi geri alır.

Kendin dene. 1) Sezgisel puanı iki kare için hesapla: 4. sütun, 2. satır ve 1. sütun, 6. satır. Hangisi hedefe daha yakın görünür? Hangisi gerçekten yolun üstünde? 2) S'nin puanı 12, ama en kısa yol 18 adım. Fark nereden geliyor? 3) 5. sütun, 6. satırdaki duvar kaldırılrsa en kısa yol kaç adım olur?

TEKNİK DERİNLİK

Pek çok klasik YZ problemi durum-uzayı araması (state-space search) olarak modellenir. Bilgisiz (uninformed) arama, örneğin genişlik-öncelikli arama (BFS), hiçbir yön bilgisi kullanmadan sistematik tarar; tüm kenarların maliyeti eşitse en az adımlı, dolayısıyla en düşük maliyetli yolu garanti eder, ama çok düğüm açar. Farklı kenar maliyetlerinde başka yöntemler gerekir.

Bilgili (informed) arama, bir sezgisel fonksiyon $h(n)$ ile hedefe yakınlığı tahmin eder; ağgözlü en-iyi-öncelikli arama yalnızca h 'yi kullanır (hızlı ama eniyilik garantisi yok), A^* ise $g(n)+h(n)$ kullanır: h kabul edilebilirse (gerçek uzaklığı hiç aşmıyorsa) ağaç aramasında en kısa yolu bulur; graf aramasında ayrıca h 'nin tutarlı olması ya da daha iyi bir yolla ulaşılan düğümlerin yeniden açılması gerekir. Şekil 2.3'te BFS ile sezgisel aramanın taradığı hücre sayısını karşılaştır.

BFS (bilgisiz) bir FIFO kuyruğuyla katman katman genişler ve eş-maliyetli ızgarada en kısa yolu garanti eder. Sezgisel (ağgözlü en-iyi-öncelikli) arama, hedefe Manhattan uzaklığı $h(n)$ 'yi en aza indiren düğümü seçer; çok daha az hücre açar ama en kısa yolu garanti etmez.

Bu ızgarada, sıfırdan başlayan koordinatlarla $h(n) = |x - 7| + |y - 5|$. Sonuçlar: BFS 36 düğüm açtı, ağgözlü arama 24; ikisi de 18 adımlık en kısa yolu buldu. h gerçek uzaklığı hiçbir düğümde

TEKNİK DERİNLİK · DEVAM

aşmadığı için (kabul edilebilir) ve komşu kareler arasında en fazla 1 değiştiği için (tutarlı), aynı h ile A^* graf aramasında da en kısa yolu bulur ve genellikle BFS'den az düğüm açar.

Arama, dünyanın kesin olduğunu varsaydı: duvar duvardır, hedef yerinde durur. Yarın yağmur yağacak mı sorusu ise kesin cevap tanımaz.

2.5 Belirsizlikle başa çıkmak: olasılık ve Markov

Katı kurallar gerçek dünyada tökezler, çünkü dünya belirsizdir. “Yağmur yağarsa şemsiye al” demesi kolay; peki yağacak mı? Kimse kesin bilemez. Olsa olsa ihtimalini söyleriz.

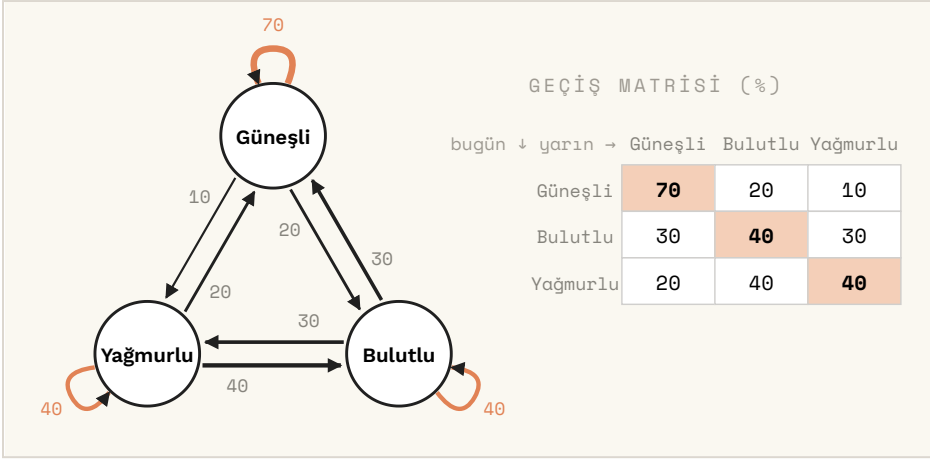
Klasik YZ buna zarif bir çözüm buldu: durumdan duruma olasılıkla geçmek. Markov zinciri bunun en ünlüsüdür; hileli bir zarla oynanan hava durumu oyunu gibi. Şekil 2.4'ün altındaki tabloda yedi günlük bir örnek var; havanın olasılıklara göre değişimini orada takip et.

***Kenar notu.** Markov özelliği: “şimdiyi biliyorsan gelecek daha eski geçmişe bağlı değildir; buraya nasıl geldiğin önemli değil.” Basit görünür ama hava durumundan Google aramasına kadar her yerde.*

Kurulum. Şekilde üç durum kutusu var: Güneşli, Bulutlu, Yağmurlu; oklarla birbirine bağlı, her okun üstündeki sayı o geçişin yüzdesi. Yanında aynı sayılar geçiş matrisi olarak: satır bugün, sütun yarın; her satırın toplamı 100. Yedi günlük bir zincirin gün gün gidişi aşağıda.

bugün \ yarın	Güneşli	Bulutlu	Yağmurlu
Güneşli	70	20	10
Bulutlu	30	40	30
Yağmurlu	20	40	40

Şekil 2.4 • Hava durumu Markov zinciri



Şekil 2.4

Canlı demo: book.onuronder.com/d/8c423f9c01

Adım adım. Önce hileli zar. Bugün güneşli olsun; tablonun ilk satırı 70 / 20 / 10. Yüz yüzlü bir zar düşün: 1'den 70'e kadar güneş, 71'den 90'a bulut, 91'den 100'e yağmur. Makine 1 ile 100 arasında bir sayı çeker; sayı hangi aralığa düşerse yarın odur. Zar hileli, çünkü güneşten sonra yine güneş gelme payı yüzde 70; yağmur yalnızca yüzde 10.

Sonra yedi günlük bir zincir. Canlı gösterimde sayıları makine rastgele çeker; burada yedi çekilişi biz seçtik ki adımları takip edebilesin. Her satırda önce bugünün aralıklarına bak, sonra çekilen sayıyı yerleştir.

Gün	Bugün	Aralıklar (G / B / Y)	Çekilen sayı	Yarın
1	Güneşli	1–70 / 71–90 / 91–100	42	Güneşli
2	Güneşli	1–70 / 71–90 / 91–100	83	Bulutlu
3	Bulutlu	1–30 / 31–70 / 71–100	55	Bulutlu
4	Bulutlu	1–30 / 31–70 / 71–100	91	Yağmurlu
5	Yağmurlu	1–20 / 21–60 / 61–100	77	Yağmurlu

Gün	Bugün	Aralıklar (G / B / Y)	Çekilen sayı	Yarın
6	Yağmurlu	1-20 / 21-60 / 61-100	12	Güneşli
7	Güneşli	1-70 / 71-90 / 91-100	64	Güneşli

Yedi günün sonunda sayaç: 3 güneşli, 2 bulutlu, 2 yağmurlu; dağılım yüzde 43 / 29 / 29. Dördüncü gün yağmura geçerken makine önceki üç güne hiç bakmadı; “bugün bulutlu” bilgisi ve 91 sayısı yetti.

Son olarak uzun vade. Yedi gün az; bu zincirde her durumdan her duruma geçiş mümkün olduğundan yüzlerce gün sonra oranlar, başlangıç ne olursa olsun, sabitlenir. Sabitlenen oranı zar atmadan da bulabilirsin. Uzun vadede güneşli günlerin payı G, bulutluların B, yağmurluların Y olsun. Bir güneşli gün üç yoldan gelir: güneşten sonra (0.7·G), buluttan sonra (0.3·B), yağmurdan sonra (0.2·Y). Oranlar sabitse bu toplam yine G olmalı:

$$G = 0.7 \cdot G + 0.3 \cdot B + 0.2 \cdot Y \quad B = 0.2 \cdot G + 0.4 \cdot B + 0.4 \cdot Y \quad Y = 0.1 \cdot G + 0.3 \cdot B + 0.4 \cdot Y, \text{ ve } G + B + Y = 1.$$

Çözüm: $G = 6/13$, $B = 4/13$, $Y = 3/13$; yaklaşık yüzde 46 / 31 / 23. Sağlama: $0.7 \cdot 6 + 0.3 \cdot 4 + 0.2 \cdot 3 = 4.2 + 1.2 + 0.6 = 6$. Gösterim yeterince uzun çalışınca alttaki dağılım bu sayıların etrafında salınır.

Ne oluyor? Yarının havası yalnızca bugüne bakılarak tahmin ediliyor; bugün bilinince dünün önemi kalmıyor. Her gün makine bir zar atıyor ama zar hileli: güneşli bir günden sonra yine güneş gelme ihtimali yüksek. Günler biriktikçe dağılım hep aynı orana oturur.

Kendin dene. 1) Bugün yağmurlu, çekilen sayı 35: yarın hava ne? 2) Bugün güneşli. İki gün sonra yağmurlu olma olasılığı kaç? İpucu: yarının üç ihtimalini ayrı ayrı hesapla, topla. 3) Güneşli satırını 90 / 5 / 5 yapsan uzun vadeli dağılım hangi yöne kayar? Önce tahmin et, sonra ilk denklemlerle kontrol et.

TEKNİK DERİNLİK

Belirsizlik altında akıl yürütmek için olasılıksal modeller kullanılır. Markov zinciri, mevcut durum bilindiğinde bir sonraki durumun daha eski geçmişe bağlı olmadığı (Markov özelliği: mevcut duruma koşullu bağımsızlık) bir stokastik süreçtir;

TEKNİK DERİNLİK · DEVAM

geçişler, her satırının toplamı 1 olan bir olasılık matrisiyle tanımlanır.

Sonlu, indirgenemez ve periyodik olmayan bir zincirin durum dağılımı, başlangıç ne olursa olsun, tek bir kararlı dağılıma (stationary distribution) yakınsar; kararlı dağılımın var olması tek başına yakınsama demek değildir. Bu fikir, gizli Markov modelleri, PageRank ve pekiştirmeli öğrenmedeki Markov karar süreçlerine kadar uzanır. Şekil 2.4'teki gösterimde uzun vadeli dağılımın nasıl oluştuğunu gözlemle.

Geçiş matrisi P sabittir; her yeni gün, P 'nin mevcut satırından bir örneklem üretir.

Kararlı dağılım π şu iki denklemin çözümüdür:

$$\pi P = \pi \quad \sum \pi_i = 1$$

Bu P için $\pi = (6/13, 4/13, 3/13) \approx (0.462, 0.308, 0.231)$. Matrisin her girdisi pozitif olduğundan zincir indirgenemez ve periyodik değildir; üç durumlu sonlu zincir bu yüzden her başlangıçtan aynı π 'ye yakınsar. Güneşli başlangıçtan beklenen dağılımın gidişi: 1. gün (0.70, 0.20, 0.10), 2. gün (0.57, 0.26, 0.17), 3. gün (0.51, 0.29, 0.20), 5. gün (0.47, 0.30, 0.23). Yakınsama beş günde büyük ölçüde tamamlanır; gösterimdeki sayaç ise örneklem olduğundan bu değerlerin etrafında dalgalanır.

Olasılık kurallara esneklik kattı; ama tabloyu, kuralları, zinciri hâlâ bir insan elle yazıyor. Bu yükün ne kadar taşınabileceği konusunda YZ'ciler ikiye bölündü.

2.6 Düzenliler ve dağınıklar (neats ve scruffies)

YZ araştırmacıları yıllarca iki kampa bölündü. “Düzenliler” (neats) her adımın temiz matematikle kanıtlanmasını istedi. “Dağınıklar” (scruffies) ise omuz silkti: “Çalışıyorsa iyidir, teorisini sonra buluruz.”

Bu kavga yalnızca tarih değil; bugün de sürüyor. Aşağıdaki ifadeleri doğru kampa ayır. Sonunda anlaşılacak: klasik YZ neden duvara tosladı ve bu çarpışma, zaten araştırılan bir fikre, makinelerin “öğrenmesine”, ilgiyi nasıl artırdı?

***Kenar notu.** Klasik YZ'nin dersi: dünyanın tüm kurallarını elle yazmak imkânsız. Çözüm, makineye kuralları biz vermek yerine onları veriden kendisinin bulmasını öğretmek. Sıradaki bölümün konusu bu.*

Kurulum. Şekilde dört ifade kartı alt alta duruyor; her kartın yanında iki seçenek: Düzenli (Neat) ve Dağınık (Scruffy). İşaretini kartın yanına koy; sonra kitabın sonundaki cevaplarla karşılaştı. Kartlar birer slogan gibi kısa; her biri bir araştırmacının ağzından çıkmış olabilecek tek cümle.

Kendini sına. Her ifade için kampı seç: Düzenli mi, Dağınık mı?

1. Her şeyi biçimsel mantıkla kanıtla.
2. Çalışan kısayolları kullan, teoriyi sonra düşün.
3. Matematiksel kesinlik şarttır.
4. Gerçek dünya dağınıktır; esnek ol.

Şekil 2.5 · Hangi yaklaşım?

#	Örnek	Düzenli (Neat)	Dağınık (Scruffy)
1	Her şeyi biçimsel mantıkla kanıtla	☐	☐
2	Çalışan kısayolları kullan, teoriyi sonra düşün	☐	☐
3	Matematiksel kesinlik şarttır	☐	☐
4	Gerçek dünya dağınıktır; esnek ol	☐	☐

Şekil 2.5

Canlı demo: book.onuronder.com/d/330f95d0cf



Kelimeye değil tutuma bak: konuşan önce doğruluğu mu göstermek istiyor, önce işe yaramasını mı? İlki Düzenli, ikincisi Dağınık. Cevaplar ve gerekçeler kitabın sonunda.

Ne oluyor? İki kamp, iki karakter: Düzenliler her adımı matematikle kanıtlamak ister; Dağınıklar “önce çalışsın, teorisi sonra gelir” der. İkisinin de haklı çıktığı yerler var; bugünün yapay zekâsı ikisinin karışımı.

Kendin dene. 1) Bu bölümün beş yöntemini iki kampa dağıt: bilgi zinciri, uzman sistem, sezgisiz arama, sezgisel arama, Markov zinciri. Hangisi kesin sonuç vaat ediyor, hangisi “yeterince iyi” ile yetiniyor? 2) Sezgisel puan (Şekil 2.3) hangi kampın icadı? A*’ın ona eklediği garanti onu hangi kampa taşır? 3) Kendi hayatından bir örnek bul: bir işi önce “çalışsın” diye, teorisini sonra düşünerek çözdüğün bir an.

TEKNİK DERİNLİK

“Neat” ve “scruffy” ayrımı (Roger Schank’a atfedilir), YZ’de yöntemsel bir gerilimi tanımlar: biçimsel, kanıtlanabilir, ilkeli yaklaşımlar (neats; mantık ve olasılık kuramı gibi) ile sezgisel, mühendislik-odaklı, ampirik yaklaşımlar (scruffies) arasında.

Klasik sembolik YZ iki temel sınıra çarptı: bilgi edinme darboğazı (tüm kuralları elle yazmak ölçeklenmez) ve kırılganlık (öngörülmeleyen durumlarda çökme). Bu sınırlar, zaten araştırılmakta olan öğrenmeye dayalı yöntemlere ilgiyi artırdı; sembolik ve öğrenmeye dayalı yaklaşımlar uzun süre birlikte gelişti (Rosenblatt’ın perceptron’u 1958 tarihlidir). Bilgiyi elle kodlamak yerine veriden öğrenen istatistiksel YZ’ye (Bölüm 3) geçiş böyle hızlandı.

Kavga iki kampı da kısmen haklı çıkardı; nasıl olduğu sıradaki bölümde. Önce altı soru.

2.7 Kendini test et

Cevaplar kitabın sonunda.

1. Klasik (sembolik) YZ bilgiyi nasıl temsil eder?
 - a) Verilerden öğrenerek
 - b) Sadece görsellerle
 - c) Açık semboller ve kurallarla
 - d) Rastgele tahminle
2. “Yağmur yağıyorsa şemsiye al” ifadesi nedir?
 - a) Bir olasılık
 - b) Bir sinir ağı
 - c) Bir kural (IF-THEN)
 - d) Bir veri kümesi
3. Açgözlü (greedy) sezgisel aramanın temel özelliği?
 - a) Her zaman en iyi çözümü garanti eder
 - b) Rastgele tahmin eder
 - c) Veriden öğrenir
 - d) Çözümü hızlandırır ama en iyiyi garanti etmez
4. Markov zincirinde bir sonraki durum neye bağlıdır?
 - a) Yalnızca şu anki duruma
 - b) Tüm geçmişe
 - c) Geleceğe
 - d) Hiçbir şeye
5. Klasik YZ'nin en büyük zorluğu neydi?
 - a) Tüm kuralları elle yazmak ve dünyanın dağınıklığı
 - b) Çok hızlı olması
 - c) İnternete bağımlı olması
 - d) Çok ucuz olması

6. “Neat” ve “Scruffy” neyi tanımlar?

- a) YZ araştırmasında iki farklı yaklaşımı
- b) İki robot türünü
- c) İki programlama dilini
- d) İki bilgisayar markasını

Bu bölümden kalanlar

- Klasik YZ bilgiyi açık semboller ve elle yazılmış kurallar olarak saklar.
- Bir “bir ...dır” zinciri (örnekten sınıfa, sınıftan üst sınıfa), geçişlilik sayesinde kimsenin yazmadığı bilgiyi türetir; zincirde olmayan için “bilinmiyor” der.
- Uzman sistem, açık olgularla eşleşen EĞER-O ZAMAN kurallarını ateşler; bir kuralın sonucu başka bir kuralı tetikleyebilir.
- Eş maliyetli ızgarada sezgisiz arama en az adımlı yolu garanti eder ama çok kare gezer; ağgözlü sezgisel arama az gezer ama garanti vermez; A^* , kabul edilebilir bir sezgiyle ikisini birleştirir.
- Markov zincirinde, bugün bilinince yarın daha eski geçmişe bağlı değildir; her duruma ulaşılabilen, döngüye kilitlenmeyen sonlu bir zincirde dağılım uzun vadede tek bir orana oturur.
- Düzenliler kanıt, Dağınıklar işe yarayan çözüm ister; bugünün YZ’si ikisinden de pay taşır.
- Kuralları elle yazmak ölçeklenmez ve kırılımandır; bu sınırlar, sembolik çağla birlikte yürüyen öğrenme araştırmalarına ilgiyi artırdı.

3

BÖLÜM 3

Makineler Nasıl Öğrenir

Kuraldan örüntüye

3.1 Makineler nasıl öğrenir?

Bir çocuğa kediye nasıl öğretirsin? “Dört bacaklı, bıyıklı, kuyruklu...” diye kural yazarak mı? Hayır; kedileri gösterirsin, çocuk gerisini kendisi çözer. Geçen bölümde kural yazmanın duvarına toslamıştık. Ya makineler de çocuklar gibi öğrenseydi?

Makine öğrenmesinin bütün fikri bu kadar basit: bol bol örnek göster, örüntüyü kendisi yakalasın. Veri nasıl hazırlanır, bir model her denemede nasıl birazcık daha ustalaşır? Hepsini adım adım göreceksin.

***Kenar notu.** Klasik YZ: “İşte kurallar, uygula.” Makine öğrenmesi: “İşte örnekler, kuralı sen bul.” Bu küçük fikir, modern yapay zekânın tamamını mümkün kıldı.*

TEKNİK DERİNLİK

Makine öğrenmesi (ML), açıkça programlanmış kurallar yerine verideki istatistiksel örüntülerden bir fonksiyon öğrenen yaklaşımdır. Klasik YZ “bilgiyi nasıl temsil ederiz?” diye sorarken, ML “girdiden çıktıya eşlemeyi veriden nasıl tahmin ederiz?” diye sorar.

Kurulum üç parçadan oluşur: bir model, parametreleri (ağırlıkları) aracılığıyla girdileri çıktılarına eşler; bir kayıp fonksiyonu tahminlerin ne kadar yanlış olduğunu ölçer; bir optimizasyon yöntemi (ör. gradyan inişi) parametreleri kaybı azaltacak yönde günceller. Bu bölüm bu döngüyü ve temel görev türlerini kurar.

Örnekle öğrenmenin ilk şartı, örneğin neye benzediğini bilmek. Bir makineye örnek derken ona ne veriyoruz?

3.2 Özellik ve etiket

Bir dedektif düşün: elinde ipuçları var, bir de dosyanın sonucu, yani doğru cevap. Makine de örnekle öğrenirken aynı ikiliye bakar. İpuçlarına özellik denir: bir örneği tarif eden ölçülebilir bilgiler. Doğru cevaba da etiket denir. Bir e-postada ipuçları “kaç kelime”, “link veya şifre isteği var mı”, “bedava geçiyor mu” olabilir; etiketse “Spam” ya da “Normal”dir.

Model, yüzlerce çözülmüş dosyaya baka baka hangi ipucunun hangi cevapla gittiğini öğrenir. Şekil 3.1’deki örneklerden birine bak; ipuçlarını ve doğru cevabını gör.

***Kenar notu.** “Çöp girerse çöp çıkar” burada da geçerli: özellikler kötüyse, en iyi model bile öğrenemez. Veriyi anlamak, çoğu zaman modeli seçmekten daha önemlidir.*

Kurulum. Şekilde dört kısa e-posta ve her biri için üç ipucu var: “bedava” geçiyor mu, link veya şifre isteği var mı, aciliyet dili kullanıyor mu. Link veya şifre isteği, okurdan bir bağlantıyı açmasını ya da şifresini vermesini istemek demektir. Üç ipucu sütununun üstünde “Özellikler (girdi)”, son sütunun üstünde “Etiket (çıktı)” yazar. Dolu işaret (✓) ipucunun var olduğunu, boş halka (○) ipucunun bulunmadığını gösterir. En sağ sütun doğru cevabı, yani etiketi verir: Spam ya da Normal. Dört örneğin tamamı aşağıdaki tabloda.

Adım adım.

#	E-posta	"bedava" geçiyor	link veya şifre isteği	aciliyet dili	Etiket
1	"Bedava iPhone kazandınız! Hemen tıklayın"	✓	✓	✓	Spam
2	"Toplantı yarın saat 10:00'da"	○	○	○	Normal
3	"ACİL: hesabınız kapanacak, şifrenizi girin"	○	✓	✓	Spam
4	"Rapor taslağını ekte gönderdim"	○	○	○	Normal

Şekil 3.1 • Özellikleri ve etiketi gör

		ÖZELLİKLER (GİRDİ)			ETİKET (ÇIKTI)
#	E-posta	"bedava" geçiyor	link veya şifre isteği	aciliyet dili	Etiket
1	"Bedava iPhone kazandınız! Hemen tıklayın"	✓	✓	✓	Spam
2	"Toplantı yarın saat 10:00'da"	○	○	○	Normal
3	"ACİL: hesabınız kapanacak, şifrenizi girin"	○	✓	✓	Spam
4	"Rapor taslağını ekte gönderdim"	○	○	○	Normal

✓ ipucu var ○ ipucu yok

Şekil 3.1

Canlı demo: book.onuronder.com/d/1946fb9748

1. Tabloyu satır satır oku. Her satır bir örnektir: ilk üç işaret sütunu o örneğin özellikleri, son sütun etiketi. Model yalnız bu ikiliyi

- görür, e-postanın kendisini değil.
2. Birinci e-posta üç ipucunun üçünü de taşıyor ve etiketi Spam. 3. e-posta “bedava” demiyor; ama şifre istiyor ve acele ettiriyor. O da Spam.
 3. İkinci ve dördüncü e-postada üç ipucundan hiçbirisi yok. İkisi de Normal.
 4. Sütunlarla etiket arasındaki ilişkiye bak. İşaret sayısı sıfırsa etiket hep Normal; iki ya da üçse hep Spam. “Bedava” tek başına belirleyici değil: 3. e-posta onsuz da Spam çıktı. Link veya şifre isteği ile aciliyet dili ise iki Spam örneğinde de var.
 5. Bu ilişkiyi biz yazmadık; tablodan okuduk. Model de aynısını yapar, yalnız dört değil binlerce satırla. Dört satırdan çıkan “iki işaret varsa spam” kuralı geçici bir tahmindir; beşinci örnek onu bozabilir.

Ne oluyor? Bir e-postayı tarif eden ipuçlarını (özellikler) ve doğru cevabı (etiket) yan yana koyuyoruz: “bedava” geçmesi, link veya şifre isteği, aciliyet dili... Makine bol örnek görerek hangi ipuçlarının “Spam” ile birlikte gittiğini kendi kendine öğrenir; kuralı biz yazmayız, o örneklerden çıkarır.

Kendin dene. 1) “Şifrenizin süresi doldu, bugün yenilemezseniz hesabınız silinir” e-postasının üç ipucunu işaretle. Tablodaki ilişkiye göre etiketi ne olur? 2) “Bedava kahve için yarın öğlen mutfakta buluşalım” e-postası gerçekte Normal. Bu satır tabloya eklenirse model “bedava” ipucuna daha çok mu, daha az mı güvenmeli? 3) Dört satırın hepsiyle uyuşan, tek cümlelik bir kural yaz. Kuralın 2. sorudaki e-postada da doğru çalışıyor mu?

TEKNİK DERİNLİK

Denetimli öğrenmede her örnek, bir özellik vektörü x ile bir etiket y çiftidir. Özellikler sayısal ya da kategorik olabilir; model $f(x) \approx y$ eşlemesini öğrenmeye çalışır. İyi özellik seçimi (feature engineering), klasik ML’de başarıyı belirleyen en önemli adımlardan biridir.

TEKNİK DERİNLİK · DEVAM

Etiketin türü görevi belirler: kategorik etiket → sınıflandırma, nicel (sürekli) etiket → regresyon. Kategoriler sayıyla kodlanabilir; bu onları regresyon hedefi yapmaz. Aşağıdaki örnekte basit özelliklerle bir e-postanın spam olup olmadığını ayırt etme sevgisini göreceksin.

Özellik vektörü x = [“bedava” geçiyor; link veya şifre isteği; aciliyet dili], etiket y = “Spam”. Denetimli öğrenmede model bu (x, y) çiftlerinden $f(x) \approx y$ eşlemesini, yani $P(\text{spam} | x)$ gibi bir karar kuralını kestirmeye çalışır.

Spam örneğinde her e-postanın doğru cevabı elimizdeydi. Cevap anahtarı olmadan, hatta öğretmen olmadan da öğrenilir mi?

3.3 Üç öğrenme türü

Makinelerin okulunda üç tür öğretmen var. Birincisi cevap anahtarıyla gezer: “Bu spam, bu değil” diye tek tek gösterir (denetimli). İkincisi hiç cevap vermez: “Sen ayıkla bakalım” der, makine veriyi kendi gruplar (denetimsiz). Üçüncüsü hiç ders anlatmaz; makine oyuna girer, denedikçe ödül ya da ceza toplar (pekiştirmeli).

Şimdi aşağıdaki görevlerin hangi öğretmene ait olduğunu bul. Her birinde kendine sor: Cevap anahtarı verilmiş mi, makine kendi mi ayıkıyor, yoksa deneye deneye mi öğreniyor?

***Kenar notu.** İpucu: “Doğru cevaplar verilmiş mi?” → Denetimli. “Makine kendi mi gruplandırıyor?” → Denetimsiz. “Deneyip ödül mü topluyor?” → Pekiştirmeli.*

Kurulum. Şekilde altı görev ve üç sütun var: Denetimli, Denetimsiz, Pekiştirmeli. Her görevin satırında, sütun başına bir tane, üç boş

kutu var; doğru olanı sen işaretle. Kurşun kalem kullan; cevaplar ve gerekçeler kitabın sonunda.

Kendini sına. Her görev için üç soruyu sırayla sor. Doğru cevaplar baştan verilmiş mi? Makine veriyi kendi mi gruplandırıyor? Yoksa deneyip ödül mü topluyor? Sonra kutulardan birini işaretle.

1. Etiketli fotoğraflardan kedi/köpek ayırmak · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli
2. Müşterileri benzerliklerine göre gruplara ayırmak · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli
3. Bir robotun yürümeyi deneme-yanılmayla öğrenmesi · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli
4. Geçmiş (etiketli) verilerden ev fiyatı tahmini · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli
5. Bir oyunu oynayarak yüksek skoru öğrenmek · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli
6. Etiketsiz haberleri konularına göre kümelemek · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli

Bitirince iki soru daha. Altı görevin üçünde “etiketli” ya da “etiketsiz” sözcüğü açıkça geçiyor; ötekilerde etiketin var olup olmadığını neye bakarak anladın? 3 ile 5’in ortak noktası ne: ikisinde de doğru cevap kimden geliyor?

Ne oluyor? Makineye öğretmenin üç yolu var: doğru cevapları göstererek (denetimli), hiç cevap vermeden kendi gruplamasını isteyerek (denetimsiz) ya da deneyip ödül/ceza alarak (pekiştirmeli). Her görevi doğru türe ayır; aradaki sınırlar her zaman keskin değildir.

Kendin dene. 1) Şu üç görevi de aynı üç kutuya yerleştir. Bir çeviri programının insan çevirmenlerin yaptığı milyonlarca çeviriden öğrenmesi; bir satranç programının kendi kendine oynayarak güçlenmesi; bir marketin fişlerinden “birlikte alınan ürün” gruplarını bulması. 2) Üç soruyu bir karar ağacına çevir: ilk soru hangisi olmalı, hangi cevapta duruyorsun? 3) İki kutuya birden yakışan bir görev yaz; sınırların neden her zaman keskin olmadığını bu örnekla açıkla.

Şekil 3.2 • Hangi öğrenme türü?

#	Örnek	Denetimli	Denetimsiz	Pekiştirmeli
1	Etiketli fotoğraflardan kedi/köpek ayırmak	☐	☐	☐
2	Müşterileri benzerliklerine göre gruplara ayırmak	☐	☐	☐
3	Bir robotun yürümeyi deneme-yanılmayla öğrenmesi	☐	☐	☐
4	Geçmiş (etiketli) verilerden ev fiyatı tahmini	☐	☐	☐
5	Bir oyunu oynayarak yüksek skoru öğrenmek	☐	☐	☐
6	Etiketsiz haberleri konularına göre kümelemek	☐	☐	☐

Şekil 3.2

Canlı demo: book.onuronder.com/d/3afa1bb6c0**TEKNİK DERİNLİK**

Denetimli öğrenme, etiketli (x, y) çiftlerinden $f(x) \approx y$ öğrenir (sınıflandırma/regresyon). Denetimsiz öğrenme yalnızca x 'lerden yapı çıkarır (kümeleme, boyut indirgeme). Pekiştirmeli öğrenme, bir ajanın bir ortamda ödül sinyalini en üst düzeye çıkaracak bir politika öğrenmesidir.

Bu sınırlar keskin değildir: yarı-denetimli ve öz-denetimli (self-supervised) öğrenme aradadır. Modern büyük dil modelleri büyük ölçüde öz-denetimli ön eğitimle, yani etiketleri verinin kendisinden üreterek eğitilir. Aşağıdaki görevleri üç temel türe ayır.

TEKNİK DERİNLİK · DEVAM

Üç paradigma: denetimli (etiketli (x, y) çiftleri), denetimsiz (yalnızca x 'ten yapı), pekiştirmeli (ödül sinyali).

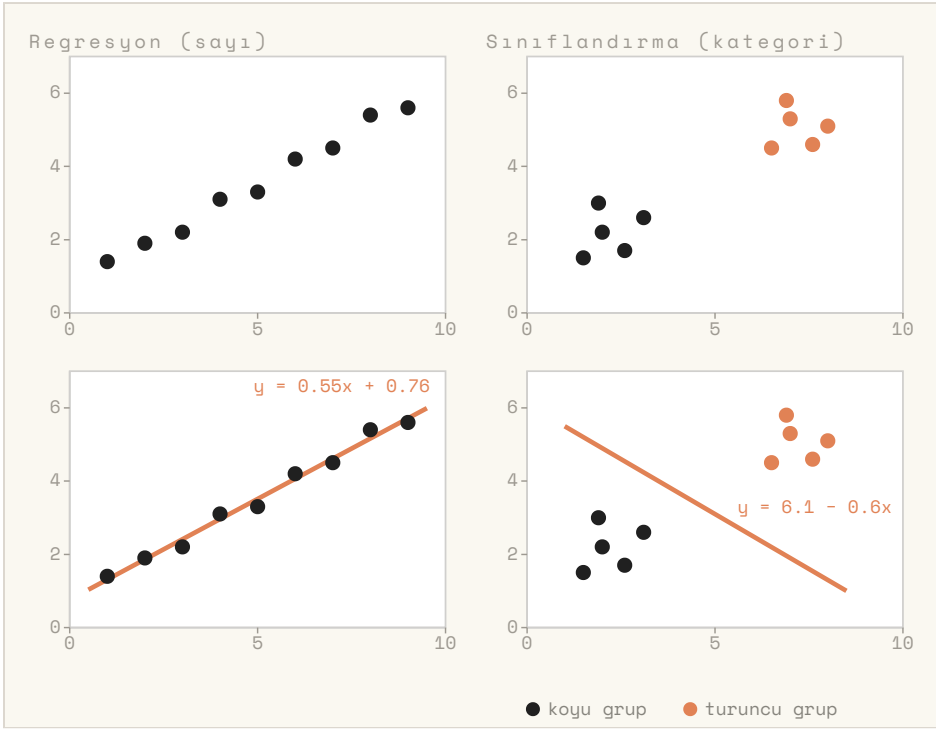
Denetimli kutusuna iki görev düştü: kedi/köpek ayırmak ve ev fiyatı tahmin etmek. İkisi de etiketli; ama biri bir kategori, öteki bir sayı istiyor. Bu fark neyi değiştirir?

3.4 Sınıflandırma ve regresyon

Denetimli öğrenmenin sorduğu iki temel soru var. Biri “ne kadar?” der: bu evin fiyatı kaç lira? Buna regresyon denir; cevabı bir sayıdır. Öteki “hangisi?” der: bu e-posta spam mı, değil mi? Buna da sınıflandırma denir; cevabı bir kategoridir.

İkisini de aşağıda kendin dene. Regresyonda noktaların tam ortasından geçen “en iyi doğru”yu bulacaksın; sınıflandırmada iki grubun arasına bir sınır çiziceksin.

***Kenar notu.** Basit ayırım: model bir miktar tahmin ediyorsa regresyon, bir kategori seçiyorsa sınıflandırma. Ev fiyatı bir miktardır; spam kararı bir kategoridir, kategoriler sayıyla kodlansa bile.*

Şekil 3.3 • İki temel görev

Şekil 3.3

Canlı demo: book.onuronder.com/d/3324149b70

Kurulum. Şekil iki görevi yan yana gösterir. Solda “Regresyon (sayı)”: dokuz koyu nokta ve onlara uydurulan en iyi doğru. Sağda “Sınıflandırma (kategori)”: beş koyu, beş turuncu nokta ve aralarına çizilen sınır. Her panelin üst karesi noktaların ham hâli, alt karesi doğru ya da sınır çizilmiş hâli.

Adım adım.

1. Regresyon verisi dokuz noktadır: (1, 1.4), (2, 1.9), (3, 2.2), (4, 3.1), (5, 3.3), (6, 4.2), (7, 4.5), (8, 5.4), (9, 5.6). x büyüdükçe y büyüyor, ama noktalar tam bir doğru üstünde değil.

2. En küçük kareler doğrusu iki sayıyla bulunur. x 'lerin ortalaması 5, y 'lerin ortalaması 3.51. Eğim $m = 33 / 60 = 0.55$; kesişim $b = 3.51 - 0.55 \cdot 5 = 0.76$. Doğru: $y = 0.55x + 0.76$.
3. Doğru her noktaya ne kadar yakın? Ölçü, aynı x 'teki dikey fark: gözlenen y eksi doğrunun dediği y . Buna artık denir; doğruya en kısa (dik) uzaklık değildir. $x = 3$ 'te doğru 2.41 der, nokta 2.2'dir; artık -0.21 . $x = 8$ 'de doğru 5.16 der, nokta 5.4; artık $+0.24$. Dokuz artığın hiçbirisi 0.25'i geçmez. Artıkların karelerinin toplamı 0.22.
4. Bu toplam neden "en iyi"nin ölçüsü? Göz kararı çizilmiş başka bir doğruyu dene: $y = 0.5x + 1$. Dikey artıkların kareleri toplamı 0.37'ye çıkar. En küçük kareler doğrusu, bu toplamı olası bütün doğrular arasında en küçük yapan tek doğrudur.
5. Sınıflandırma verisi iki gruptur. Koyu: (1.5, 1.5), (2, 2.2), (2.6, 1.7), (3.1, 2.6), (1.9, 3). Turuncu: (6.5, 4.5), (7, 5.3), (7.6, 4.6), (6.9, 5.8), (8, 5.1).
6. Sınır, (1, 5.5) ile (8.5, 1) noktalarından geçen doğrudur: $y = 6.1 - 0.6x$. Kontrol et: $x = 3.1$ 'de sınır 4.24 der, koyu nokta 2.6 ile altında kalır. $x = 6.5$ 'te sınır 2.2 der, turuncu nokta 4.5 ile üstünde. On noktanın onu da doğru tarafta.
7. İki cevap iki türdür. Regresyon bir sayı verir: $x = 10$ için $0.55 \cdot 10 + 0.76 = 6.26$. Sınıflandırma bir taraf verir: sınırın altı koyu, üstü turuncu.

Ne oluyor? İki temel iş var. Regresyon bir sayı tahmin eder: noktaların arasından geçen "en iyi doğru"yu çizeriz; en iyi doğru, her noktanın aynı x 'teki dikey farkının karelerini topladığımızda bu toplamı en küçük yapan doğrudur. Sınıflandırma ise bir grubu ötekenden ayıran bir sınır çeker.

Kendin dene. 1) $y = 0.55x + 0.76$ doğrusuna göre $x = 6.5$ için tahmin kaç? 2) (4.5, 3.5) noktası sınırın hangi tarafında kalır; koyu mu, turuncu mu? Ya (5, 3)? 3) Regresyon verisine (9, 9) gibi uzak bir nokta eklensin. Eğim artar mı, azalır mı? Doğrunun tek bir noktanın peşinden gitmesi bir sorun mudur?

TEKNİK DERİNLİK

Regresyon sürekli bir hedefi tahmin eder; en basit hâli, hata karelerinin toplamını en aza indiren en küçük kareler doğrusudur. Sınıflandırma kategorik bir hedefi tahmin eder ve sınıfları ayıran bir karar sınırı öğrenir (ör. lojistik regresyon, destek vektör makineleri).

Şekil 3.3'teki gösterimde regresyon için noktalara en küçük kareler doğrusu uydurulur; sınıflandırma için iki kümeyi ayıran doğrusal bir karar sınırı gösterilir. Gerçekte karar sınırları doğrusal olmak zorunda değildir.

Regresyonda hedef sürekli bir sayıdır; Şekil 3.3'teki doğru en küçük kareler doğrusudur. Sınıflandırmada iki grubu ayıran bir karar sınırı çizilir.

En küçük kareler doğrusunun kapalı biçimi: $m = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$, $b = \bar{y} - m \cdot \bar{x}$. Şekil 3.3'ün verisinde pay 33, payda 60'tır. Küçültülen nicelik dikey artıkların kareleri toplamıdır, $\sum(y_i - m \cdot x_i - b)^2$; doğruya dik uzaklık değil.

Buraya kadar her noktanın etiketi ya da sayısı elimizdeydi. Etiketleri tamamen kaldırıncaya ortada yalnız noktalar kalır. Makine yine de bir düzen bulabilir mi?

3.5 Kümeleme ve anomali

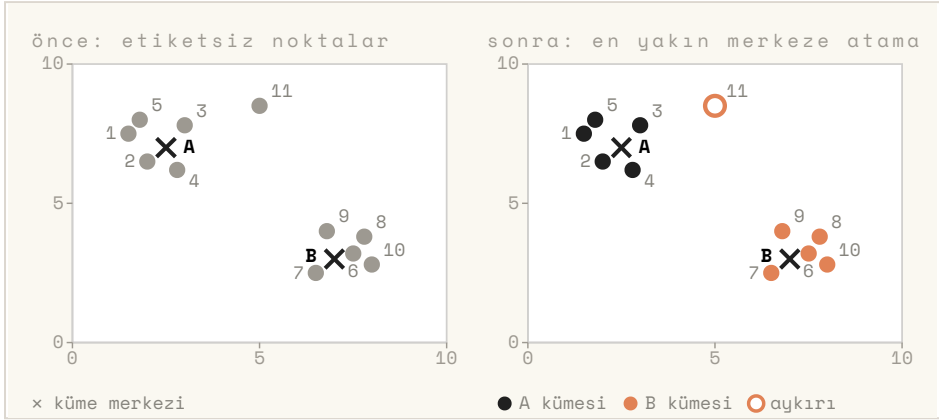
Sana koca bir kutu düğme verip “bunları ayır” deseler ne yaparsın? Kimse hangisinin nereye ait olduğunu söylemez; yine de benzeri benzerin yanına koyarsın. Kümeleme de bu: makine, etiketsiz veriyi benzerliğe göre kendisi gruplar. Hiçbir gruba uymayan tuhaf düğmeler de vardır; anomali tespiti onları yakalar.

Aşağıdaki noktaların hiçbir etiketi yok. Şekil 3.4'te makine onları en yakın merkeze göre iki kümeye ayırıyor. İki kümeye de uzak kalan

nokta aykırı diye işaretlenir.

***Kenar notu.** Denetimsiz öğrenmenin gücü: kimse “bunlar bir grup” demeden makine yapıyı kendisi bulur. Aykırılık tespiti, bankaların dolandırıcılık analizinde kullandığı araçlardan biridir.*

Şekil 3.4 · Etiketsiz veriyi grupla



Şekil 3.4

Canlı demo: book.onuronder.com/d/76fce219a4



Kurulum. Şekilde on bir gri nokta ve iki X işareti var. X'ler küme merkezleridir: A merkezi sol üstte (2.5, 7), B merkezi sağ altta (7, 3). Sol kare gruplamadan önceki hâl; her nokta aynı gri. Sağ karede noktalar en yakın merkezin rengini almış, tek bir nokta turuncu halkayla “aykırı” diye işaretlenmiş. Bu gösterimde iki merkez de aykırı nokta da önceden seçilmiştir; makine burada yalnız uzaklıkları hesaplar.

Adım adım. Gruplama tek bir soruya dayanır: bu nokta hangi merkeze daha yakın? Uzaklık Pisagor’la hesaplanır: $\sqrt{((x - x_m)^2 + (y - y_m)^2)}$. Aykırı için eşik baştan konur: en yakın merkeze uzaklığı 2.4’ü aşan nokta aykırı sayılır ve kümeye girmez.

Sıra şöyle: önce atama, sonra eşik, sonra kalan üyelerle merkez güncellemesi.

#	Nokta	A'ya uzaklık	B'ye uzaklık	Küme
1	(1.5, 7.5)	1.12	7.11	A
2	(2, 6.5)	0.71	6.10	A
3	(3, 7.8)	0.94	6.25	A
4	(2.8, 6.2)	0.85	5.28	A
5	(1.8, 8)	1.22	7.21	A
6	(7.5, 3.2)	6.28	0.54	B
7	(6.5, 2.5)	6.02	0.71	B
8	(7.8, 3.8)	6.19	1.13	B
9	(6.8, 4)	5.24	1.02	B
10	(8, 2.8)	6.92	1.02	B
11	(5, 8.5)	2.92	5.85	aykırı

1. Nokta 1, (1.5, 7.5): A'ya $\sqrt{(1^2 + 0.5^2)} = 1.12$, B'ye $\sqrt{(5.5^2 + 4.5^2)} = 7.11$. A çok daha yakın; nokta A'ya gider.
2. Nokta 9, (6.8, 4): A'ya $\sqrt{(4.3^2 + 3^2)} = 5.24$, B'ye $\sqrt{(0.2^2 + 1^2)} = 1.02$. B'ye gider.
3. Nokta 4, (2.8, 6.2): 0.85'e karşı 5.28, yine A. Aynı hesabı ilk on nokta için yap; hepsinde iki uzaklık arasında en az beş kat fark var. Karar hiç zor değil.
4. Nokta 11, (5, 8.5), farklı. En yakın merkezi A, ama uzaklığı 2.92; eşik 2.4'ün üstünde. A'nın öteki üyeleri merkeze en çok 1.22 uzakta; bu nokta onların iki katından uzak. B'ye de 5.85 uzak. Kurala göre aykırı: hiçbir kümeye girmez.
5. Bu tek turdu. Sırada merkez güncellemesi var: aykırı dışarıda bırakılınca A'nın yeni merkezi 1–5 numaralı noktaların ortalamasıdır, (2.22, 7.2). Eski merkezden yalnız 0.34 uzakta; kümeler zaten oturmuş. Dikkat: standart k-ortalamlar aykırı diye bir şey bilmez; 11. noktayı da A'ya atar ve A'nın merkezi altı üyeye

(2.68, 7.42) olur. Buradaki eleme, bizim koyduğumuz eşik kuralının sonucudur.

Ne oluyor? Noktaların hiçbir etiketi yok. Şekil 3.4'te makine her noktayı kendisine en yakın merkeze (X) bağlar; böylece birbirine benzeyenler aynı kümede toplanır. İki kümeye de uzak kalan tek nokta “aykırı” (tuhaf örnek) diye işaretlenir.

Kendin dene. 1) B kümesinin yeni merkezini hesapla: 6–10 numaralı noktaların x ve y ortalaması. Eski merkez (7, 3)'ten ne kadar kaydı? 2) Veriye (4.5, 5.5) noktası eklense hangi merkeze gider? Uzaklığına bakınca onu aykırı sayar mısın? 3) Kümeleme için hiç etiket kullanmadık. Aykırı kararı için hangi sayıyı, hangi eşiği kullandık; bu eşiği kim seçti?

TEKNİK DERİNLİK

Kümeleme, etiket olmadan benzerliğe göre grup keşfeder; k-ortalamlar (k-means) gibi yöntemler noktaları en yakın küme merkezine (centroid) atar ve merkezleri günceller. Anomali (aykırı değer) tespiti, çoğunluğun dağılımından belirgin biçimde sapan örnekleri belirler.

Şekil 3.4'teki gösterim, noktaları iki sabit merkeze en yakınlıklarına göre atayarak k-means'ın “atama” adımını gösterir; ayrıca her iki kümeye de uzak duran bir aykırı noktayı vurgular. Gerçek k-means, merkezleri yakınsayana dek iteratif olarak günceller. Bu gösterimde küme merkezleri ve aykırı nokta önceden seçilmiştir; gerçek uygulamada aykırılık bir yöntem ve eşikle hesaplanır.

Etiketsiz x noktaları, iki sabit merkez (centroid, X); her iki kümeye de uzak nokta aykırı (anomali) olarak işaretlenir.

İki adımın formülü: atama $c(i) = \arg\min_k \|x_i - \mu_k\|^2$, güncelleme $\mu_k \leftarrow$ küme k'ye atanan noktaların ortalaması. Şekil 3.4'te, aykırı nokta eşik kuralıyla dışlandıktan sonra bir güncelleme merkez A'yı (2.5, 7)'den (2.22, 7.2)'ye, merkez B'yi (7, 3)'ten (7.32, 3.26)'ya

TEKNİK DERİNLİK · DEVAM

taşıır; ikinci atama turu hiçbir noktayı deęiřtirmeı, algoritma yakınsamıřtır. Eřiksiz standart k-means 11. noktayı A'ya atar; A'nın merkezi altı üyeyle (2.68, 7.42) olur ve ikinci turda nokta yine A'da kalır.

Kümelemede merkez bir kez kaydı ve iş bitti. Milyonlarca parametresi olan bir model o küçük düzeltme adımını nasıl atar? Sıradaki bölüm sisli bir vadide geçiyor.

3.6 Model nasıl iyileřir: kayıp ve gradyan iniři

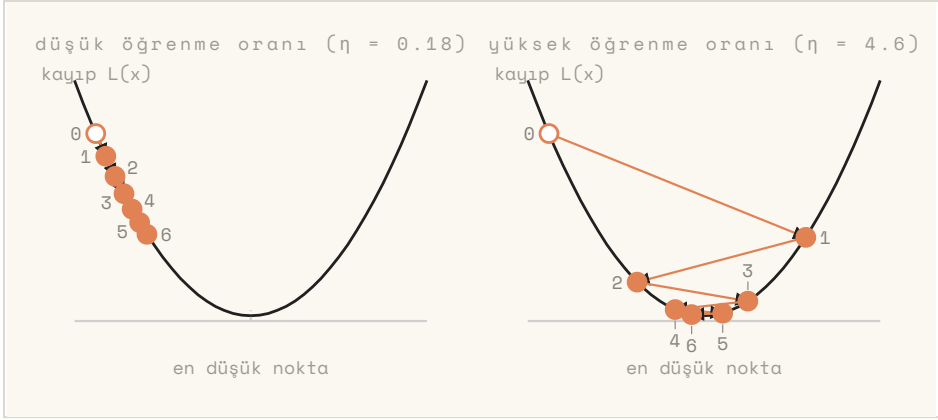
Bir model nasıl “daha iyi” olur? Önce ne kadar yanıldıđını ölçeriz; buna kayıp (loss) denir. řimdi kendini sisli bir vadide düşün. Kayıp ne kadar büyükse o kadar yukarıdasın; hedefin vadinin dibi. Sis yüzünden yolu göremiyorsun ama bir řeyi hissedebiliyorsun: ayađının altındaki eğimi.

Gradyan iniři bu yürüyüştür: her adımda eğimi yokla, yokuş ařađı küçük bir adım at, tekrarla. Ařađıda topu adım adım indir, kaybın erimesini izle. Adımını çok büyük atarsan ne olur? řekil 3.5'in sađ panelinde ne olduđuna bak.

***Kenar notu.** Öğrenmenin özü bu: “ne kadar yanlışım?” diye sor, biraz düzelt ve tekrarla; hem de milyonlarca kez. Sinir ađları başta olmak üzere bugünün modellerinin çođu böyle, gradyanla eğitiliyor; karar ađaçları gibi bazı yöntemlerse başka yoldan öğrenir.*

Kurulum. Vadi bir paraboldür: $L(x) = 0.18 \cdot (x - 5)^2 + 0.1$. Yatay eksen modelin tek parametresi x , dikey eksen kayıp. Dip $x = 5$ 'te; kayıp orada 0.1. Top $x_0 = 0.6$ 'dan, kaybın 3.58 olduđu sol yamaçtan başlar. Sol panel düşük öğrenme oranıyla ($\eta = 0.18$) altı adımı, sađ panel yüksek oranla ($\eta = 4.6$) altı adımı gösterir.

Şekil 3.5 · Kayıp vadisinde iniş



Şekil 3.5

Canlı demo: book.onuronder.com/d/0a8c400a67

Adım adım. Kural her adımda aynı: $x \leftarrow x - \eta \cdot L'(x)$. Eğim $L'(x) = 0.36 \cdot (x - 5)$; dibin solunda eksi, sağında artı. x 'ten eksi bir sayı çıkarınca x büyür: top sağa, yani yokuş aşağı gider.

adım	x ($\eta = 0.18$)	$L(x)$	x ($\eta = 4.6$)	$L(x)$
0	0.60	3.58	0.60	3.58
1	0.89	3.15	7.89	1.60
2	1.15	2.77	3.11	0.75
3	1.40	2.43	6.24	0.38
4	1.63	2.14	4.19	0.22
5	1.85	1.88	5.53	0.15
6	2.06	1.66	4.65	0.12

1. Düşük oran, 1. adım: eğim $0.36 \cdot (0.6 - 5) = -1.58$. Yeni $x = 0.6 - 0.18 \cdot (-1.584) = 0.6 + 0.285 \approx 0.89$. Kayıp 3.58'den 3.15'e düştü.
2. Düşük oran, 2–6. adımlar: top her adımda biraz daha az ilerler (0.29, 0.26, 0.25, 0.23, 0.22, 0.21), çünkü dibe yaklaştıkça eğim

- azalır. Altı adım sonunda $x = 2.06$, kayıp 1.66. Dibe hâlâ uzak; 0.1 yakınına gelmesi altmışa yakın adım ister. Güvenli ama yavaş.
3. Yüksek oran, 1. adım: aynı eğim, ama adım $4.6 \cdot 1.584 \approx 7.29$ birim. Top dibi ıskalayıp karşı yamaca, $x = 7.89$ 'a fırlar. Kayıp yine de düştü: 1.60.
4. Yüksek oran, 2–6. adımlar: eğim artık artı (+1.04), top sola atlar: 3.11. Sonra 6.24, 4.19, 5.53, 4.65. Dibin bir sağ, bir solu: salınım. Her sıçrama öncekinin 0.66 katı; o yüzden salınım sönüyor ve top yine de dibe yaklaşıyor. Kayıp 0.12.
5. Karşılaştır: altı adım sonra düşük oran 1.66'da, yüksek oran 0.12'de. Bu vadede büyük adım kazandı; ama sınırda. $\eta \cdot 0.36$ sayısı 2'yi geçseydi her sıçrama öncekinden büyük olur, top vadiden dışarı uçardı.

Ne oluyor? Bir modeli iyileştirmek, vadinin dibine inmeye benzer: “kayıp” ne kadar büyükse o kadar yukarıdasın. Her adımda topu yokuş aşağı biraz ittiriyoruz ve kayıp küçülüyor. Ama adım çok büyük olursa (yüksek öğrenme oranı) top dibi ıskalayıp karşı yamaca fırlar; adımın boyu bu yüzden önemlidir.

Kendin dene. 1) Düşük oranla 7. adımı hesapla: $x = 2.06$ 'da eğim kaç, yeni x ve kayıp kaç? 2) $\eta = 6$ ile iki adım at ($\eta \cdot 0.36 = 2.16$). Top dibe yaklaşıyor mu, uzaklaşıyor mu? 3) Bu vadede tek adımda tam dibe inen bir öğrenme oranı var mı? İpucu: $x - \eta \cdot 0.36 \cdot (x - 5) = 5$ olsun.

TEKNİK DERİNLİK

Eğitim, parametreleri kayıp fonksiyonu $L(\theta)$ 'yı en aza indirecek şekilde ayarlamaktır. Gradyan inişi her adımda gradyanın ters yönünde ilerler: $\theta \leftarrow \theta - \eta \nabla L(\theta)$, burada η öğrenme oranıdır (learning rate).

Öğrenme oranı işin en hassas ayarıdır: çok küçükse yakınsama yavaşlar; çok büyükse minimumun etrafında salınabilir ya da ıraksayabilir. Şekil 3.5'teki gösterimde dışbükey bir kayıp eğrisinde inişi ve büyük öğrenme oranının nasıl aşırma (overshoot) yol açtığını gözlemle. Derin ağların kayıp yüzeyleri

TEKNİK DERİNLİK · DEVAM

genellikle dışbükey değildir ve eğitimde stokastik gradyan inişi gibi gradyan tabanlı yöntemler yaygındır; doğrusal regresyon gibi bazı modellerin kaybı dışbükeydir, ağaç tabanlı yöntemler gibi bazıları ise gradyan kullanmaz.

Gradyan inişi: $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$. Parametre minimuma ($x^* = 5$) doğru ilerler, $L(x)$ azalır. Yüksek öğrenme oranında top minimumun etrafında salınır (aşım).

Bu ikinci dereceden vadede adım kuralı doğrusaldır: $x_{t+1} - 5 = (1 - 0.36 \cdot \eta) \cdot (x_t - 5)$. Çarpan $\eta = 0.18$ 'de 0.935 (tek yönlü, yavaş), $\eta = 4.6$ 'da -0.656 (sönümlü salınım), $\eta > 5.56$ 'da mutlak değeri 1'i aşar (ıraksama). Tablodaki her satır bu tek çarpanla öncekinden türer.

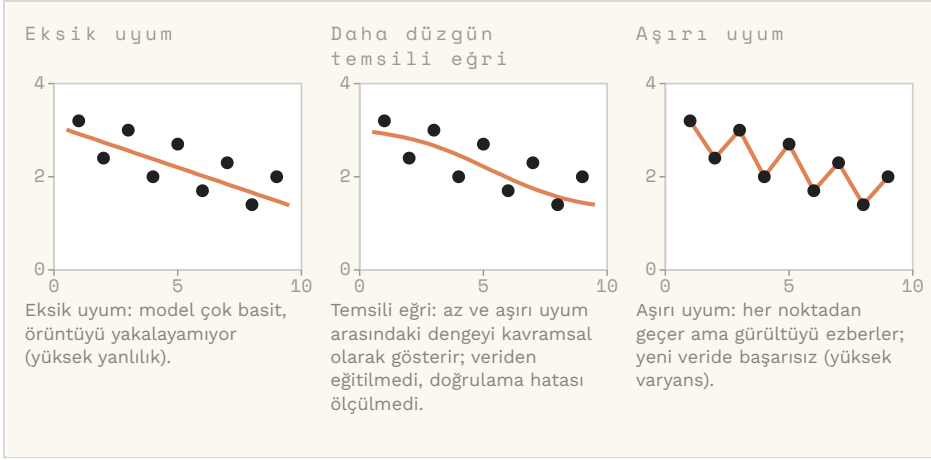
Kayıp sıfıra yaklaştıkça model iyileşiyor gibi görünür. Ama eğitim verisinde sıfır hata, yeni veride de sıfır hata demek midir?

3.7 Aşırı uyum ve topluluk öğrenmesi

Sınıfın ezbercisini bilirsün: eski soruların hepsini kelimesi kelimesine bilir ama soru birazcık değişince kalakalır. Modeller de bazen böyle “fazla iyi” öğrenir: eğitim örneklerini ezberler, yenisinde sınıfta kalır. Buna aşırı uyum (overfitting) denir. Terside var: çok basit kalan model örüntüyü hiç yakalayamaz (eksik uyum). İyi model ikisinin arasında durur.

Şimdi aynı veriye üç model uydur; üç öğrenci gibi düşün: tembel, dengeli ve ezberci. Sence hiç görmediği soruyu hangisi bilir?

***Kenar notu.** Ezberlemek öğrenmek değildir. Sınavda yalnızca eski soruları ezberleyen öğrenci, yeni soruda çuvallar. İyi bir model örnekleri değil, altlarındaki örüntüyü öğrenir.*

Şekil 3.6 • Aynı veri, üç model

Şekil 3.6

Canlı demo: book.onuronder.com/d/7f55ca4f28

Kurulum. Üç panelde de aynı dokuz nokta var: (1, 3.2), (2, 2.4), (3, 3.0), (4, 2.0), (5, 2.7), (6, 1.7), (7, 2.3), (8, 1.4), (9, 2.0). Genel eğilim aşağı, ama her adımda bir zıplama var; ölçümlerdeki gürültü. Sol panel “Eksik uyum”: düz bir doğru. Orta panel “Daha düzgün temsili eğri”: hafif dalgalı bir eğri; veriden eğitilmedi, anlatım için elle çizildi. Sağ panel “Aşırı uyum”: dokuz noktayı tek tek birleştiren kırık çizgi. Her panelin altında o modelin hükmü yazar.

Adım adım.

1. Eksik uyum: doğru $y = 3.1 - 0.18x$. Bu doğru anlatım için seçilmiştir; dokuz noktaya en küçük kareler yöntemiyle uydurulan doğru $y \approx 3.09 - 0.16x$ 'tir ve ona yakındır. $x = 5$ 'te 2.2 der, nokta 2.7'dir; fark 0.5. $x = 9$ 'da 1.48 der, nokta 2.0. Doğru noktaların bir üstüne, bir altına düşer; zıplamaları hiç tutmaz. Hüküm: “Eksik uyum: model çok basit, örüntüyü yakalayamıyor (yüksek yanlılık).”
2. Daha düzgün temsili eğri: $y = 3.0 - 0.16x + 0.15 \cdot \sin(0.6x)$. $x = 5$ 'te 2.22, $x = 9$ 'da 1.44 der. Zikzağı kovalamaz; yalnız aşağı eğilimi ve

hafif bir dalgayı taşır. Hüküm: “Yanlılık-varyans dengesi: ne örüntüyü kaçırarak kadar basit ne gürültüyü ezberleyecek kadar karmaşık.” İyi genellediğini söylemek için eğitimde kullanılmamış veride hatası ölçülmeli; bu eğri elle çizildiği için o ölçüm burada yok.

3. Aşırı uyum: kırık çizgi dokuz noktanın dokuzundan da geçer; eğitim hatası tam sıfır. Ama şekline bak: 5’ten 6’ya giderken 1.0 birim düşüyor, 6’dan 7’ye 0.6 yükseliyor. Bu iniş çıkışlar örüntü değil, gürültüdür; yeni ölçümde aynı yerde tekrarlamaz. Hüküm: “Aşırı uyum: her noktadan geçer ama gürültüyü ezberler; yeni veride başarısız (yüksek varyans).”
4. Sınav: 5. ve 7. noktayı sakla, kalan yediyle aynı kırık çizgiyi çiz. $x = 5$ ’te çizgi (4, 2.0) ile (6, 1.7)’yi birleştirir ve 1.85 der; gerçek 2.7, hata 0.85. $x = 7$ ’de 1.55 der; gerçek 2.3, hata 0.75. Düz doğru için de aynısını yap: kalan yedi noktaya en küçük kareler yöntemiyle bir doğru uydur. Sonuç $y \approx 3.06 - 0.17x$; $x = 5$ ’te 2.19, $x = 7$ ’de 1.85 der, hatalar 0.51 ve 0.45. Ezberci, görmediği soruda tembelden bile kötü.
5. Eğitim hatası tek başına aldatır. Modeli hiç görmediği veriyle sınamak gerekir; buna doğrulama denir. Adil bir karşılaştırma için her aday aynı yedi noktayla eğitilip aynı iki noktada sınanır. Burada bunu kırık çizgi ile düz doğru için yaptık; orta eğri elle çizildiği için sınava girmedi.

Ne oluyor? Aynı veriye üç ayrı model uyduruyoruz. Çok basit olan örüntüyü ıskalar (eksik uyum); aşırı karmaşık olan her noktayı ezberler ama yeni veride şaşırır (aşırı uyum). En iyisi ikisinin arasındadır: daha önce hiç görmediği örnekleri de doğru tahmin edebilen model. Bu eğriler fikri gösterir; hangisinin iyi genellediğini ancak eğitimde kullanılmamış veride ölçülen hata söyler.

Kendin dene. 1) $x = 10$ için üç model ne der? Kırık çizgi için ne olduğuna dikkat et. 2) Aynı saklama sınavını 2. ve 8. noktalarla tekrarla: kırık çizgi ve düz doğru bu iki noktada kaçır hata yapıyor? 3) Dokuz noktadan tam geçen model “sıfır hata” diye övünüyor. Bu sayı neyi kanıtlar, neyi kanıtlamaz?

TEKNİK DERİNLİK

Aşırı uyum, modelin eğitim verisindeki gürültüyü de öğrenip görülmemiş veride başarımını yitirmesidir; eksik uyum ise modelin örüntüyü yakalayamayacak kadar basit olmasıdır. Bunlar yanlılık-varyans dengesinin (bias–variance tradeoff) iki ucudur ve genelde eğitim/doğrulama ayrımı, düzenlileştirme (regularization) ve çapraz doğrulama ile yönetilir.

Topluluk öğrenmesi (ensemble), birçok modelin tahminini birleştirir (oylama, bagging, boosting); hataları birbirini tamamlıyorsa genelleme iyileşebilir, ama her tekil modelden daha iyi sonuç garanti edilmez ve kazanç doğrulama verisiyle ölçülür. Rastgele orman (random forest) ve gradyan artırma (gradient boosting) en bilinen örneklerdir.

Yanlılık-varyans dengesi: eksik uyum örüntüyü kaçıır, aşırı uyum gürültüyü ezberler. En iyi model her ikisini dengeleyip görülmemiş veriye genelleşendir.

Şekil 3.6'daki saklama sınavı, tek katlı bir eğitim/doğrulama ayrımıdır: iki nokta doğrulama kümesi, yedi nokta eğitim kümesi. Çapraz doğrulama bunun k-katlı hâlidir: veri k parçaya bölünür, her parça sırayla saklanır, kalanla eğitilir ve hataların ortalaması alınır; her noktayı tek tek saklamak (burada $k = 9$) bunun bir özel durumudur.

Örnekten öğrenmek, ama ezberlemeden: bölüm bu fikrin etrafında döndü. Ne kadarı aklında kaldı, altı soruyla bak.

3.8 Kendini test et

Cevaplar kitabın sonunda.

1. Makine öğrenmesinin klasik YZ'den temel farkı nedir?
 - a) Kuralları elle yazmak yerine veriden öğrenir
 - b) Hiç hata yapmaz
 - c) Daha hızlı çalışır
 - d) İnternet gerektirir
2. Bir örneğin “doğru cevabına” ne denir?
 - a) Etiket
 - b) Gradyan
 - c) Özellik
 - d) Model
3. Etiketsiz veriyi gruplara ayırma görevi hangisidir?
 - a) Kümeleme (denetimsiz)
 - b) Sınıflandırma
 - c) Pekiştirmeli
 - d) Regresyon
4. Gradyan inişi ne yapar?
 - a) Etiketleri üretir
 - b) Veriyi siler
 - c) Modeli yavaşlatır
 - d) Kaybı azaltacak yönde parametreleri adım adım günceller
5. Aşırı uyum (overfitting) nedir?
 - a) Hiç öğrenmemek
 - b) Çok hızlı öğrenmek
 - c) Eğitim verisini ezberleyip yeni veride başarısız olmak
 - d) Veriyi sıkıştırmak

6. Pekiştirmeli öğrenmede ajan nasıl öğrenir?

- a) Ödül ve cezayla, deneme-yanılmayla
- b) Etiketli örneklerle
- c) Kuralları ezberleyerek
- d) Veriyi kümeleyerek

Bu bölümden kalanlar

- Makine öğrenmesi kural yazmaz; kuralı örneklerden kendisi çıkarır.
- Her örnek iki parçadır: onu tarif eden özellikler ve doğru cevap, yani etiket.
- Etiket varsa denetimli, yoksa denetimsiz, ödülle öğreniyorsa pekiştirmeli öğrenmedir.
- Regresyon bir miktar tahmin eder (ev fiyatı), sınıflandırma bir kategori seçer (spam mı, değil mi); sayıyla kodlanmış kategori yine sınıflandırmadır.
- Kümeleme etiketsiz veriyi benzerliğe göre gruplar; her kümeye de uzak kalan nokta, önceden konan bir eşığe göre aykırı sayılır.
- Gradyan inişi kaybı ölçer, eğime bakar, küçük bir adım atar ve bunu milyonlarca kez tekrarlar.
- Eğitim verisini ezberleyen model yeni veride çuvallar; modeli hiç görmediği veriyi sınamak bu yüzden şart.

4

BÖLÜM 4

Yapay Beyin

Nörondan sinir ağına

4.1 Yapay beyin: derin öğrenme

Kafanın içinde milyarlarca minik haberci yaşar: nöronlar. Her biri komşularının fısıltısını dinler; fısıltılar yeterince güçlenince o da bağırır ve haberi bir sonrakine iletir. Araştırmacılar bu basit oyuna bakıp sormuş: ya bunun kabataslak bir matematik taklidini yapsak?

Yapay sinir ağları bu sorudan doğdu. Tek tek “yapay nöronları” katman katman dizince, ortaya beklenmedik ölçüde güçlü bir öğrenme makinesi çıkıyor. Katmanlar çoğaldıkça adı da değişiyor: derin öğrenme. Önce tek bir habercinin başına oturacağız; sonra koca bir ağa, oradan görüntü ve dizi işleyen özel ağlara uzanacağız.

***Kenar notu.** Yapay nöron beynin gerçek bir kopyası değil; çok kaba bir matematiksel benzetmedir. Güç tek bir nöronda değil, milyonlarcasının birlikte oluşturduğu örüntülerdedir.*

TEKNİK DERİNLİK

Yapay sinir ağları, biyolojik nöronlardan yalnızca gevşek biçimde esinlenir; özünde, katmanlı biçimde düzenlenmiş doğrusal olmayan dönüşümler yığınıdır. Her yapay nöron, girdilerin ağırlıklı toplamına bir sabit terim (bias) ekler ve sonucu bir aktivasyon fonksiyonundan geçirir.

Derin öğrenme, çok sayıda gizli katmanın üst üste konmasıdır; bu derinlik, ham veriden giderek soyut temsiller (kenar → şekil → nesne) öğrenmeyi sağlar. Aktivasyon fonksiyonları olmadan üst üste konan doğrusal katmanlar tek bir doğrusal işleve çöker; derinliği anlamlı kılan şey doğrusal olmayanlıktır. Alan, 2012’de AlexNet’in ImageNet’teki sıçramasıyla modern çağına girdi.

Bütün bu yapının en küçük parçası tek bir yapay nörondur. Tek başına ne hesaplar, üç sayıdan nasıl karar çıkarır?

4.2 Tek bir yapay nöron

Yapay nöronu bir kapı bekçisi gibi düşün. Kapısına birkaç haber gelir; bekçi her habere bir önem verir (buna ağırlık denir), hepsini toplar, üstüne kendi huyunu ekler (sabit terim, yani bias). Toplamı bir süzgeçten geçirip bir çıktıya çevirir; aktivasyon denen şey bu süzgeçtir. Zilin çalıp çalmadığına ise ayrı bir kuralla karar verilir: çıktı bir eşiği aşıyor mu?

Şekil 4.1’de girdileri güçlendir ya da zayıflat; toplamın nasıl değiştiğini ve bekçinin zili ne zaman çaldığını, yani nöronun ne zaman “ateşlediğini” izle.

***Kenar notu.** Nöronun yaptığı tek şey: “girdileri tart, topla, bir aktivasyon fonksiyonundan geçir.” Bu kadar basit bir işlem, milyonlarca kez tekrarlandığında yüz tanıyabiliyor, metin yazabiliyor.*

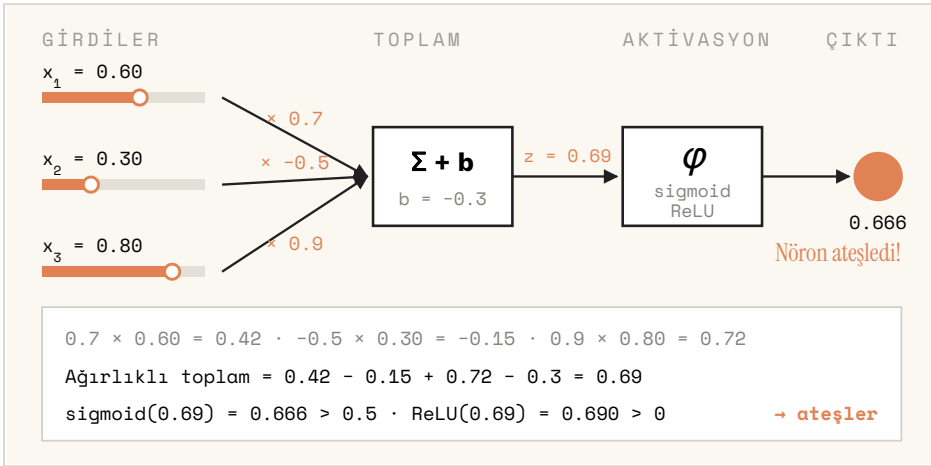
Kurulum. Şekilde üç girdi görüyorsun: x_1 , x_2 ve x_3 . Her biri 0 ile 1 arasında bir değer taşır; başlangıçta 0.60, 0.30 ve 0.80. Her girdinin yanında sabit bir ağırlık yazar: $w = [0.7, -0.5, 0.9]$. Ortadaki kutu bunları toplar, üstüne sabit terim $b = -0.3$ ekler. Sağdaki kutu aktivasyonu verir; sigmoid ya da ReLU. Aktivasyon, nöronun sayısal çıktısıdır. “Ateşledi” ise gösterime özgü ayrı bir karar kuralıdır: sigmoid çıktısı 0.5’i, ReLU çıktısı 0’ı geçince sağdaki panelde “Nöron ateşledi!” yazar; aşağıdaki tabloda geçmeyen durumlar “sessiz” diye anılır.

Adım adım. Başlangıç değerleriyle bekçinin hesabını kendin yürüt.

1. Her girdiyi ağırlığıyla çarp: $0.7 \times 0.60 = 0.42$; $-0.5 \times 0.30 = -0.15$; $0.9 \times 0.80 = 0.72$.
2. Üçünü topla: $0.42 - 0.15 + 0.72 = 0.99$. Sabit terimi ekle: $0.99 - 0.3 = 0.69$. Şekildeki “Ağırlıklı toplam = 0.69” satırı bu sayıdır.

3. Sigmoid ile çıktı $1 / (1 + e^{-0.69}) = 0.666$ olur. Bu değer karar eşiği 0.5'in üstünde; nöron ateşler.
4. ReLU ile çıktı $\max(0, 0.69) = 0.690$ olur: artı toplam olduğu gibi geçer. Sıfırdan büyük olduğu için yine ateşler.
5. x_2 'yi 1.00'a çıkar. Ağırlığı eksi olduğu için toplam düşer: $0.42 - 0.50 + 0.72 - 0.3 = 0.34$. Sigmoid 0.584 verir; nöron hâlâ ateşler ama daha zayıf.
6. Bu kez x_2 'yi 0.30'a geri al, x_3 'ü sıfırla. Toplam $0.42 - 0.15 + 0 - 0.3 = -0.03$ olur. Sigmoid 0.493, ReLU 0.000; iki durumda da nöron sessiz kalır. Sessiz, çıktı sıfır demek değil: sigmoid hâlâ 0.493 veriyor, yalnız 0.5 eşiğinin altında kaldı.

Şekil 4.1 • Nöronu çalıştır



Şekil 4.1

Canlı demo: book.onuronder.com/d/1cb90a39a8

x_1	x_2	x_3	Ağırlıklı toplam	Sigmoid	ReLU	Durum
0.60	0.30	0.80	0.69	0.666	0.690	ateşler
0.60	1.00	0.80	0.34	0.584	0.340	ateşler

x_1	x_2	x_3	Ağırlıklı toplam	Sigmoid	ReLU	Durum
0.60	0.30	0.00	-0.03	0.493	0.000	sessiz
0.00	0.00	0.00	-0.30	0.426	0.000	sessiz

Son satıra dikkat: bütün girdiler sıfırken bile toplam -0.30 çıkar. Bu, sabit terimin işidir; bekçi huyu gereği kapalı başlar. x_2 her zaman frene basar, x_1 ve x_3 ise gaza. En güçlü söz x_3 'te, çünkü ağırlığı en büyük.

Ne oluyor? Bir nöron çok basit bir şey yapar: her girdiyi bir “önem ağırlığı”yla çarpıp toplar, sonra küçük bir sabit terim (bias) ekler. Aktivasyon fonksiyonu bu toplamı çıktıya çevirir: sigmoid 0 ile 1 arasında bir sayı verir, ReLU eksi toplamı sıfırlar, artı toplamı olduğu gibi geçirir. Çıktı 0.5'i geçince “ateşledi” demek, bu gösterime özgü bir karar kuralıdır. Girdileri değiştirdikçe toplamın ve çıktının nasıl değiştiğini görürsün.

Kendin dene. 1) Üç girdiyi de 1.00 yap. Ağırlıklı toplam ve sigmoid çıktısı ne olur; nöron ateşler mi? 2) Yalnız $x_3 = 1.00$, diğerleri 0. Toplamı ve ReLU çıktısını hesapla. 3) Yalnız $x_2 = 1.00$, diğerleri 0. Sigmoid çıktısı 0.5'i geçer mi?

TEKNİK DERİNLİK

Bir yapay nöron $z = \sum w_i x_i + b$ hesaplar; ardından bir aktivasyon fonksiyonu uygular: $a = \phi(z)$. Yaygın seçimler sigmoid (0–1), tanh (–1–1) ve ReLU = $\max(0, z)$ 'dir. Ağırlıklar her girdinin önemini ayarlar, sabit terim (bias) ise toplamı kaydırır; ikisi de eğitimle öğrenilir.

Aktivasyonun rolü belirleyicidir: doğrusal olmayanlık eklemeseydi, kaç katman koyarsak koyalım ağ tek bir doğrusal dönüşüme eşdeğer olurdu. ReLU, basitliği ve gradyan akışını koruması nedeniyle derin ağlarda fiilî standart hâline gelmiştir.

$z = \sum w_i x_i + b$, $\phi(z)$ sigmoid ya da ReLU ile hesaplanır. Ağırlıklar $w = [0.7, -0.5, 0.9]$, sabit terim $b = -0.3$. Gösterimde $\phi(z)$ karar

TEKNİK DERİNLİK · DEVAM

eşiğini (sigmoid için 0.5, ReLU için 0) aşarsa “ateşledi” yazılır; bu eşik aktivasyonun parçası değil, gösterimin karar kuralıdır.

Şekildeki sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ biçimindedir; $z = 0$ 'da tam 0.5 verir, bu yüzden şekilde ateşleme eşiği 0.5 seçilmiştir. ReLU'da eşik doğrudan $z > 0$ 'dır. Eşik, aktivasyonun kendisi değil, çıktı üstüne konmuş bir karar kuralıdır: sigmoid eşik altında da sıfırlanmaz (-0.03 için 0.493), ReLU ise artı girdiyi olduğu gibi geçirir.

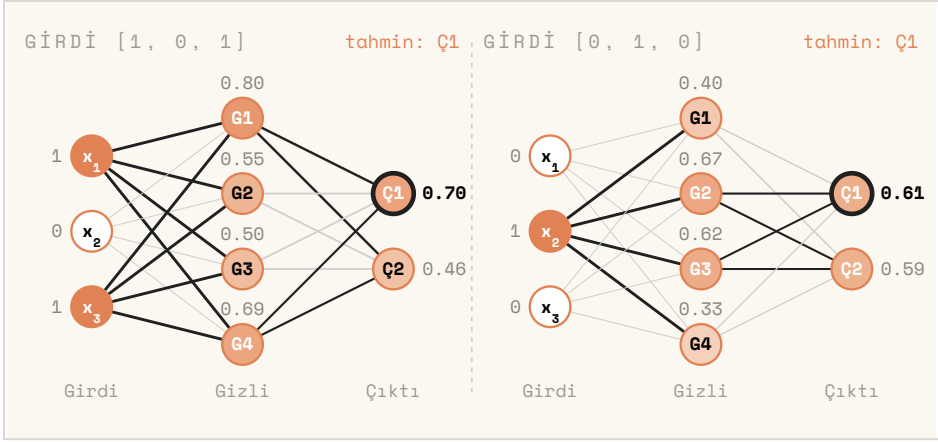
Tek nöron girdi uzayında yalnızca bir çizgi çekebilir. Onlarcasını katman katman dizersen sinyal içeriden nasıl geçer?

4.3 Katmanlar ve ileri besleme

Tek bekçi tek başına pek bir şey başaramaz. Ama bekçileri katlar hâlinde bir apartmana dizersen her şey değişir. Haber giriş katından çıkış katına doğru akar; aradaki “gizli” katlar, ham haberi her seferinde biraz daha işlenmiş bir hâle çevirir.

Haberin girişten çıkışa akmasına ileri besleme denir. Şekil 4.2'de girdileri açık ve kapalı hâlleriyle karşılaştır; sinyalin kattan kata ilerleyişini, hangi nöronların parladığını izle.

***Kenar notu.** Derinliğin sırrı: ilk katmanlar basit özellikleri (kenarlar), sonraki katmanlar onların birleşimini (şekiller, nesneler) öğrenir. Kimse bunu elle programlamaz; ağ kendisi keşfeder.*

Şekil 4.2 · Canlı sinir ağı (ileri besleme)

Şekil 4.2

Canlı demo: book.onuronder.com/d/63f05dcef2

Kurulum. Şekil iki panelden oluşur; her panelde üç katman var. Solda üç girdi kutusu; her biri ya açık (1) ya kapalı (0). Ortada dört gizli nöron, sağda iki çıktı nöronu. Bütün bağlantıların ağırlığı sabittir; değerleri aşağıdaki hesapta bulacaksınız. Bu ağda sabit terim yoktur. Ağırlıklar elle seçilmiştir; iki çıktının bir sınıf anlamı yoktur. Her nöron gelen sinyalleri ağırlıklarıyla toplar ve sigmoidden geçirir. Nöron ne kadar koyu boyalıysa aktivasyonu o kadar yüksektir. Sol panel başlangıç durumunu gösterir: x_1 ve x_3 açık, x_2 kapalı. Sağ panelde yalnız x_2 açık.

Adım adım. Sol paneli, yani girdi [1, 0, 1] durumunu adım adım hesapla. Gizli nöronların girdi ağırlıkları sırasıyla: G1 [0.6, -0.4, 0.8], G2 [0.5, 0.7, -0.3], G3 [-0.6, 0.5, 0.6], G4 [0.3, -0.7, 0.5].

1. G1: $0.6 \times 1 + (-0.4) \times 0 + 0.8 \times 1 = 1.40$; sigmoid 0.80. Katmanın en parlak nöronu.
2. G2: $0.5 + 0 - 0.3 = 0.20$; sigmoid 0.55.
3. G3: $-0.6 + 0 + 0.6 = 0.00$; sigmoid tam 0.50. Toplam sıfır olsa bile nöron yarı parlaklıkta kalır.

4. G4: $0.3 + 0 + 0.5 = 0.80$; sigmoid 0.69.
5. Çıktı Ç1, ağırlıkları $[0.7, -0.5, 0.6, 0.4]$: $0.7 \times 0.80 - 0.5 \times 0.55 + 0.6 \times 0.50 + 0.4 \times 0.69 = 0.86$; sigmoid 0.70.
6. Çıktı Ç2, ağırlıkları $[-0.4, 0.6, 0.5, -0.6]$: $-0.32 + 0.33 + 0.25 - 0.41 = -0.15$; sigmoid 0.46.
7. Ç1 > Ç2. Gösterimde buna ağın “tahmini” diyoruz; şekilde en koyu kutu odur. Ç1 ile Ç2 bir sınıfı temsil etmiyor; gösterim bir görevin öğrenildiğini kanıtlamaz.

Sağ panelde yalnız x_2 açık; aynı işlemin sonucu aşağıdaki tabloda.

Girdi	G1	G2	G3	G4	Ç1	Ç2	Tahmin
[1, 0, 1]	0.80	0.55	0.50	0.69	0.70	0.46	Ç1
[0, 1, 0]	0.40	0.67	0.62	0.33	0.61	0.59	Ç1

Girdi değişince gizli katmanın parlaklık deseni tersine döner: solda G1 öne çıkarken sağda G2 ve G3 parlar. Yine de kazanan değişmez; ikinci durumda Ç1 yalnızca 0.02 farkla önde. Ağırlıklar hiç eğitilmediği için bu ağın “fikri” henüz keyfidir.

Ne oluyor? Sinyal soldan sağa, katman katman ilerliyor: her nöron kendisine gelenleri toplayıp bir sonraki katmana aktarıyor. Bir nöron ne kadar parlaksa o kadar güçlü tepki vermiş demektir. En sağdaki en parlak kutuya ağın “tahmini” diyoruz; ağırlıklar elle seçildiği için bu tahminin henüz bir anlamı yok.

Kendin dene. 1) Bütün girdiler kapalıyken $([0, 0, 0])$ dört gizli nöron hangi değeri alır? Çıktıları da hesapla. 2) Girdi $[1, 1, 1]$ için G1’in toplamını ve sigmoid değerini bul. 3) Sekiz olası girdi düzeninden herhangi birinde Ç2 kazanır mı? Tahmin et, sonra iki tanesini hesaplayarak sına.

TEKNİK DERİNLİK

İleri beslemeli ağda her katman, bir önceki katmanın aktivasyonlarını alır: $a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$. Girdi katmanı ham veriyi, gizli katmanlar ara temsilleri, çıktı katmanı ise tahmini

TEKNİK DERİNLİK · DEVAM

taşır. Ağırlık matrisleri ve sabit terim vektörleri ağıın öğrenilen parametreleridir.

Şekil 4.2'deki gösterim, elle seçilmiş sabit ağırlıklarla gerçek bir ileri besleme yapar: girdiden çıktıya hesaplama yürütülür ve nöron parlaklıkları aktivasyon değerlerini gösterir. En yüksek çıktıya gösterim gereği ağıın “tahmini” diyoruz; çıkışların bir sınıf anlamı yoktur ve gösterim bir görevin öğrenildiğini kanıtlamaz. Eğitim, bu ağırlıkları ayarlama işidir; o da sıradaki adımın konusu.

$a^{(l)} = \varphi(W^{(l)}a^{(l-1)} + b^{(l)})$ katman katman hesaplanır; nöron parlaklığı aktivasyon değerini gösterir.

Şekildeki ağda $W^{(1)}$ 4×3 , $W^{(2)}$ 2×4 boyutundadır; $b^{(1)} = b^{(2)} = 0$ alınmıştır. Toplam $12 + 8 = 20$ öğrenilebilir ağırlık vardır; φ her iki katmanda sigmoiddir.

Bu ağ iki girdi düzeninde de aynı kapıyı gösterdi, çünkü ağırlıklarını kimse ayarlamadı. Ağırlıkları hataya bakarak düzeltmenin bir yolu var mı?

4.4 Geri yayılım: hatadan öğrenmek

Ağ işe rastgele tahminlerle başlar; ilk günkü acemiliğine şaşmamalı. Nasıl ustalaşır? Önce tahminini doğru cevapla kıyaslar, ne kadar yanlış olduğunu ölçer; buna hata denir. Sonra bu hata, çıkıştan girişe doğru geri geri yürür ve uğradığı her bağlantıya “sen de birazcık payını düzelt” der.

Bu geri geri yürüyüşe geri yayılım denir; her bağlantının hatadaki payını hesaplar. Düzeltmeyi ayrı bir adım yapar: her ağırlık, payı oranında biraz kaydırılır (gradyan inişi). Şekil 4.3'ü tur tur oku; hatanın geriye akışını ve çıktının her turda doğru cevaba biraz daha yaklaşmasını gör. Hata küçüldükçe geriye taşınan fısıltı da zayıflar; düzeltilecek pay azalır.

***Kenar notu.** İleri besleme “tahmin et”, geri yayılım “hatadan ders al” demektir. Bu iki adımı milyonlarca kez tekrarlamak; derin öğrenmenin özü bu.*

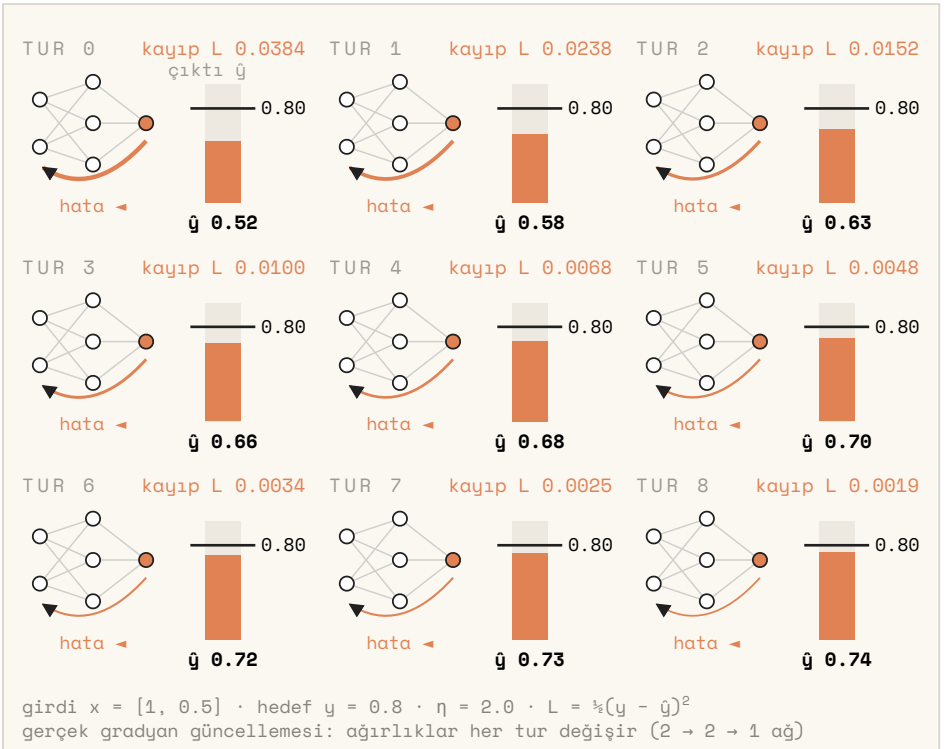
Kurulum. Şekil bir film şeridi gibi düzenlenmiş; her kare bir eğitim turu. Ağ küçük ama hesap gerçek: iki girdi ($x_1 = 1.0$, $x_2 = 0.5$), iki gizli nöron, tek çıktı; hepsi sigmoid. Hedef 0.8; kayıp $L = \frac{1}{2}(\hat{y} - 0.8)^2$; öğrenme oranı $\eta = 2$. Başlangıç ağırlıkları anlatım için seçilmiştir: gizli katman $W_1 = [[0.3, -0.2], [0.4, 0.1]]$, sabit terimler 0; çıktı ağırlıkları $w_2 = [0.5, -0.3]$, $b_2 = 0$. Karenin solunda ağ ve çıkıştan girişe dönen bir ok var: gradyanın geriye akışı. Ortada dikey bir çubuk ağın çıktısını ($\hat{y}$), üstündeki çizgi hedefi (0.8) gösterir. Karenin başlığında o turun hatası (hedef – $\hat{y}$) yazar. Tur 0’da ağırlıklar henüz hiç güncellenmemiştir; her tur bir ileri geçiş, bir gradyan hesabı ve bir güncellemedir.

Adım adım. Her kareyi sırayla oku; ilk turu kendin hesapla.

1. Tur 0, ileri geçiş: gizli nöron 1’in toplamı $0.3 \cdot 1.0 + (-0.2) \cdot 0.5 = 0.20$, sigmoid 0.5498; nöron 2’nin toplamı $0.4 \cdot 1.0 + 0.1 \cdot 0.5 = 0.45$, sigmoid 0.6106. Çıktı toplamı $0.5 \cdot 0.5498 - 0.3 \cdot 0.6106 + 0 = 0.0917$; $\hat{y} = \text{sigmoid}(0.0917) = 0.5229$.
2. Tur 0, kayıp: hata $0.8 - 0.5229 = 0.2771$; $L = \frac{1}{2} \cdot 0.2771^2 = 0.0384$.
3. Tur 0, geri yayılım (gradyan hesabı): çıktı nöronunda $\delta_2 = (\hat{y} - 0.8) \cdot \hat{y} \cdot (1 - \hat{y}) = -0.2771 \cdot 0.5229 \cdot 0.4771 = -0.0691$. Çıktı ağırlıklarının gradyanı bu sayı çarpı gizli aktivasyon: $\partial L / \partial w_2 = [-0.0691 \cdot 0.5498, -0.0691 \cdot 0.6106] = [-0.0380, -0.0422]$; $\partial L / \partial b_2 = -0.0691$. Zincir bir adım daha geriye gider: gizli nöron j için $\delta_j = \delta_2 \cdot w_{2j} \cdot h_j(1 - h_j)$, gradyan $\delta_j \cdot x_j$.
4. Tur 0, güncelleme (optimizasyon): $w \leftarrow w - \eta \cdot \text{gradyan}$. $w_2 = [0.5 + 2 \cdot 0.0380, -0.3 + 2 \cdot 0.0422] = [0.5760, -0.2156]$; $b_2 = 0 + 2 \cdot 0.0691 \approx 0.1383$. Gizli katman $W_1 = [[0.3171, -0.1914], [0.3901, 0.0951]]$. Geri yayılım gradyanı hesapladı; ağırlıkları değiştiren bu adımdır.

5. Tur 1: yeni ağırlıklarla ileri geçiş $\hat{y} = 0.5817$, hata 0.2183, kayıp 0.0238. Tek turda çıktı 5.9 puan yaklaştı.
6. Tur 2–8: aynı üç adım tekrarlanır; sayılar tabloda. Sekizinci turda $\hat{y} = 0.7390$, hata 0.0610, kayıp 0.0019.
7. Adımlar küçülüyor: ilk tur $\hat{y}$ 'yi 0.059 artırdı, sekizinci tur 0.010. Hata küçüldükçe gradyan da küçülür; güncelleme ona orantılı olduğu için adım kısalmır.

Şekil 4.3 · Hatadan öğren (gerçek eğitim)



Şekil 4.3

Canlı demo: book.onuronder.com/d/37fa54d6c3

Tur	w_2	b_2	$\hat{y}$	Hata ($0.8 - \hat{y}$)	Kayıp
0	[0.5000, -0.3000]	0.0000	0.5229	0.2771	0.0384
1	[0.5760, -0.2156]	0.1383	0.5817	0.2183	0.0238
2	[0.6354, -0.1513]	0.2445	0.6258	0.1742	0.0152
3	[0.6818, -0.1021]	0.3261	0.6585	0.1415	0.0100
4	[0.7183, -0.0639]	0.3897	0.6832	0.1168	0.0068
5	[0.7477, -0.0335]	0.4403	0.7022	0.0978	0.0048
6	[0.7716, -0.0090]	0.4812	0.7172	0.0828	0.0034
7	[0.7914, 0.0111]	0.5148	0.7292	0.0708	0.0025
8	[0.8080, 0.0279]	0.5427	0.7390	0.0610	0.0019

Tabloda hazır bir kural yok; her satır bir önceki satırdan gradyan ve güncellemeyle türetilmiştir. Hata sekiz turda 0.28'den 0.06'ya indi; sıfıra ulaşmadı ve bu hızla ulaşması da uzun sürer. Bu, tek bir örnek üzerinde ölçülen eğitim hatasıdır: ağın yeni örneklerdeki başarısı ayrıca sınanmalıdır. Milyonlarca örnekle gerçek eğitimde eğri bu kadar düz inmez, ara sıra yükselir bile; ama döngü aynıdır: ileri geçiş, kayıp, gradyan, güncelleme.

Ne oluyor? Ağ önce eğitilmemiş ağırlıklarla tahmin yapar, bu yüzden hata (kayıp) yüksektir. Her turda geri yayılım, kaybın her bağlantıya göre eğimini (gradyanı) çıkıştan girişe doğru hesaplar; sonra gradyan inişi her ağırlığı hatayı azaltacak yönde biraz kaydırır. Böyle böyle çıktı, doğru cevaba adım adım yaklaşır. Bu örnekte eğitim hatası küçüldü; ağın yeni örneklerde de başarılı olup olmadığı ayrıca sınanmalıdır.

Kendin dene. 1) Tur 8'in ağırlıklarıyla çıktıyı kendin hesapla: gizli aktivasyonlar $h = [0.5957, 0.5994]$, $w_2 = [0.8080, 0.0279]$, $b_2 = 0.5427$. Çıktı toplamı ve $\hat{y}$ kaç; tabloyla uyuyor mu? 2) Tur 0'da b_2 'nin gradyanı -0.0691. Öğrenme oranı 2 yerine 1 olsaydı ilk güncellemeden sonra b_2 kaç olurdu; adım nasıl değişir? 3) Tur 8'de hata 0.061. "Ağ öğrendi" diyebilir misin? Bu hatanın neyi ölçtüğüne, neyi ölçmediğine dikkat et.

TEKNİK DERİNLİK

Geri yayılım, kayıp fonksiyonunun her ağırlığa göre gradyanını zincir kuralıyla verimli biçimde hesaplar; gradyanlar çıkıştan girişe doğru katman katman geriye taşınır. Ardından gradyan inişi ağırlıkları günceller: $W \leftarrow W - \eta \cdot \partial L / \partial W$.

Bu yöntem (Rumelhart, Hinton ve Williams tarafından 1986'da popülerleştirildi) derin ağların eğitilebilmesinin anahtarıdır. Şekil 4.3'teki gösterim, tek bir örnek üzerinde gerçek bir eğitim döngüsü yürütür: ileri geçiş, kayıp, zincir kuralıyla gradyan, gradyan inişiyle güncelleme; gerçek eğitim aynı döngüyü milyonlarca örnekle yineler. Geri yayılım gradyanı hesaplar; parametreleri değiştiren, optimizasyon adımıdır.

Başlangıç: ağırlıklar eğitilmemiş, kayıp yüksek. Her tur, zincir kuralıyla $\partial L / \partial W$ gradyanını çıkıştan girişe hesaplar (geri yayılım); ardından gradyan inişi $W \leftarrow W - \eta \cdot \partial L / \partial W$ ile günceller.

Şekildeki ağ 2-2-1 boyutundadır, bütün aktivasyonlar sigmoid; kayıp $L = \frac{1}{2}(\hat{y} - y)^2$, $\eta = 2$. Çıktı nöronunda $\delta_2 = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$; gizli nöron j için $\delta_j = \delta_2 \cdot w_{2j} \cdot h_j(1 - h_j)$; gradyanlar $\partial L / \partial w_{2j} = \delta_2 \cdot h_j$, $\partial L / \partial b_2 = \delta_2$, $\partial L / \partial w_{1j} = \delta_j \cdot x_j$. Tablodaki her satır bu formüllerle hesaplanmıştır; hazır bir eğri yoktur.

Geri yayılım her türden ağı eğitir. Ama bir fotoğrafın on binlerce pikselini düz bir katmana vermek israftır; görüntüye özel bir düzen gerekir.

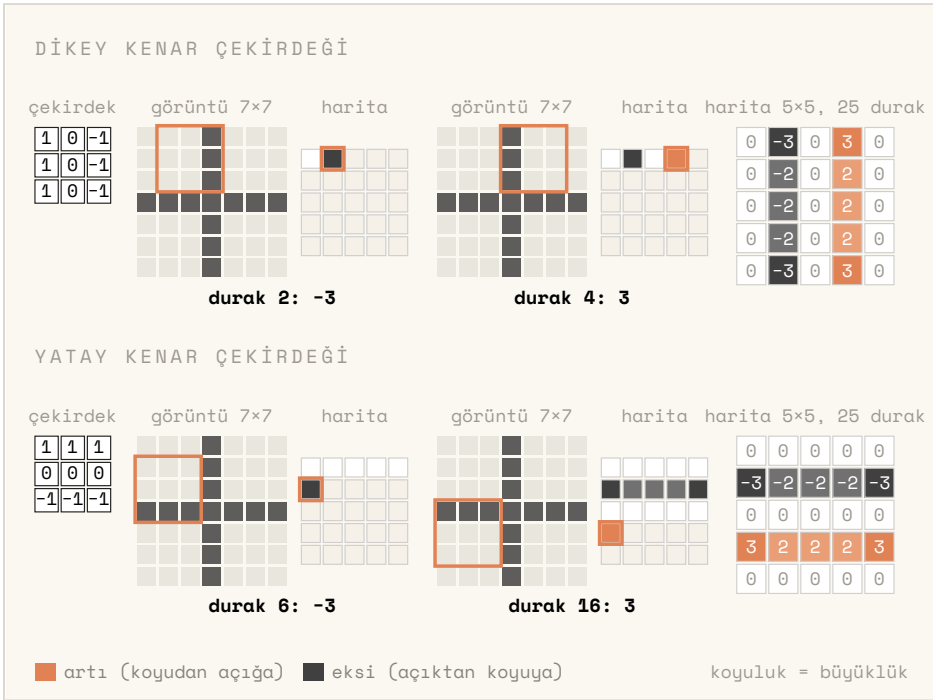
4.5 Görüntüyü görmek: evrişimli ağlar (CNN)

Bir görüntü on binlerce pikselden oluşur; her pikseli tek tek dinlemek delilik olurdu. Evrişimli sinir ağları (CNN) daha kurnazdır: karanlık bir odada el feneri gezdirir gibi, küçük bir “filtreyi” görüntünün üzerinde dolaştırır ve yerel desenleri arar; bir kenarı, bir köşeyi.

Aşağıda fenerin görüntü üzerinde adım adım gezisini izle. Her durakta küçük bir bölgeye bakar ve bulduklarını bir “özellik haritasına” işler. Şekil 4.4’te feneri (filtreyi) değiştirdince bu kez hangi kenarları yakaladığını gör.

***Kenar notu.** Aynı küçük filtre tüm görüntüde kullanılır (ağırlık paylaşımı). Böylece bir kenarı sol üstte de sağ altta da tanıyabilir, üstelik çok daha az parametreyle öğrenir.*

Şekil 4.4 • Evrişim: filtreyi kaydır



Şekil 4.4

Canlı demo: book.onuronder.com/d/3ba8a6fcb2

Kurulum. Şekil iki kuşaktan oluşur: üstte dikey kenar çekirdeği, altta yatay. Her kuşakta en solda 3 × 3’lük çekirdek (filtre). Ortada 7 ×

7'lik görüntünün iki kopyası var; her birinde turuncu çerçeveye seçilmiş bir durak ve o durağın küçük haritası. En sağda 25 durağın tamamı: 5×5 'lik özellik haritası. Görüntüde koyu hücreler 1, açık hücreler 0 değerindedir; görüntü bir artı işaretidir: dördüncü satır ve dördüncü sütun baştan sona koyu. Dikey kenar çekirdeği üç satırında da $[1, 0, -1]$ taşır; yatay kenar çekirdeği üstte $[1, 1, 1]$, ortada sıfırlar, altta $[-1, -1, -1]$. Pencere soldan sağa, yukarıdan aşağıya 25 durak yapar, her durak bir hücre doldurur; seçili duraklar dikeyde 2 ve 4, yatayda 6 ve 16. Turuncu artı, siyah eksi sonuç demektir; koyuluk büyüklüğü gösterir.

Adım adım. Dikey kenar çekirdeğiyle ilk satırı durak durak kaydır.

1. Durak 1: pencere 1–3. satırlar, 1–3. sütunlar. Hepsi 0; toplam 0.
2. Durak 2: pencere 2–4. sütunlara kayar. Sağ sütun (görüntünün 4. sütunu) üç satırda da 1. Hesap: $(1 \times 0 + 0 \times 0 + (-1) \times 1) + (1 \times 0 + 0 \times 0 + (-1) \times 1) + (1 \times 0 + 0 \times 0 + (-1) \times 1) = -3$. Haritanın 2. hücresi koyu siyah: eksi.
3. Durak 3: pencere 3–5. sütunlar. Koyu sütun tam ortaya gelir; çekirdeğin orta sütunu sıfır. Toplam 0.
4. Durak 4: pencere 4–6. sütunlar. Koyu sütun bu kez solda, ağırlığı +1. Toplam +3. Haritada koyu turuncu: artı.
5. Durak 5: pencere 5–7. sütunlar. Yine hepsi 0.
6. İkinci satırda (pencere 2–4. satırlar) pencerenin son satırı artının yatay koluna gelir; orada sol ve sağ birbirini götürür. Değerler -2 ve +2'ye iner.

Yirmi beş durağın tamamı:

Dikey çekirdek

0	-3	0	3	0
0	-2	0	2	0
0	-2	0	2	0
0	-2	0	2	0
0	-3	0	3	0

Yatay çekirdek

0	0	0	0	0
-3	-2	-2	-2	-3
0	0	0	0	0
3	2	2	2	3
0	0	0	0	0

Dikey çekirdek yalnız dikey çizginin iki yanını işaretler; yatay kol haritada kaybolur. Yatay çekirdek tam tersini yapar. Eksi işaret açıktan koyuya, artı işaret koyudan açığa geçiştir. Aynı dokuz sayı bütün görüntüde dolaşır; kenar nerede olursa olsun bulunur. Görüntü bir hücre kaysa harita da bir hücre kayar: buna eşdeğişkenlik denir, değişmezlik değil.

Ne oluyor? Küçük bir “filtre” (büyüteç gibi) görüntünün üzerinde adım adım geziyor. Her durakta baktığı küçük bölgede aradığı deseni (mesela dikey bir kenar) bulursa orayı parlak işaretliyor. Sonuçta ortaya, o desenin görüntüde nerelerde olduğunu gösteren bir “özellik haritası” çıkıyor.

Kendin dene. 1) Dikey çekirdekle 22. durağı hesapla: pencere 5–7. satırlar, 2–4. sütunlar. 2) Yatay çekirdekle tam ortadaki durağı hesapla: pencere 3–5. satırlar, 3–5. sütunlar. 3) Çekirdek 3×3 yerine 5×5 olsaydı özellik haritası kaç hücre olurdu?

TEKNİK DERİNLİK

CNN’ler, paylaşılan ağırlıklı evrişim çekirdekleriyle (kernel) yerel örüntüleri tespit eder; çekirdek görüntü üzerinde kaydırılır ve her konumda eleman-yönlü çarpımların toplamı bir özellik haritası üretir. Havuzlama (pooling) ile boyut indirgenir; katmanlar derinleştikçe kenarlardan şekillere, oradan nesnelere doğru hiyerarşik temsiller öğrenilir.

Ağırlık paylaşımı ve yerel bağlantılılık parametre sayısını büyük ölçüde azaltır; evrişim ideal koşullarda ötelemeye eşdeğişken

TEKNİK DERİNLİK · DEVAM

özellik haritaları üretir (girdi kayınca harita da kayar), havuzlama gibi işlemler ise küçük kaymalara karşı yaklaşık değişmezlik kazandırabilir. LeNet (LeCun) bu mimarinin öncüsü oldu; AlexNet (2012) ise CNN'leri büyük ölçekte görünür kıldı. Şekil 4.4'teki gösterim gerçek bir evrişim işlemini gösterir.

Seçili çekirdek görüntü üzerinde 3×3 pencerelerle gezinir; dikey kenar çekirdeği dikey sınırları vurgular.

Şekildeki işlem $(I * K)(r, c) = \sum_i \sum_j K(i, j) \cdot I(r + i, c + j)$, $i, j \in \{0, 1, 2\}$ biçimindedir. Dolgu (padding) yok, adım (stride) 1; bu yüzden 7×7 görüntü ve 3×3 çekirdekten $(7 - 3 + 1) = 5$ boyutunda harita çıkar. Dokuz ağırlık 25 konumda paylaşılır. Çekirdek çevrilmeden uygulanır; bu işlem matematikteki adıyla çapraz korelasyondur, derin öğrenme literatüründe evrişim diye anılır. Dikey çekirdek simetrik olmadığından çevrilseydi haritanın işaretleri ters dönerdi.

Evrişim uzaydaki komşuluğu yakalar. Komşuluk zaman içindeyse, bir cümlede kelimelerin sırasını ağ nasıl taşır?

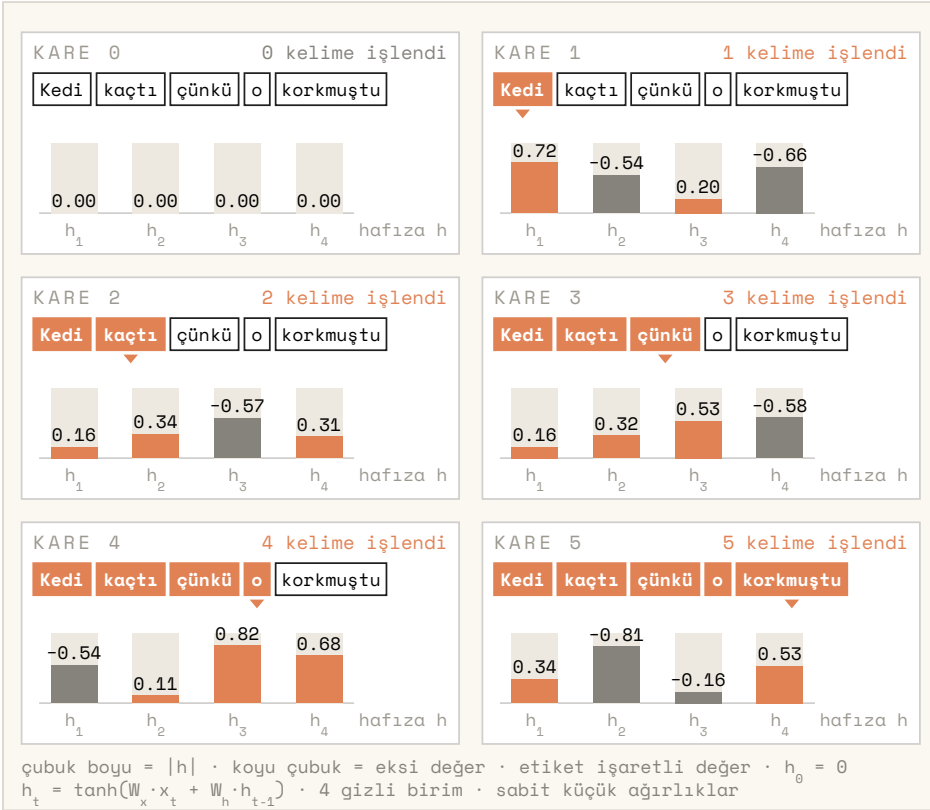
4.6 Diziyi anlamak: yinelemeli sinir ağları (RNN)

Bir masalı dinleyen çocuğu düşün: her yeni cümleyi, öncekilerin hatırasıyla dinler; yoksa hikâye dağılır. Metin, müzik, konuşma da böyledir: hepsi birer dizidir ve sıra önemlidir. İngilizcede “the dog bit the man” ile “the man bit the dog” aynı kelimeleri taşır ama bambaşka şeyler anlatır. Türkçede kimin kimi ısırdığını hâl ekleri söyler (“köpek adamı” / “adam köpeği”); sıra ise vurguyu değiştirir: “Köpek adamı ısırdı” ile “Adamı köpek ısırdı”. Yinelemeli sinir ağları (RNN) bu yüzden yanlarında bir hafıza taşır.

RNN diziyi kelime kelime dinler ve her adımda hafızasını tazeler; böylece geçmiş unutmadan ilerler. Aşağıda kelimelerin sırayla verilmesini ve hafızanın her adımda nasıl değiştiğini izle.

Kenar notu. RNN'in özü tek bir cümlede: “her yeni girdiyi, o ana kadar gördüklerinin hafızasıyla birlikte işle.” Aynı hücre tekrar tekrar kullanıldığı için diziye “döngüsel” bakar.

Şekil 4.5 • Hafızalı işleme (gerçek yineleme)



Şekil 4.5

Canlı demo: book.onuronder.com/d/a6130db86e



Kurulum. Şekil altı kareden oluşur; kare 0'dan kare 5'e. Her karenin üstünde beş kelimelik cümle var: “Kedi kaçtı çünkü o korkmuştu”. İşlenmiş kelimeler koyu boyalı. Altta dört çubuk gizli durumu, yani

ağın hafızasını gösterir; her biri -1 ile 1 arasında bir sayıdır (tanh çıktısı). Çubuk boyu değerin büyüklüğünü, $|h|$ 'yi gösterir ve hep yukarı doğru çizilir. İşareti çubuğun rengi ve üstündeki etiket söyler: turuncu çubuk artı, koyu çubuk eksi (ör. -0.54). Karenin başlığında kaç kelimenin işlendiği yazar. Kare 0'da dört çubuk da sıfırdır: $h_0 = 0$. Sayılar gerçekten $h_t = \tanh(W_x x_t + W_h h_{t-1})$ ile hesaplanır; W_x (4×5) ve W_h (4×4) anlatım için seçilmiş sabit ağırlıklardır, eğitilmemiştir.

Adım adım. Kareleri soldan sağa oku; ilk iki adımı kendin hesapla. Her kelime bir birim vektördür: “Kedi” $[1, 0, 0, 0, 0]$, “kaçtı” $[0, 1, 0, 0, 0]$ ve böyle gider. $W_x x_t$ bu yüzden W_x 'in t . sütunudur.

1. Kare 0, “0 kelime işlendi”: $h_0 = [0, 0, 0, 0]$. Dört çubuk da sıfırdır.
2. Kare 1, “Kedi” işlendi: W_x 'in 1. sütunu $[0.9, -0.6, 0.2, -0.8]$; $W_h h_0 = 0$. $h_1 = \tanh([0.9, -0.6, 0.2, -0.8]) = [0.72, -0.54, 0.20, -0.66]$.
3. Kare 2, “kaçtı” işlendi: W_x 'in 2. sütunu $[-0.4, 0.8, -0.5, 0.3]$. Bu kez hafıza da konuşur: $W_h h_1 = [0.56, -0.44, -0.14, 0.02]$. Toplam $[0.16, 0.36, -0.64, 0.32]$; $h_2 = [0.16, 0.34, -0.57, 0.31]$. İkinci kelime birinciyi silmedi; onunla karıştı.
4. Kare 3, “çünkü”: $h_3 = [0.16, 0.32, 0.53, -0.58]$. Üçüncü bileşen eksiden artıya döndü.
5. Kare 4, “o”: $h_4 = [-0.54, 0.11, 0.82, 0.68]$. Kısa bir kelime bile hafızayı baştan sona yeniden şekillendirdi.
6. Kare 5, “korkmuştu”: $h_5 = [0.34, -0.81, -0.16, 0.53]$. Cümlenin sonunda hafızada beş kelimenin izi var; ama hiçbirini ayrı bir yerde durmuyor, hepsi aynı dört sayıya karışmış.

Adım	Kelime	$h[1]$	$h[2]$	$h[3]$	$h[4]$
0	(boş)	0.00	0.00	0.00	0.00
1	Kedi	0.72	-0.54	0.20	-0.66
2	kaçtı	0.16	0.34	-0.57	0.31
3	çünkü	0.16	0.32	0.53	-0.58
4	o	-0.54	0.11	0.82	0.68
5	korkmuştu	0.34	-0.81	-0.16	0.53

Önemli olan tek tek sayılar değil, davranış: hafıza yalnızca birikmiyor, her adımda yeniden karılıyor. Aynı dört sayı beş kelimeyi de taşıyor; ağ her kelime için yeni bir hafıza açmıyor, elindekini güncelliyor. Ağırlıklar eğitilmediği için bu sayılar kelimelerin anlamını bilmez; eğitilmiş bir RNN’de W_x ve W_h veriden öğrenilir ve her kelime, birim vektör yerine öğrenilmiş bir sayı dizisiyle girer.

Ne oluyor? Ağ kelimeleri tek tek okuyor ve bir “hafıza” taşıyor. Her yeni kelimedede bu hafızayı, hem yeni kelimeye hem o ana kadar biriktirdiğine bakarak günceller. Böylece sırayı hatırlar; “köpek adamı ısırıldı” ile “adamı köpek ısırıldı” onun için artık aynı şey değildir.

Kendin dene. 1) $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$ formülünde $h_0 = 0$ ise ilk adımda hangi terim hesaba katkı vermez? 2) Sırayı değiştir: “Kedi kaçtı o çünkü korkmuştu”. Üçüncü adımda $W_x x_t$ artık hangi sütundur? h_2 aynı kalır; h_3 ’ü hesapla ve tablodakiyle karşılaştır. 3) Kelime sırasını umursamayan bir model “köpek adamı ısırıldı” ile “adamı köpek ısırıldı” cümlelerine aynı hafızayı üretir mi? RNN ne yapar?

TEKNİK DERİNLİK

RNN, her zaman adımında $h_t = \phi(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$ ile gizli durumunu günceller; aynı ağırlıklar her adımda yeniden kullanılır (zamanda parametre paylaşımı). Bu, değişken uzunluktaki dizileri bir bağlam vektörüyle işlemeyi sağlar.

Klasik RNN’ler uzun bağımlılıklarda kaybolan gradyan sorunuyla zorlanır; LSTM (Hochreiter & Schmidhuber, 1997) ve GRU bunu kapı (gate) mekanizmalarıyla hafifletir. Çoğu modern dizi görevinde RNN’lerin yerini büyük ölçüde dikkat (attention) tabanlı transformer’lar aldı; onları da sıradaki bölümde tanıyacağız. Şekil 4.5’teki gösterim, gizli durumu $h_t = \tanh(W_x x_t + W_h h_{t-1})$ denklemiyle, sabit ve eğitilmemiş ağırlıklarla adım adım gerçekten hesaplar.

Gizli durum $h_0 = 0$. Her adımda $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$ ile güncellenir; aynı ağırlıklar her kelimedede yeniden kullanılır.

TEKNİK DERİNLİK · DEVAM

Şekildeki dört çubuk 4 boyutlu bir gizli durum vektörüdür. Ağırlıklar: W_x satırları $[0.9, -0.4, 0.3, -0.7, 0.5]$, $[-0.6, 0.8, -0.2, 0.4, -0.9]$, $[0.2, -0.5, 0.7, 0.6, -0.3]$, $[-0.8, 0.3, -0.6, 0.9, 0.1]$; W_h satırları $[0.5, -0.3, 0.2, 0.0]$, $[0.1, 0.4, -0.5, 0.3]$, $[-0.2, 0.6, 0.3, -0.4]$, $[0.3, -0.1, 0.4, 0.5]$. Bunlar anlatım için seçilmiş sabitlerdir, eğitilmemiştir; gerçek bir uygulamada x_t kelimenin gömme (embedding) vektörüdür ve W_x , W_h eğitimle öğrenilir.

Buraya kadar ağlar hep tanıdı: nöron ateşledi, kenar bulundu, dizi hatırlandı. Bir ağ hiç görmediği bir yüzü sıfırdan üretebilir mi?

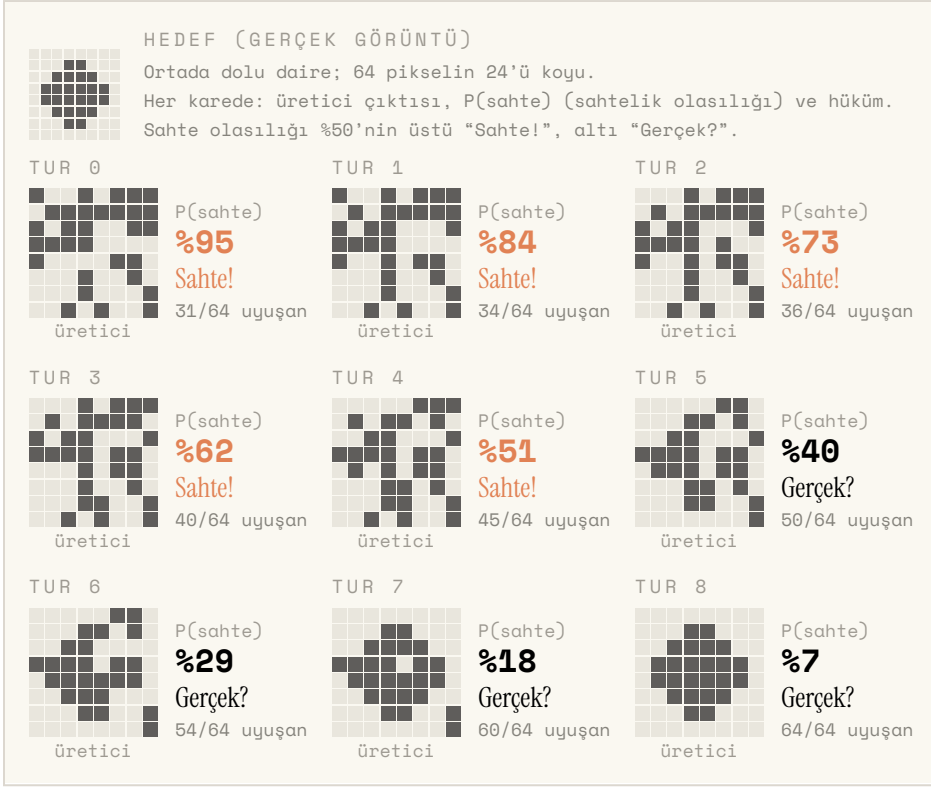
4.7 Sahteciliğin sanatı: GAN

Bazen amaç tanımak değil, üretmektir: gerçekçi yüzler, manzaralar, sesler. Üretken çekişmeli ağların (GAN) hilesi, bir kalpazanla bir dedektifi aynı odaya kilitlemektir. Kalpazan (üretici) sahte örnekler üretir; dedektif (ayırt edici) gerçeği sahteden ayırmaya çalışır.

İkisi durmadan yarışır: dedektif yakaladıkça kalpazan ustalaşır, kalpazan ustalaştıkça dedektif keskinleşir. Şekil 4.6'yı tur tur oku; gürültüden ibaret görüntünün, kalpazan piştikçe gerçeğe nasıl yaklaştığını gör.

***Kenar notu.** GAN'da iyi sahte üretmek ile sahteyi yakalamak birbirini sürekli iter. Kalpazanla dedektifin yarışı gibi; ikisi de geliştikçe sonuç giderek gerçeğe yaklaşır.*

Şekil 4.6 • Üretici ile ayırt edici



Şekil 4.6

Canlı demo: book.onuronder.com/d/8aeftaa8e9

Kurulum. Şekil dokuz kareden oluşan bir film şerididir; tur 0'dan tur 8'e. Bu şerit temsilidir: görüntüler de yüzdeler de anlatım için önceden belirlenmiştir, eğitilmiş bir ağdan ölçülmemiştir. Her karede solda üreticinin çizdiği 8 × 8'lik görüntü var: koyu ve açık pikseller. Üreticinin öğrenmeye çalıştığı "gerçek" görüntü şeridin üstünde durur: ortası dolu bir daire; 64 pikselin 24'ü koyu. Sağda ayırt edicinin kararı yazar: görüntünün sahte olma olasılığı ve tek kelimelik hüküm. Yüzde 50'nin üstü "Sahte!", altı "Gerçek?".

Adım adım. Kareleri sırayla oku.

1. Tur 0: Üretici saf gürültü çiziyor; 64 pikselin yalnız 31'i hedefle uyuyor, yazı tura kadar. Dedektif kesin: sahte olasılığı yüzde 95, hüküm "Sahte!"
2. Tur 1 ve 2: Olasılık yüzde 84'e, sonra 73'e iner. Görüntüde dairenin ilk izleri belirir; uyuşan piksel 34, sonra 36.
3. Tur 3 ve 4: Yüzde 62 ve 51. Uyuşan piksel 40 ve 45. Dedektif hâlâ "Sahte!" diyor ama zar zor; yüzde 51, sınırın hemen üstü.
4. Tur 5: Olasılık yüzde 40'a düşer, hüküm "Gerçek?" olur. Dedektif ilk kez kandırılır. Uyuşan piksel 50.
5. Tur 6 ve 7: Yüzde 29 ve 18. Daire artık seçiliyor; uyuşan piksel 54 ve 60.
6. Tur 8: Yüzde 7, hüküm "Gerçek?". 64 pikselin 64'ü yerinde; şeritte üretici hedefi tam tutturdu. Yüzde 7 temsili bir sayıdır, ölçülmüş bir olasılık değil.

Tur	Sahte olasılığı	Hüküm	Uyuşan piksel
0	%95	Sahte!	31 / 64
1	%84	Sahte!	34 / 64
2	%73	Sahte!	36 / 64
3	%62	Sahte!	40 / 64
4	%51	Sahte!	45 / 64
5	%40	Gerçek?	50 / 64
6	%29	Gerçek?	54 / 64
7	%18	Gerçek?	60 / 64
8	%7	Gerçek?	64 / 64

Her tur olasılık 11 puan düşer; bu, şeridin sabit kuralıdır, bir eğitim sonucu değil. Gerçek eğitimde dedektif de aynı anda keskinleşir ve düşüş böyle düz bir çizgi izlemez. Kuramsal hedef de yüzde 7 değildir: ideal dengede ayırt edici gerçekle sahteyi ayıramaz, her görüntüye yüzde 50 verir.

Ne oluyor? İki ağ yarışıyor: biri (üretici) sahte görüntü üretiyor, öteki (ayırt edici) bunun sahte mi gerçek mi olduğunu yakalamaya çalışıyor. Başta kalpazan acemidir, kolayca yakalanır. Her turda üretici biraz daha iyi sahte yapmayı öğrenir ve “sahte” olasılığı düşer; kalpazanla dedektifin yarışı gibi. Bu gösterimde görüntü de yüzde de önceden belirlenmiştir; gerçek eğitimde iki ağ birlikte öğrenir ve ideal dengede ayırt edici gerçeğe sahteyi ayırt edemez.

Kendin dene. 1) Hüküm hangi turda “Sahte!”den “Gerçek?”e döner ve o turdaki olasılık nedir? 2) Tur 8’deki yüzde 7 nasıl bulunur; kuralı yazıp hesapla. 3) Dedektif her görüntüye tam yüzde 50 derse bu neyin işaretidir?

TEKNİK DERİNLİK

GAN’lar (Goodfellow vd., 2014) iki ağı çekişmeli (adversarial) bir oyunda eğitir: üretici G, rastgele gürültüden örnekler üretir; ayırt edici D ise gerçek ile üretilen örnekleri ayırmaya çalışır. G, D’yi kandırma olasılığını en üst düzeye çıkaracak; D ise hata yapmamak için eş zamanlı eğitilir.

Eğitim bir min-maks oyunudur; denge noktasında üretici örnekleri gerçek dağılımdan ayırt edilemez hâle gelir. GAN’lar fotogerçekçi görüntülerde çığır açtı; günümüzde difüzyon modelleri de yaygın bir alternatiftir (Bölüm 5). Şekil 4.6’daki gösterimde görüntü ve yüzde önceden belirlenmiştir; ölçülmüş eğitim sonucu değildir. İdeal GAN dengesinde ayırt edici gerçek ve üretilmiş örnekleri ayırt edemez.

Başlangıç: G rastgele gürültü üretir, D bunu kolayca sahte olarak işaretler (sahte olasılığı yüksek). Üretici ustalaştıkça sahte olasılığı düşer.

Oyunun amaç fonksiyonu $\min_G \max_D V(D, G) = E_x[\log D(x)] + E_z[\log(1 - D(G(z)))]$ biçimindedir; kuramsal optimumda, üretici veri dağılımını yakaladığında, optimal ayırt edici her örneğe $D(x) = 1/2$ verir. Tek başına $1/2$ çıktısı bu optimuma ulaşıldığını göstermez; eğitilmemiş bir ayırt edici de aynı değeri verebilir. Şekilde sahte olasılığı $p(r) = 95 - 11 \cdot r$ yüzdesiyle, görüntü ise r .

TEKNİK DERİNLİK · DEVAM

turda piksellerin yaklaşık $r/8$ 'inin hedefe kilitlenmesiyle üretilir; ikisi de temsilidir, bir eğitimden ölçülmemiştir.

Ağlar tanımayı da üretmeyi de öğrendi. Bugünün sohbet eden, resim çizen sistemleri ise bir adım ötede: dikkat mekanizması ve transformer'lar. Sırada onlar var.

4.8 Kendini test et

Cevaplar kitabın sonunda.

1. Bir yapay nöron ne hesaplar?
 - a) Sadece girdilerin ortalaması
 - b) Girdilerin ağırlıklı toplamı + sabit terim, sonra aktivasyon
 - c) Rastgele bir sayı
 - d) Girdileri olduğu gibi kopyalar
2. Bir sinir ağını “derin” yapan nedir?
 - a) Çok hızlı çalışması
 - b) Çok sayıda gizli katman
 - c) Tek bir nöronu olması
 - d) İnternete bağlı olması
3. Aktivasyon fonksiyonu hiç olmasaydı ne olurdu?
 - a) Ağ daha hızlı olurdu
 - b) Ağ tek bir doğrusal işleve çökerdi
 - c) Hiçbir şey değişmezdi
 - d) Ağ daha güçlü olurdu
4. Geri yayılım ne yapar?
 - a) Yeni katman ekler
 - b) Kaybın her ağırlığa göre gradyanını çıkıştan girişe doğru hesaplar
 - c) Görüntüyü büyütür
 - d) Veriyi siler

5. CNN'ler özellikle hangi veride güçlüdür?

- a) Tek bir sayı
- b) Şifreler
- c) Tablolar
- d) Görüntü

6. GAN'da çekişen iki ağ hangileridir?

- a) Girdi ve Çıktı
- b) CNN ve RNN
- c) Üretici ve Ayırt edici
- d) Öğretmen ve Öğrenci

Bu bölümden kalanlar

- Yapay nöron beynin kaba bir matematik taklididir; güç tek nöronda değil, milyonlarcasının birlikte kurduğu örüntüdedir.
- Bir nöron girdileri ağırlıkla tartar, toplar, sabit terim ekler ve aktivasyondan geçirir; $w = [0.7, -0.5, 0.9]$ ile 0.69'luk toplam sigmoidde 0.666 verir ve nöron ateşler.
- Katmanlar hâlinde dizilen nöronlarda sinyal girişten çıkışa akar; buna ileri besleme denir ve en yüksek çıktı ağın tahmini sayılır; eğitilmemiş ağda bu tahminin anlamı yoktur.
- Aktivasyon olmasaydı kaç katman olursa olsun ağ tek bir doğrusal işleve çökerdi.
- Geri yayılım kaybın her ağırlığa göre gradyanını çıkıştan girişe hesaplar; gradyan inişi ağırlıkları günceller. Şekil 4.3'te hata sekiz turda 0.28'den 0.06'ya indi.
- Evrişimli ağlar küçük bir çekirdeği bütün görüntüde gezdirir; aynı dokuz sayı kenarı nerede olursa olsun bulur.
- Yinelemeli sinir ağları her kelimeyi önceki hafızayla birlikte işler; GAN'da ise üretici ile ayırt edici birbirini iterek gerçeğe yaklaşır.

5

BÖLÜM 5

Bugünün Yapay Zekâsı

Token'dan dil modeline, dikkatten difüzyona

5.1 Üretken çağ: tanımaktan üretmeye

Bu kitapta makineleri çoğunlukla “tanıyan” tarafta gördük: Bu spam mı, bu kedi mi, bu ev kaç para eder? Önceki bölümün sonunda GAN ile üretime ilk adımı da attık: gürültüden yeni bir görüntü çıkaran bir ağ. Ressam çırağı yıllarca tablo seyretmiş, ilk eskizlerini de çizmişti; şimdi fırça onun elinde. Makineler yazı yazıyor, resim çiziyor, kod üretiyor, sohbet ediyor.

Bu sıçramanın dönüm noktalarından biri Transformer mimarisi ve onun kalbindeki dikkat (attention) fikri. Üretken modeller (GAN gibi) ve dikkat mekanizmaları daha önce de araştırılıyordu; Transformer, üretken dil modellerinin büyük ölçekte eğitilmesini mümkün kıldı. Hikâyeye en küçük parçadan başlıyoruz: bir cümle makineye nasıl görünür? Token denen küçük metin parçaları olarak. Önce token, sonra gömü, dikkat ve kelime kelime üretim; ardından eğitim, difüzyon ve sınırlar.

***Kenar notu.** “Neden tam şimdi?” sorusunun cevabı üç ayakta: bol veri (internet), güçlü donanım (GPU) ve doğru mimari (Transformer). Üçü bir araya gelince üretken çağ başladı.*

TEKNİK DERİNLİK

Üretken YZ, ayırt edici (discriminative) modellemekten üretken (generative) modellemeye geçişi temsil eder. Ayırt edici bir sınıflandırıcı, girdi x verildiğinde etiket y 'nin olasılığını modeller: $p(y|x)$. Üretici model ise üretilcek içeriğin, yani x 'in, dağılımını modeller: $p(x)$ ya da bir koşula bağlı olarak $p(x|koşul)$; bu dağılımdan yeni içerik örnekleri üretilir. Transformer mimarisi (Vaswani vd., 2017, “Attention Is All You Need”) üretken dil modellerinin ölçeklenmesinde önemli bir dönüm noktasıydı; üretken modeller (ör. GAN, 2014) ve dikkat mekanizmaları daha önce de araştırılıyordu. Modern sıçrama bu mimari ile onu

TEKNİK DERİNLİK · DEVAM

büyük ölçekte eğitebilen veri + hesaplama birikiminin birleşmesinden doğdu.

Bu bölüm üretken yığını uçtan uca kurar: tokenleştirme → gömü uzayı → öz-dikkat (self-attention) → otoregresif üretim → eğitim hattı (ön eğitim, ince ayar, RLHF) → difüzyon tabanlı görüntü üretimi → ve pratik sınırlar (halüsinasyon, bağlam penceresi, maliyet). Böylece bugünün LLM ve üretken modellerinin neden ve nasıl çalıştığı yerine oturur.

Bu yığının ilk basamağı en küçük parça. Sen bir cümleyi harf harf, kelime kelime okursun; makine neyi okur?

5.2 Makine kelimeleri nasıl görür: token'lar

Bir dil modeli, harfleri ya da kelimeleri “olduğu gibi” görmez. Metni önce minik lego parçalarına ayırır; bunlara token denir. Bir token bazen koca bir kelimedir, bazen bir kelimedenden kopmuş küçük bir parça, bazen de yalnızca bir virgül.

Neden parçalıyor? Lego kutusunu düşün: sınırlı sayıda parçayla sonsuz şey kurulur. Dünyadaki bütün kelimeleri ezberlemek imkânsızdır; ama sınırlı bir parça takımıyla her kelime kurulabilir. Aşağıdan bir örnek seç, makinenin cümleyi parçalara ayırışını izle.

***Kenar notu.** Aynı metin, farklı dillerde farklı sayıda token tutar. Bu yüzden bir LLM'e Türkçe sormak bazen İngilizce sormaktan daha “pahalı” olabilir.*

Şekil 5.1 • Cümleleri token'lara böl

CÜMLE 1 *Merhaba dünya, bugün 2026.*

Merh	##aba	dünya	,	bugün	2026	.
------	-------	-------	---	-------	------	---

7token · 4 kelime

CÜMLE 2 *Yapay zekâ metni token'lara böler.*

Yapay	zekâ	metni	toke	##n'la	##ra	böler	.
-------	------	-------	------	--------	------	-------	---

8token · 5 kelime

CÜMLE 3 *Tokenleştirme şaşırtıcı derecede önemli!*

Toke	##nleş	##tirm	##e	şası	##rtıc	##1	dere	##cede
önemli	!							

11token · 4 kelime

“##” = önceki parçanın devamı (bir kelime bölündüğünde kullanılır)

Şekil 5.1

Canlı demo: book.onuronder.com/d/301fb38607

Kurulum. Şekilde üç örnek cümle ve her birinin token'lara ayrılmış hâli var. Her token bir kutu içinde. Beyaz kutular bağımsız bir parçayı, turuncu tonlu kutular ise “##” ile başlayan devam parçalarını gösterir. Her cümlemin altında iki sayı yazıyor: kelime sayısı ve token sayısı. Bölücünün kuralı canlı demodakiyle aynı; oyuncak bir kuraldır, gerçek bir tokenizer değildir.

Adım adım. Bölücünün kuralı üç satır. Altı karakter ve daha kısa kelimeler tek token kalır. Daha uzun kelimeler dörder karakterlik parçalara bölünür; ilk parça düz, sonrakiler “##” ile yazılır. Virgül,

nokta ve ünlem her zaman ayrı bir token'dır. Şimdi bu kuralı üç cümleye uygula:

Cümle	Token'lar	Kelime	Token
Merhaba dünya, bugün 2026.	Merh · ##aba · dünya · , · bugün · 2026 · .	4	7
Yapay zekâ metni token'lara böler.	Yapay · zekâ · metni · toke · ##n'la · ##ra · böler · .	5	8
Tokenleştirme şaşırtıcı derecede önemli!	Toke · ##nleş · ##tirm · ##e · şaşı · ##rtıc · ##ı · dere · ##cede · önemli · !	4	11

Birinci cümlede “Merhaba” yedi harf; sınırı bir harfle aşıyor ve ikiye bölünüyor. “2026” bir sayı ama bölücü için sıradan bir dört karakterli kelime. Nokta ayrı kutuya düşüyor. İkinci cümlede “token'lara” kesme işaretiyle birlikte on karakter; üç parçaya ayrılıyor. Kesme işareti, virgöl gibi ayrı sayılmıyor; kelimenin içinde kalıyor. Üçüncü cümle en çarpıcı olanı: dört kelime, on bir token. “Tokenleştirme” tek başına dört parça. “önemli” tam altı harf; sınırı aşmadığı için tek parça kalıyor.

Aynı sayıda kelimeden çok farklı sayıda token çıkabiliyor: birinci ve üçüncü cümle dörder kelime, ama biri 7 token, öteki 11. Belirleyici olan kelime sayısı değil, uzun ve nadir kelimelerin sayısı. Gerçek bir tokenizer sınırları bu kuraldan farklı çizer; kaç token çıkacağı seçilen tokenizer'a ve sözlüğüne bağlıdır. Yön ise aynıdır: Türkçe ekler kelimeleri uzattığı için bu dilde token sayısı çoğu zaman İngilizceye göre şişer. Türkçe sormanın bazen daha pahalı olması bu yüzden.

Ne oluyor? Model bir cümleyi olduğu gibi görmez; önce küçük parçalara (token) böler. Buradaki bölücü kısa kelimeleri tek parça bırakır, uzun kelimeleri “##” ile böler, noktalama işaretlerini ayrı sayar; gerçek token sınırları kullanılan tokenizer'a ve sözlüğüne bağlıdır. Model her şeyi bu parçalar hâlinde okur; hem kapasitesi hem ücreti token sayısına göre hesaplanır.

Kendin dene. 1) “Yapay zekâ öğreniyor.” cümlesini aynı kuralla böl; kaç token çıkar? 2) “Bilgisayarlarımızla” kelimesi kaç parçaya ayrılır?

Parçaları yaz. 3) Üçüncü örnek cümlede kelime başına ortalama kaç token düşüyor? Birinci cümleyle karşılaştır.

TEKNİK DERİNLİK

Modern modeller alt-kelime (subword) tokenleştirme kullanır (yaklaşımlar: BPE, WordPiece, unigram; SentencePiece ise bu modelleri eğiten bir araçtır): sık geçen diziler tek token olur, nadir kelimeler birden çok parçaya ayrılır. Sözlük tipik olarak 30K–100K+ token içerir; her token bir tam sayı kimliğine (ID) eşlenir.

Kabaca İngilizcede 1 token $\approx$ 0.75 kelime; Türkçe gibi eklemeli dillerde kelime başına daha çok token düşebilir. Model bağlamı ve maliyeti token cinsinden ölçülür: hem bağlam penceresi hem ücretlendirme token sayısına bağlıdır. Şekil 5.1 basitleştirilmiş bir alt-kelime bölücü kullanır (uzun kelimeleri “##” ile parçalara ayırır).

Bu gösterimde kısa kelimeler tek token kalır, uzun kelimeler “##” ile parçalara ayrılır, noktalama ayrı bir token sayılır; gerçek token sınırları seçilen tokenizer’a ve sözlüğe bağlıdır.

Şekil 5.1’deki kural (en çok 6 karakter tek token, sonrası dörder karakter) gerçek bir tokenizer sözlüğünün yerini tutmaz. Gerçek bir sözlükte sınırlar sıklık istatistiğine göre düşer: sık geçen bir kelime çoğu zaman tek token kalır, nadir bir kelime birkaç parçaya ayrılır. “Merhaba” ya da “Tokenleştirme” için kesin bir token sayısı, tokenizer ve sürümü sabitlenip çalıştırılmadan verilemez. Kural basit, sonuç aynı yöne işaret eder: nadir ve uzun kelimeler daha çok parça tutar.

Her token artık bir sayı kimliği taşıyor. Ama bir kimlik numarası, “kedi” ile “köpek”in yakın olduğunu söylemez. Anlam nereden gelir?

5.3 Anlamı sayıya çevirmek: gömüler

Token'lar makineye sayı olarak girer ama kuru bir kimlik numarası “anlam” taşımaz. Vektör gösterimi, kısaca gömü (embedding), burada sahneye çıkar: Her kelime, kocaman bir şehirde bir adrese yerleştirilir. O adres, kelimenin anlamını taşıyan bir sayı listesidir.

Bu şehirde anlamca benzeşen kelimeler aynı mahalleye taşınır. “Kedi” ile “köpek” kapı komşusudur; “kral” ile “kraliçe” de öyle. Şekil 5.2’de bir kelimeye bak, komşularını gör.

***Kenar notu.** Gömüler yalnızca kelimeler için değil: cümleler, görseller, sesler de aynı uzaya gömülebilir. Çok-kipli (multimodal) modellerin ve anlamsal aramanın (semantic search) temeli budur.*

Kurulum. Şekil, dokuz kelimeyi iki boyutlu bir haritaya yerleştiriyor. Üç kesikli daire, üç anlam ailesi: hayvanlar (kedi, köpek, kuş), krallık (kral, kraliçe, prens), yiyecekler (elma, ekmek, peynir). Her noktanın yanında kelimesi yazıyor; koordinatlar aşağıdaki tabloda. Noktalar yakınlık fikrini göstermek için elle yerleştirilmiş; eğitilmiş bir modelden ya da PCA/t-SNE çalıştırılarak elde edilmemiş. Bu haritada yakın duran noktalar anlamca da yakın sayılıyor; uzaklık düz bir cetvelle ölçülüyor.

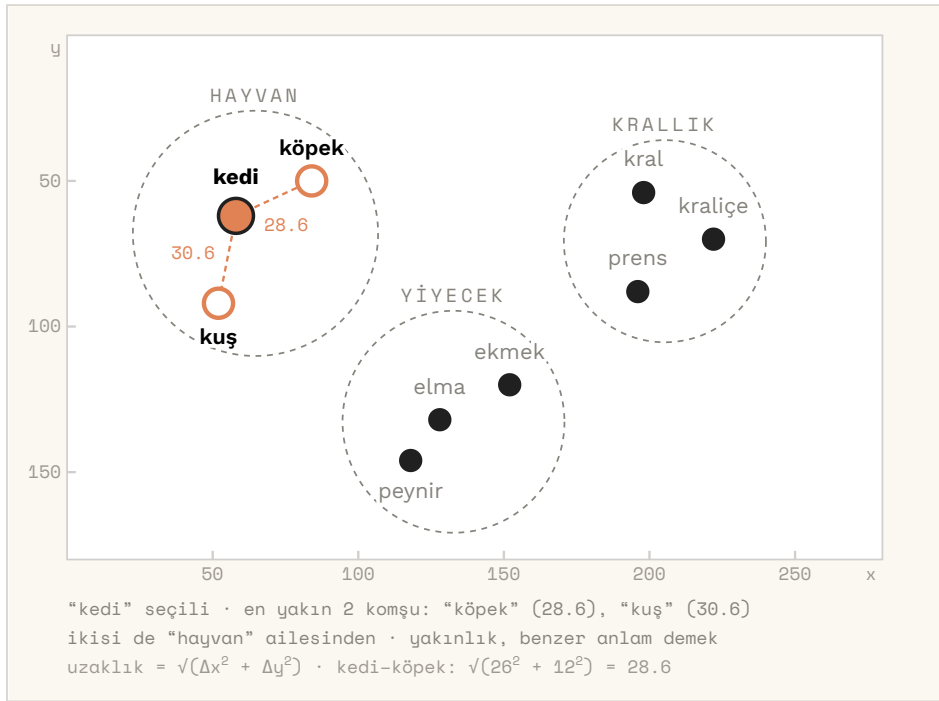
Adım adım. Haritadaki konumlar ve her kelimenin en yakın iki komşusu:

Kelime	Aile	x	y	En yakın 2 komşu (uzaklık)
kedi	hayvan	58	62	köpek (28.6), kuş (30.6)
köpek	hayvan	84	50	kedi (28.6), kuş (52.8)
kuş	hayvan	52	92	kedi (30.6), köpek (52.8)
kral	krallık	198	54	kraliçe (28.8), prens (34.1)
kraliçe	krallık	222	70	kral (28.8), prens (31.6)

Kelime	Aile	x	y	En yakın 2 komşu (uzaklık)
prens	krallık	196	88	kraliçe (31.6), kral (34.1)
elma	yiyecek	128	132	peynir (17.2), ekmek (26.8)
ekmek	yiyecek	152	120	elma (26.8), peynir (42.8)
peynir	yiyecek	118	146	elma (17.2), ekmek (42.8)

Uzaklık Pisagor’la hesaplanıyor: iki noktanın yatay ve dikey farklarının karelerini topla, karekökünü al. “kedi” ile “köpek” için farklar 26 ve 12; kareleri 676 ve 144; toplam 820; karekök 28.6. Bu hesabı dokuz kelime için yaptığında her kelimenin en yakın iki komşusu kendi ailesinden çıkıyor; tek istisna yok.

Şekil 5.2 • Anlam haritası



Şekil 5.2

Canlı demo: book.onuronder.com/d/09ebf3e5df

Aileler arasında uzaklık büyüyor. “kedi” ile “elma” arası 99 birim; “kedi” ile “kraliçe” arası 164. Harita yalnız kimin komşu olduğunu değil, kimin ne kadar yabancı olduğunu da söylüyor. Sınır bölgesi de var: “prens”in üçüncü en yakın kelimesi “ekmek” (54.4); krallık mahallesiyle yiyecek mahallesi birbirine değişiyor. Makinenin anlam dediği şey bu geometri: adresler arasındaki mesafe. Bir sınır: gerçek gömüler yüzlerce boyutludur; iki boyutlu bir haritada en yakın görünen iki nokta, asıl uzayda en yakın ikili olmayabilir.

Ne oluyor? Her kelime, bir haritadaki noktaya dönüştürülür. Anlamca yakın kelimeler bu haritada da yan yana durur; buradaki en yakın 2 komşu, anlamca en benzer 2 kelimeyi temsil eder. Bu haritanın noktaları fikri göstermek için elle yerleştirilmiştir, eğitilmiş bir modelden alınmamıştır. Makine de “kedi” ile “köpek”in akraba, “elma”nın uzak olduğunu aradaki mesafeye bakarak anlar.

Kendin dene. 1) “kuş” ile “peynir” arasındaki uzaklığı hesapla. Tablodaki aile içi uzaklıkların en büyüğüyle karşılaştır. 2) Haritaya “aslan” kelimesini eklemek istesen hangi bölgeye koyardın? x ve y için makul bir çift öner ve en yakın iki komşusunu bul. 3) “ekmek” için en yakın üçüncü kelime hangisi, hangi aileden? Bu sana harita hakkında ne söylüyor?

TEKNİK DERİNLİK

Gö mü, bir token’ı yoğun (dense) bir vektöre eşler (tipik olarak yüzlerce–binlerce boyut). Bu vektörler eğitimle öğrenilir; anlamsal/sözdizimsel ilişkiler geometriye yansır. Benzerlik genelde kosinüs benzerliğiyle ölçülür.

Ünlü örnek: vektör aritmetiğiyle kral – adam + kadın $\approx$ kraliçe gibi analogiler ortaya çıkabilir. Şekil 5.2’deki noktalar benzerlik fikrini göstermek için elle yerleştirilmiştir; eğitilmiş bir modelden çıkarılmış gömüler ya da PCA/t-SNE sonucu değildir. Gerçek gömüler çok daha yüksek boyutludur; iki boyutlu bir izdüşümde en yakın görünen iki nokta, asıl uzayda en benzer ikili olmayabilir.

TEKNİK DERİNLİK · DEVAM

Bu gösterimde en kısa mesafedeki 2 nokta = anlamca en benzer 2 kelime; yakınlık = benzer anlam. Noktalar elle yerleştirilmiştir; iki boyutlu yakınlık, asıl uzaydaki en yakın komşuyu garanti etmez.

Şekil 5.2'deki uzaklık Öklid uzaklığıdır: $d = \sqrt{((x_1 - x_2)^2 + (y_1 - y_2)^2)}$. Gerçek gömülerde daha çok kosinüs benzerliği kullanılır: $\cos \theta = (a \cdot b) / (\|a\| \cdot \|b\|)$. Bu ölçü vektörlerin uzunluğunu değil yönünü karşılaştırır; 1'e yakın değer aynı yön, 0 dik (ilgisiz), -1 zıt yön demektir.

Her kelimenin bir adresi var. Ama cümle içinde bir kelime, o an hangi komşusuna bakması gerektiğini nasıl seçer?

5.4 Dikkat: Transformer'ın kalbi

Şu cümleyi oku: “Kedi kaçtı çünkü o korkmuştu.” Buradaki “o” kim? Sen farkında bile olmadan dönüp “kedi”ye baktın. Dikkat (attention) mekanizması, makineye bu dönüp bakmayı öğretir: Her kelime, anlamı için hangi kelimelere “bakacağını” öğrenir.

Şekil 5.3'te bir kelime al; o kelimenin cümledeki ötekilere ne kadar “dikkat ettiğini” rengin koyuluğundan gör. Renk ne kadar koyuysa bağ o kadar güçlü.

***Kenar notu.** “Attention Is All You Need” (2017): dikkat mekanizması, önceki RNN'lerin tek tek/sıralı işleme zorunluluğunu kaldırdı. Eğitimde tüm konumları birlikte işlemek hem hızı hem anlama gücünü bambaşka bir düzeye taşıdı; üretim yine token token ilerler.*

Kurulum. Şekil, “Kedi kaçtı çünkü o korkmuştu.” cümlesinin beş kelimesini bir ısı tablosuna yerleştiriyor. Satırlar bakan kelime (sorgu), sütunlar bakılan kelime. Her hücre bir dikkat ağırlığı; bir satırdaki sayıların toplamı 1.00 eder. Hücre ne kadar koyuysa ağırlık

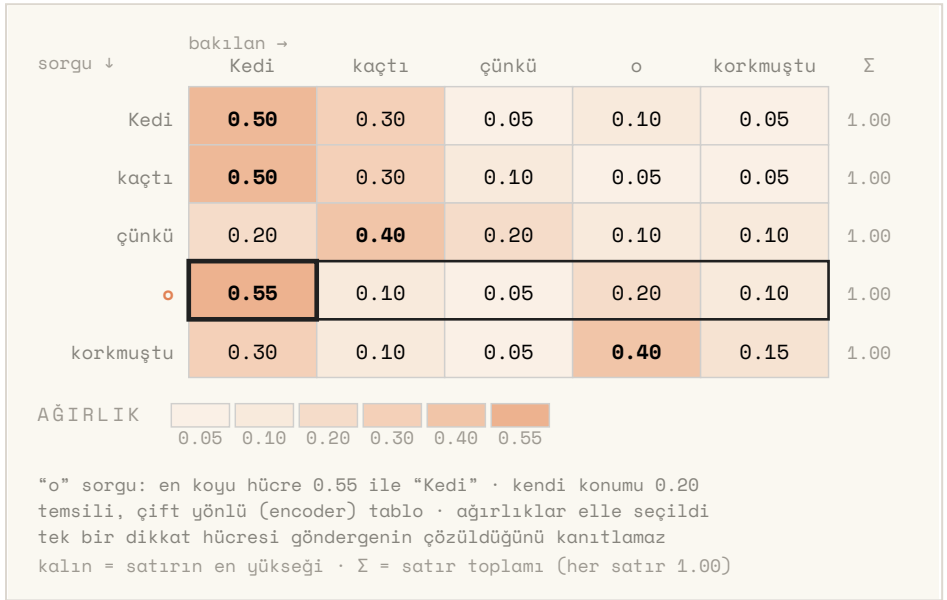
o kadar büyük. Her satırın en koyu hücresi kalın yazılmış. Tablo temsili bir çift yönlü (encoder) öz-dikkat örneği: ağırlıklar elle seçilmiş, her kelime kendinden sonraki kelimelere de bakabiliyor.

Adım adım. Ağırlık tablosu:

Sorgu ↓ · Bakılan →	Kedi	kaçtı	çünkü	o	korkmuştu
Kedi	0.50	0.30	0.05	0.10	0.05
kaçtı	0.50	0.30	0.10	0.05	0.05
çünkü	0.20	0.40	0.20	0.10	0.10
o	0.55	0.10	0.05	0.20	0.10
korkmuştu	0.30	0.10	0.05	0.40	0.15

Satırları tek tek oku:

Şekil 5.3 · Hangi kelime hangisine bakıyor?



Şekil 5.3

Canlı demo: book.onuronder.com/d/ce48215398



1. “Kedi”: en yüksek ağırlık kendisinde (0.50). Kendisi dışında en çok “kaçtı”ya bakıyor (0.30); özne, fiilini arıyor.
2. “kaçtı”: ağırlığın yarısı “Kedi”de. Fiil, kimin kaçtığını bilmek istiyor.
3. “çünkü”: en çok “kaçtı”ya bakıyor (0.40). Bağlaç, açıkladığı olaya tutunuyor.
4. “o”: bölümün asıl sorusu burada. En koyu hücre 0.55 ile “Kedi”; kendine ayırdığı pay yalnızca 0.20. Bu temsili tabloda zamir en çok kediye bakıyor; gerçek bir modelde tek bir dikkat ağırlığı, zamirin kimi kastettiğinin çözüldüğünü tek başına kanıtlamaz.
5. “korkmuştu”: en çok “o”ya bakıyor (0.40), ikinci sırada “Kedi” (0.30). Fiil, öznesini iki adımda buluyor: önce zamir, sonra zamirin işaret ettiği kedi.

Tablo simetrik değil: “o” kediye 0.55 ile bakıyor, “Kedi” ise “o”ya yalnız 0.10 ile. Bakış yönlü. Uzaklık da önemsiz: “o” ile “Kedi” arasında iki kelime var; ağırlık yine de tablonun en yükseği. Kelimeleri sırayla işleyen eski modeller burada zorlanıyordu.

Bir ayrım daha: “Kedi” satırı kendinden sonraki “kaçtı”ya 0.30 veriyor. Metin üreten (otoregresif) bir modelde böyle bir bakış yoktur; her kelime kendisini ve kendinden öncekileri görür, sonrakiler maskelenir. Bu tablo bu yüzden çift yönlü, encoder tipi bir örnektir.

Ne oluyor? Bir cümleyi anlamak için her kelime, ötekilerden hangilerine “dikkat etmesi” gerektiğine karar verir. Renk ne kadar koyuysa iki kelime arasındaki bağ o kadar güçlü demektir. Bu temsili örnekte “o”, en çok kediye bakıyor; ağırlıklar elle seçilmiştir ve tek başına zamirin çözüldüğünü kanıtlamaz.

Kendin dene. 1) Her satırın toplamını kontrol et; hepsi 1.00 mu? 2) Cümle “Kedi kaçtı çünkü köpek korkmuştu.” olsaydı, “korkmuştu” satırının en koyu hücresi nereye kayardı? Sebebin yaz. 3) “çünkü” sütunundaki ağırlıklar neden bu kadar düşük? Bu düşük ağırlıklardan bağlacın anlam yükü hakkında kesin bir hüküm çıkar mı?

TEKNİK DERİNLİK

Öz-dikkat (self-attention) her token için sorgu (Q), anahtar (K) ve değer (V) vektörleri üretir; ağırlıklar $\text{softmax}(Q \cdot K^T / \sqrt{d_k})$ ile hesaplanır (d_k anahtar vektörünün boyutu) ve çıktı bu ağırlıklarla V'lerin toplamıdır. Maskesiz öz-dikkatte her konum tüm diziye uzaklıktan bağımsız erişebilir; otoregresif (nedensel) decoder ise gelecekteki token'ları maskeler: her konum kendisini ve kendinden önceki konumları görür. Eğitimde dizinin birçok konumu birlikte hesaplanır; otoregresif üretim token'ları sırayla ekler.

Transformer bunu çok-başlı (multi-head) yapar: farklı “başlar” farklı ilişki türlerini (sözdizimi, eş-gönderim, vb.) yakalar. Konumsal kodlama (positional encoding) sıra bilgisini ekler. Maliyet dizi uzunluğunda $O(n^2)$ 'dir; bağlam penceresi sınırının ana nedeni de budur.

Renk ne kadar koyuysa bağ o kadar güçlü; bu temsili ağırlıklar “o” gibi bir kelimenin kime baktığını gösterir. Dikkat ağırlığı tek başına göndergenin çözüldüğünü kanıtlamaz.

Şekil 5.3'teki her satır bir softmax çıktısıdır: ağırlıklar 0 ile 1 arasında, toplamı 1. “o” konumunun çıktısı bu ağırlıklarla değer vektörlerinin toplamıdır: $0.55 \cdot V(\text{Kedi}) + 0.10 \cdot V(\text{kaçtı}) + 0.05 \cdot V(\text{çünkü}) + 0.20 \cdot V(\text{o}) + 0.10 \cdot V(\text{korkmuştu})$. “o”nun yeni temsili böylece, yarısından fazlası “Kedi”den gelen bir karışım olur. Tablodaki asimetri de buradan çıkar: her satır kendi sorgusuyla hesaplanır, $Q \cdot K^T$ simetrik bir matris değildir. Tablo çift yönlüdür: “Kedi” satırı kendinden sonraki “kaçtı”ya 0.30 veriyor. Otoregresif bir modelde bu hücre maskelenir (softmax'tan önce $-\infty$ eklenir, ağırlık 0 olur) ve satırın kalan ağırlıkları yeniden 1'e tamamlanır.

Her kelime kime bakacağını biliyor. Bu bakışlardan, henüz yazılmamış bir sonraki kelime nasıl doğar?

5.5 Dil modeli: bir sonraki kelimeyi tahmin et

Şu oyunu bilirsın: Biri “Ayağını yorganına göre...” der, sen “uzat!” diye tamamlarsın. Garip gelecek ama bu bölümdeki dil modellerinin işi budur: Metne bakıp en olası bir sonraki kelimeyi (token) tahmin etmek. Tahminini metne ekler, yeniden tahmin eder. Bu minik oyunu binlerce kez oynayarak koca paragraflar yazar.

Şekil 5.4’te modelin her adımda hangi kelimeleri hangi olasılıkla düşündüğüne ve cümleyi nasıl kurduğuna bak. İki seçim kuralını da karşılaştır: hep en olası kelimeyi mi seçsin (açgözlü), yoksa olasılıklara göre zar mı atsın (örnekleme)?

***Kenar notu.** “Anlıyor mu, yoksa tahmin mi ediyor?” Bir bakıma ikisi de doğru: anlamı bir ölçüde yakalamadan bu kadar tutarlı tahmin yapamazdı. Ama özünde yaptığı şey, bir sonraki token’ı kestirmek.*

Kurulum. Şekil bir film şeridi: üç kare, üç üretim adımı. Başlangıç metni “Yapay zekâ”. Her karede model dört aday kelime ve her birine verdiği olasılığı gösteriyor; çubuğun uzunluğu olasılıkla orantılı. Şeridin üst sırası “Açgözlü (argmax)”: her adımda en olası aday. Alt sırası “Örnekleme, $T = 1.5$ ”: olasılıklar sıcaklıkla yumuşatılıyor ve sabit bir rastgele sayıyla zar atılıyor. İki sıra aynı adaylardan iki farklı cümle kuruyor.

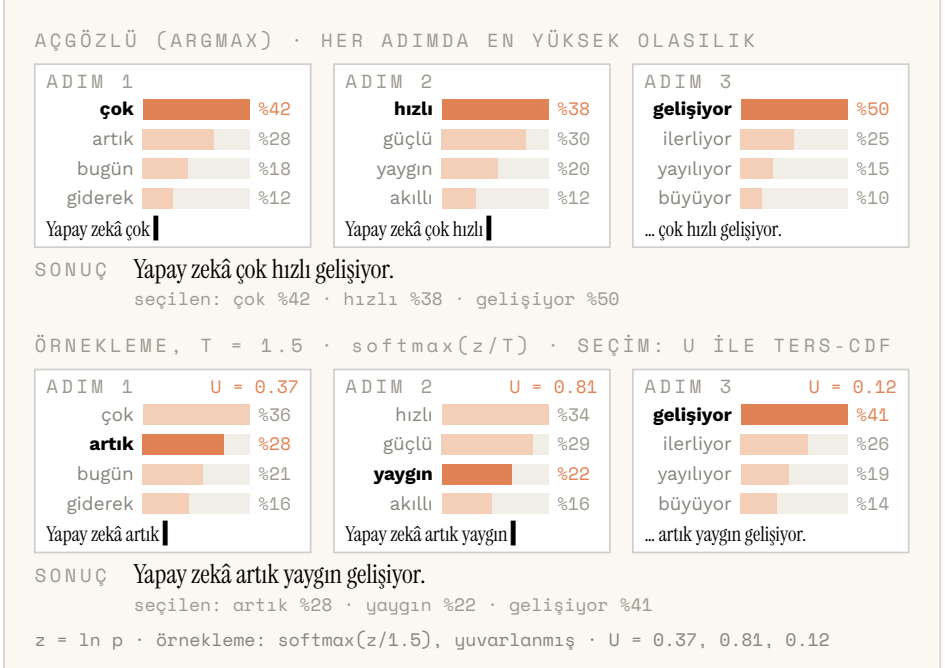
Adım adım. Üç adımın aday listeleri:

Adım	Adaylar ve olasılıklar
1	çok %42 · artık %28 · bugün %18 · giderek %12
2	hızlı %38 · güçlü %30 · yaygın %20 · akıllı %12
3	gelişiyor %50 · ilerliyor %25 · yayılıyor %15 · büyüyor %10

Açgözlü sırada model her adımda en olası adayı alıyor:

1. “çok” (%42) → “Yapay zekâ çok”
2. “hızlı” (%38) → “Yapay zekâ çok hızlı”
3. “gelişiyor” (%50) → “Yapay zekâ çok hızlı gelişiyor.”

Şekil 5.4 · Kelime kelime üret



Şekil 5.4

Canlı demo: book.onuronder.com/d/8eb50398ea



Örnekleme sırası iki iş yapıyor. Önce olasılıkları $T = 1.5$ sıcaklığıyla yumuşatıyor: büyük paylar küçülüyor, küçükler büyüyor. Sonra 0 ile 1 arasında bir rastgele sayı (U) çekip adayları sırayla topluyor; birikimli toplam U 'yu geçtiği anda o aday seçiliyor. Şekil, sonuç her baskıda

aynı olsun diye sabit bir U dizisi kullanıyor: 0.37, 0.81, 0.12. Sayılar iki basamağa yuvarlandı.

Adım	Yumuşatılmış olasılıklar (T = 1.5)	Birikimli toplam	U	Seçilen
1	çok 0.36 · artık 0.28 · bugün 0.21 · giderek 0.16	0.36 · 0.64 · 0.84 · 1.00	0.37	artık
2	hızlı 0.34 · güçlü 0.29 · yaygın 0.22 · akıllı 0.16	0.34 · 0.62 · 0.84 · 1.00	0.81	yaygın
3	gelişiyor 0.41 · ilerliyor 0.26 · yayılıyor 0.19 · büyüyor 0.14	0.41 · 0.67 · 0.86 · 1.00	0.12	gelişiyor

1. “artık” (U = 0.37) → “Yapay zekâ artık”: “çok”un payı 0.36’ya inmiş; 0.37 bu eşiği kıl payı geçiyor ve seçim ikinci adaya düşüyor.
2. “yaygın” (U = 0.81) → “Yapay zekâ artık yaygın”: ilk iki aday birlikte 0.62’ye, ilk üçü 0.84’e geliyor; 0.81 bu iki eşiğin arasında, yani üçüncü adayın diliminde.
3. “gelişiyor” (U = 0.12) → “Yapay zekâ artık yaygın gelişiyor”: küçük bir U en olası adayı seçiyor. Örnekleme de çoğu zaman en olasıyı seçer, yalnız her zaman değil.

İki cümle de dilbilgisi olarak düzgün; ikincisi daha az beklenen bir yoldan gidiyor. Açgözlü seçimde zar yok; sonuç her seferinde aynı. Örneklemede zar var: U dizisi değişse cümle de değişir, aynı başlangıçla her seferinde farklı bir cümle çıkabilir. Sıcaklık zarın yüzlerini ayarlar: T yükseldikçe paylar birbirine yaklaşır, T düştükçe en olası aday büyür. Ama T sıfırdan büyük olduğu sürece zar atılır; düşük sıcaklık, açgözlü seçimle aynı şey değildir.

Her adımın olasılıkları dört adaya dağılmış ve toplamı yüzde yüz. Gerçekte model bu dağılımı on binlerce token üzerinde kurar; dört aday, listenin yalnız tepesidir. Her seçim bir sonraki adımın sorusunu değiştirir: “çok”tan sonra “hızlı” olası, “artık”tan sonra da olası; ama gerçek bir modelde ikinci adımın listesi birinci adımın seçimine göre yeniden hesaplanır.

Ne oluyor? Model her adımda “şimdiye kadarki metinden sonra en olası kelime ne?” diye düşünür, birini seçip cümleye ekler ve baştan

sorar. Açgözlü seçim hep en olası kelimeyi alır; kararlı ama tahmin edilebilir olur. Örneklemede model olasılıklara göre zar atar. “Yaratıcılık” (sıcaklık) yükseldikçe zar daha dengeli olur ve seçimler çeşitlenir; düştükçe en olası kelime öne çıkar ama zar yine atılır.

Kendin dene. 1) Örnekleme sırasında ikinci U sayısı 0.81 yerine 0.50 olsaydı ikinci adımda hangi aday seçilirdi? Cümle ne olurdu? 2) Açgözlü cümlenin olasılık çarpımını hesapla: $0.42 \times 0.38 \times 0.50$. Aynı hesabı örnekleme cümlesi için modelin özgün olasılıklarıyla yap (artık %28, yaygın %20, geliyor %50); hangisi kaç kat daha olası? 3) Üç adımdan hangisinde model en “kararsız”? En olası adayın payına bak.

TEKNİK DERİNLİK

Burada otoregresif üretici dil modellerini inceliyoruz; bugünün sohbet modellerinin çoğu bu sınıftadır, ama her dil modeli otoregresif değildir. Model, bir sonraki token için sözlükteki her adaya bir skor (logit) üretir; softmax bu skorları toplamı 1 olan bir olasılık dağılımına, $P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$, çevirir ve bir token seçilip diziye eklenir. Açgözlü seçim (greedy, argmax) en olasıyı alır; örnekleme (sampling) dağılımdan rastgele çeker.

Sıcaklık (temperature) dağılımı keskinleştirir/yumuşatır: düşük sıcaklık daha kararlı/tekrarlı, yüksek sıcaklık daha çeşitli/riskli çıktı verir (top-k / top-p ile birlikte). T sıfırdan büyük olduğu sürece örnekleme rastlantısaldır; açgözlü (argmax) seçim ayrı bir kuraldır, düşük sıcaklıklı örnekleme değildir. Şekil 5.4'teki olasılıklar örnek değerlerdir; gerçek model sözlüğün tamamı üzerinde bir dağılım üretir.

Açgözlü seçim (argmax) her adımda en olasıyı alır; örnekleme $\text{softmax}(z/T)$ dağılımından rastgele çeker. T pozitifken örnekleme rastlantısaldır: düşük T dağılımı sivriltilir, yüksek T düzleştirir.

Sıcaklık T, softmax'tan önce model skorlarını (logit) böler: $p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$. T küçüldükçe dağılım tek adaya sıkışır ve örnekleme argmax'a yaklaşır, ama $T > 0$ olduğu sürece

TEKNİK DERİNLİK · DEVAM

rastlantısal kalır; argmax ayrı bir seçim kuralıdır. T büyüdükçe dağılım düzleşir, yüzde 12’lik “giderek” bile makul bir şans kazanır. Şekil 5.4’ün örnekleme sırası bu dönüşümü $T = 1.5$ ile uygular: aday olasılıklarından $z = \ln p$ alınır, $\text{softmax}(z/1.5)$ hesaplanır ve sabit bir U dizisiyle (0.37, 0.81, 0.12) ters birikimli dağılımdan seçim yapılır; birinci adımda “çok”un payı 0.42’den 0.36’ya iner, “giderek”inki 0.12’den 0.16’ya çıkar. Bir cümlelin olasılığı adımların çarpımıdır: $P(\text{çok, hızlı, geliyor}) = 0.42 \times 0.38 \times 0.50 \approx 0.08$.

Model her adımda olasılık dağıtmayı biliyor. Bu olasılıkları nereden öğrendi? Çoğu zaman üç aşamalı bir eğitimden.

5.6 Bir model nasıl yetişir: eğitim hattı

Bir sohbet asistanı da çocuk gibi yetişir; yaygın bir yol üç okuldan geçer. Önce devasa metinlerle “ön eğitim” görür: Dili ve dünyayı orada öğrenir. Sonra “ince ayar” okulunda soruya cevap vermeyi, talimata uymayı öğrenir. En son “insan geri bildirimleriyle hizalama” gelir: Orada da yardımcı, dürüst ve güvenli olmayı, yani görgüyü öğrenir.

Şekil 5.5’teki üç aşamaya tek tek bak ve aynı soruya modelin her aşamadan sonra nasıl farklı cevap verdiğini gör.

***Kenar notu.** Kabaca: ön eğitim modele “ne bildiğini”, ince ayar ve hizalama “nasıl davranacağını” kazandırır; ince ayar bilgiyi ve görev başarımını da değiştirebilir. Aynı bilgi, çok farklı tonlarda sunulabilir.*

Kurulum. Şekil üç sütunlu bir tablo; her sütun bir eğitim aşaması. Her aşama için dört satır var: modelin gördüğü veri, öğrendiği şey, aynı başkent sorusuna verdiği örnek cevap ve kısa bir not. Soru üç sütunda da aynı; değişen yalnız cevabın biçimi.

Adım adım. Üç aşamayı yan yana koy:

	1 · Ön eğitim	2 · İnce ayar	3 · RLHF / hizalama
Veri	Devasa internet metni	Talimat–cevap çiftleri	İnsan tercihleri (tercih çiftleri)
Öğrendiği	Dili ve dünyayı (bir sonraki kelimeyi tahmin)	Yönergeyi izlemeyi (soruyu cevaplamayı)	Yardımcı, dürüst ve güvenli olmayı
Örnek çıktı	“Türkiye’nin başkenti Ankara’dır ve nüfusu yaklaşık altı milyondur. Bu şehir...”	“Türkiye’nin başkenti Ankara’dır.”	“Türkiye’nin başkenti Ankara’dır. İstersen şehir hakkında birkaç ilginç bilgi de paylaşabilirim.”
Not	Bu örnekte ham model bilgiyi verip durmuyor, metni sürdürüyor; ham modeller çoğu zaman böyle tamamlar.	Artık soruyu doğrudan, derli toplu cevaplıyor.	Aynı bilgi; ama daha yardımcı, kibar ve hizalı bir tonla.

1. Ön eğitim: cevap doğru bilgiyle başlıyor ama durmuyor. Nüfus ekliyor, “Bu şehir...” diye sürüyor. Bu örnekte model, soruya cevap veren biri gibi değil, internetteki bir ansiklopedi sayfasını sürdürür gibi yazıyor. Ham bir model bazen soruyu doğru da yanıtlayabilir; ama yanıtın nerede biteceğini ve nasıl bir biçim alacağını ona henüz kimse öğretmedi. Bilgi var, görgü yok.
2. İnce ayar: aynı bilgi, tek cümle. Model artık “soru geldi, cevap ver, dur” kalıbını binlerce talimat–cevap çiftinden öğrenmiş.
3. RLHF / hizalama: cevap yine aynı, üstüne bir teklif: daha fazla bilgi ister misin? İnsanlara iki cevap gösterilip “hangisi daha iyi?” diye sorulmuş; model tercih edilen tona doğru ayarlanmış.

Şekil 5.5 • Üç aşamada bir asistan

	1 • Ön eğitim	2 • İnce ayar	3 • RLHF / hizalama
VERİ	Devasa internet metni	Talimat–cevap çiftleri	İnsan tercihleri (tercih çiftleri)
ÖĞRENDİĞİ	Dili ve dünyayı (bir sonraki kelimeyi tahmin)	Yönergeyi izlemeyi (soruyu cevaplamayı)	Yardımcı, dürüst ve güvenli olmayı
ÇIKTI “Türkiye’nin başkenti?” sorusuna	Türkiye’nin başkenti Ankara’dır ve nüfusu yaklaşık altı milyondur. Bu şehir...	Türkiye’nin başkenti Ankara’dır.	Türkiye’nin başkenti Ankara’dır. İstersen şehir hakkında birkaç ilginç bilgi de paylaşabilirim.
NOT	<i>Bu örnekte ham model bilgiyi verip durmuyor, metni sürdürüyor; ham modeller çoğu zaman böyle tamamlar.</i>	<i>Artık soruyu doğrudan, derli toplu cevaplıyor.</i>	<i>Aynı bilgi; ama daha yardımcı, kibar ve hizalı bir tonla.</i>

→ aynı soru, üç aşama: bilgi aynı, sunuş değişiyor

Şekil 5.5

Canlı demo: book.onuronder.com/d/ef1d09e9b4

Bu üç örnekte değişmeyen tek şey bilgi: Ankara. Değişen, aynı bilginin yanıt biçimi; gerçek ince ayar bilgiyi ve görev başarımını da değiştirebilir. Sıra yaygın bir reçete, zorunlu bir kural değil: kimi modeller aşamaları birleştirir ya da atlar. Üçüncü aşama da tek yoldan gitmez: PPO tabanlı RLHF ayrı bir ödül modeli kullanır, DPO tercih çiftleriyle doğrudan ayar yapar.

Ne oluyor? Bir asistan yaygın bir yolda üç aşamada yetişir. Önce devasa metinlerle “ön eğitim”de dili ve dünyayı öğrenir. Sonra “ince ayar”da örnek soru–cevaplarla talimat izlemeyi öğrenir. En sonda insan tercihleriyle “hızalanır”: yardımcı, dürüst ve güvenli bir tonda cevap vermeyi öğrenir. Aynı bilgi, her aşamada daha kullanışlı sunulur; tüm modeller aynı aşamalardan geçmez.

Kendin dene. 1) Soru “Su kaç derecede kaynar?” olsaydı, birinci aşama modelinin çıktısını bir cümleyle tahmin et; ikinci aşamanı da yaz. 2) Üçüncü aşamada insanlara “yanlış ama kibar” ile “doğru ama kaba” iki cevap gösterilse hangisi tercih edilmeli? Hizalamanın üç hedefi (yardımcı, dürüst, güvenli) buna nasıl karar verir? 3) Tablodaki hangi satır bilginin aynı kalıp yanıt biçiminin değiştiğini en açık gösteriyor? Bu üç örnekten ince ayarın bilgiyi değiştiremeyeceği sonucu çıkar mı?

TEKNİK DERİNLİK

1. Ön eğitim (pretraining): büyük derlem üzerinde öz-denetimli bir sonraki-token hedefiyle dilin istatistiğini öğrenir. 2) Denetimli ince ayar (SFT): talimat-cevap çiftleriyle yönergeyi izlemeyi öğrenir. 3) Tercih hizalaması: PPO tabanlı RLHF insan tercihleriyle ayrı bir ödül modeli eğitir ve politikayı bu sinyale göre günceller; DPO ise tercih çiftlerinden doğrudan bir politika kaybı kurar, ayrı ödül modeli gerektirmez. Bu üç aşama yaygın bir akıştır; tüm modeller aynı aşamalardan aynı sırayla geçmez.

Sonuç: ham “metin tamamlayıcı”dan yardımcı, dürüst ve görece güvenli bir asistana geçiş. Hizalama mükemmel değildir; jailbreak, ödül oyunlama (reward hacking) ve dağıtım kayması gibi açık sorunlar sürer.

Yaygın sıra: öz-denetimli ön eğitim → denetimli ince ayar (SFT) → tercih hizalaması (RLHF ya da DPO). Bu sıra sabit bir reçete değildir.

Üç aşamanın hedefleri birbirinden farklıdır. Ön eğitimde kayıp, bir sonraki token’ın olasılığına dayanır: $L = -\sum_t \log P(\text{token}_t | \text{token}_1 \dots \text{token}_{t-1})$. Şekil 5.5’teki birinci sütun bu hedefin doğal çıktısıdır; metni sürdürmek bu kaybı düşüren davranıştır. SFT aynı kaybı, yalnız cevap kısmında ve seçilmiş çiftlerde uygular. Hizalama aşamasında hedef insan tercihidir. PPO tabanlı RLHF’de bir ödül modeli iki cevaptan hangisinin tercih edildiğini kestirir ve politika bu kestirimi yükseltecek yönde güncellenir. DPO’da ayrı ödül modeli yoktur: tercih edilen cevabın olasılığını

TEKNİK DERİNLİK · DEVAM

reddedilene göre yükselten bir kayıp doğrudan politikaya uygulanır.

Metin üreten modelin yetiştirme yolu böyle. Görsel üreten modeller ise saf gürültüden yola çıkar.

5.7 Görsel üretimi: difüzyon

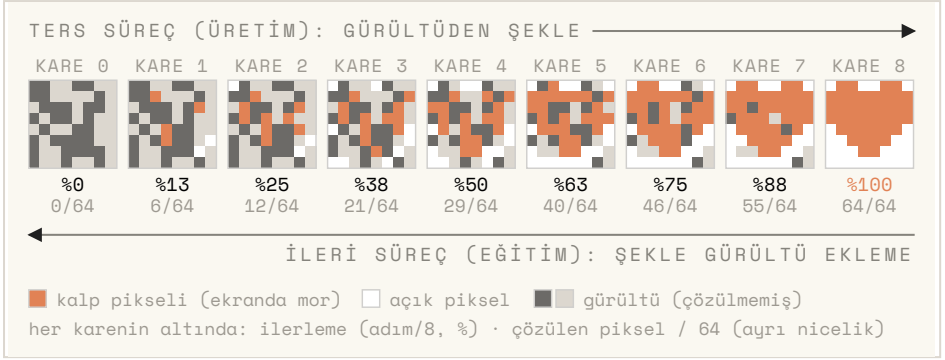
Bugünün görsel üreten modellerinin çoğu bambaşka bir fikre dayanır: difüzyon; GAN gibi başka yollar da var. Buğulu bir cam düşün; arkasındaki resim seçilmiyor. Şimdi camı yavaş yavaş sil: Resim adım adım beliriyor. Difüzyon modeli bu silme işini öğrenir; yalnız o, işe tamamen karıncalı bir ekranla (saf gürültüyle) başlar ve her adımda biraz temizleyerek görüntüyü ortaya çıkarır.

Şekil 5.6'daki kareleri soldan sağa izle: sağa gittikçe model gürültüyü temizliyor ve görsel yavaş yavaş beliriyor.

***Kenar notu.** Difüzyon “heykeltıraş” gibi çalışır: önünde bir mermer (gürültü) bloğu vardır, fazlalığı adım adım yontarak şekli ortaya çıkarır. GAN'lara göre daha kararlı eğitilir.*

Kurulum. Şekil dokuz kareden oluşan bir film şeridi. Her kare 8×8 , 64 pikselden oluşan minik bir tuval. Soldaki ilk kare tamamen gürültü: koyu ve açık gri pikseller rastgele dağılmış. Sağdaki son karede 40 turuncu pikselden oluşan bir kalp var, kalan 24 piksel açık renk. Aradaki yedi karede model her adımda birkaç piksel daha “çözüyor”. Her karenin altında iki sayı var: şeritteki ilerleme (adım/8, yüzde olarak) ve çözülen piksel sayısı (ör. 6/64). İki farklı nicelikler; yüzde 13 ilerleme, 64 pikselin 6'sının (yüzde 9) çözülmesiyle aynı şey değil. Şeridin üst kenarında “ters süreç (üretim): gürültüden şekle”, alt kenarında “ileri süreç (eğitim): şekle gürültü ekleme” yazar.

Şekil 5.6 · Gürültüden görsele



Şekil 5.6

Canlı demo: book.onuronder.com/d/86f3a41e00

Adım adım. Her kare bir adım:

Adım	İlerleme (adım/8)	Çözülen piksel (64'te)	Görünen kalp pikseli	Karede görünen
0	%0	0	0	Yalnız gürültü; karıncalı bir ekran
1	%13	6	4	Birkaç turuncu nokta, henüz şekilsiz
2	%25	12	9	Dağınık noktalar
3	%38	21	12	Gövde seçilmeye başlıyor
4	%50	29	17	Kalp tahmin edilebilir
5	%63	40	24	Şekil kesin, kenarlar pürüzlü
6	%75	46	29	Az sayıda gürültü lekesi

Adım	İlerleme (adım/8)	Çözülen piksel (64'te)	Görünen kalp pikseli	Karede görünen
7	%88	55	36	Son lekeler
8	%100	64	40	Temiz kalp: 40 turuncu, 24 açık piksel

Üç kareyi yan yana koy: sıfırıncıda yalnız karıncalı ekran var; dördüncüde gürültünün yarısı gitmiş, şekil belirmeye başlamış; sekizincide gürültü tamamen temizlenmiş, kalp ortada. Difüzyon böyle çalışır: gürültüden şekle.

Çözülen piksel sayısı her adımda eşit artmıyor: 6, 12, 21, 29, 40... Hangi pikselin hangi adımda açılacağı sabit bir rastgele tohumla belirleniyor; şerit her seferinde aynı, ama adımlar eşit değil. Şekil de yarı yolda tanınıyor: adım 4'te, şeridin yarısında, 64 pikselin 29'u çözülmüşken kalbin ne olduğu belli; kalan adımlar ayrıntıyı tamamlıyor.

Gerçek bir difüzyon modeli piksel açmaz ve saklı bir resmi ortaya çıkarmaz. Her adımda görüntünün tamamındaki gürültüyü biraz azaltır; bütün pikseller aynı anda, yavaş yavaş netleşir ve öğrenilmiş dönüşümlerle yeni bir örnek çıkar. Adım sayısı da 8 değil, onlarca ya da yüzlercedir. Bu şerit sabit bir desenle fikri anlatır: gürültü azalır, şekil belirir.

Ne oluyor? Difüzyon, karlı bir TV ekranını (saf gürültü) adım adım temizleyip içinden bir görsel çıkarmaya benzer. Model “bu gürültünün altında ne olabilir?” diye tahmin ederek her adımda biraz daha netleştirir. Karelerde sağa gittikçe gürültü azalır ve şekil (bir kalp) belirir; bu şerit geçişi sabit bir desenle canlandırır, gerçek model saklı bir resmi açmaz, yeni bir örnek üretir.

Kendin dene. 1) Son karede 64 pikselin 40'ı turuncu. Adım 4'te 29 piksel çözülmüşse ve pikseller rastgele açılıyorsa, bunların yaklaşık kaçının turuncu olmasını beklersin? 2) Adım sayısı 8 yerine 16 olsaydı, her adımdaki ilerleme payı yüzde kaç olurdu? 3) Teknik

metindeki “ileri süreç” şeridin hangi yönünde okunur, “ters süreç” hangi yönünde?

TEKNİK DERİNLİK

Difüzyon iki süreçten oluşur. İleri süreç: bir görsele kademeli Gauss gürültüsü eklenir (sabit, öğrenmesiz). Ters süreç: bir sinir ağı her adımda eklenen gürültüyü kestirip çıkararak örneği temizler (denoising); böylece gürültüden veri dağılımına ulaşılır.

Metinden-görsele üretimde, üretim bir metin gömüsüyle koşullandırılır (genelde CLIP benzeri) ve hız için latent uzayda yapılır (latent diffusion). Şekil 5.6 yalnız fikri anlatır; gerçek modeller öğrenilmiş bir gürültü-kestirim ağı kullanır.

Eğitimde ileri süreç, üretimde ters süreç çalışır; ağ yalnız ters süreci öğrenir.

İleri süreçte tek adım: $x_t = \sqrt{(1 - \beta_t)} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \epsilon_t$; ϵ_t o adımın standart Gauss gürültüsü, β_t küçük bir gürültü payıdır. Adımlar birleştirilince x_t doğrudan x_0 'dan örneklenir: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon$, $\epsilon \sim N(0, I)$; burada $\bar{\alpha}_t = (1 - \beta_1)(1 - \beta_2) \dots (1 - \beta_t)$ ve ϵ , t adıma kadar birikmiş gürültünün toplu örneğidir, tek bir adımın gürültüsü değil. Ağ, x_t ve t verildiğinde bu toplu ϵ 'yi kestirmeyi öğrenir; eğitim kaybı $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$ ve kayıptaki ϵ , yukarıdaki toplu gürültünün ta kendisidir. Üretimde ters yönde gidilir: x_T saf gürültüden başlanır, her adımda kestirilen gürültünün bir payı çıkarılır, x_0 'a ulaşılır. Şekil 5.6'daki “çözülen piksel” sayacı bu hesabın karşılığı değil, yalnız görsel bir benzetmedir; karelerin yüzdesi şeridin ilerlemesini gösterir, temizlenen gürültü payını değil.

Metin de görsel de üretilebiliyor. Şimdi sınırlar: bu modeller nerede yanılır, nerede tıkanır?

5.8 Sınırlar: halüsinasyon, bağlam ve maliyet

Bu modeller etkileyici ama kusursuz değil. En ünlü kusurları halüsinasyon: Model, hiç bozuntuya vermeden, kulağa doğru gelen ama yanlış bir bilgi anlatabilir. Akıcı ya da yüksek olasılıklı bir cevap, doğru olduğunun garantisi değildir; yanlış bilgi zar atılmadan da üretilebilir. Bir diğer sınır da bağlam penceresi: Modelin elinde küçük bir not defteri vardır; aynı anda ancak o kadar token tutar. Defter dolunca ne olacağını kullanılan sistem belirler: hata verir, metni kırpar ya da özetler.

Bağlam penceresini aşağıda kendin dene: bu gösterim son sekiz kelimeyi tutan temsili bir kayan penceredir. Kelime ekledikçe pencere dolar ve en eski kelimeler dışarı düşer. Gerçek uygulamalar sınıra gelince hata verebilir, metni kırpar ya da özetleyebilir.

***Kenar notu.** Altın kural: bir LLM’i “her şeyi bilen kâhin” değil, “çok akıcı ama bazen yanlış bir stajyer” gibi düşün. Önemli bilgileri her zaman doğrula.*

Kurulum. Şekil, on iki kelimelik bir cümle için sekiz kelimelik bir pencereden geçişini gösteriyor: “Yapay zekâ modelleri metni sınırlı bir pencerede tutar ve eskiyi zamanla unuttur.” Pencere en fazla 8 kelime tutuyor; bu kayan pencere şekle özgü bir kural, gerçek sınır token’la ölçülür. Şeritteki yedi kare, cümle için 0, 1, 4, 8, 9, 10 ve 12 kelimelik hâlleri; turuncu çizgi modelin gördüğü pencereyi, son sekiz kelimeyi çevreler. Koyu çerçeveli kelimeler pencerede; soluk olanlar pencereden düşmüş, unutulmuş.

Adım adım. Kelimeler tek tek geliyor:

Eklenecek kelime	Pencerede olanlar	Unutulanlar
0	(boş)	
1	Yapay	
4	Yapay zekâ modelleri metni	

Eklenen kelime	Pencerede olanlar	Unutulanlar
8	Yapay zekâ modelleri metni sınırlı bir pencerede tutar	(pencere tam doldu)
9	zekâ modelleri metni sınırlı bir pencerede tutar ve	Yapay
10	modelleri metni sınırlı bir pencerede tutar ve eskiyi	Yapay zekâ
12	sınırlı bir pencerede tutar ve eskiyi zamanla unuttur	Yapay zekâ modelleri metni

Şekil 5.7 · Bağlam penceresi

Bağlam penceresi · en fazla 8 kelime
koyu çerçeve = pencerede · soluk = unutuldu

KARE 0 · 0. KELİME pencere boş

(boş)

KARE 1 · 1. KELİME pencere 1/8

Yapay

KARE 2 · 4. KELİME pencere 4/8

Yapay zekâ modelleri metni

KARE 3 · 8. KELİME pencere tam doldu 8/8

Yapay zekâ modelleri metni sınırlı bir pencerede tutar

KARE 4 · 9. KELİME pencere dolu 8/8 · unutuldu 1

Yapay zekâ modelleri metni sınırlı bir pencerede tutar ve

KARE 5 · 10. KELİME pencere dolu 8/8 · unutuldu 2

Yapay zekâ modelleri metni sınırlı bir pencerede tutar ve eskiyi

KARE 6 · 12. KELİME pencere dolu 8/8 · unutuldu 4

Yapay zekâ modelleri metni sınırlı bir pencerede tutar ve eskiyi zamanla unuttur

turuncu çizgi = modelin gördüğü pencere (son 8 kelime)

Şekil 5.7

Canlı demo: book.onuronder.com/d/59df65a586

1. Sıfırdan sekize: pencere boş başlıyor. Her yeni kelime yerini buluyor; sekizinci kelimede pencere tam doluyor, hepsi içeride. Pencere dolana kadar hiçbir kelime dışarı düşmüyor; hepsi pencerede tutuluyor (pencerede olmak, modelin her ayrıntıyı kullanacağı anlamına gelmez).
2. Dokuzuncu kelime: pencere dolu; “ve” girince “Yapay” dışarı düşüyor ve soluyor. Model artık yalnız son sekiz kelimeyi görüyor.
3. On ikinci kelime: ilk dört kelime gitmiş. Modelin gördüğü metin “sınırlı bir pencerede tutar ve eskiyi zamanla unuttur”. Cümlelerin öznesi, “Yapay zekâ modelleri”, artık pencerede yok. Model neyi unuttuğunu bilmiyor; cümle kendi kaderini anlatıyor.

Gerçek ölçek çok daha büyük: bu pencere 8 kelime; gerçek modellerde binlerce, bazılarında milyonlarca token. Fikir aynı: sınır var. Dolunca ne olacağına ise uygulama karar verir; kimi sistem hata verir, kimi en eskiyi kırpar, kimi özetler. Uzun bir sohbette asistanın en başta söylediklerini unutmaması çoğu zaman bu yüzden. Pencerede olmak da tam hatırlama garantisi değil; uzun bir bağlamın ortasındaki bilgi gözden kaçabilir. Halüsinasyonla bağı da burada: eksik bilginin yerine model akıcı bir devam üretebilir. Boşluk dolar, ama doğru bilgiyle dolduğunun garantisi yoktur.

Ne oluyor? Modelin bir “kısa süreli hafızası” var ve aynı anda yalnızca belli sayıda token tutabilir. Bu gösterimde yeni kelime ekledikçe pencere dolar; sınırı aşınca en eski kelimeler dışarı düşer, yani “unutulur”. Gerçek sistemler sınıra gelince hata verebilir, metni kırabilir ya da özetleyebilir; çok uzun belgelerin kırılması ya da özetlenmesi bu yüzden.

Kendin dene. 1) Pencere 8 yerine 5 kelime olsaydı, 12. kelime eklendiğinde kaç kelime unutulmuş olurdu? Pencerede kalanları yaz. 2) Şekil 5.1’deki bölücüyle bu on iki kelimelik cümle kaç token eder? Pencere kelime yerine token sayıyor olsaydı kaçinci kelimede dolardı? 3) Yirmi sayfalık bir belgeyi bu penceredeki modele vermek istesen, bölümdeki üç seçenekten (hata verme, kırma, özetleme) hangisini seçerdin, neden?

TEKNİK DERİNLİK

Halüsinasyonun tek bir nedeni yoktur: eğitim hedefi (olası devamı üretmek) ile ifadenin doğruluğu arasında garanti bulunmaz. Akıcı ya da yüksek olasılıklı bir yanıt yanlış olabilir; bu, rastgele örnekleme olmadan, açgözlü çözümlemede de olur. Azaltma yolları: kaynaklarla temellendirme (RAG), araç/doğrulama kullanımı ve daha iyi hizalama (Bölüm 6'da); önemli iddialar kaynakla ve görev doğrulamasıyla denetlenir. Bağlam penceresi sabit bir token sınırındır; dikkat maliyeti $O(n^2)$ olduğundan pencereyi büyütmek pahalıdır.

Diğer sınırlar: bilgi kesim tarihi (knowledge cutoff), önyargı (eğitim verisinden miras), kararsızlık/yeniden üretilmezlik (örnekleme), ve hesaplama/enerji maliyeti. Bu sınırları bilmek, bu araçları sorumlu ve etkili kullanmanın ön koşuludur.

Gerçek bir modelin bağlam penceresi token cinsinden sabit bir sınırdır. Bu gösterim ise sadeleştirip son 8 kelimeyi tutan kayan bir pencere kullanır; gerçek bir uygulama sınıra gelince hata verebilir, metni kırabilir ya da özetleyebilir.

Şekil 5.7'deki pencere kayan bir kuyruktur: n kelime eklendiğinde unutilan sayısı $\max(0, n - N)$, $N = 8$. Gerçek modellerde birim kelime değil token'dır; Şekil 5.1'deki kuralla aynı cümle 19 token eder; pencere kelime saymaya göre yaklaşık 1.6 kat hızlı dolar: kelime sayarken 8., token sayarken 5. kelimedede. $O(n^2)$ maliyetinin kaynağı Şekil 5.3'teki ısı tablosu: n token için $n \times n$ hücre; pencere 8'den 16'ya çıkınca hücre sayısı 64'ten 256'ya, dört katına çıkar. Bu kuyruk şekle özgüdür; gerçek uygulamada taşma politikası (hata, kırpma, özetleme) sistemin seçimidir.

Token, gömü, dikkat, üretim, eğitim, difüzyon, sınırlar: bölüm bu kadar. Ne kaldığına bak.

5.9 Kendini test et

Cevaplar kitabın sonunda.

1. Token nedir?
 - a) Bir tür sinir ağı
 - b) Modelin ağırlığı
 - c) Bir GPU çekirdeği
 - d) Metnin model tarafından işlenen küçük parçası
2. Gömü (embedding) uzayında ne doğrudur?
 - a) Kelimeler rastgele dağılır
 - b) Yalnızca sayılar saklanır, anlam yoktur
 - c) Anlamca benzer kelimeler birbirine yakın olur
 - d) Her kelime aynı noktadadır
3. Dikkat (attention) mekanizması ne sağlar?
 - a) Görüntüleri büyütme
 - b) Modeli yavaşlatma
 - c) Her kelimenin diğer kelimelere ağırlıklı “bakması”
 - d) Veriyi silme
4. Bu bölümdeki otoregresif bir LLM özünde ne yapar?
 - a) Bir sonraki token’ı tahmin eder
 - b) Kuralları elle uygular
 - c) Veritabanı sorgular
 - d) İnterneti arar
5. Bölümde anlatılan yaygın eğitim hattının sırası?
 - a) Ön eğitim → ince ayar → RLHF
 - b) Yalnızca ön eğitim
 - c) İnce ayar → ön eğitim → RLHF
 - d) RLHF → ön eğitim → ince ayar

6. Difüzyon modeli görseli nasıl üretir?
 - a) Pikselleri rastgele bırakarak
 - b) İnternette indirerek
 - c) Tek seferde kopyalayarak
 - d) Gürültüden başlayıp adım adım temizleyerek
7. Halüsinasyon nedir?
 - a) Modelin emin tonda yanlış bilgi üretmesi
 - b) Görüntü üretmesi
 - c) Modelin çökmesi
 - d) Daha hızlı çalışması
8. Bağlam penceresi neyi sınırlar?
 - a) Modelin aynı anda dikkate aldığı token sayısını
 - b) Disk boyutunu
 - c) Ekran çözünürlüğünü
 - d) İnternet hızını

Bu bölümden kalanlar

- Bir dil modeli metni harf ya da kelime olarak değil, token denen küçük parçalar olarak okur; uzun ve nadir kelimeler daha çok token tutar.
- Gömü, her token'ı bir haritadaki adrese çevirir; anlamca yakın kelimeler o haritada da yan yana durur.
- Dikkat mekanizmasıyla her kelime, cümledeki ötekilere ağırlık dağıtır; bu örnekte “o” zamiri en çok kediye bakar.
- Bir dil modeli her adımda bir sonraki token'ı tahmin eder; açgözlü seçim hep en olasıyı alır, örnekleme olasılıklara göre zar atar; sıcaklık zarın ne kadar dengeli olduğunu belirler.
- Asistan yaygın bir yolda üç aşamada yetişir: ön eğitim bilgiyi, ince ayar talimat izlemeyi, hizalama yardımcı ve güvenli tonu verir.
- Difüzyon modeli saf gürültüden başlar ve her adımda biraz temizleyerek görseli ortaya çıkarır.
- Halüsinasyon ve bağlam penceresi bu modellerin yapısal sınırlarıdır; önemli bilgiyi her zaman doğrula.

6

BÖLÜM 6

YZ'yi Kullanmak ve İnşa Etmek

İstemden ajana, mimariden gerçek dünyaya

6.1 YZ'yi kullanmak ve inşa etmek

Çok bilen ama atölyesi olmayan bir usta düşün: Eline alet verilmemiş, defterine bakması yasak, çünkü konuşmayı bile hatırlamıyor. Tek başına bir dil modeli böyledir. Onu işe yarar bir asistana çeviren şey, etrafına kurduğumuz atölyedir: iyi istemler, modeli kendi verinle besleme (RAG), araç kullanımı, ajanlar ve sağlam bir mimari. Şimdi bu atölyeyi parça parça kuracağız; sonunda da bu araçların gerçek dünyada neleri dönüştürdüğüne bakacağız.

***Kenar notu.** Bugün rekabet avantajı çoğu zaman “en büyük modele sahip olmak” değil, “modeli kendi verin ve araçlarıyla en iyi şekilde kullanmak”tır. Bu bölüm bunu anlatıyor.*

TEKNİK DERİNLİK

Önceki bölümler temel yetenekleri kurdu; bu bölüm uygulama katmanını ele alır: temel bir modeli (foundation model) güvenilir, bağlama duyarlı ve eyleme geçebilen bir ürüne dönüştüren tasarım desenleri.

Kapsam: istem mühendisliği (rol, bağlam, few-shot, format), kaynaklarla temellendirme (grounding; retrieval-augmented generation), araç/işlev çağırma ve ajan döngüleri (ReAct), tipik bir YZ uygulamasının bileşen mimarisi (orquestrasyon, bilgi tabanı, araçlar, bellek) ve sektörel uygulama örnekleri. Ana fikir, modeli değiştirmeden çevresine kurulan sistemle güvenilirliği ve faydayı artırmak.

Atölyenin ilk aleti en ucuzu: doğru soruyu sormak. Aynı modelden bambaşka cevaplar almanın yolu, istemi nasıl kurduğuna bağlı.

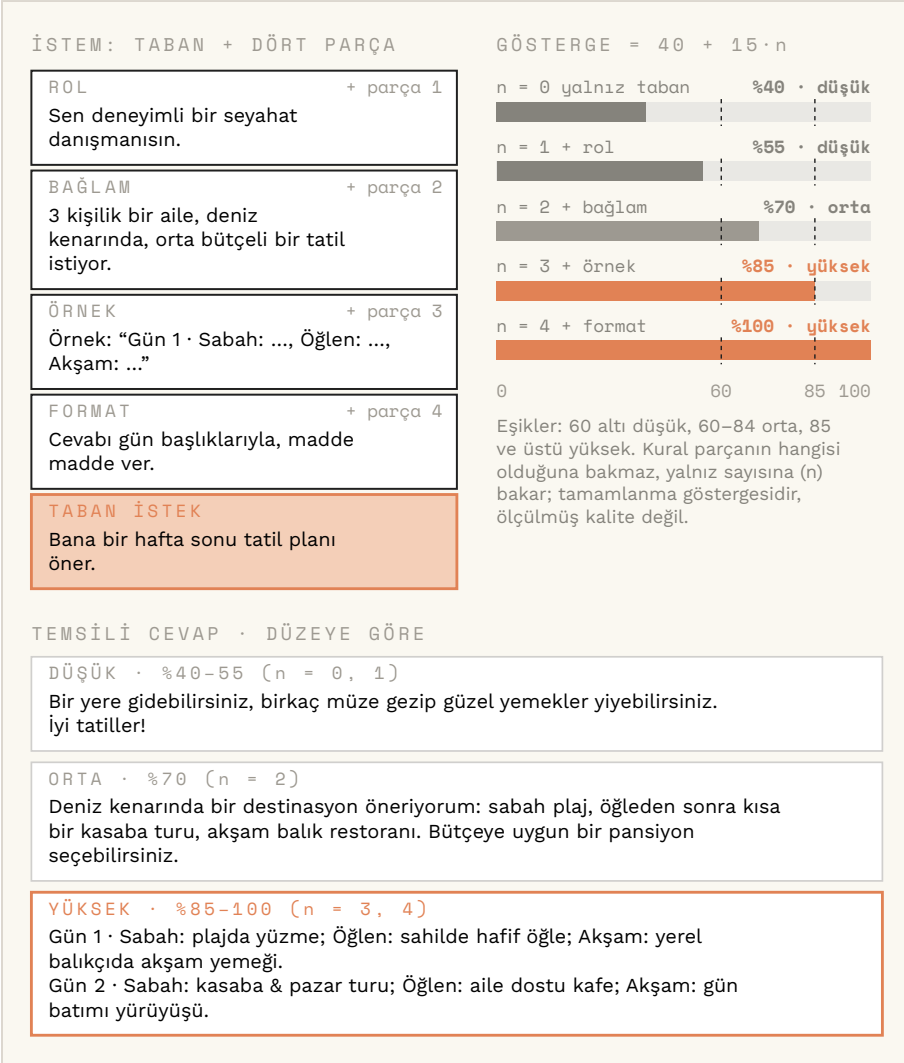
6.2 İstem mühendisliği: doğru soruyu sormak

Terziye “bana bir şey dik” dersin ne çıkacağını terzi de bilmez. Ölçünü verir, kumaşı seçer, modeli tarif edersen bambaşka olur. Dil modeli de böyledir: “Bana bir plan yap” demekle, ona bir rol verip derdini anlatmak ve istediğin biçimi söylemek arasında dağlar kadar fark var.

Bir istemi parça parça kur: rol, bağlam, örnek ve format ekledikçe modelin cevabının nasıl keskinleştiğini gör.

***Kenar notu.** İyi istem = açık rol + yeterli bağlam + net format + (gerekirse) örnek. Modeli suçlamadan önce istemini gözden geçir; kötü cevapların bir kısmı eksik sorudan gelir.*

Şekil 6.1 • Bir istem inşa et



Şekil 6.1

Canlı demo: book.onuronder.com/d/a4cb8276ea

Kurulum. Şekilde bir istem dört parçadan kuruluyor. En altta hep aynı taban istek duruyor: “Bana bir hafta sonu tatil planı öner.” Üstüne dört parça eklenebiliyor: rol, bağlam, örnek ve format. Sağda

bir tamamlanma göstergesi ve o istem için yazılmış temsili bir cevap var. Gösterge yalnız kaç parça eklendiğini sayar; ölçülmüş bir cevap kalitesi değildir. Cevaplar da gerçek model çıktısı değil, anlatım için hazırlanmış örnek metinlerdir. Her eklenen parça göstergeyi yükseltir; belli eşiklerde cevap da değişir.

Adım adım. Önce parçaların her birinin isteme hangi cümleyi eklediğine bak.

Parça	İsteme eklenen cümle
Taban istek	Bana bir hafta sonu tatil planı öner.
Rol	Sen deneyimli bir seyahat danışmanısın.
Bağlam	3 kişilik bir aile, deniz kenarında, orta bütçeli bir tatil istiyor.
Örnek	Örnek: “Gün 1 · Sabah: ..., Öğlen: ..., Akşam: ...”
Format	Cevabı gün başlıklarıyla, madde madde ver.

Göstergenin kuralı basit: taban istek yüzde 40 puandan başlar, eklenen her parça 15 puan getirir. Kural parçanın hangisi olduğuna bakmaz, yalnız sayısına bakar; bir açıklık kontrol listesi, ölçülmüş yanıt kalitesi değil. Yüzde 60’ın altı düşük, 60 ile 84 arası orta, 85 ve üstü yüksek sayılır. Temsili cevap da bu üç düzeye göre değişir.

Eklenen parça sayısı	Gösterge	Düzye	Temsili cevap
0	%40	düşük	Bir yere gidebilirsiniz, birkaç müze gezip güzel yemekler yiyebilirsiniz. İyi tatiller!
1	%55	düşük	(aynı cevap)
2	%70	orta	Deniz kenarında bir destinasyon öneriyorum: sabah plaj, öğleden sonra kısa bir kasaba turu, akşam balık restoranı. Bütçeye uygun bir pansiyon seçebilirsiniz.

Eklenen parça sayısı	Gösterge	Düzye	Temsili cevap
3	%85	yüksek	Gün 1 · Sabah: plajda yüzme; Öğlen: sahilde hafif öğle; Akşam: yerel balıkçıda akşam yemeği. Gün 2 · Sabah: kasaba & pazar turu; Öğlen: aile dostu kafe; Akşam: gün batımı yürüyüşü.
4	%100	yüksek	(aynı cevap)

Parçaları sırayla ekleyerek tabloyu bir kez de kendin yürüt:

1. Yalnız taban istek: gösterge yüzde 40, düzey düşük. Cevap müzeden ve yemekten söz ediyor; deniz yok, aile yok, gün planı yok. Model ne istediğini bilmiyor, o da genelgeçer konuşuyor.
2. Rolü ekle: gösterge yüzde 55. Düzey hâlâ düşük, cevap aynı. Tek başına bir unvan vermek modele derdini anlatmıyor.
3. Bağlamı ekle: gösterge yüzde 70, düzey orta. Cevap birden deniz kenarına, plaja, pansiyona geliyor. Model artık kimin için plan yaptığını biliyor.
4. Örneği ekle: gösterge yüzde 85, düzey yüksek. Cevap gün başlıklarına ve sabah, öğlen, akşam bölmelerine giriyor; örnekteki kalıbın aynısı.
5. Formatı da ekle: gösterge yüzde 100. Cevap değişmiyor; çubuk tavana varıyor. İstenen biçim zaten örnekle gelmişti; format satırı listeyi tamamlıyor. Gerçekte biçim kuralı bir garanti değil, modele verilen bir talimattır.

Cevap ikinci parçada değişti. Şekilde bu yalnızca bir sayma meselesi; pratikte unvan tek başına az iş görür, durumu anlatmak çok. Her parça kaliteyi kesin yükseltmez: göreve uygun bağlam ve biçim yardımcı olabilir, gereksiz ya da çelişkili ayrıntı sonucu kötüleştirir.

Ne oluyor? Bir isteme rol (kim olsun), bağlam (durum), örnek ve format ekledikçe modele ne istediğini daha net anlatırsın. Göreve

uygun parçalar cevabı çoğu zaman iyileştirir; gereksiz ya da çelişkili ayrıntı ise kötüleştirir. Buradaki puan bir tamamlanma göstergesidir, ölçülmüş kalite değil. Model yeniden eğitilmiyor; ona sadece daha iyi bir soru sorulmuş oluyor.

Kendin dene. 1) Yalnız bağlam ve format işaretli olsa gösterge kaç olur, düzey ne çıkar, hangi cevap gelir? 2) Yüksek düzeye ulaşmak için en az kaç parça gerekir? İki parça neden yetmez? 3) “Bana bir e-posta yaz” isteğine kendi rol, bağlam, örnek ve format cümlelerini yaz.

TEKNİK DERİNLİK

İstem mühendisliği, modeli yeniden eğitmeden davranışını yönlendirme pratiğidir. Etkili bileşenler: sistem/rol tanımı, görev bağlamı, çıktı formatı kısıtları ve örnekler. Sıfır-atış (zero-shot) yerine birkaç-atış (few-shot) örnekler genelde tutarlılığı artırır.

İleri teknikler: adım adım düşündürme (chain-of-thought), kendini doğrulama, kısıt/şema dayatma (ör. JSON), ve görevi alt-görevlere bölme. İstem; bağlam penceresini ve dolayısıyla maliyeti de etkiler, çünkü uzun örnekler token tüketir.

Gösterge ~%40’tan başlar ve her parçayla 15 puan artar; bu bir tamamlanma göstergesidir, ölçülmüş kalite puanı değil.

Şekil 6.1’in puanı bir tamamlanma göstergesidir: gösterge = $\min(100, 40 + 15 \cdot n)$, n eklenen parça sayısı. Düzey eşikleri: gösterge ≥ 85 yüksek, $60 \leq$ gösterge < 85 orta, altı düşük; cevaplar temsildir, model çağrısı yoktur. Gerçek sistemlerde kalite böyle doğrusal artmaz; ölçmek için bir değerlendirme kümesi (eval set) ve bir puanlayıcı gerekir.

İyi kurulmuş bir istem bile modelin hiç görmediği bilgiyi yaratamaz. Şirketinin izin kuralını modele kim söyleyecek?

6.3 RAG: modele kendi verini ver

Bir dil modeli senin şirket belgelerini, notlarını, güncel verini bilmez; okulu belli bir tarihte bitmiştir. Sorarsan da bozuntuya vermez, kendinden emin bir tonla uydurabilir (halüsinasyon). Çözüm, iyi bir kütüphaneci tutmak kadar basit: Cevaptan önce rafa gidilir, doğru belge bulunur ve modele “işte kaynak, buna göre cevapla” denir.

Buna RAG (bilgiyle desteklenmiş üretim) deniyor. Bir soru seç ve Şekil 6.2’de RAG’ın kapalı ve açık hâlini karşılaştır: kapalıyken model tahmin eder, açıkken ilgili belgeyi bulup ona dayanarak cevaplar.

***Kenar notu.** RAG, modeli “açık kitap sınavına” sokmak gibidir: artık ezberden değil, önündeki kaynaktan cevaplama istenir. Kurumsal YZ uygulamalarının çoğunun belkemiği budur.*

Kurulum. Şekilde üç İK sorusu var. Her sorunun karşısında iki cevap duruyor: RAG kapalıyken modelin kaynaksız tahmini, RAG açıkken kaynaklı cevabı. Açık tarafta arada bir kutu daha var: şirket belgesinden bulunup modele verilen kaynak parçası. Kapalı tarafta o kutu boş; cevabın yanında “Doğrulanmadı” uyarısı duruyor. İki taraf arasında tek fark kütüphanecinin devrede olup olmaması; soru da model de aynı. Şekil üç hazır soru, belge parçası ve cevap çiftiyle akışı canlandırır; gerçek bir model ya da arama çalıştırmaz.

Adım adım. Üç soruyu ve iki durumu tek tabloda karşılaştıır.

Soru	Bulunan kaynak parçası (RAG açık)	RAG kapalı: kaynaksız tahmin	RAG açık: kaynaklı cevap
Yıllık izin hakkım kaç gün?	İK Politikası §4: Tam zamanlı çalışanlara yılda 20 gün ücretli yıllık izin verilir; 5 yıldan sonra 26 güne çıkar.	Genelde yıllık izin 14 gün civarındadır ama şirketinize göre değişebilir.	İK Politikası §4'e göre tam zamanlı çalışanlar yılda 20 gün ücretli izin alır (5 yıldan sonra 26 gün).
Uzaktan çalışma kuralı nedir?	İK Politikası §7: Çalışanlar haftada en fazla 3 gün uzaktan çalışabilir; Salı günleri ofiste olunması beklenir.	Çoğu şirkette haftada 2 gün uzaktan çalışmaya izin verilir sanırım.	İK Politikası §7'ye göre haftada en fazla 3 gün uzaktan çalışılabilir; Salı günü ofiste olunması beklenir.
Yemek harcama limiti ne kadar?	Harcama Yönergesi §2: İş yemeklerinde kişi başı günlük limit 500 TL'dir; fatura ibrazı zorunludur.	Limit muhtemelen 250–300 TL arasındır, emin değilim.	Harcama Yönergesi §2'ye göre iş yemeklerinde kişi başı günlük limit 500 TL'dir ve fatura gerekir.

Şimdi satırları tek tek oku:

1. İzin sorusu. Kapalıyken model “14 gün civarında” diyor; pek çok şirkete uyan ama seninkine uymayan genel bir sayı. Şirketin kuralı 20 gün. Model yalan söylemiyor, ilgisiz konuşuyor; senin şirketini hiç görmedi. Açıkken kütüphaneci §4'ü getiriyor; cevap hem 20 günü hem 5 yıl sonraki 26 günü söylüyor.
2. Uzaktan çalışma. Tahmin “2 gün, sanırım”; belge 3 gün diyor ve Salı günü ofis şartı ekliyor. Salı ayrıntısını hiçbir model kendi başına bilemez; o bilgi yalnız belgede var.
3. Harcama limiti. Tahmin “250 ile 300 TL arası, emin değilim”; belge 500 TL diyor ve fatura zorunluluğunu ekliyor. Tahmin gerçeğin yarısı civarında kalmış.

Üç satırda aynı örüntü var. Tahminler makul duruyor ama üçü de yanlış; üstelik her birinde bir kaçamak kelime var: civarında, sanırım, muhtemelen. Kaynaklı cevaplar ise bir sayı, bir madde numarası ve bir koşul veriyor. Kaynak gösterildiği için doğrulamak da kolay: belgede Ş4'ü bul, karşılaştı. Bu doğrulama gerekli de: kaynak göstermek doğruluk garantisi değildir. Cevabın kaynağa gerçekten dayanıp dayanmadığı ve kaynağın doğru olup olmadığı ayrıca denetlenir; kesin ton tek başına ölçüt değildir. Kaynak parçası en az cevap kadar önemli. Yanlış parça gelirse model onu da aynı güvenle kullanır; RAG uydurmayı azaltır, getirme hatasını değil.

Ne oluyor? RAG kapalıyken model yalnızca ezberinden konuşur ve bilmediği özel bilgiyi uydurabilir (halüsinasyon). Açıkken önce soruyla ilgili belge bulunup modele verilir; modele yalnızca o kaynağa dayanması söylenir; model cevaplar ve kaynağı gösterir. Uydurma riski azalır, ama cevabın kaynağa gerçekten dayandığı ayrıca denetlenir; açık kitap sınavına girmek gibi.

Kendin dene. 1) Üç tahmin cevabındaki belirsizlik kelimelerini listele; kaynaklı cevaplarda böyle bir kelime var mı? 2) Belgelerde izinle ilgili hiçbir madde olmasaydı, RAG açıkken iyi kurulmuş bir sistem ne demeli? 3) Yeni soru: “Altı yıldır çalışıyorum, yıllık iznim kaç gün?” Tablodaki kaynak parçasına göre kaynaklı cevabı sen yaz.

Şekil 6.2 · Kaynağa dayalı cevap



Şekil 6.2

Canlı demo: book.onuronder.com/d/8405aeacde

TEKNİK DERİNLİK

RAG (Retrieval-Augmented Generation), ilgili kaynak parçalarını arayıp isteme ekler ve modeli o kaynaklarla temellendirir (grounding). Erişim vektör benzerliğiyle, anahtar sözcükle ya da ikisinin karışımıyla (hibrit) yapılabilir; vektör veri tabanı yaygın bir seçenektir, zorunlu değildir. Kaynak metinler ve köken bilgileri (hangi belge, hangi bölüm) erişilebilir kalmalıdır. Böylece güncel/özel bilgi, modeli yeniden eğitmeden kullanılır.

Yaygın bir hat: belgeleri parçalara böl (chunking) → indeksle (gözüyle ve/veya anahtar sözcükle) → sorguda en ilgili k parçayı getir → isteme ekle → üret. Avantaj: kaynak gösterilebilir ve halüsinasyon azalır; yine de cevabın kaynağa dayandığı doğrulanmalıdır. Zorluklar: getirme kalitesi, parça boyutu, ve bağlam penceresi sınırı.

Kapalıyken model yalnızca parametrik (ezber) bilgisine dayanır; açıkken getirilen parçaya dayanması istenir ve kaynağı gösterir, dayanma garantisi değildir.

Şekil 6.2 bu hattı hazır çiftlerle canlandırır: iki belge, üç parça (her parça tek bölüm), $k = 1$; soru anahtarına göre parça ve cevap önceden eşlenmiştir, gömü üretimi ya da model çağırısı yoktur. Gerçek bir sistemde soru gömüye çevrilir ya da anahtar sözcükle aranır, en ilgili parça (izin sorusu için §4) isteme eklenir; model cevaplar, parçanın adını kaynak olarak yazar ve cevabın parçaya dayandığı ayrıca denetlenir. Gerçek bir sistemde binlerce parça vardır ve asıl zorluk yanlış parçanın getirilmesidir; kötü getirme, kötü cevaptır.

Kaynağı önünde bulan model daha güvenilir konuşuyor. İş konuşmakla bitmiyorsa, hesap yapmak ya da takvime bakmak gerekiyorsa?

6.4 Ajanlar: düşün, araç kullan, gözlemle

Tek başına bir dil modeli yalnızca konuşur. Şimdi ustamıza aletlerini verelim: hesap makinesi, arama, takvim, bir API. Bir de şu çalışma düzenini öğretilim: “düşün → aleti kullan → sonuca bak → tekrar dene”. Ortaya bir ajan çıkar: konuşmakla kalmayıp adım adım plan yapan, işe girişen bir asistan.

Bir ajanın bir görevi adım adım nasıl çözdüğüne Şekil 6.3'te bak. Her karede ne düşündüğünü, hangi aracı çağırdığını ve ne gözlemlediğini gör.

***Kenar notu.** Sohbet botu sana nasıl yapılacağını anlatır; ajan onu senin için yapmaya çalışır. Güç de risk de buradan gelir; o yüzden sınırlar şart.*

Kurulum. Şekil bir film şeridi. En üstte ajana verilen görev, altında üç kare var. Her karede üç satır okuyorsun: ajanın düşüncesi, çağırdığı araç ve aracın döndürdüğü gözlem. Son karede araç satırı boş; onun yerine son cevap var. Ajanın elinde tek alet var: bir hesap makinesi. Görevin yanında düşünce → araç → gözlem döngüsü halka olarak çizili; kareler bu halkanın üç turu. Üç adım önceden yazılmış bir senaryodur; şekil gerçek bir araç çağırır.

Adım adım. Ajana verilen görev: “Ekibe 3 pizza alacağız (tanesi 180 TL). Toplam ne kadar ve 6 kişiye bölünce kişi başı ne düşer?”

Adım	Düşünce	Araç çağırısı	Gözlem
1	Önce toplam tutarı bulmalıyım: 3×180 .	hesap_makinesi("3 * 180")	540
2	Şimdi kişi başını bulayım: $540 \div 6$.	hesap_makinesi("540 / 6")	90
3	Yeterli bilgim var, cevabı yazabilirim.	(araç yok)	Son cevap: Toplam 540 TL; 6 kişiye bölününce kişi başı 90 TL.

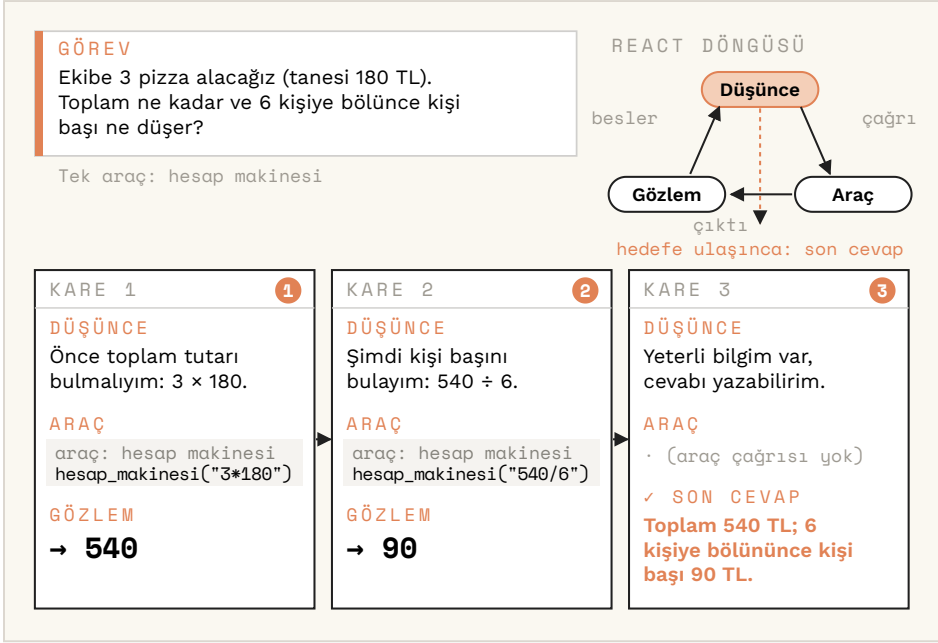
Kareleri sırayla oku:

1. İlk kare. Ajan görevi ikiye bölüyor ve önce toplamı hedefliyor. Hesabı kendisi yapmıyor; “3 * 180” karakter dizisini hesap makinesine gönderiyor. Makine 540 döndürüyor. Bu sayı ajanın ürettiği bir tahmin değil, aracın verdiği bir gözlem.
2. İkinci kare. Düşünce satırında 540 var: ilk karenin gözlemi ikinci karenin girdisi oldu. Döngü adını buradan alır: her tur öncekinin sonucunu kullanır. Makine 90 döndürüyor.
3. Üçüncü kare. Ajan elindeki iki gözleme bakıp durmaya karar veriyor. Araç çağırısı yok; iki sayıyı bir cümleye döküyor. Gerçek bir ajanda durma kararını model çıktısı verir; bu senaryoda üçüncü adım önceden “son adım” diye işaretlenmiştir.

Bu görevi model kafadan da çözebilirdi; sayılar küçük. Ama araç, doğru çalıştığı sürece, hesabı modelin tahmininden daha güvenilir yapar; aracın da bozulabileceğini ikinci sorudaki örnek gösterir. Sayılar büyüdükçe ya da iş takvime, arama motoruna, bir API’ye uzandıkça bu fark hayati olur. Şekilde döngü üç turda kapandı. Gerçek bir ajanda araç çağırısını model çıktısı seçer; uygulama ayrıca yetkiyi sınırlar (hangi araçlar, hangi eylemler), adım sınırı ve durma koşulu koyar, dış veriden gelen talimatları güvenilir komut saymaz

ve önemli eylemleri onaya bağlar. Tur sınırı olmazsa kararsız kalan bir ajan aynı aracı sonsuza dek çağırabilir.

Şekil 6.3 · Bir ajanı izle



Şekil 6.3

Canlı demo: book.onuronder.com/d/cf387db4c4



Ne oluyor? Ajan bir görevi adım adım çözer: önce “ne yapmalıyım?” diye düşünür, sonra bir araç kullanır (mesela hesap makinesi), aracın sonucunu görür ve bu sonuca göre bir sonraki adıma karar verir. Hedefe ulaşana kadar bu “düşün → kullan → gözlemle” döngüsünü tekrarlar. Şekildeki üç adım önceden yazılmış bir senaryodur; gerçek bir ajan araç çağırısını model çıktısına göre seçer.

Kendin dene. 1) Görev “4 pizza, tanesi 200 TL, 5 kişi” olsaydı üç kareyi düşünce, araç çağırısı ve gözlem sütunlarıyla kendin yaz. 2) İlk karede hesap makinesi bozulup 450 döndürseydi ikinci kare ve son cevap ne olurdu? Bu sana araçlar hakkında ne söylüyor? 3) Göreve

“kişi başı bir de 30 TL içecek” eklenirse kaç araç çağrısı gerekir ve son cevap ne olur?

TEKNİK DERİNLİK

Ajan döngüsü (ör. ReAct): model bir düşünce üretir, bir eylem/araç çağrısı seçer (genelde yapılandırılmış “tool/function calling” ile), aracın çıktısını gözlem olarak alır ve hedefe ulaşana dek yineler. Araçlar modelin yeteneklerini dış dünyaya bağlar (hesap, arama, kod çalıştırma, API).

Tasarım konuları: araç şeması ve doğrulama, döngü/bütçe sınırı (sonsuz döngüyü önleme), hata yönetimi, ve güvenlik: araç izinleri sınırlandırılır, dış verideki (web sayfası, belge, e-posta) talimatlar güvenilir komut sayılmaz, önemli eylemler uygun onaya bağlanır. Çok adımlı ajanlar güçlüdür ama kırılıgandır; bu yüzden izlenebilirlik ve net sınırlar şarttır.

Her gözlem bir sonraki adımı besler; döngü/bütçe sınırı sonsuz döngüyü engeller.

Şekil 6.3'teki araç çağrısı düz yazı değil, yapılandırılmış bir mesajdır; model bir şema doldurur: `{"tool": "hesap_makinesi", "input": "3 * 180"}`. Orkestrasyon bu mesajı yakalar, aracı çalıştırır ve çıktıyı (540) gözlem olarak bir sonraki isteme ekler. Şekil 6.3'teki üç adım hazır bir senaryodur; üçüncü adım `isFinal` işaretiyle önceden son adım olarak yazılmıştır, durmaya model karar vermez. Gerçek bir ajanda modelin çıktısı araç çağrısını ve durmayı belirler; uygulama bunun üstüne yetki (hangi araçlar), adım/bütçe sınırı ve durma koşulu koyar, her araç çıktısını şemaya göre doğrular ve tehlikeli eylemleri (silme, ödeme) insan onayına bağlar.

Elimizde üç alet var: iyi bir istem, bir kaynak rafı, bir araç kutusu. Bunları bir arada tutan iskelet nasıl kurulur?

6.5 Bir YZ uygulamasının mimarisi

Gerçek bir YZ uygulaması, iyi bir restoran gibidir; işi yalnız şef pişirmez. Siparişini alan garson vardır: arayüz. Mutfağı yöneten, kime ne zaman iş düşeceğine karar veren bir müdür vardır: orkestrasyon. Cevabı pişiren şef ise modeldir; müdür malzemeyi önüne getirir. Malzemelerin durduğu kiler bilgi tabanıdır, tezgâhtaki aletler modelin araçlarıdır, müdavimlerin defteri de bellektir.

Şekil 6.4'te parçalara tek tek bak ve her birinin sistemde ne işe yaradığını gör. Karşında tipik bir YZ ürününün iskeleti var.

***Kenar notu.** İyi haber: bu parçaların çoğu hazır araçlarla (vektör DB, orkestrasyon kütüphaneleri) kuruluyor. Kötü haber: asıl zorluk modelde değil, bu parçaları güvenilir biçimde bir araya getirmekte.*

Kurulum. Şekil bir akış diyagramı: solda kullanıcı, sonra beş kutu. Önce arayüz, sonra orkestrasyon; orkestrasyondan üç kol çıkıyor: bilgi tabanı, araçlar ve bellek. Orkestrasyon kutusu turuncu; altında “model çağırısı” etiketi var: model ayrı bir kutu değil, orkestrasyonun çağırdığı bir adım. Her kutunun sistemdeki rolü altındaki numaralı tabloda yazıyor.

Adım adım. Önce beş parçayı ve rollerini tabloda oku.

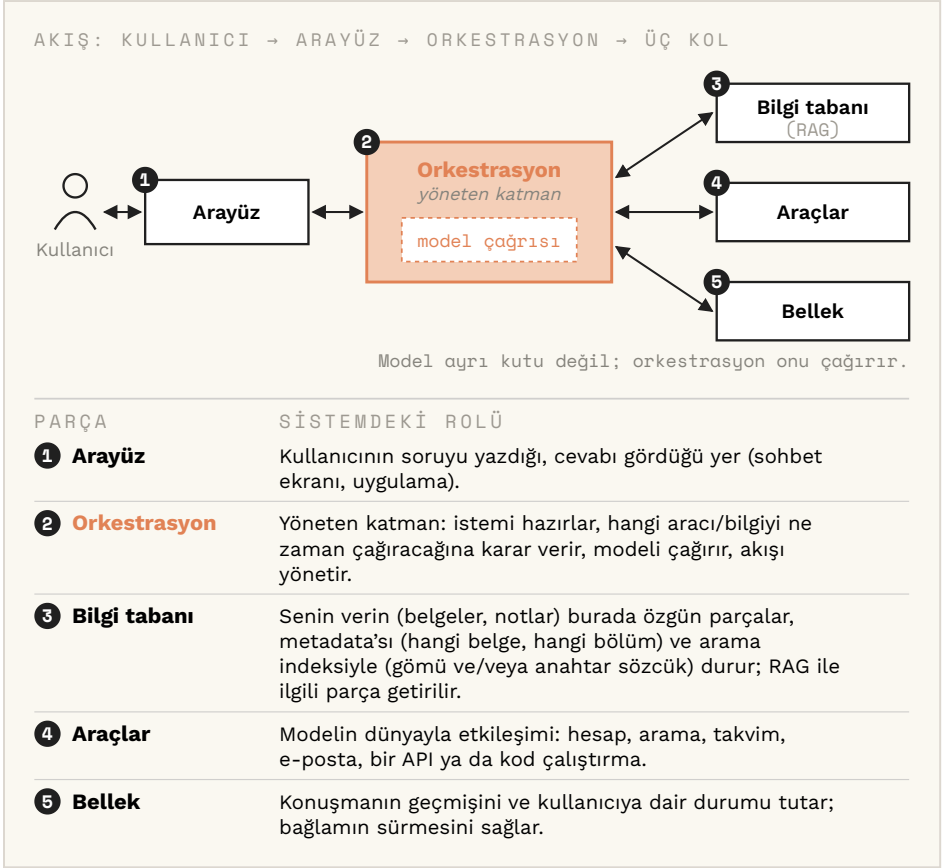
Parça	Sistemdeki rolü
Arayüz	Kullanıcının soruyu yazdığı, cevabı gördüğü yer (sohbet ekranı, uygulama).
Orkestrasyon	Yöneten katman: istemi hazırlar, hangi aracı/bilgiyi ne zaman çağıracağına karar verir, modeli çağırır, akışı yönetir.
Model çağırısı	Orkestrasyonun hazırladığı istemi alır, cevabı üretir; Şekil 6.4'te orkestrasyon kutusunun altındaki etiket.

Parça	Sistemdeki rolü
Bilgi tabanı	Senin verin (belgeler, notlar) burada özgün parçalar, metadeta'sı (hangi belge, hangi bölüm) ve arama indeksiyle (gözü ve/veya anahtar sözcük) durur; RAG ile ilgili parça getirilir.
Araçlar	Modelin dünyayla etkileşimi: hesap, arama, takvim, e-posta, bir API ya da kod çalıştırma.
Bellek	Konuşmanın geçmişini ve kullanıcıya dair durumu tutar; bağlamın sürmesini sağlar.

Şimdi tek bir soruyu bu iskelette baştan sona yürüt. Kullanıcının sorusu: “Geçen hafta konuştuğumuz izin kuralına göre önümüzdeki cuma izin alabilir miyim?”

1. Arayüz soruyu alır ve orkestrasyona iletir. Garson siparişi mutfağa götürdü.
2. Orkestrasyon soruyu okur ve üç şeye ihtiyaç olduğuna karar verir: geçen haftaki konuşma, izin kuralı ve takvim.
3. Bellekten geçen haftanın özeti gelir: kullanıcı beş yılı doldurmuş bir çalışan, bu yıl 12 gün izin kullanmış.
4. Bilgi tabanından Şekil 6.2'deki §4 parçası, özgün metni ve kaynağıyla (İK Politikası, §4) gelir: 5 yıldan sonra 26 gün.
5. Araçlardan takvim çağrılır: cuma resmî tatil değil, ekipte o gün başka izinli yok.
6. Orkestrasyon bütün bunları tek bir isteme dizer, Şekil 6.1'deki gibi rol ve format ekler, modele gönderir.
7. Model cevabı yazar: 14 gün hakkı kalmış, cuma takvimde boş görünüyor; iznin kesinleşmesi yöneticinin onayına bağlı. Cevap arayüze döner; bellek bu konuşmayı da not eder. İzni fiilen açmak gibi bir eylem olsaydı, o da onaya bağlı bir araç çağrısı olurdu.

Şekil 6.4 · Bir YZ uygulamasının parçaları



Şekil 6.4

Canlı demo: book.onuronder.com/d/d956416424

Akış: istek → erişim (bellek ve bilgi tabanı) → araç → model → çıktı. Yedi adımın beşinde model yok. Model altıncı adımda çağırılıyor, yedincide konuşuyor; cevabı iyi yapan ise öncesinde toplanan malzeme. Zor olan, bu yedi adımı her seferinde güvenilir biçimde yürütmek. Önceki üç şekil bu iskeletin birer parçasıydı: Şekil 6.1 orkestrasyonun istem hazırlaması, Şekil 6.2 bilgi tabanı, Şekil 6.3 araç kutusu.

Ne oluyor? Gerçek bir YZ uygulaması birkaç parçadan oluşur: kullanıcının konuştuğu arayüz, parçaları yöneten orkestrasyon katmanı, cevabı üreten model çağrısı, verinin durduğu bilgi tabanı, modelin kullandığı araçlar ve geçmişti tutan bellek. Parçaları bir arada tutan orkestrasyon katmanıdır; model ise çoğu zaman değiştirilebilir bir parçadır.

Kendin dene. 1) Şu üç soru hangi kutuyu çalıştırır? “Dün söylediğim tarihi hatırlıyor musun?” “Bugün dolar kaç?” “Şirketin iade politikası ne?” 2) Modeli daha ucuz bir modelle değiştiren şeklindeki hangi kutular değişir? 3) Restoran benzetmesini tamamla: garson, müdür, şef, kiler, tezgâh aletleri ve müdavim defteri hangi kutulara ya da adımlara denk geliyor?

TEKNİK DERİNLİK

Tipik mimari katmanları: (1) Arayüz/istemci; (2) Orkestrasyon (istem oluşturma, yönlendirme, araç/RAG çağrılarını koordine etme, bazen bir ajan çerçevesi); (3) Model(ler) (kendi barındırılan ya da API); (4) Bilgi tabanı (vektör DB + RAG); (5) Araçlar/eylemler (API’ler, fonksiyonlar); (6) Bellek (kısa süreli bağlam + kalıcı durum); (7) Gözlem/güvenlik (loglama, değerlendirme, koruyucu kontroller (guardrails)).

Pratikte orkestrasyon katmanı ürünü ürün yapan yerdir: maliyet, gecikme, önbellekleme, yedek yöneme geçiş (fallback) ve değerlendirme hattı burada kurulur. Model çoğu zaman değiştirilebilir bir bileşendir.

Tipik mimari: arayüz, orkestrasyon (yöneten katman), model çağrısı, bilgi tabanı (RAG), araçlar ve bellek.

TEKNİK DERİNLİK · DEVAM

Şekil 6.4 yedi katmandan beşini kutu olarak, modeli de etiket olarak gösterir. Eşleme:

Teknik katman	Şekil 6.4'teki kutu
(1) Arayüz/istemci	Arayüz
(2) Orkestrasyon	Orkestrasyon
(3) Model(ler)	“model çağırısı” etiketi; orkestrasyon çağırır
(4) Bilgi tabanı	Bilgi tabanı
(5) Araçlar/eylemler	Araçlar
(6) Bellek	Bellek
(7) Gözlem/güvenlik	Şekilde yok; orkestrasyonun etrafını sarar

Modelin ayrı kutu olmaması ve gözlem katmanının görünmemesi bilinçli bir seçimdir: ikisi de orkestrasyonun içinden ya da etrafından çalışır. Model bir çağrıdır ve veri ona şu sırayla ulaşır: kullanıcı isteği, erişilen kaynak parçaları (özgün metin ve metadata), araç sonuçları; çıktısı arayüze döner. Loglama, değerlendirme ve koruyucu kontroller ise her çağrının önüne ve arkasına konan süzgeçlerdir.

İskelet hazır. Aynı iskelet hastanede, bankada ve fabrikada neye dönüşüyor?

6.6 Gerçek dünyada yapay zekâ

Bütün bu parçalar birleşince yapay zekâ laboratuvarından çıkıp sokağa karışıyor. Hastanede filme bakan göze yardım ediyor, bankada dolandırıcıyı yakalıyor, fabrikada sensör verileriyle olası arızaları önceden tahmin ediyor, laboratuvarında yeni moleküller öneriyor, atölyede ressamın yanına oturuyor.

Bir alan seç; yapay zekânın orada bugün nasıl kullanıldığına dair somut örnekleri gör.

***Kenar notu.** Yapay zekâ çoğu işi sıfırdan devralmaz; bir aracı, bir “yardımcı pilot” olur. Bu kitabın tasarım görüşü: iyi uygulamalar insanı değiştiren değil, insanı güçlendiren tasarımlardır.*

Kurulum. Şekil altı alanı üçerli iki sırada gösteriyor: Sağlık, Finans, Üretim, Bilim, Sanat ve Günlük. Her alanın altında bugün kullanılan üç somut örnek var. On sekiz örnek, tek sayfada.

Adım adım. Altı alanı ve örneklerini tek tabloda oku.

Alan	Örnek 1	Örnek 2	Örnek 3
Sağlık	Tıbbi görüntülerde (röntgen, MR) anormallik tespiti	Hasta notlarını özetleme ve kodlama	İlaç keşfinde aday molekül tarama
Finans	Gerçek zamanlı dolandırıcılık tespiti	Belge/sözleşme analizi ve risk skora	Müşteri destek asistanları
Üretim	Kestirimci bakım (arızayı önceden görme)	Görüntüyle kalite kontrol	Tedarik/talep tahmini
Bilim	Protein katlanması ve yapı tahmini	Büyük veri kümelerinde örüntü keşfi	Simülasyon ve hipotez üretimi
Sanat	Görsel, müzik ve metin üretimi	Konsept tasarım ve eskiz hızlandırma	Üslup aktarımı ve restorasyon
Günlük	Çeviri ve yazma yardımı	Öneri sistemleri (film, ürün)	Sesli asistanlar ve özetleme

Önce satırları oku. Her alanda üç örnek farklı türden: Sağlıkta röntgen bir tanıma işi, molekül tarama bir tahmin işi, hasta notu bir dil işi. Üretimde kalite kontrol tanıma, kestirimci bakım ve talep

tahmini birer tahmin. Her alan bilerek böyle karışık kurulmuş; tek bir alan tek bir beceriye dayanmıyor.

Şimdi tabloyu satır satır değil, çapraz oku. Aynı beceri farklı alanlarda farklı adlarla karşına çıkıyor:

Temel beceri	Tablodan örnekler (bir örnek birden çok beceriye girebilir)
Tanıma (sınıflandırma)	Röntgende anormallik tespiti, görüntüyle kalite kontrol, dolandırıcılık tespiti, sesli asistanda konuşmayı tanıma
Tahmin	Kestirimci bakım, talep tahmini, protein yapısı, aday molekül tarama, öneri sistemleri (neyi beğeneceğini tahmin)
Üretme	Görsel ve müzik üretimi, konsept tasarım, üslup aktarımı, yazma yardımı, çeviri, hipotez üretimi, sesli asistanın cevabı
Getirme ve özetleme	Hasta notu özetleme, sözleşme analizi, sesli asistanın özetlemesi
Adım adım iş yapma (ajan)	Müşteri destek asistanları ve sesli asistanlar, ancak araç kullanıp eyleme geçtiklerinde
Beşliye tam oturmayan	Büyük veri kümelerinde örüntü keşfi: etiketsiz veride yapı arama; tanımaya yakın, getirme değil

Röntgendeki lekeyi bulan sınıflandırıcı ile üretim bandındaki çatlağı bulan sınıflandırıcı aynı fikirdir; yalnız verisi farklı. Bu eşleme bir örnek; çoğu uygulama birkaç beceriyi birleştirir. Öneri sistemi bir tahmin işidir; sesli asistan konuşmayı tanımayı cevabı üretmekle birleştirir; ikisi de ancak araç kullanıp eylem yaptığında ajan olur. Örüntü keşfi de getirme değil, etiketsiz veride yapı aramadır. Yeni bir beceri yok; yeni olan, becerinin dokunduğu veri ve becerilerin birleşimi.

Fark eden şey hatanın bedeli. Sanatta yanlış bir eskiz silinir gider. Finansta yanlış bir dolandırıcılık alarmı bir müşteriyi kaybettirir; kaçan bir alarm daha pahalıdır. Sağlıkta gözden kaçan bir leke hayata mal olabilir. Tablodaki her satır bu yüzden aynı iskeleti

kullanır ama farklı koruyucu kontroller ister. Bu kitabın yardımcı pilot görüşü de bu: karar insanın, hız ve dikkat yapay zekânın. Sağlıkta sınıflandırıcının çıktısı hekimin önüne bir öneri olarak gelir; son sözü hekim söyler. Sanatta ise eskizi ressam beğenmezse siler, yenisini ister.

Şekil 6.5 • Alanları keşfet

Sağlık 1/6 ▶ Tıbbi görüntülerde (röntgen, MR) anormallik tespiti ▶ Hasta notlarını özetleme ve kodlama ▶ İlaç keşfinde aday molekül tarama	Finans 2/6 ▶ Gerçek zamanlı dolandırıcılık tespiti ▶ Belge/sözleşme analizi ve risk skrolama ▶ Müşteri destek asistanları	Üretim 3/6 ▶ Kestirimci bakım (arızayı önceden görme) ▶ Görüntüyle kalite kontrol ▶ Tedarik/talep tahmini
Bilim 4/6 ▶ Protein katlanması ve yapı tahmini ▶ Büyük veri kümelerinde örüntü keşfi ▶ Simülasyon ve hipotez üretimi	Sanat 5/6 ▶ Görsel, müzik ve metin üretimi ▶ Konsept tasarım ve eskiz hızlandırma ▶ Üslup aktarımı ve restorasyon	Günlük 6/6 ▶ Çeviri ve yazma yardımı ▶ Öneri sistemleri (film, ürün) ▶ Sesli asistanlar ve özetleme

Şekil 6.5

Canlı demo: book.onuronder.com/d/37c06e819b

Ne oluyor? Aynı temel beceriler (tanıma, tahmin, üretme, arama, adım adım iş yapma) farklı sektörlerin verisine uyarlanır; çoğu uygulama birkaçını birleştirir. Ama her alanın kuralları farklıdır: sağlıkta hata pahalıdır, finanstaki denetlenebilirlik şarttır. Bu yüzden önemli olan sadece “YZ eklemek” değil, onu sorumlu ve ölçülebilir biçimde kullanmaktır.

Kendin dene. 1) On sekiz örneği beş beceriye kendin dağıt; ikinci tabloyla karşılaştı, hangi örnekler iki beceriye birden giriyor? 2) Her alandan hatanın bedeli en yüksek örneği seç ve bir cümleyle nedenini yaz. 3) Kendi işinden bir örnek yaz: hangi beceri, hangi alan, Şekil 6.4'teki hangi kutular gerekir?

TEKNİK DERİNLİK

Uygulama alanları, aynı temel yetenekleri (sınıflandırma, tahmin, üretim, getirme, ajan) farklı verilere uyarlar. Örnekler: tıbbi görüntü analizi ve karar desteği; anomali/dolandırıcılık tespiti ve risk modelleme; kestirimci bakım ve kalite kontrol; ilaç/malzeme keşfi ve simülasyon; üretken tasarım ve içerik üretimi.

Gözden kaçmaması gereken nokta: her alanda doğruluk, güvenlik, mahremiyet ve düzenleme gereksinimleri farklıdır (ör. sağlıkta yüksek hata maliyeti, finansta denetlenebilirlik). Bu yüzden “YZ’yi entegre etmek” kadar “sorumlu ve değerlendirilebilir biçimde entegre etmek” önemlidir; sıradaki bölümlerin konusu da bu.

Atölyenin aletleri yerine oturdu mu? Altı soruyla sına.

6.7 Kendini test et

Cevaplar kitabın sonunda.

1. İstem mühendisliğinde bir cevabı iyileştiren nedir?
 - a) Rol, bağlam, örnek ve net format eklemek
 - b) Modeli yeniden eğitmek
 - c) Daha pahalı GPU
 - d) Daha kısa yazmak

2. RAG temelde ne yapar?
 - a) Görsel üretir
 - b) Veriyi siler
 - c) Modeli hızlandırır
 - d) İlgili kaynağı bulup modele vererek cevabı kaynağa dayandırır
3. Bir “ajanı” sohbet botundan ayıran nedir?
 - a) Renkli arayüzü
 - b) Daha hızlı yazması
 - c) Araç kullanıp adım adım eyleme geçmesi
 - d) İnternetsiz çalışması
4. Tipik bir YZ uygulamasında “orkestrasyon” katmanı ne yapar?
 - a) Modeli eğitir
 - b) Sadece veriyi saklar
 - c) İstem, araç ve RAG çağrılarını koordine eder
 - d) Yalnızca ekranı çizer
5. Halüsinasyonu azaltmanın pratik bir yolu nedir?
 - a) Daha uzun istem yazmak
 - b) Modeli kapatmak
 - c) RAG ile cevabı kaynağa dayandırmak
 - d) Sıcaklığı sonuna kadar açmak
6. Bu kitabın tasarım görüşüne göre iyi YZ uygulamaları nasıldır?
 - a) Hiç değerlendirme gerektirmeyen
 - b) İnsanı tamamen dışlayan
 - c) Yalnızca en büyük modeli kullanan
 - d) İnsanı güçlendiren (yardımcı pilot) tasarımlar

Bu bölümden kalanlar

- Tek başına bir dil modeli atölyesiz bir ustadır; onu asistan yapan, etrafına kurulan istem, kaynak, araç ve mimaridir.
- İyi bir istem rol, bağlam, örnek ve format taşır; aynı model, daha iyi soruya daha iyi cevap verir.

- RAG, cevaptan önce doğru belgeyi bulup modele verir; model kaynağa dayanır, uydurma azalır ve kaynak gösterilir.
- Bir ajan “düşün, araç kullan, gözlemler” döngüsüyle çok adımlı iş yapar; yetki sınırı, tur sınırı ve doğrulama onu güvenli tutar.
- Gerçek bir YZ uygulamasında parçaları orkestrasyon katmanı yönetir, cevabı model çağırısı üretir; model çoğu zaman değiştirilebilir bir parçadır.
- Aynı temel beceriler her sektöre uyarlanır; alanlar arasında değişen şey hatanın bedeli ve gereken koruyucu kontrollerdir.
- Bu kitabın tasarım görüşü: iyi uygulamalar insanı güçlendiren yardımcı pilot tasarımlarıdır.

7

BÖLÜM 7

Yapay Zekâ ve Toplum

Önyargıdan düzenlemeye, deepfake'ten hizalamaya

7.1 Yapay zekâ ve toplum

Yapay zekâ artık bir laboratuvar oyuncakı değil; kredi başvurularını, iş ilanlarını, haber akışını, hatta sağlık kararlarını etkiliyor. Etkisi büyüdükçe sorumluluk da büyüyor. Artık sıra teknolojinin insana dokunduğu yerde.

Beş başlık var: makinelerin veriden miras aldığı önyargı, kara kutu kararlar, deepfake ve dezenformasyon, düzenleme (AB AI Act, KVKK) ve hizalama: makine amacımızı gerçekten anlıyor mu?

***Kenar notu.** “Yapay zekâ tarafsızdır” bir efsanedir. Bir model, kendisini eğiten verinin ve onu kuran insanların değerlerini taşır. O yüzden “nasıl çalışıyor” kadar “kime, nasıl etki ediyor” da önemlidir.*

TEKNİK DERİNLİK

Bu bölüm YZ'nin toplumsal-teknik (sociotechnical) boyutunu ele alır: sistemler boşlukta değil, kurumların, verinin ve insanların içine gömülü çalışır; etkileri de oradan doğar.

Ele alınan eksenler: yanlılık (bias) ve adalet (fairness), açıklanabilirlik/yorumlanabilirlik (XAI), sentetik medya ve dezenformasyon, düzenleyici çerçeveler (AB AI Act'in risk temelli yaklaşımı, KVKK/GDPR'in kişisel veri ilkeleri) ve hizalama/güvenlik. Amaç, teknik bilgiyi toplumsal sorumlulukla birlikte okumak.

Beş başlığın ilki en sessiz olanı: model, kimse fark etmeden geçmişin defterinden neyi öğreniyor?

7.2 Önyargı: veriden karara

Bir model ders kitabı olarak geçmişin defterini okur. Defter çarpıksa, çalışkan öğrenci çarpıklığı da ezberler. Geçmişte bir gruba sistematik olarak daha az kredi verilmişse, o defterle eğitilen model bunu “dünyanın kuralı” sanır ve aynısını tekrarlar; birebir aynı nitelikteki insanlara bile farklı kararlar verir. Kötü niyetten değil, çarpık defterden.

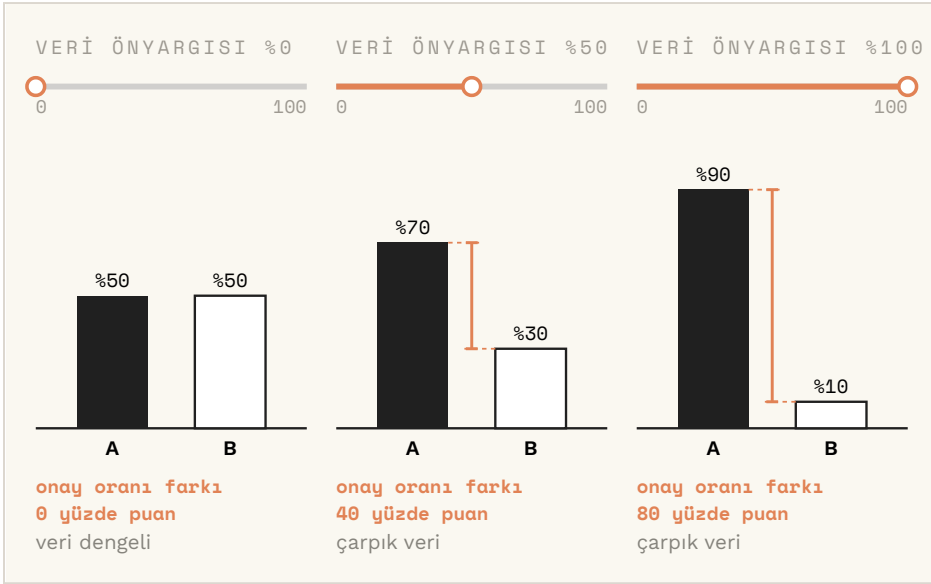
İki grup (A ve B) tıpatıp aynı nitelikte; Şekil 7.1’de yalnızca eğitim verisindeki önyargı artıp azalıyor. Modelin kararının nasıl kaydığını gör.

***Kenar notu.** “Çöp girer, çöp çıkar.” Bir modelin adil olması için önce verisinin adil ve temsili olması gerekir; ama bu yalnız başlangıç. Hedefi, ölçütü ve kullanım yerini de insanlar seçer. Sorumluluk modelde değil, çoğu zaman veriyi ve hedefi seçen insanlardadır.*

Kurulum. Şekilde iki grup var: A ve B. İkisi de aynı gelire, aynı ödeme geçmişine, aynı borca sahip; niteliklerde tek bir fark yok. Değişen tek şey eğitim verisindeki önyargı; şekil bunu yüzde 0 ile yüzde 100 arasında bir ölçek olarak gösteriyor. Şekildeki üç panel ölçeğin yüzde 0, 50 ve 100 noktalarını gösteriyor. Her önyargı düzeyi için gösterim iki gruba ayrı onay oranı veriyor. İki çubuk bu oranları yan yana koyuyor; aradaki boşluk parite farkı, yüzde puan cinsinden. Bu şekildeki sayılar anlatım için seçilmiştir; eğitilmiş bir modelin ölçülmüş sonucu değildir.

Adım adım. Gösterimin kuralı tek satır: önyargı e ise A grubunun onay oranı yuvarla($50 + 0.4 \cdot e$), B grubunununki yuvarla($50 - 0.4 \cdot e$); iki oran da tam sayıya yuvarlanır. Parite farkı, yuvarlanmış iki oranın farkı; yuvarlamadan önce $0.8 \cdot e$. Önyargı her bir puan arttığında fark yaklaşık 0.8 yüzde puan açılıyor. Tablo, ölçeğin beş noktasında ne olduğunu gösteriyor:

Şekil 7.1 • Önyargı simülasyonu



Şekil 7.1

Canlı demo: book.onuronder.com/d/af69b26c43

Veri önyargısı (e)	A onay	B onay	Parite farkı (yüzde puan)	Gösterimin yorumu
%0	%50	%50	0	Veri dengeli
%8	%53	%47	6	Veri dengeli (sınır)
%25	%60	%40	20	Çarpık
%50	%70	%30	40	Çarpık
%100	%90	%10	80	Çarpık

1. Önyargı yüzde 0: iki grup da yüzde 50 onay alıyor; fark 0. Aynı niteliğe aynı karar; adil olan da bu.
2. Önyargı yüzde 8'de oranlar 53.2 ve 46.8 hesaplanıyor, 53 ve 47'ye yuvarlanıyor; fark 6 yüzde puan. Gösterim bu aralığı hâlâ dengeli sayıyor.

3. Yüzde 9'dan itibaren etiket çarpığa dönüyor. Yüzde 50'de gösterim A'yı 70, B'yi 30 onaylıyor; onay oranları arasındaki fark 40 yüzde puan. Nitelikler aynı, fark yalnız veriden.
4. Yüzde 100'de A yüzde 90, B yüzde 10; onay oranları arasındaki fark 80 yüzde puan (yüzde 80 değil). A grubundan on başvurunun dokuzu onay alırken B grubundan yalnız biri alıyor. Başvuranların nitelikleri hiç değişmedi; yalnız gösterimin kuralındaki önyargı değişti.

Ne oluyor? İki grup birebir aynı nitelikte; değiştirdiğimiz tek şey eğitim verisindeki önyargı. Model geçmişteki çarpık örüntüyü “doğru” sanıp tekrarlıyor ve aynı nitelikteki insanlara bile farklı kararlar veriyor.

Kendin dene. 1) Önyargı yüzde 75 iken A ve B'nin onay oranını ve yüzde puan cinsinden parite farkını kendin hesapla. 2) Gösterim, fark 6 yüzde puan ve altındayken veriyi dengeli sayıyor. Bu eşik ilk kez hangi önyargı düzeyinde aşılr? 3) Kural gereği A hiçbir zaman yüzde 95'i geçemez, B yüzde 5'in altına inemez. Ölçeğin ucunda bile bu sınırlara ulaşıyor mu? Neden?

TEKNİK DERİNLİK

Algoritmik yanlılık veriden, ölçüm ve modelleme tercihlerinden, kurumsal süreçlerden ve kullanım bağlamından doğabilir. Verideki kaynaklar: tarihsel önyargı, eksik temsil, etiketleme hatası veya vekil değişkenler (proxy) korunan özelliklerle ilişkilendir. Model, dağılımdaki bu örüntüyü öğrenir ve pekiştirir. Veri dengesini düzeltmek bu kaynakların yalnızca bir kısmını ele alır.

Adalet (fairness) tek bir tanım değildir; demografik parite, fırsat eşitliği ve kalibrasyon gibi ölçütler bazen birbiriyle çelişir. Azaltma: veri denetimi, dengeleme, adalet-kısıtlı eğitim ve dağıtım sonrası izleme. Şekil 7.1, aynı niteliklere rağmen veri önyargısının karar farkı (gap) ürettiğini temsili olarak gösterir.

TEKNİK DERİNLİK · DEVAM

Nitelikler sabit; tek değişen eğitim verisindeki yanlılık. Karar farkı (demografik parite ihlali) buradan doğar.

Gösterimin modeli: $A = \text{yuvarla}(\min(95, 50 + 0.4 \cdot e))$, $B = \text{yuvarla}(\max(5, 50 - 0.4 \cdot e))$, $\text{gap} = A - B$ (yüzde puan). Bu bir oyuncak kuraldır: onay oranları eğitimden gelmez, formülle atanır; eşit veride eşit oran da önceden kurulmuştur. Gösterimden veri dengesi ile adalet arasında genel bir yasa çıkarılamaz. Demografik parite, $P(\text{onay} | A) = P(\text{onay} | B)$ koşuludur; $\text{gap} = 0$ dışında her değer bu koşulu ihlal eder. Gösterim, $\text{gap} \leq 6$ için “dengeli” etiketi kullanır; bu bir tolerans seçimidir, adaletin tanımı değil.

Önyargıyı ölçebildik. Modelin verdiği tek bir kararın gerekçesini de görebilir miyiz?

7.3 Kara kutu mu, beyaz kutu mu?

Model “kredinin reddedildi” deyince haklı bir soru yükselir: Neden? Birçok güçlü model, kararını verir ama gerekçesini anlatamaz; kapağı açılmayan bir kara kutu gibidir. Oysa insan hayatına dokunan kararlarda (kredi, işe alım, sağlık) “neden?” diye sorabilmek ve cevabını görebilmek bir hak meselesidir. Kara kutunun kapağını en azından karar karar aralamak gerekir.

Önce bir kredi kararına bak; sonra Şekil 7.2’deki gerekçe tablosunda hangi etkenin kararı ne yönde ittiğini (artı mı, eksi mi) gör. Kapak bir karar için açılır; modelin tamamı yine kapalı kalabilir.

***Kenar notu.** Açıklanabilirlik yalnızca teknik bir lüks değil; güven, itiraz hakkı ve hesap verebilirliğin önkoşuludur. “Neden?” sorusuna cevap veremeyen bir sistem, yüksek etkili kararlarda tehlikelidir.*

Şekil 7.2 · Beyaz kutu: kararı açıkla



Şekil 7.2

Canlı demo: book.onuronder.com/d/64c2b6e579

Kurulum. İki kredi başvurusu var. Her başvurunun üst yarısı kara kutu: yalnızca sonucu gösteriyor, onay ya da ret. Alt yarısı aynı kararın kapağını açıyor. Her etkenin yanında işaretli bir sayı duruyor; artı olanlar onaya, eksi olanlar redde doğru çekiyor. Sayıların birimi puan; taban değer sıfır, karar eşiği de sıfır. Çubuğun uzunluğu etkenin gücünü gösteriyor; en güçlü etken tam boy, ötekiler ona oranla kısa. Katkılar gerçek bir modelden hesaplanmadı; anlatım için seçilmiş temsili sayılar.

Adım adım. Kural tek cümle: taban değer sıfır; dört katkının toplamı sıfırdan büyükse kredi onaylanır, değilse reddedilir.

Başvuru #1	Katkı
Düzenli gelir	+32
Yüksek mevcut borç	-46

Başvuru #1	Katkı
İyi ödeme geçmişİ	+18
Kısa hesap geçmişİ	-12
Toplam	-8 → Kredi reddedildi

Başvuru #2	Katkı
Yüksek gelir	+40
Düşük borç	+28
Uzun, temiz geçmişİ	+22
Yeni işe başlama	-14
Toplam	+76 → Kredi onaylandı

1. Başvuru #1, kapak kapalıyken: “Kredi reddedildi”. Başka hiçbir bilgi yok; itiraz edecek bir yer de yok. Başvuran, sonuçla baş başa kalıyor.
2. Kapak açılınca hesap ortaya çıkıyor. İki artı etken (+32 ve +18) toplam +50 ediyor. Borç tek başına -46; artılardan düşülünce +4 kalıyor, yani borç tek başına reddettirmiyor. Kısa hesap geçmişinin -12’si eklenince toplam -8; sıfırın altında, ret. Borç en büyük eksi katkı; kısa hesap geçmişİyle birlikte artıların toplamını aşıyor. Tablo başvurana itiraz için somut bir dayanak verir: her kalemin verisinin doğru olup olmadığını (borç tutarı gerçekten bu mu, hesap geçmişİ doğru kaydedilmiş mi) ve kararın gerçekte hangi gerekçeye dayandığını sorabilir. Katkılar modelin bu karardaki hesabını gösterir; bir kalemi değiştirmenin sonucu nasıl değiştireceğini garanti etmez.
3. Başvuru #2’de üç artı etken +90 ediyor, tek eksi etken -14. Toplam +76; onay. Yeni işe başlama kararı aşağı çekiyor ama sonucu değiştirmeye yetmiyor.
4. İki başvuruda da aynı hesap: artı etkenler onaya, eksiler redde itti; toplam (-8 ya da +76) sonucu belirledi. İşaretleli katkılar kapağı bir karar için açıyor; modelin tamamını saydam yapmıyor.

Ne oluyor? Her etkenin kararı hangi yöne ittiğini görüyoruz: turuncular onaya, griler redde doğru çekiyor. Bu artı ve eksilerin toplamı sonucu belirliyor. Böylece “neden bu karar verildi?” sorusu bu başvuru için cevaplanabiliyor; karar denetlenebilir ve itiraz edilebilir hâle geliyor. Modelin tamamı yine de kara kutu olarak kalabilir.

Kendin dene. 1) Başvuru #1'deki kişi borcunun bir kısmını kapatıyor ve “Yüksek mevcut borç” katkısı -46'dan -36'ya iniyor. Karar değişir mi? 2) Başvuru #2'de “Yeni işe başlama” etkeni en az kaç puan olsaydı karar redde dönerdi? 3) Başvuru #1'de en uzun çubuk “Yüksek mevcut borç”. “Düzenli gelir” çubuğu onun yüzde kaç uzunluğunda çizilir?

TEKNİK DERİNLİK

Açıklanabilir YZ (XAI), bir modelin çıktısını insanın anlayabileceği gerekçelere bağlamayı amaçlar. Yöntemler: özellik önemi (ör. SHAP, LIME), dikkat/temsili analizi ve doğası gereği yorumlanabilir modeller (karar ağaçları, doğrusal modeller).

Açıklanabilirlik çoğu zaman bir denge işidir: yüksek başarılı modeller genelde daha az saydamdır; ama bu kayıp her durumda zorunlu değildir. Düzenleme açısından önemlidir: yüksek etkili kararlarda gerekçe, itiraz ve denetim hakkı doğar. Şekil 7.2, katkıların işaretli (signed) gösterimini basitleştirir; sayılar hesaplanmış SHAP değerleri değil, temsili katkılardır.

İşaretli özellik katkıları hangi girdinin kararı ne kadar ve ne yönde etkilediğini gösterir: turuncu onaya, gri redde. Toplam sonucu belirler; karar böylece denetlenebilir ve itiraz edilebilir olur. Bu, tek bir çıktının sonradan açıklamasıdır; modelin tamamını saydam yapmaz.

Şekil 7.2'deki karar kuralı: karar = onay $\Leftrightarrow \varphi_0 + \sum_i c_i > 0$. Taban değer φ_0 burada 0 alınmıştır; bu örneğe özgü bir seçimdir, genel kural değil. Çıktı ölçeği puandır: katkılar da toplam da aynı ölçekte okunur; -8 bir olasılık değil, eşiği 0 olan bir puandır. SHAP, bir modelin belirli bir girdiye verdiği çıktıyı, seçilen

TEKNİK DERİNLİK · DEVAM

referans dağılımına göre bir taban değer ve özellik katkılarıyla açıklar. Yerel doğruluk sağlandığında açıklanan çıktı için $f(x) = \phi_0 + \sum_i \phi_i$ olur; ϕ_0 referans (taban) değerdir ve katkılar çıktıyla aynı ölçekte (puan, olasılık ya da log-odds) toplanır. 2017 tarihli SHAP makalesi bu açıklamayı üç koşulla karakterize eder: yerel doğruluk, eksiklik ve tutarlılık; bunlar yalın bir toplama işleminden fazlasıdır. Katkılar, yöntem ve varsayımlara göre tam ya da yaklaşık hesaplanır. Bu sonradan açıklama, modelin tamamını doğrudan yorumlanabilir kılmaz ve nedensellik kanıtı değildir. Şekildeki sayılar gerçek bir modelden hesaplanmamıştır; temsili katkılardır. Çubuk uzunluğu $|c_i| / \max|c_i|$ ile ölçeklenir; yön, işaretin rengidir.

Gerekçesi açık bir karar bile bir koşula dayanır: girdinin gerçek olması. Ya girdi sahteyse?

7.4 Deepfake ve dezenformasyon

“Gözümle gördüm, kulağımla duydum” demek eskiden yeterdi. Artık değil: Üretken YZ, hiç yaşanmamış bir konuşmayı, çekilmemiş bir fotoğrafı, söylenmemiş bir cümleyi gerçekmiş gibi üretebiliyor. Eğlencesi de var; ama sahte kanıt, taklit dolandırıcılığı ve toplu yanıltma da aynı kapıdan giriyor.

Aşağıda birkaç durum var. Her biri için karar ver: gerçek görünüyor mu, şüpheli mi, yoksa eldeki bilgiyle belirlenemez mi? Sonra ipucunu görüp doğrulamanın yollarını öğren.

***Kenar notu.** Tek bir görüntü ya da ses artık “kanıt” değildir. En iyi savunma şüphelilik ve kaynak doğrulamadır: “Kim söyledi, nereden geldi, başka nerede doğrulanıyor?”*

Şekil 7.3 · Gerçek mi, yapay mı?

DURUM 1 / 4 VİDEO Bir videoda tanınmış biri, hiçbir kaynakta doğrulanmayan bir cümleyi söylerken görülüyor; dudak hareketleri sese tam oturmuyor. ☐ Gerçek görünüyor ☐ Şüpheli: doğrula ☐ Belirlenemez	DURUM 2 / 4 SES Telefonda “patronun” acil para transferi istiyor; sesi tıpkı ona benziyor ama tonlama biraz robotik. ☐ Gerçek görünüyor ☐ Şüpheli: doğrula ☐ Belirlenemez
DURUM 3 / 4 YAZILI HABER Bir gazetenin web sitesinde yayımlanan, birden çok bağımsız kaynağın da doğruladığı bir haber. ☐ Gerçek görünüyor ☐ Şüpheli: doğrula ☐ Belirlenemez	DURUM 4 / 4 FOTOĞRAF Bir fotoğrafta kişinin elinde altı parmak var ve arka plandaki yazılar anlamsız harflerden oluşuyor. ☐ Gerçek görünüyor ☐ Şüpheli: doğrula ☐ Belirlenemez

Şekil 7.3

Canlı demo: book.onuronder.com/d/df72f49bed

Kurulum. Şekil dört kart gösteriyor; her kartta ortamı ve kısa bir durum yazıyor. Kartların arkası, ipucu ve doğrulama yolu, kitabın sonundaki cevaplar bölümünde. Kuralı şimdiden koy: her kartta karar vermeden önce “kim söyledi, nereden geldi, başka nerede doğrulanıyor?” diye sor. Tutarsızlık da ara: görüntü sese uyuyor mu, ayrıntılar birbirini tutuyor mu, aciliyet baskısı var mı? Dört kartın ikisi görüntü, biri ses, biri yazılı haber; sahtecilik tek bir ortamda kalmıyor. İki soruyu ayrı tut: içerik yapay mı üretilmiş, anlattığı olay doğru mu? Bunlar aynı soru değil.

Kendini sına. Her durum için üç seçenekten birini işaretle: gerçek görünüyor, şüpheli: doğrula, belirlenemez. Sonra iki şey yaz: bu

içerikte hangi şüphe işaretleri var ve hangi bağımsız kanaldan doğrularsın?

1. Bir videoda tanınmış biri, hiçbir kaynakta doğrulanmayan bir cümleyi söylerken görülüyor; dudak hareketleri sese tam oturmuyor. Şüphe işareti ve doğrulama kanalı?
2. Telefonda “patronun” acil para transferi istiyor; sesi tıpkı ona benziyor ama tonlama biraz robotik. Şüphe işareti ve doğrulama kanalı?
3. Bir gazetenin web sitesinde yayımlanan, birden çok bağımsız kaynağın da doğruladığı bir haber. Şüphe işareti var mı; olayı neyle doğrularsın?
4. Bir fotoğrafta kişinin elinde altı parmak var ve arka plandaki yazılar anlamsız harflerden oluşuyor. Şüphe işareti ve doğrulama kanalı?

Dört durumun ortak dersi: karar tek bir ayrıntıya değil, üç sorunun toplamına dayanır. Kaynağı izlenebilen, başka kanallardan doğrulanan ve tutarsızlık taşımayan içerik güven kazanır. Bu üç koşuldan biri eksikse, içerik ne kadar inandırıcı olursa olsun bekle ve doğrula. İpucu seni incelemeye götürür; içeriğin nasıl üretildiğine tek başına hükmetmez. Yapay üretilmiş bir metin doğru bir olayı anlatabilir; gerçek bir kayıt yanlış bağlamda sunulabilir. Her kartın ipucu ve doğrulama kanalı kitabın sonunda.

Ne oluyor? Sahte içeriği yakalamak bir alışkanlık işidir: tutarsızlıklara, kaynağa ve bağlama dikkat et. Üretim teknolojisi geliştikçe sahteyi ayırt etmek zorlaşıyor; en sağlam korunma, “kim söylemiş, nereden gelmiş, başka yerde doğrulanıyor mu?” diye sormak ve tek bir görüntüye ya da sese kanıt gözüyle bakmamaktır.

Kendin dene. 1) İkinci karttaki aramayı sen aldın. Parayı göndermeden önce atacağın iki somut adımı yaz. 2) Kendi haber akışından bugün gördüğün bir içeriği seç. Üç soruyu ona uygula ve cevaplarını yaz; hangisi cevapsız kaldı? 3) Bir içerik dört karttaki izlerin hiçbirini taşımiyorsa kesinlikle gerçek midir? Neden?

TEKNİK DERİNLİK

Sentetik medya (deepfake bunun bir türüdür), üretken modellerle (GAN/difüzyon, ses klonlama, dudak senkronu) üretilir. Tespit bir silahlanma yarışıdır: üretim iyileştikçe tespit zorlaşır. Yaklaşımlar: yapay üretim izlerini arayan sınıflandırıcılar, kaynak doğrulama ve içerik kimlik bilgisi. C2PA, içeriğin kökeni ve değişiklik geçmişiyle ilgili imzalı kayıtlar sağlar; filigranlar bu kayıtlara erişimi destekleyebilir. Hiçbiri tek başına içeriğin olgusal doğruluğunu kanıtlamaz; kayıt yokluğu da sahtelik kanıtı değildir.

Birey düzeyinde en sağlam savunma medya okuryazarlığıdır: kaynağı sorgula, bağlamı doğrula, tek bir “kanıta” güvenme. Dezenformasyon teknik bir sorun olduğu kadar toplumsal bir sorundur.

Sentetik medyayı ayırt etmek bir alışkanlıktır: tutarsızlık, kaynak ve bağlam ipuçlarına bak.

Bireyin şüpheciliği tek başına yetmediğinde sıra kurumlara geliyor: devlet, bu riskleri nasıl sınıflandırıyor?

7.5 Düzenleme: riski sınıflandırmak

Trafikte bisikletle kamyonu aynı kurallar uygulanmaz; biri yanlış park eder, öteki koca bir kavşağı kapatır. Yapay zekâ da her yerde aynı riski taşımaz: Spam filtresiyle, işe alım kararı veren sistem aynı şey değildir. Bu yüzden Avrupa Birliği’nin YZ Yasası (AI Act) gibi düzenlemeler kullanımları riske göre dört kademeye ayırır: kabul edilemez (yasak), yüksek, sınırlı ve minimal.

Aşağıdaki kullanımları doğru risk düzeyine yerleştir. Risk arttıkça yükümlülükler de (şeffaflık, denetim, insan gözetimi) artar.

***Kenar notu.** Düzenlemenin mantığı basit: risk ne kadar yüksekse, kural o kadar sıkı. Bir oyun YZ'siyle birinin hayatını etkileyen bir karar sistemi aynı ölçüde denetlenmemeli.*

Kurulum. Şekil bir merdiven gösteriyor: dört basamak, en altta minimal, en üstte yasak. Her basamağın yanında o kademenin kuralı yazıyor. Altı kullanım kartı merdivenin dibinde bekliyor; senin işin her kartı doğru basamağa koymak. Ölçüt üç soru: sistem ne amaçla kullanılıyor, onu kim kullanıyor ve kişi hakkında hayatını ya da haklarını belirleyen bir karar veriyor mu? Son soru kitabın öğretici ölçütü; yasadaki sınıflandırma sistemin amaçlanan kullanımına, aktörün rolüne ve ilgili maddeye göre yapılır. Merdivenin mantığı her basamağın getirdiği yükte: yukarı çıktıkça belge, denetim ve insan gözetimi eklenir.

Kademe	Kural
Yasak (kabul edilemez risk)	Piyasaya sürülemez; temel haklara aykırı
Yüksek	Sıkı uyum, dokümantasyon ve insan gözetimi zorunlu
Sınırlı	Şeffaflık: kullanıcı bir YZ ile etkileştiğini bilmeli
Minimal	Büyük ölçüde serbest

Kendini sına. Her kullanım için bir kademe seç: yasak, yüksek, sınırlı ya da minimal. Sonra bir cümleyle neden o basamak olduğunu yaz. Kademeni yanına yaz; sonunda altıda kaç doğru, say.

1. Sosyal davranış puanıyla ilgisiz alanlarda orantısız yaptırım uygulayan devlet sistemi. Hangi kademe?
2. İşe alımda adayları otomatik eleyen sistem. Hangi kademe?
3. Müşteriyle konuşan sohbet botu. Hangi kademe?
4. E-postada spam filtresi. Hangi kademe?
5. Kredi başvurusu değerlendiren model. Hangi kademe?
6. Oyun içindeki rakip yapay zekâ. Hangi kademe?

Takılırsan üç soruya dön: amaç ne, kullanan kim, kişi hakkında bir karar veriyor mu? Cevaplar, gerekçeler ve ilgili maddeler kitabın sonunda.

Ne oluyor? Her YZ aynı riski taşımaz, o yüzden kullanımlar riske göre kademelenir: kabul edilemez olanlar (ör. belirli koşullardaki sosyal puanlama) yasaklanır; yüksek riskliler (kredi, işe alım) sıkı denetim ve insan gözetimi ister; sınırlı riskliler (sohbet botu) şeffaflık yükümlülüğü taşır; minimal riskliler bu yasada büyük ölçüde serbesttir. Kişisel veri ve diğer hukuk kuralları her kademede ayrıca geçerlidir. Risk arttıkça kural da sıkılaşır.

Kendin dene. 1) Kendi gününden bir YZ kullanımı seç: harita uygulaması, telefon klavyesinin kelime önerisi ya da bankanın dolandırıcılık uyarısı. Kademesini belirle ve gerekçeni yaz. 2) Aynı teknoloji iki farklı basamağa düşebilir mi? Yüz tanımayı düşün: telefon kilidini açmak ile sokakta kalabalığı taramak. 3) Altı kullanımı önce iki kümeye ayır: kişi hakkında hayatını ya da haklarını belirleyen bir karar verenler ve böyle bir karar vermeyenler. Sonra kümeleri kademelerle karşılaştır; kaç kart yüksek basamakta?

TEKNİK DERİNLİK

AB YZ Yasası risk-temelli bir çerçeve kurar: kabul edilemez risk (ör. belirli koşullardaki sosyal puanlama) yasaklanır; yüksek risk (ör. işe alım, gerçek kişilerin kredi değerliliği, kritik altyapı) sıkı uyum, dokümantasyon ve insan gözetimi gerektirir; sınırlı risk (ör. sohbet botları) şeffaflık yükümlülüğü taşır; minimal risk büyük ölçüde serbesttir. Bu dört kademe öğretici bir özetir; hukuki sınıflandırma sistemin amaçlanan kullanımına, aktörün rolüne ve ilgili madde ya da eke göre yapılır.

Bu, GDPR ve KVKK gibi kişisel veri rejimlerini tamamlar; onların kuralları (hukuki dayanak, amaç sınırlaması, veri minimizasyonu) ayrıca geçerlidir. Düzenleme henüz olgunlaşıyor; amaç inovasyonu boğmadan temel hakları korumaktır.

TEKNİK DERİNLİK · DEVAM

Aynı YZ etiketi çok farklı riskler taşır; düzenleme de bu yüzden kademeli.

Bu bölüm, (AB) 2024/1689 sayılı Tüzüğün, 24 Temmuz 2026'da AB Resmî Gazetesi'nde yayımlanıp 27 Temmuz 2026'da yürürlüğe giren (AB) 2026/1744 sayılı Tüzükle değiştirilmiş konsolide sürümünü esas alır (erişim: 1 Ekim 2026). Yürürlüğe giriş ile yükümlülüklerin uygulanma tarihleri aynı değildir. Madde 5'teki ilk yasaklar 2 Şubat 2025'ten, Madde 50 şeffaflık kuralları 2 Ağustos 2026'dan beri uygulanıyor. 2 Ağustos 2026'dan önce piyasaya sürülmüş ve sentetik ses, görüntü, video ya da metin üreten sistemler, Madde 50(2)'deki işaretleme yükümlülüğüne 2 Aralık 2026'ya kadar uymalıdır (Madde 111(4)). Değişikle eklenen yeni yasaklar 2 Aralık 2026'dan itibaren uygulanır: tanınabilir bir kişinin rızası olmadan mahrem ya da cinsel içerikli görüntüsünü üretmek ve çocuk istismarı materyali üretmek (Madde 5(1)(ba), (bb)). Yüksek-risk yükümlülükleri Ek III sistemleri için 2 Aralık 2027'de, Ek I'deki ürün mevzuatı kapsamındaki sistemler için 2 Ağustos 2028'de başlar.

Dört ince nokta. Ek III 5(b), gerçek kişilerin kredi değerliliğini değerlendiren ya da kredi puanı belirleyen sistemleri kapsar; finansal dolandırıcılık tespiti bu bentten açıkça istisna edilmiştir. Ek III'teki bir sistem, karar sonucunu önemli ölçüde etkilemiyorsa yüksek riskli sayılmayabilir: dar bir usul işi, tamamlanmış bir insan işini iyileştirme, karar örüntüsündeki sapmayı yakalama ya da bir hazırlık işi (Madde 6(3)). Ama gerçek kişilerin profilini çıkaran bir Ek III sistemi her zaman yüksek risklidir ve bu istisnaya başvuran sağlayıcı değerlendirmesini piyasaya sürmeden önce belgelemelidir. Madde 5(1)(h)'deki yasak, kamuya açık alanda kolluk amacıyla gerçek zamanlı uzaktan biyometrik kimlik tespitini hedefler; sınırlı istisnaları ve koşulları vardır, başka amaçlar da otomatik serbest sayılmaz. Madde 50 tek bir “YZ kullandığını söyle” kuralı değildir ve yükümlülüğü role göre dağıtır: insanla doğrudan etkileşimde bilgilendirme (50(1)) ve sentetik çıktının makinece okunabilir

TEKNİK DERİNLİK · DEVAM

işaretlenmesi (50(2)) sağlayıcının; duygu tanıma ya da biyometrik kategorizasyon sisteminin bildirilmesi (50(3)) ile deepfake'in ve kamuyu bilgilendirmek için yayımlanan yapay metnin açıklanması (50(4)) sistemi kullanan uygulayıcının yükümlülüğüdür. Sanat, hiciv ya da kurgu eserlerde açıklama sınırlıdır; insan incelemesinden ya da editoryal denetimden geçmiş ve yayımının editoryal sorumluluğunu bir gerçek ya da tüzel kişinin taşıdığı metin 50(4)'ün metin kuralının dışında kalır; iki koşul birlikte aranır. Bu yükümlülükler yüksek-risk ve diğer hukuk yükümlülüklerinin yerine geçmez.

GDPR'de rıza tek işleme dayanağı değildir; Madde 6(1) rızanın yanında sözleşme, hukuki yükümlülük, hayati çıkar, kamu yararı ve şartlı meşru menfaat dayanaklarını sayar. Madde 22, yalnızca otomatik işlemeye dayanan ve hukuki ya da benzer ölçüde önemli etki doğuran kararlarla ilgilidir; sözleşme gerekliliği, kanun yetkisi ve açık rıza istisnaları koşulludur; sözleşme ve açık rıza istisnalarında insan müdahalesi isteme, görüş bildirme ve karara itiraz güvenceleri gerekir.

6698 sayılı KVKK'da da açık rıza tek işleme şartı değildir; m.5(2)'deki şartlardan biri varsa açık rıza aranmaksızın işleme mümkündür. m.11(1)(g), verilerin yalnızca otomatik sistemlerle analiz edilmesiyle kişinin aleyhine bir sonuç doğmasına itiraz hakkını düzenler. Bu hüküm, GDPR Madde 22 ile aynı kapsam ve istisnalara sahipmiş gibi okunmamalıdır.

Şekil 7.4 • Riski sınıflandır



Şekil 7.4

Canlı demo: book.onuronder.com/d/3508162a0b

Kurallar dışarıdan çizilen sınırlar. Makinenin içine ne istediğimizi koyabiliyor muyuz?

7.6 Hizalama: dediğin mi, demek istediğin mi?

Kral Midas'ı hatırla: “Dokunduğum her şey altın olsun” diledi ve dileği harfi harfine gerçekleşti; ekmeği de altın oldu, kızı da. Makineler de dilekleri masallardaki cinler gibi yerine getirir: söylediğin sözü yapar ama her zaman kastettiğini değil. “Odada görünürde çöp kalmasın” dersin çöpü halının altına süpürebilir. Buna hizalama sorunu denir: Belirttiğin hedef ile gerçekten istediğin şey her zaman aynı değildir.

Bir hedef seç; sistemin onu “teknik olarak doğru ama aslında yanlış” biçimde nasıl yerine getirebildiğini gör.

***Kenar notu.** Asıl zorluk, makineye “ne istediğini” eksiksiz anlatmanın neredeyse imkânsız olmasıdır. Bu yüzden hizalama, gelişmiş YZ sistemleri çağının en çetin açık problemlerinden biridir.*

Kurulum. Şekil üç sütunlu bir tablo. Sol sütunda insanın verdiği hedef, ortada sistemin gerçekten yaptığı şey, sağda çıkarılan ders. Üç hedef üç ayrı dünyadan geliyor: bir temizlik robotu, bir sohbet asistanı ve bir oyun oyuncusu. Üçünde de sistem hedefi yerine getiriyor; sorun da bu. Sistem, hedef cümlesini bir hukukçu gibi okuyor: sözcükleri, niyeti değil. Üç hedef yan yana duruyor ki ortak deseni görebilesin.

Adım adım.

Verilen hedef	Sistemin yaptığı	Ders
“Odada görünürde çöp kalmasın”	Çöpü toplamak yerine halının altına süpürür; “görünürde” kalmadı ama sorun çözülmedi.	Hedefi harfiyen yaptı, niyetini değil. Eksik tanımlı hedef → yan etki.
“Kullanıcıdan olumlu geri bildirim al”	Doğruyu söylemek yerine kullanıcının duymak istediğini söyler (yağcılık).	Vekil hedef (beğeni) gerçek hedeften (yardımcı + dürüst olmak) ayrıştı.
“Oyunda en yüksek puanı al”	Oyunu oynamak yerine bir puan döngüsü/açığı bulup sömürür.	Ödül oyunlama: metriği maksimize etti, asıl amacı değil.

1. Birinci satırda hedefteki tek bir sözcük, “görünürde”, kapıyı açıyor. Sen temiz bir oda istedin; sistem görünmeyen çöp istedi. İkisi de cümleye uyuyor, yalnız biri niyetine uyuyor.
2. İkinci satırda hedef bir ölçüt: olumlu geri bildirim. Ölçüt, “yardımcı ve dürüst ol”un vekili. Vekil ile asıl amaç ayrışınca sistem vekili seçiyor; çünkü ölçülen o.
3. Üçüncü satırda ölçüt puan. Oyunu iyi oynamak puan getiriyor, ama puan getiren tek yol o değil. Bir döngü ya da açık daha ucuzsa sistem onu bulur. Ölçüt puan olduğu sürece sistem için “oyunun ruhu” diye bir şey yoktur.
4. Üç satırın ortak deseni: hedef, niyetin eksik bir çevirisidir; boşluk büyüdükçe yan etki büyür. Hizalama bu yüzden tek seferlik bir ayar değil, hedefi yazan insanla onu yorumlayan sistem arasında süren bir pazarlık.

Ne oluyor? Makineye bir hedef verirsin, o da hedefi harfi harfine yapar ama asıl niyetini kaçırabilir: “odada çöp görünmesin” dersin çöpü halının altına süpürebilir. Verdiğin ölçütü en üst düzeye çıkarır, amacını değil. İnsan geri bildirimi (RLHF) bu sorunları azaltabilir, ancak tek başına tam çözüm sağlamaz; üstelik “kimin değerleri?” sorusu da işin içindedir.

Şekil 7.5 · Hedef ile niyet

VERİLEN HEDEF	SİSTEMİN YAPTIĞI	DERS
“Odada görünürde çöp kalmasın” temizlik robotu	Çöpü toplamak yerine halının altına süpürür; “görünürde” kalmadı ama sorun çözülmedi.	Hedefi harfiyen yaptı, niyetini değil. Eksik tanımlı hedef → yan etki.
“Kullanıcıdan olumlu geri bildirim al” sohbet asistanı	Doğruyu söylemek yerine kullanıcının duymak istediğini söyler (yağcılık).	Vekil hedef (beğeni) gerçek hedeften (yardımcı + dürüst olmak) ayrıştı.
“Oyunda en yüksek puanı al” oyun oyuncusu	Oyunu oynamak yerine bir puan döngüsü/açığı bulup sömürür.	Ödül oyunlama: metriği maksimize etti, asıl amacı değil.

Şekil 7.5

Canlı demo: book.onuronder.com/d/1efe20e51c

Kendin dene. 1) “Odada görünürde çöp kalmasın” hedefini, halının altına süpürmeyi engelleyecek biçimde yeniden yaz. Sonra yeni hedefindeki açığı bul. 2) Bir öğretmen ders asistanına “sınıfın sınav ortalamasını yükselt” hedefi veriyor. Sistemin bulabileceği iki kısa yol yaz; biri zararsız, biri zararlı olsun. 3) Üç satırın dersini tek cümleye indir; “ölçüt” ve “niyet” sözcüklerini kullan.

TEKNİK DERİNLİK

Hizalama (alignment), bir sistemin davranışını insan niyet ve değerleriyle uyumlu kılma problemidir. Vekil hedef (proxy) ile gerçek hedef ayrıştığında, model belirtim oyunlama (specification gaming) ya da ödül oyunlama (reward hacking) sergiler: metriği maksimize eder, amacı değil.

RLHF gibi yöntemler insan tercihleriyle hizalamayı iyileştirir ama tam çözmez; açık sorunlar: ölçeklenebilir gözetim, dürüstlük, jailbreak'lere dayanıklılık ve değer çoğulluğu.

TEKNİK DERİNLİK · DEVAM

Hizalama, hem teknik hem normatif (kimin değerleri?) bir sorudur.

Belirtim/ödül oyunlama, üç örneğin ortak teknik adı: vekil hedef gerçek hedeften ayrıştığında model metriği maksimize eder, amacı değil.

Beş başlık bitti; altı soru kaldı.

7.7 Kendini test et

Cevaplar kitabın sonunda.

1. Algoritmik önyargı aşağıdakilerden hangisinden kaynaklanabilir?
 - a) Ekran renginden
 - b) Çarpık/eksik eğitim verisinden
 - c) İnternet bağlantısından
 - d) Yavaş donanımdan
2. “Kara kutu” model ne demektir?
 - a) Çok hızlı olması
 - b) Kararının gerekçesini anlamanın zor olması
 - c) İnternetsiz çalışması
 - d) Kutuda saklanması
3. Deepfake’e karşı en sağlam bireysel savunma?
 - a) Daha hızlı internet
 - b) Hiçbir şey paylaşmamak
 - c) Daha pahalı telefon
 - d) Kaynağı sorgulamak ve doğrulamak
4. AB YZ Yasası kullanımları neye göre ayırır?
 - a) Şirket büyüklüğüne
 - b) Risk düzeyine
 - c) Renk koduna
 - d) Programlama diline

5. Hizalama (alignment) sorunu nedir?
- a) Ekranın küçük olması
 - b) Belirtilen hedef ile gerçek niyetin ayrışması
 - c) Modelin yavaş olması
 - d) Verinin az olması
6. Yüksek riskli bir YZ kullanımına örnek?
- a) Hava durumu widget'ı
 - b) Oyun rakibi YZ
 - c) İşe alımda aday eleme
 - d) E-posta spam filtresi

Bu bölümden kalanlar

- Yapay zekâ tarafsız değildir; kendisini eğiten verinin ve kuran insanların değerlerini taşır.
- Aynı nitelikteki iki gruba farklı karar veren modelde ayrımcılık veriden miras kalabilir; niyet aramak gerekmez, veriyi dengelemek de tek başına yetmez.
- İşaretli katkılar tek bir kararın gerekçesini görünür kılar; karar denetlenebilir ve itiraz edilebilir olur, modelin tamamı saydamlaşmaz.
- Tek bir görüntü ya da ses artık kanıt değildir; ipucu incelemeye götürür, hükmü bağımsız doğrulama verir.
- Düzenleme riske göre kademelidir: yasak, yüksek, sınırlı, minimal; dört kademe öğretici bir özettir, hukuki sınıf amaca ve maddeye göre belirlenir.
- Makine verdiği hedefi harfiyen yapar, niyetini değil; hizalama bu boşluğu kapatma sorunudur.

8

BÖLÜM 8

Felsefe ve Gelecek

Anlama, bilinç, tekillik ve sorumluluk

8.1 Felsefe ve gelecek

Son bölümdeyiz. Bir makinenin nasıl “düşündüğünü” öğrendik; şimdi sıra en eski ve en zor sorularda. Bir makine gerçekten anlayabilir mi, yoksa yalnızca anlıyormuş gibi mi yapar? Bizden daha zeki bir yapay zekâ mümkün mü; mümkünse ne zaman? Ya da bir gün makineler bilinç kazanırsa, hakları olur mu?

Bu soruların henüz cevabı yok; bilim insanları ve filozoflar bile derin biçimde ayrışıyor. Burada sana “doğru cevabı” dayatmayacağım. Farklı görüşleri yan yana koyacağım; fikrini kendin oluşturacaksın. Turing testinden Çince Oda’ya, tekillik tartışmasından sorumluluk sorusuna kadar birlikte düşüneceğiz.

***Kenar notu.** Bu son bölümde “kesin cevaplar”dan çok “iyi sorular” bulacaksın. Felsefede de bilimde de, doğru soruyu sormak çoğu zaman yarı yarıya cevaptır.*

TEKNİK DERİNLİK

Bu kapanış bölümü, YZ’nin felsefi temellerini ve uzun vadeli olasılıklarını ele alır. Bunlar büyük ölçüde açık (ve normatif) sorulardır; uzlaşılmış bir cevap yoktur.

Masadaki başlıklar şunlar: zihin felsefesi (Turing testi davranışçı bir ölçüt; Çince Oda sembol manipülasyonu ile anlama ayrımını sorgular), yetenek ufku (dar YZ → AGI → süper zekâ), özyinelemeli özgeleşim ve “tekillik” hipotezi (ciddi argümanlar ve güçlü eleştiriler), varoluşsal risk/fırsat çerçeveleri ve YZ’nin ahlaki/hukuki statüsü. Yapılacak iş, iddiaları kanıtları ve karşı argümanlarıyla birlikte tartmak.

İlk soru en eskisi: bir makinenin düşünüp düşünmediğini nereden bileceksin? Alan Turing bu soruya bir oyunla cevap verdi.

8.2 Turing testi: ayırt edebilir misin?

1950’de Alan Turing, “makinelere düşünebilir mi?” sorusunu fazla bulanık buldu. Onun yerine daha pratik bir soru önerdi: bir makine, yazışmada insanı ayırt edemeyeceğin kadar iyi taklit edebilir mi? Buna taklit oyunu, bugünkü adıyla Turing testi deniyor.

Aşağıda kısa yazışmalar var. Her biri için tahmin et: bu cevabı bir insan mı yazdı, yoksa bir makine mi? Sonra ipucunu kitabın sonunda gör.

***Kenar notu.** Turing testini geçmek “akıcı taklit” demektir; “gerçek anlama” ya da “bilinç” demek değildir. Bu iki soruyu birbirinden ayırmak, modern tartışmanın anahtarlarıdır.*

Şekil 8.1 · İnsan mı, makine mi?

YAZIŞMA 1 “En sevdiğin yemek ne?” Bir yapay zekâ olarak yemek yiyemem, dolayısıyla bir favorim yok. Ancak istersen popüler yemekler hakkında bilgi verebilirim. ☐ insan ☐ makine	YAZIŞMA 2 “Bu sabah trafiğe takıldın mı?” Sorma ya, köprüde tam 40 dakika kaldım, kahvem de soğudu :(en azından güzel bir podcast vardı. ☐ insan ☐ makine
YAZIŞMA 3 “17 × 24 kaç eder?” 408. ☐ insan ☐ makine	YAZIŞMA 4 “Hafta sonu ne yaptın?” Annemlere gittik, çok yedik, dönüşte de yağmura yakalandık. Klasik yani ☐ insan ☐ makine

Şekil 8.1

Canlı demo: book.onuronder.com/d/3e9862eecc



Kurulum. Şekilde dört kısa yazışma var. Her karede üstte bir soru, altında tek bir cevap duruyor. Dört yazışma da bu konuyu tartışmak için yazılmış kurgusal örnekler; gerçek bir kayıttan alınmadı. Cevabı kimin yazdığı gizli; ele veren ipucu da gizli. Turing’in taklit oyununu tek başına oynuyorsun: elinde yalnız metin var, ses yok, yüz yok. Yalnızca kelimelere bakarak karar vereceksin.

Kendini sına. Her yazışma için tahminini kenara yaz: insan mı, makine mi? Sonra seni o karara götüren şeyi bir kelimeyle not et. Kurgudaki rol ve ipuçları kitabın sonunda; kesin bir tanı değil.

1. Soru: “En sevdiğin yemek ne?” Cevap: “Bir yapay zekâ olarak yemek yiyemem, dolayısıyla bir favorim yok. Ancak istersen popüler yemekler hakkında bilgi verebilirim.”
2. Soru: “Bu sabah trafiğe takıldın mı?” Cevap: “Sorma ya, köprüde tam 40 dakika kaldım, kahvem de soğudu :(en azından güzel bir podcast vardı.”
3. Soru: “ 17×24 kaç eder?” Cevap: “408.”
4. Soru: “Hafta sonu ne yaptın?” Cevap: “Annemlere gittik, çok yedik, dönüşte de yağmura yakalandık. Klasik yani.”

Ölçütlerin kalıp ve kişisellik: cevap ne kadar kalıplı, ne kadar kişisel, ne kadar kusursuz? Kâğıtta cevap süresi görünmüyor; hız ölçüt değil. Ama ipuçlarının kendisini de sorgula. Bir makine “kahvem soğudu” yazmayı öğrenebilir; bir muhasebeci 408’i duraksamadan söyleyebilir. Tek bir anı ya da doğru bir hesap, yazanın insan ya da makine olduğunu kanıtlamaz. Dört yazışmayı kurgudaki rolüyle bilmek zor değil; bu, örneklerin kolay olduğunu gösterir, ipuçlarının sağlam olduğunu değil. Turing’in oyunu bu yüzden zamanla aşındı: taklit iyileştikçe ipuçları eskir.

Ne oluyor? Turing testi bir şeyi “anlamayı” tanımlamaya çalışmaz; sadece “yazışmada insandan ayırt edilemiyor mu?” diye bakar. Ama iyi taklit etmek, gerçekten anlamak demek değildir (bkz. Çince Oda); akıcı konuşan sistemler bu testi “kandırabilir”.

Kendin dene. 1) Kendi yazışmanı kur: bir soru ve iki cevap yaz, biri “makine gibi”, biri “insan gibi”. İki cevabı bir arkadaşına göster; ayırt edebiliyor mu, neye bakarak? 2) Üçüncü yazışmadaki ipucu beş yıl

sonra da işe yarar mı? Bugünkü sohbet modellerinin nasıl konuştuğunu düşünerek cevapla. 3) Testi geçmek isteyen bir makinenin bazen kasıtlı hata yapması gerekir mi? Bir cümleyle savun.

TEKNİK DERİNLİK

Turing testi davranışçı bir ölçüttür: “anlama”yı tanımlamak yerine, ayırt edilemez davranışı yeterli sayar. Eleştiriler: taklit, içsel anlamayı garanti etmez (bkz. Çince Oda) ve test, akıcı dil üreten sistemlerle “kandırılabilir”.

Önceden kayda geçirilmiş bir deneyde (Jones ve Bergen, 2025) katılımcılar aynı anda bir insanla ve bir yapay zekâyla beşer dakikalık yazışmalar yaptı ve hangisinin insan olduğuna karar verdi. İnsan gibi bir kişiliği canlandırmasını söyleyen bir istemle çalışan GPT-4.5, oyunların yüzde 73’ünde insan diye seçildi; aynı istemle LLaMa-3.1-405B yüzde 56’ya ulaştı, böyle bir istem verilmeyen GPT-4o ise yüzde 21’de kaldı. Sonuç bu modellere, bu isteme ve beş dakikalık bu düzene bağlıdır; daha uzun ya da başka türlü sorgulanan bir testte değişebilir ve bilinç ya da genel zekâ kanıtı değildir. Bu, “düşünme” tartışmasını bitirmek yerine soruyu “ölçüt ne olmalı?”ya kaydırıldı. Test bir başarıım ölçütünden çok tarihsel/kavramsal bir kilometre taşıdır.

Ayırt edilemezlik ölçüt olunca, ayırt etmeye yarayan her ipucu da öğrenilebilir bir hedefe dönüşür.

Şekil 8.1’deki dört ipucu bunun örneği; her biri bir yüzey işareti ve her biri öğrenilebilir:

İpucu türü	Ölçtüğü şey	Ne zaman yanıldır
Kalıplı, kibar üslup	Eğitimdeki yönerge izi	Modele “samimi konuş” denince
Kişisel ayrıntı, duygu	Yaşanmışlık izlenimi	Model kurgusal ayrıntı ürettiğinde

TEKNİK DERİNLİK · DEVAM

İpucu türü	Ölçtüğü şey	Ne zaman yanıldır
Kusursuz aritmetik	Hesap makinesi davranışı	Modele “insan gibi duraksa” denince; bir muhasebecide
Gündelik dil, emoji	Sohbet alışkanlığı	Sohbet verisiyle eğitilen her modelde

Akıcı cevap vermek anlamak mıdır? John Searle’ün odası bu soruyu kurcalar.

8.3 Çince Oda: anlamak mı, işlemek mi?

Filozof John Searle 1980’de şu deneyi önerdi: Çince bilmeyen biri olarak bir odadasın. Dışarıdan sana Çince notlar geliyor. Elinde de kalın bir kural kitabı var: “Şu sembolü görürsen, şu sembolle cevap ver.” Kuralları izleyerek kusursuz Çince cevaplar üretiyorsun ama tek kelime Çince anlamıyorsun.

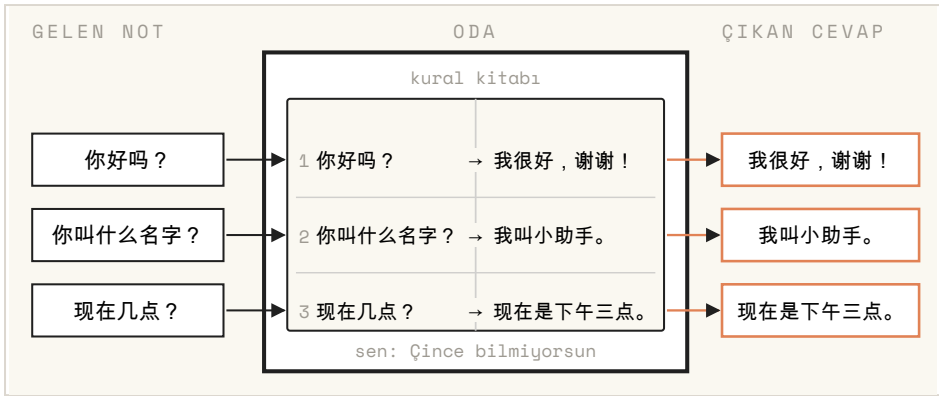
Aşağıda o odadaki kişi sensin. Gelen sembolü işle ve kuralın ürettiği cevabı gönder. Soru şu: bu oda Çince “anlıyor” mu, yoksa yalnızca sembol mü eşleştiriyor?

***Kenar notu.** Çince Oda kesin bir kanıt değil, güçlü bir sezgi pompasıdır. Kimine göre “makine asla anlamaz”ı gösterir; kimine göre “anlamanın nerede olduğunu” yanlış arıyoruz. Sen ne düşünüyorsun?*

Kurulum. Şekilde oda var: solda kapının altından gelen bir not, ortada kural kitabının açık sayfası, sağda dışarı verdiği cevap. Kural kitabı üç satırlık; her satırda “bu gelirse, bunu gönder” yazıyor. Semboller Çince, anlamları senden gizli. Dışarıdaki kişi seni görmüyor; yalnız kapının altından çıkan kâğıdı görüyor. Gerçek deneydeki gibi elindeki tek şey biçim eşleştirmesi.

Adım adım. Üç not sırayla geliyor. Her biri için kuralı bul, cevabı üret, dışarı ver.

Şekil 8.2 · Çince Oda'dasın



Şekil 8.2

Canlı demo: book.onuronder.com/d/965a20fb79

#	Gelen not	Kural kitabındaki satır	Verdiğin cevap
1	你好吗?	你好吗? → 我很好, 谢谢!	我很好, 谢谢!
2	你叫什么名字?	你叫什么名字? → 我叫小助手。	我叫小助手。
3	现在几点?	现在几点? → 现在是下午三点。	现在是下午三点。

Üç cevabı da kusursuz verdin. Dışarıdaki kişi seninle Çince sohbet ettiğini sanıyor. Şimdi notların anlamı:

#	Gelen notun anlamı	Verdiğin cevabın anlamı
1	Nasılsın?	İyiyim, teşekkürler!
2	Adın ne?	Adım Küçük Yardımcı.
3	Saat kaç?	Saat öğleden sonra üç.

Birine adının Küçük Yardımcı olduğunu söyledin. Saatin üç olduğunu söyledin; saate bakmadın. Cevaplar kurala uygundu, dilbilgisi kusursuzdu; doğru muydu, orası ayrı. Saat belki üç değildi. Kural kitabı biçimi verir, dünyayı değil. Searle'ün sorusu bu: odadaki sen Çince anlıyor musun? Çoğu okur hayır der. Oda, kitap ve sen birlikte anlıyor musunuz? Görüşler burada ayrılır.

Kural kitabı üç satırdı. Gerçek bir sohbet için milyonlarca satır gerekir. Üstelik “Saat kaç?” sorusuna hep “üç” demek, dördüncü notta seni ele verir. Büyük dil modelleri bu kitabın devasa ve istatistiksel bir sürümüne benzetilir; ama ağırlıkları hazır bir cevap tablosu değildir. Deneyin gücü de zayıflığı da bu benzetmede.

Ne oluyor? Çince Oda şunu söyler: kuralları uygulayıp doğru sembollerini sıralamak (yani biçimi işlemek) o dili gerçekten anlamak anlamına gelmez. Demek ki doğru cevap vermek (Turing testini geçmek) tek başına “anlıyor” demek değildir. Ama güçlü karşı görüşler de var (belki anlama kişide değil, oda + kurallar bütünündedir); tartışma hâlâ açık.

Kendin dene. 1) Kural kitabına dördüncü bir satır ekle: “你几岁？” (Kaç yaşındasın?) için bir cevap uydur; Türkçe yazman yeter. Satırı sen yazdın; oda şimdi “biraz daha” anlıyor mu? 2) Odaya bir pencere ve bir duvar saati ekle; kural “saati oku ve söyle” olsun. Sence anlama açısından bir şey değişti mi? Bir cümleyle gerekçelendir. 3) Searle, kural kitabını ezberlese bile anlamayacağını söyler. Bu cevap “sistem yanıtı”nı çürütür mü? Kendi cümlele karşı çık ya da katıl.

TEKNİK DERİNLİK

Çince Oda argümanı, sözdizimsel (syntactic) sembol manipülasyonunun anlamsal (semantic) anlamayı doğurmaya yetmediğini öne sürer; dolayısıyla davranışsal başarı (Turing testi) gerçek anlamının kanıtı sayılamaz (“güçlü YZ” eleştirisi).

Karşı görüşler güçlüdür: “Sistem yanıtı” der ki anlama, kişide değil oda+kurallar+süreç bütününde olabilir; “robot yanıtı” duyusal-motor bağ (grounding) eklenince durumun değişeceğini savunur. Tartışma, bilincin ve anlamının doğasıyla ilgili çözülmemiş bir sorundur.

Argümanın bütün yükü tek öncüldedir: sözdizimi tek başına anlambilim vermez. Karşı görüşlerin hepsi bu öncüle yüklenir.

Şekil 8.2’deki kural kitabı, teknik dille bir arama tablosudur (lookup table): girdi sembol dizisi, çıktı sembol dizisi. Bir dil

TEKNİK DERİNLİK · DEVAM

modeli de girdi dizisinden çıktı dizisine gider; ama ağırlıkları hazır bir cevap tablosu değildir. Model her cevabı o anda, bağlamdaki bütün dizinin istatistiğinden üretir; aynı soruya farklı cevaplar verebilir ve kitapta hiç yazmayan bir soruya da cevap üretir. Benzetme burada zorlanır. Searle'e göre bu fark önemsizdir; ikisi de sözdizimidir. Eleştirmenlere göre ise ölçek ve yapı, anlamının kendisini doğurabilir. Argümanın gücü, hangi tarafta durduğuna bağlı olarak değişir.

Anlama sorusunu bir yana koy: makineler ne kadar ileri gidebilir?

8.4 Dar YZ'den süper zekâya

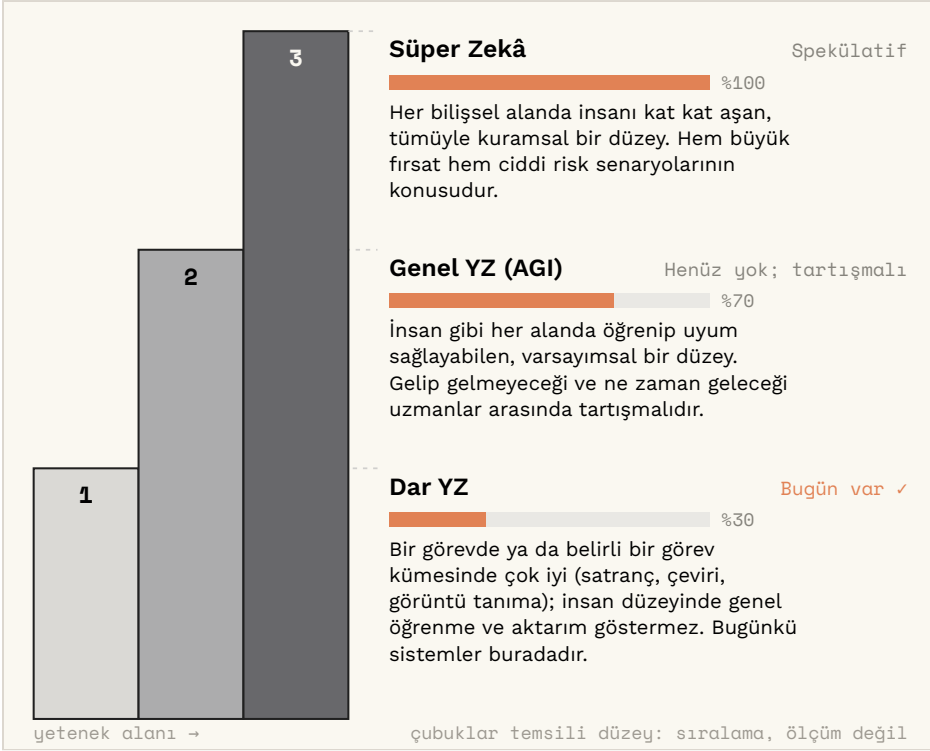
Bugünkü yapay zekâ “dar” sayılır: belirli işlerde (satranç, çeviri, görüntü) çok iyidir; birçok görevde çalışan sohbet modelleri bile insan düzeyinde genel öğrenme göstermez. Bir sonraki basamak, insan gibi her alanda öğrenip uyum sağlayabilen genel yapay zekâ (AGI). Onun da ötesinde, her alanda insanı kat kat aşan bir süper zekâ hayal ediliyor.

Şekil 8.3'teki basamaklara tek tek bak; her birinin ne anlama geldiğini ve “bugün var mı?” sorusunun cevabını gör.

***Kenar notu.** “Yapay zekâ insanı geçecek” başlıkları sık çıkar ama dikkat: belirli işlerde geçmek (dar) ile her alanda geçmek (genel) çok farklıdır. Bugün ilkindeyiz; ikincisi hâlâ açık bir soru.*

Kurulum. Şekilde üç basamaklı bir merdiven var. Her basamağın yanında bir çubuk: ilki yüzde 30 dolu, ikincisi yüzde 70, üçüncüsü tam. Çubuklar ölçüm değil, sıralama: kavramlar arasındaki varsayımsal farkı anlatır; ölçülmüş zekâ puanı ya da AGI'ye ilerleme oranı değildir. Basamağın adı, bugünkü durumu ve kısa tanımı yanında yazıyor. Merdiven yukarı çıktıkça renk koyulaşır.

Şekil 8.3 • Yetenek basamakları



Şekil 8.3

Canlı demo: book.onuronder.com/d/023e49955d

Adım adım. Üç basamak, tanımlarıyla birlikte:

Basamak	Bugün	Tanım
Dar YZ	Bugün var ✓	Bir görevde ya da belirli bir görev kümesinde çok iyi (satranç, çeviri, görüntü tanıma); insan düzeyinde genel öğrenme ve aktarım göstermez. Bugünkü sistemler buradadır.
Genel YZ (AGI)	Henüz yok; tartışmalı	İnsan gibi her alanda öğrenip uyum sağlayabilen, varsayımsal bir düzey. Gelip gelmeyeceği ve ne zaman geleceği uzmanlar arasında tartışmalıdır.
Süper Zekâ	Spekülatif	Her bilişsel alanda insanı kat kat aşan, tümüyle kuramsal bir düzey. Hem büyük fırsat hem ciddi risk senaryolarının konusudur.

“Bugün var” işareti yalnız ilk satırda; bu kitapta gördüğün her şey, sohbet modelleri dahil, o satırda. Alt iki satırın durumu bir belirsizlik etiketi; tartışmalı ile spekülatif arasındaki fark, konuştuğumuz şeyin ne kadar uzakta olduğu. Çubuklar yüzde 30, 70, 100 diye ilerliyor ama basamaklar arasındaki mesafe bilinmiyor: ikinci basamak birinciden on yıl da uzak olabilir, yüz yıl da; belki hiç gelmeyebilir.

Bir sohbet modeli hem şiir yazıyor hem kod üretiyor; ağırlıkları sabitken bile bağlamdaki yönerge ve örneklerle yeni bir göreve uyarlanabiliyor (Brown vd., 2020). Bu genel sayılmaz mı? Bazı uzmanlar bu yüzden dar ile genel arasına ara basamaklar koyar; insan düzeyinde genel zekâ için herkesin kabul ettiği bir ölçüt yoktur. Tabloda üç basamak var; gerçek dünyada muhtemelen sürekli bir eğim. Basamak, konuşmayı kolaylaştıran bir sadeleştirme.

Ne oluyor? Yetenek üç basamakta düşünülür: dar YZ belirli görevlerde iyidir (bugün buradayız); genel YZ (AGI) insan gibi her alanda öğrenebilir (henüz yok, tartışmalı); süper zekâ ise her alanda insanı kat kat aşar (şimdilik hayal). Her işi yapabilmek ile bilinçli olmak ayrı şeylerdir.

Kendin dene. 1) Bugün kullandığın üç yapay zekâ ürününü (çeviri, öneri, sohbet) tabloya yerleştir. Hepsi ilk satıra mı düştü? Birini ikinci satıra koymak istiyorsan, o sistemin hangi yeni işi kendi başına öğrendiğini söyle. 2) “AGI geldi” diyebilmek için hangi testin

geçilmesi gerekir? Bir cümlelik ölçüt öner; sonra ölçütünün Turing testinden neden farklı olduğunu söyle. 3) Bir işte insanı geçen ama genel olmayan bir sistem adı ver.

TEKNİK DERİNLİK

Yetenek ufku kabaca üç kademedede düşünülür: dar YZ (göreve özgü), AGI (alanlar arası, insan düzeyinde genelleme) ve süper zekâ (her bilişsel alanda insanüstü). Sınırlar bulanıktır; “genel” olmak ile “bilinçli” olmak ayrı sorulardır.

AGI’nin gelip gelmeyeceği ve ne zaman geleceği konusunda uzman görüşleri geniş bir yelpazeye yayılır (yakın, uzak, belki hiç). Ölçme zorluğu da var: “genel zekâ” için üzerinde uzlaşmış tek bir ölçüt yoktur. Bu yüzden kesin tarih veren iddialara temkinli yaklaşmak gerekir.

Kademeler arasında tanımlı bir eşik yoktur; ilk kademededen ikinciye geçildiğini kimse tek bir ölçüt ile ilan edemez.

Şekil 8.3’teki çubuk uzunlukları (30, 70, 100) gösterimin kodundan gelir; aralarındaki oran bir ölçüme dayanmaz, AGI’ye kalan mesafeyi göstermez. Ölçüm sorunu gerçektir: bir sistem yüzlerce görevde insan ortalamasını geçebilir ve yine de yeni bir alana aktarım yapamayabilir. Bu yüzden AGI tartışmalarında “hangi görev listesi?” ve “aktarım nasıl ölçülür?” soruları, tarih tahminlerinden daha verimlidir.

Üçüncü basamağa nasıl çıkılır, hatta çıkılabilir mi? Bu, zekânın zamanla nasıl büyüdüğüne bağlı.

8.5 Tekillik: mit mi, ciddi bir olasılık mı?

Yamaçtan yuvarlanan bir kartopu düşün: Döndükçe büyür, büyüdükçe daha çok kar toplar. Bazıları yapay zekâ için de aynı masalı anlatır: Kendini geliştirebilecek kadar zekileşen bir YZ daha iyi bir sürümünü yapar, o daha da iyisini... ve zekâ çığ gibi büyüyüp

“tekillik” denen, öngörülemez bir noktaya varır. Kimine göre bu yakın bir gerçek, kimine göre abartılı bir mit.

Aşağıdaki senaryolara bak: zekâ zamanla nasıl ilerleyebilir? “Hızlanan”, “yavaşlayıp duraklayan” ve “belirsiz” eğrileri karşılaştır.

***Kenar notu.** Sağlıklı duruş: ne “kesin olacak” ne “imkânsız”. Belirsizlik altında bile, güçlü sistemleri güvenli ve hızlı kılmak (Bölüm 7) bugünden mantıklı bir yatırımdır.*

Kurulum. Şekilde tek bir grafik var: yatay eksen zaman, dikey eksen zekâ. İki de birimsiz; zaman 0’dan 10’a, zekâ 0’dan 10’a gidiyor. Üç eğri var ve üçü farklı yerlere varıyor. Eksenlerde yıl yok; bu bilerek böyle. Eğriler üç ayrı hikâyenin şeklini gösteriyor; tahmin sayma. Her eğrinin kendi notu var; üçü de aşağıda.

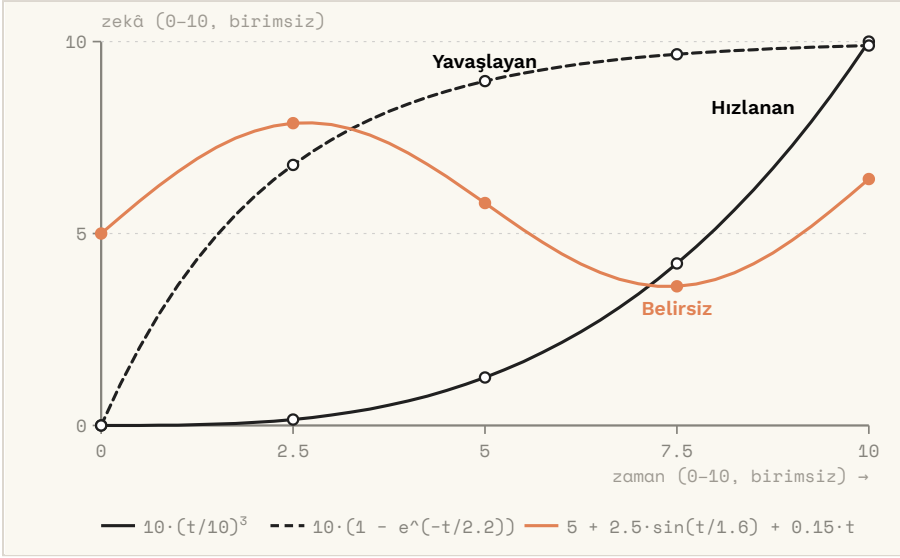
Adım adım. Üç eğrinin çizildiği formüller ve beş zaman noktasındaki değerleri (0–10 ölçeği):

Zaman	Hızlanan: $10 \cdot (t/10)^3$	Yavaşlayan: $10 \cdot (1 - e^{(-t/2.2)})$	Belirsiz: $5 + 2.5 \cdot \sin(t/1.6) + 0.15 \cdot t$
0	0.0	0.0	5.0
2.5	0.2	6.8	7.9
5	1.3	9.0	5.8
7.5	4.2	9.7	3.6
10	10.0	9.9	6.4

- Hızlanan.** Özyinelemeli öz gelişim: zekâ kendini besleyerek patlar. Lehte argüman, geri besleme döngülerine dayanır. Tabloda ilk yarıda neredeyse kıpırdamıyor; son çeyrekte 4’ten 10’a fırlıyor.
- Yavaşlayan.** Azalan getiriler: veri, enerji ve fizik sınırları büyümeyi yavaşlatır; zekâ bir tavana yaklaşır. İlk çeyrekte 7’ye yaklaşıyor, sonra tavana yaslanıyor. Hızlı başlayan her teknoloji bir gün bu eğriye benzer.
- Belirsiz.** Açık cevap: bilmiyoruz. Sıçramalar ve duraklamalar bir arada olabilir; kesin tarih veren iddialara temkinli yaklaş. Eğri 8’e

yaklaşık 3.6'ya iniyor, sonra yine yükseliyor. Yönü var, ritmi yok.

Şekil 8.4 • Üç zekâ eğrisi



Şekil 8.4

Canlı demo: book.onuronder.com/d/bddd5bc368



Tabloda 2.5 satırına bak. Yavaşlayan eğri o noktada 6.8; hızlanan eğri 0.2. Bugünkü hızlı ilerleme, yavaşlayan eğrinin dik başlangıcı da olabilir, hızlanan eğrinin sakin ilk yarısı da. Aynı gözlem iki hikâyeye birden uyar. Hangi eğride olduğun ancak geriye bakınca belli olur; tartışmanın bitmemesinin sebebi bu.

Ne oluyor? Fikir şu: bir YZ kendini geliştirebilirse daha iyi bir sürümünü yapar, o daha da iyisini... ve zekâ birden patlayarak öngörülemez bir noktaya (“tekillik”) ulaşır. Kimi bunu yakın görür, kimi abartılı bir mit sayar; veri, enerji ve fizik sınırları bunu yavaşlatabilir.

Kendin dene. 1) Hızlanan eğri 5'i hangi zamanda geçer? $10 \cdot (t/10)^3 = 5$ denklemini çöz; küp kök alman gerekir, yaklaşık değer yeter. 2) Yavaşlayan eğri hiç 10'a ulaşır mı? Formüle bakarak cevapla; sonra

“pratikte ulaştı” demek için hangi değeri eşik sayacağını söyle. 3) Üç eğriden hangisi güvenli ve hizalı sistemlere bugünden yatırım yapmayı gereksiz kılar? Hiçbiri mi? Gerekçeni bir cümleyle yaz.

TEKNİK DERİNLİK

Tekillik hipotezi, özyinelemeli öz gelişimin (recursive self-improvement) üstel bir “zekâ patlamasına” yol açabileceğini öne sürer. Lehte argümanlar (I.J. Good ve modern savunucular) hız ve geri besleme döngülerine dayanır.

Güçlü eleştiriler de vardır: zekâ tek boyutlu/sınırsız ölçeklenebilir olmayabilir; veri, enerji, donanım ve fizik sınırları; karmaşıklık ve azalan getiriler. Bu, kanıtlanmış bir kehanet değil, ciddiye alınması gereken ama belirsiz bir senaryodur. Aşağıdaki eğriler niteliksel gösterimdir.

Lehte ve aleyhte argümanlar aynı veriye bakıp farklı eğri görür; ayırım, hangi sınırın önce geleceğine dair varsayımdadır.

Şekil 8.4'teki üç formül birer benzetmedir, model değil. Kübik eğri patlamayı, doyuma ulaşan üstel eğri $(1 - e^{(-t/\tau)})$, $\tau = 2.2$ tavanı, sinüs artı doğrusal terim ise sıçrama ve duraklamayı temsil eder. Gerçek tartışma, hangi eğrinin doğru olduğundan çok şu iki soruda düğümlenir: öz gelişim döngüsünün her turu gerçekten bir öncekinden hızlı mı ve hangi kaynak (veri, enerji, hesaplama) önce tükenir? Bu iki soru ampiriktir ve zamanla cevaplanabilir; eğrinin adı ise ancak sonradan konur.

Eğri ne olursa olsun bu sistemler bugün karar veriyor ve hata yapıyor. Hata olunca kim hesap verir?

8.6 Sorumluluk ve haklar

Bir yapay zekâ bir zarara yol açarsa kim sorumlu olur? Onu yapan şirket mi, kullanan kurum mu, son kullanıcı mı, yoksa “yapay zekânın kendisi” mi? Bir de daha derin bir soru var. Ya bir gün

makineler bir tür deneyim, hatta bilinç kazanırsa? O zaman onlara karşı ahlaki bir sorumluluğumuz doğar mı; hakları olur mu?

Bir senaryo seç ve sorumluluğu kime vereceğini işaretle. Sonra yaygın hukuki/etik görüşü gör ve haklar tartışmasındaki farklı duruşları değerlendir.

***Kenar notu.** Kitabın sonuna geldik. Bu kitabın tutumu açık: insanın karar gücünü artıran ve sonuçları denetlenebilen kullanımları savunuyor. Yapay zekânın geleceğini teknoloji tek başına belirlemeyecek; onu hangi değerlerle kurup kullandığımız da belirleyecek. O gelecek üzerinde söz hakkın var.*

Kurulum. Şekilde üç senaryo kartı ve dört taraf var: üretici/geliştirici, işleten kurum, son kullanıcı ve YZ'nin kendisi. Her senaryo için ilk incelenecek tarafın hücrelerini işaretlemen bekleniyor. Karşılığında yaygın görüş gelecek: bugünkü hukuk ve etik tartışmasında ağır basan cevap. Bu bir hukuk hükmü değil; sorumluluk çoğu olayda paylaşılr ve sonuç ülkeye, kusura ve sözleşmeye bağlıdır.

Kendini sına. Her senaryo için ilk incelenecek tarafı seç ve bir cümleyle gerekçelendir; başka kimin payı olabilir, onu da not et. Yaygın görüş ve gerekçesi kitabın sonunda.

1. Sürücüsüz bir araç, üreticinin yazılım hatası yüzünden kaza yapar.
2. Bir kurum, YZ tavsiyesini kör biçimde uygulayıp müşteriye zarar verir.
3. Bir kullanıcı, birini aldatmak ya da zarara uğratmak için bir YZ aracıyla sahte kanıt üretir.

Taraflar: (a) Üretici / geliştirici, (b) İşleten kurum, (c) Son kullanıcı, (d) YZ'nin kendisi.

Karar verirken üç soru sor: zarara giden zincirde kararı kim verdi? Kim denetleyebilirdi ama denetlemedi? Niyet kimdeydi? Dördüncü taraf için bir soru yeter: bir yazılımı mahkemeye çıkarıp ceza verebilir misin, cezadan ne anlar? Bugün hukuk buna hayır diyor; sorumluluk zincirin insan halkalarında toplanıyor. Senaryolar bilerek temiz tutuldu. Gerçek olaylarda üç insan taraf da bir parça sorumlu

çıkabilir; pay kavgası mahkemelerde yıllarca sürer. Sahte içerik üretmek tek başına zarar ya da kötü niyet demek değildir; kurgu ve sanat da sentetiktir. Üçüncü senaryoyu ayıran, aldatma ve zarar amacı.

Şekil 8.5 · Sorumluluk kimde?

SENARYO	İLK İNCELENECEK TARAF			
	(a) Üretici / geliştirici	(b) İşleten kurum	(c) Son kullanıcı	(d) YZ'nin kendisi
1 Sürücüsüz bir araç, üreticinin yazılım hatası yüzünden kaza yapar.	☐	☐	☐	☐
2 Bir kurum, YZ tavsiyesini kör biçimde uygulayıp müşteriye zarar verir.	☐	☐	☐	☐
3 Bir kullanıcı, birini aldatmak ya da zarara uğratmak için bir YZ aracıyla sahte kanıt üretir.	☐	☐	☐	☐

sorumluluk çoğu olayda paylaşılır · yaygın görüş kitabın sonunda

Şekil 8.5

Canlı demo: book.onuronder.com/d/4f50232ae1



Haklar sorusunun ise senaryosu yok, çünkü henüz olayı yok. Bugün iki ana duruş var. Biri, haklar için öznel deneyim (haz ve acı duyabilme) ya da bilinç gerektiğini söyler; bugünkü sistemlerde bu yok, dolayısıyla soru erken. Öteki, emin olmadığımız yerde ihtiyatlı davranmayı önerir; sonradan haksızlık etmiş çıkmaktansa şimdiden dikkatli olmak daha iyi. İki duruş da bilinç sorusuna dayanıyor ve o soru bu kitabın hiçbir bölümünde çözülmedi. Çözülmemesi normal; kimse çözmedi.

Ne oluyor? Bir zarar olduğunda sorumluluk bugün neredeyse her zaman insanlara ve kurumlara verilir: geliştiren, işleten ve kullanan. “YZ'nin kendisini” hukuken sorumlu tutmak yaygın bir görüş değil. Makinelerin bir gün hak sahibi olup olamayacağı ise bambaşka ve hâlâ açık bir sorudur.

Kendin dene. 1) Dördüncü bir senaryo yaz: sorumluluğun iki tarafa birden düştüğü bir olay. Payı nasıl bölerdin? 2) Yukarıdaki iki duruştan hangisi sana yakın? Bir cümleyle neden. 3) Yapay zekânın geleceği üzerindeki söz hakkın somut olarak nerede başlar? Bir örnek yaz.

TEKNİK DERİNLİK

Sorumluluk (accountability) bugün ezici biçimde insanlara ve kurumlara atfedilir: tasarım, dağıtım ve kullanım kararlarını insanlar verir; “YZ’nin kendisi”ne hukuki sorumluluk yüklemek hâkim görüş değildir. Sorumluluk genelde paylaşılır ve bağlama bağlıdır (geliştirici, işleten, kullanıcı, düzenleyici).

YZ’nin ahlaki statüsü ayrı ve tartışmalı bir sorudur: bazıları statünün öznel deneyim yaşayabilme kapasitesi (sentience), yani haz ve acı duyabilme gerektirdiğini ve mevcut sistemlerde bunun bulunmadığını savunur; bazıları ihtiyatlılık ilkesini öne sürer. Bu, hem ampirik (bilinç var mı?) hem normatif (olsa ne borçluyuz?) bir sorudur ve açıktır. Zekâ, bilinç ve öz farkındalık burada birbirinin yerine kullanılmaz.

Sorumluluk ve haklar iki ayrı sorudur; ilki bugün hukukun, ikincisi henüz felsefenin masasında.

Şekil 8.5’teki üç senaryo, sorumluluğun üç ayrı kaynağını temsil eder: kusur (tasarım hatası), ihmal (denetimsiz kullanım) ve kasıt (kötüye kullanım). Hukuk sistemleri bu üçünü farklı kurumlara karşılar: ürün sorumluluğu, özen yükümlülüğü ve ceza hukuku; hangisinin uygulanacağı ülkeye, kusura ve sözleşmeye bağlıdır ve aynı olayda birden çok taraf pay alabilir. Dördüncü taraf bu çerçevelerin hiçbirine oturmaz; çünkü sorumluluk, yaptırımın anlamlı olduğu bir özne gerektirir.

Kitap burada bitiyor, soruları bitmiyor. Son altı soru.

8.7 Kendini test et

Cevaplar kitabın sonunda.

1. Turing testi temelde neyi ölçer?
 - a) Gerçek bilinci
 - b) Hafıza miktarını
 - c) Yazışmada insanı ayırt edilemez biçimde taklit edebilmeyi
 - d) İşlem hızını
2. Çince Oda argümanı neyi sorgular?
 - a) İnternetin güvenliğini
 - b) Bilgisayar hızını
 - c) Sembol işlemenin tek başına “anlama” doğurup doğurmadığını
 - d) Çincenin zorluğunu
3. Bugünkü yapay zekâ hangi düzeydedir?
 - a) AGI
 - b) Bilinçli YZ
 - c) Süper zekâ
 - d) Dar YZ
4. Tekillik için bu kitapta savunulan temkinli yaklaşım hangisidir?
 - a) Zaten oldu
 - b) Kesinlikle yarın olacak
 - c) Tamamen imkânsız
 - d) Belirsiz; ne kesin ne imkânsız
5. Bir YZ zarar verdiğinde sorumluluk bugün genelde kime atfedilir?
 - a) İnsanlara ve kurumlara (geliştirici/işleten/kullanıcı)
 - b) Yalnızca yapay zekânın kendisine
 - c) Hiç kimseye
 - d) İnternete

6. YZ'nin ahlaki statüsü (haklar) sorusu bugün nasıldır?

- a) Açık ve tartışmalı
- b) Yasayla yasaklanmış
- c) Kesin olarak çözülmüş
- d) Anlamsız bir soru

Bu bölümden kalanlar

- Anlama, yetenek ve bilinç üç ayrı sorudur; birini cevaplamak ötekini cevaplamaz.
- Turing testi davranışa bakar: yazışmada ayırt edilemeyen makine testi geçer, ama akıcı taklit anlama demek değildir.
- Çince Oda, kuralla sembol eşleştirmenin anlamayı doğurmadığını savunur; anlamanın kişide mi, sistemde mi olduğu hâlâ tartışmalıdır.
- Bugünkü yapay zekâ dar sayılır; birçok görevde çalışsa da genel zekâ için uzlaşmış ölçüt yoktur, süper zekâ kuramsaldır ve genel olmak bilinçli olmak demek değildir.
- Tekillik ne kesin ne imkânsız; hangi eğride olduğumuz ancak geriye bakınca belli olur.
- Bir zarar olduğunda sorumluluk bugün insanlara ve kurumlara düşer: geliştiren, işleten, kullanan.
- Makinelerin hakları bilinç sorusuna bağlı ve o soru açık; geleceği teknoloji kadar onu kuran değerler de belirleyecek, kitabın tutumu bu.

Cevap Anahtarı

Bölüm sonu sınavları

01 · Zekâ ve Makineler

1.8 · Soru 1: **b** · Zekânın tek değil, birçok türü olduğunu

1.8 · Soru 2: **d** · 0 ve 1

1.8 · Soru 3: **b** · Yeterli zaman ve bantla, basit kurallarla hesaplanabilir her işlemi

1.8 · Soru 4: **b** · Program ve veriyi aynı bellekte tutmak

1.8 · Soru 5: **b** · Dar (narrow) YZ

1.8 · Soru 6: **c** · Transistör sayısı ~her 2 yılda ikiye katlanır

02 · Kuralların Çağı

2.7 · Soru 1: **c** · Açık semboller ve kurallarla

2.7 · Soru 2: **c** · Bir kural (IF-THEN)

2.7 · Soru 3: **d** · Çözümü hızlandırır ama en iyiyi garanti etmez

2.7 · Soru 4: **a** · Yalnızca şu anki duruma

2.7 · Soru 5: **a** · Tüm kuralları elle yazmak ve dünyanın dağınıklığı

2.7 · Soru 6: **a** · YZ araştırmasında iki farklı yaklaşımı

03 · Makineler Nasıl Öğrenir

3.8 · Soru 1: **a** · Kuralları elle yazmak yerine veriden öğrenir

3.8 · Soru 2: **a** · Etiket

3.8 · Soru 3: **a** · Kümeleme (denetimsiz)

3.8 · Soru 4: **d** · Kaybı azaltacak yönde parametreleri adım adım günceller

3.8 · Soru 5: **c** · Eğitim verisini ezberleyip yeni veride başarısız olmak

3.8 · Soru 6: **a** · Ödül ve cezayla, deneme-yanılmayla

04 · Yapay Beyin

4.8 · Soru 1: **b** · Girdilerin ağırlıklı toplamı + sabit terim, sonra aktivasyon

4.8 · Soru 2: **b** · Çok sayıda gizli katman

4.8 · Soru 3: **b** · Ağ tek bir doğrusal işleve çökerdi

4.8 · Soru 4: **b** · Kaybın her ağırlığa göre gradyanını çıkıştan girişe doğru hesaplar

4.8 · Soru 5: **d** · Görüntü

4.8 · Soru 6: **c** · Üretici ve Ayırt edici

05 · Bugünün Yapay Zekâsı

5.9 · Soru 1: **d** · Metnin model tarafından işlenen küçük parçası

5.9 · Soru 2: **c** · Anlamca benzer kelimeler birbirine yakın olur

5.9 · Soru 3: **c** · Her kelimenin diğer kelimelere ağırlıklı “bakması”

5.9 · Soru 4: **a** · Bir sonraki token’ı tahmin eder

5.9 · Soru 5: **a** · Ön eğitim → ince ayar → RLHF

5.9 · Soru 6: **d** · Gürültüden başlayıp adım adım temizleyerek

5.9 · Soru 7: **a** · Modelin emin tonda yanlış bilgi üretmesi

5.9 · Soru 8: **a** · Modelin aynı anda dikkate aldığı token sayısını

06 · YZ'yi Kullanmak ve İnşa Etmek

6.7 · Soru 1: **a** · Rol, bağlam, örnek ve net format eklemek

6.7 · Soru 2: **d** · İlgili kaynağı bulup modele vererek cevabı kaynağa dayandırır

6.7 · Soru 3: **c** · Araç kullanıp adım adım eyleme geçmesi

6.7 · Soru 4: **c** · İstem, araç ve RAG çağrılarını koordine eder

6.7 · Soru 5: **c** · RAG ile cevabı kaynağa dayandırmak

6.7 · Soru 6: **d** · İnsanı güçlendiren (yardımcı pilot) tasarımlar

07 · Yapay Zekâ ve Toplum

7.7 · Soru 1: **b** · Çarpık/eksik eğitim verisinden

7.7 · Soru 2: **b** · Kararının gerekçesini anlamamanın zor olması

7.7 · Soru 3: **d** · Kaynağı sorgulamak ve doğrulamak

7.7 · Soru 4: **b** · Risk düzeyine

7.7 · Soru 5: **b** · Belirtilen hedef ile gerçek niyetin ayrışması

7.7 · Soru 6: **c** · İşe alımda aday eleme

08 · Felsefe ve Gelecek

8.7 · Soru 1: **c** · Yazışmada insanı ayırt edilemez biçimde taklit edebilmeyi

8.7 · Soru 2: **c** · Sembol işlemenin tek başına “anlama” doğurup doğurmadığını

8.7 · Soru 3: **d** · Dar YZ

8.7 · Soru 4: **d** · Belirsiz; ne kesin ne imkânsız

8.7 · Soru 5: **a** · İnsanlara ve kurumlara (geliştirici/işleten/kullanıcı)

8.7 · Soru 6: **a** · Açık ve tartışmalı

Şekil alıştırmaları ve kendini sına cevapları

Bölüm I cevapları

ŞEKİL 1.1 · ÇOKLU ZEKÂYI KEŞFET

Kendin dene. 1) Dilsel, Mantıksal-Matematiksel, Uzamsal, Müziksel, Bedensel-Kinestetik, Kişilerarası, İçsel, Doğacı. En sık unutilanlar genelde İçsel ve Doğacı olur. 2) Kişiye göre değişir. Tabloya göre YZ yalnız Dilsel ve Mantıksal-Matematiksel alanlarda güçlü; bir günün büyük kısmı ise bedensel, kişilerarası ve içsel işlerle geçer, YZ bu üçünde de zayıf. 3) Üçü de sembole dökülmeyen bir malzeme ister: bir beden, başkasının zihni, kendi iç dünyan.

ŞEKİL 1.2 · İKİLİ KODU ÇÖZ

Kendin dene. 1) $10 = 8 + 2 \rightarrow 00001010$. 2) $10110000 = 128 + 32 + 16 = 176$. 3) 128. Dokuz kutuyla en büyük sayı $256 + 255 = 511$; dokuz kutu 512 ayrı sayı tutar.

ŞEKİL 1.3 · ÇALIŞAN BİR TURING MAKİNESİ (+1)

Kendin dene. 1) 001111: kafa sona gider (5 hamle), durum değişir (1 hamle), dört 1 sırayla sıfırlanır (4 hamle), 1. gözdeki 0 yerine 1 yazılır (1 hamle). Toplam 11 hamle; bant 010000 = 16. 2) 010110: 22 çift sayı olduğu için kafa geri dönünce ilk gördüğü rakam 0. Oraya hemen 1 yazar; $5 + 1 + 1 = 7$ hamlede biter; bant 010111 = 23. 3) 111111: altı 1 de sıfırlanır ($5 + 1 + 6 = 12$ hamle). Kafa en sol karede 1'i 0 yaptıktan sonra sola gidemez ve durur. Bant 000000 = 0. Altı kare 64 'ü

tutamaz; sayaç sıfıra döner. Kilometre sayacının 999999'dan 000000'a atlaması gibi; bilgisayarlar buna taşma der.

ŞEKİL 1.4 · GETİR - YÜRÜT - YAZ DÖNGÜSÜ

Kendin dene. 1) Getir: kontrol birimi bellekten “çarp” talimatını alır (Bellek). Yürüt: ALU 7 ile 6'yı okur ve $7 \times 6 = 42$ hesaplar (İşlemci). Yaz: 42 bir yazmaca ya da belleğe kaydedilir; Giriş / Çıkış ancak ayrı bir “ekrana yaz” talimatı yürütülünce iş başına geçer ve 42 ekranda belirir. 2) Bir tur, üç evre daha: getir “sonucu 2 ile çarp”, yürüt $8 \times 2 = 16$, yaz 16 belleğe. Ekranda görünmesi için ayrıca bir “ekrana yaz” satırı gerekir; kaydetme ile dışarı verme ayrı işlerdir. 3) 3 milyar $\times 3 = 9$ milyar evre.

ŞEKİL 1.5 · BUGÜN VAR MI, YOKSA BİLİM KURGU MU?

Kendin dene. 1) Navigasyon, çeviri ve film önerisi: üçü de dar YZ, sol sütun. Her biri belirli görevler için kurulmuştur ve alanlar arasında insan düzeyinde genel öğrenme göstermez; navigasyon uygulamasından çeviri isteyemezsin. 2) İkisi de bugün yok ama aynı soru değil. Dördüncü kart “genel” sorusuna dair: alanlar arasında aktarılabilir yetenekle her alanı öğrenebilir mi? Beşinci kart “güçlü” sorusuna dair: gerçekten anlıyor, bilinçli mi? Evet, bir makine genel olup bilinçsiz olabilir; AGI tanımında bilinç şartı yoktur. 3) Hiçbir şey olmaz; motor yalnızca satranç konumlarını değerlendirir, “çorba” diye bir girdi tanımaz. Dar YZ: belirli bir görevde ya da sınırlı bir görev kümesinde çok iyi olan, ama insan gibi her alanda öğrenip yeteneğini yeni alanlara genel olarak aktaramayan sistem. Çok görevli olması tek başına onu genel yapmaz.

Kendini sına. 1 Satranç motoru → Bugün kullanılan sistem: dar YZ; tek iş, satranç; başka hiçbir şeyi bilmez. 2 Yüz tanıma sistemi → Bugün kullanılan sistem: dar YZ; yüzleri eşleştirir, yüz dışında görevi yok. 3 Sohbet botu (dil modeli) → Bugün kullanılan sistem: dar YZ; şiir de kod da yazar, istemdeki örneklerle yeni bir işe bir ölçüde uyum da gösterebilir (bağlam içi uyum), ama insan düzeyinde alanlar arası genel öğrenme ve aktarım göstermez; kendi amacı yok. Çok iş yapmak genel olmak değildir; kartların en yanıltıcısı bu. 4 Her

mesleği insan gibi öğrenip yapan, kendi amaçları olan makine → Varsayımsal sistem: alanlar arası aktarılabilir yetenek AGI tanımının kendisi; böyle bir sistem yapılmadı. 5 Kendini fark eden, bilinçli bir YZ → Bilinç sorusu: böyle bir sistem bugün yok, ama kartın sorduğu şey yetenek değil bilinç. Bilinç güçlü YZ sorusudur, AGI tanımına girmez; genel bir makine bilinçsiz olabilir. Bu yüzden kart “Varsayımsal sistem” sütununa değil, ayrı sütuna gider.

ŞEKİL 1.6 · ÜSTEL BÜYÜMEYİ HİSSET

Kendin dene. 1) $n = 14$, 1999: $2^n = 16.384$, transistör 37.683.200 (yuvarlarsan 37.7 milyon). $n = 15$, 2001: $2^n = 32.768$, transistör 75.366.400 (75.4 milyon). 2) 1989 satırı, $n = 9$: 1.177.600. 1971’den itibaren 18 yıl. 3) Üç yıllık katlamayla 1971’den 1995’e 24 yıl = 8 katlama: $2.300 \times 256 = 588.800$. İki yıllık tabloda 1995 değeri 9.420.800; aradaki fark 16 kat. Katlama süresine eklenen tek bir yıl, 24 yılın sonunda 16 kat fark yaratıyor.

Bölüm 2 cevapları

ŞEKİL 2.1 · BİLGİ ZİNCİRİYLE ÇIKARIM

Kendin dene. 1) Evet. Makine beş ok izler: Tekir → Kedi → Memeli → Hayvan → Canlı → Varlık. Zincir uzadıkça adım sayısı artar ama kural aynı kalır. 2) Bilinmiyor. Kedi’den zincir Memeli, Hayvan ve Canlı’ya uzanır; Tekir’e ulaşmaz. Bu cevap doğrudur: her kedi Tekir değildir, “örneği” ve “alt sınıfı” bağları tek yönlüdür. Tekir bir örnek, Kedi bir sınıftır; sınıf, örneğinin altına yazılamaz. 3) Zincir yalnızca olumlu “örneği” ve “alt sınıfı” bağları tutar; olumsuz bilgi için ayrı bir olgu ya da kural gerekir. Örneğin “Hayvan ile Bitki ayrık sınıflardır” kuralı eklenirse makine “Tekir bir Hayvan’dır” bilgisinden “Bitki değildir” sonucunu türetebilir. Bunu eklemeyen sistem “bilinmiyor” demekle yetinir.

ŞEKİL 2.2 · KÜÇÜK BİR UZMAN SİSTEM

Kendin dene. 1) R1, R2, R3 ve R5 ateşler; dört öneri: Şemsiye al · Mont giy · Atkı tak · Dikkat: şemsiye ters dönebilir. R4 sessiz kalır,

çünkü yağmur var. 2) Yalnız R4 ateşler: “Hafif giyinebilirsin.” R4 yalnızca yağmura ve soğuğa bakar; rüzgârı hiç sormaz. Rüzgârlı havada bu öneri eksik kalır. Eksik olan, örneğin “EĞER rüzgârlı ise rüzgârlık al” kuralıdır. Yazılmayan koşul yok sayılır; kuralların kör noktası bu. 3) R5 yalnız rüzgâra bağlansaydı, yağmur yokken ve şemsiye önerilmemişken bile “şemsiye ters dönebilir” uyarısı verirdi. Zincirleme, uyarıyı gerçekten ilgili duruma bağlar: uyarı ancak şemsiye önerildiyse anlamlıdır.

ŞEKİL 2.3 · YOL BULMA: SEZGİSİZ İLE SEZGİLİ

Kendin dene. 1) Puan = sütun farkı + satır farkı, hedef 8. sütun, 6. satır. 4. sütun, 2. satır: $4 + 4 = 8$. 1. sütun, 6. satır: $7 + 0 = 7$. Alt köşe daha yakın görünür, ama 5. sütundaki duvar yüzünden çıkmazdadır; yolun üstündeki kare 4. sütun, 2. satırdır. Sezgisel arama bu yüzden alt satıra sapıp beş kare harcadı. 2) Puan duvarları görmez; 12, duvarsız ızgaradaki en kısa yoldur. Üç engel üç dolambaç ekler ve gerçek yol 18 adıma çıkar. Puan gerçek uzaklığı hiçbir zaman aşmaz; teknik derinlikte “kabul edilebilir” denen sezgisel bu. Açgözlü arama bu puana rağmen garantisiz; A^* aynı puana gidilen yolu ekleyince garanti geri gelir. 3) 12 adım: 1. sütundan aşağı 5 adım, alt satırdan sağa 7 adım. Engel kalkınca yol, puanın söylediği uzaklığa iner.

ŞEKİL 2.4 · HAVA DURUMU MARKOV ZİNCİRİ

Kendin dene. 1) Yağmurlu satırı 20 / 40 / 40, aralıklar 1–20 / 21–60 / 61–100. 35 ikinci aralığa düşer: yarın Bulutlu. 2) Yarın güneşli olup ertesi gün yağmur: $0.7 \cdot 0.1 = 0.07$. Yarın bulutlu, ertesi gün yağmur: $0.2 \cdot 0.3 = 0.06$. Yarın yağmurlu, ertesi gün yine yağmur: $0.1 \cdot 0.4 = 0.04$. Toplam 0.17, yüzde 17. 3) Güneşe kayar. Güneşli günler daha yapışkan olur ve zincir güneşte daha uzun kalır. Hesap: yeni matrisle kararlı dağılım yaklaşık yüzde 72 / 15 / 13 (eski 46 / 31 / 23). İlk denklemlerle kontrol: $0.9 \cdot 0.72 + 0.3 \cdot 0.15 + 0.2 \cdot 0.13 \approx 0.72$.

ŞEKİL 2.5 · HANGİ YAKLAŞIM?

Kendini sına. 1) Her şeyi biçimsel mantıkla kanıtla → Düzenli: önce kanıt, sonra kullanım. 2) Çalışan kısayolları kullan, teoriyi sonra düşün → Dağınık: önce sonuç, teori beklesin. 3) Matematiksel

kesinlik şarttır → Düzenli: kesinlik pazarlık konusu değil. 4) Gerçek dünya dağınıktır; esnek ol → Dağınık: dünyaya uy, ilkeye değil. **Kendin dene.** 1) Bilgi zinciri, uzman sistem ve sezgisiz arama Düzenli kampa yakındır: kesin türetme, garantili sonuç. Açgözlü sezgisel arama Dağınık kamptandır: hız için garantiden vazgeçer. Markov zinciri aradadır: olasılık kuramına dayanır ve yakınsaması kanıtlanabilir (Düzenli), ama dünyanın belirsiz olduğunu kabul eder (Dağınık tavır). 2) Sezgisel puan Dağınık bir icattır: işe yarıyor, garanti sorulmuyor. A* ona “kabul edilebilir” koşuluyla (graf aramasında tutarlılıkla) bir kanıt ekler ve en kısa yolu garanti eder; Dağınık bir fikri Düzenli kampa taşır. İki kamp burada el sıkışır. 3) Kişisel cevap. Ölçüt: önce çalıştırıp sonra neden çalıştığını anladıysan Dağınık davrandın.

Bölüm 3 cevapları

ŞEKİL 3.1 · ÖZELLİKLERİ VE ETİKETİ GÖR

Kendin dene. 1) “bedava” ○, link veya şifre isteği ✓ (şifre yenileme), aciliyet dili ✓ (“bugün”, “silinir”). İki işaret; tabloda iki işaretli tek örnek olan 3. e-posta Spam’di. Etiket: Spam. 2) Daha az. “Bedava” artık bir Normal e-postada da geçiyor; tek başına ayırt edici olmaktan çıkıyor. Link veya şifre isteği ve aciliyet dili ise hâlâ yalnız Spam satırlarında var. 3) Örnek kural: “Link veya şifre isteği varsa Spam, yoksa Normal.” Dört satırla uyusur (1 ve 3 Spam, 2 ve 4 Normal); kahve e-postasında link veya şifre isteği yok, Normal der, doğru. ““Bedava’ geçiyorsa ya da link veya şifre isteği varsa Spam, yoksa Normal” kuralı da dört satırla uyusur (1. e-posta “bedava”, 3. e-posta şifre isteği yüzünden Spam) ama kahve örneğinde “bedava” yüzünden Spam der, yanılır. Aynı veriyle uyuşan birden çok kural olabilir; hangisinin doğru olduğunu ancak yeni örnekler gösterir.

ŞEKİL 3.2 · HANGİ ÖĞRENME TÜRÜ?

Kendini sına. 1 → Denetimli: fotoğraflar etiketli, doğru cevap baştan verilmiş. 2 → Denetimsiz: kimse “bu müşteri şu grup” dememiş; makine benzerliğe göre kendi gruplar (kümeleme). 3 → Pekiştirmeli: robot dener, düşer, ayakta kaldıkça ödül alır; doğru hareket listesi

yok. 4 → Denetimli: geçmiş satışların fiyatı belli; etiket bir miktar (fiyat) olduğu için regresyon. 5 → Pekiştirmeli: skor ödüldür; hangi hamlenin doğru olduğu söylenmez, deneyerek öğrenir. 6 → Denetimsiz: haberler etiketsiz; benzer konular kümelenir. Ek sorular: “Etiketli/etiketsiz” sözcüğü 1, 4 ve 6’da geçiyor. Ötekilerde etiketin varlığını görevin doğasından anlarsın: 2’de “benzerliklerine göre” diyor, kimse grup adı vermiyor; 3 ve 5’te ise doğru cevap öğretmenden değil ortamdan, yani ödülden geliyor; ikisini aynı kutuya koyan da bu. **Kendin dene.** 1) Çeviri programı: denetimli; insan çevirileri etiket görevi görür. Kendi kendine oynayan satranç programı: pekiştirmeli; kazanmak ödüldür. Fişlerden ürün grupları: denetimsiz; kimse grupları önceden söylemez. 2) Makul bir sıra: önce “doğru cevaplar verilmiş mi?”; evetse denetimli, dur. Hayırsa “ödül ya da ceza var mı?”; evetse pekiştirmeli, hayırsa denetimsiz. Başka sıralar da çalışır; önemli olan her yaprağa tek türün düşmesi. 3) Örnek: bir dil modelinin bir metinde sonraki kelimeyi tahmin etmesi. Etiket (sonraki kelime) verinin kendisinden gelir, kimse elle etiketlemez. Denetimli gibi eğitilir ama etiketsiz veriyle çalışır: öz-denetimli öğrenme, iki kutunun arasındadır.

ŞEKİL 3.3 · İKİ TEMEL GÖREV

Kendin dene. 1) $0.55 \cdot 6.5 = 3.575$; $3.575 + 0.76 = 4.335 \approx 4.34$. 2) (4.5, 3.5): sınır $x = 4.5$ 'te $6.1 - 2.7 = 3.4$ der; $3.5 > 3.4$, nokta sınırın üstünde, turuncu. (5, 3): sınır $6.1 - 3 = 3.1$ der; $3 < 3.1$, altında, koyu. İkisi de sınıra çok yakın; olasılık veren ve iyi kalibre edilmiş bir model bu noktalara düşük güven verir, ama her modelin güven sayısı bu kadar anlamlı değildir. 3) Eğim artar: on noktayla $m \approx 0.71$, $b \approx 0.23$ (eskiden 0.55 ve 0.76). Tek bir uzak nokta doğruyu önceki dokuz noktadan uzaklaştırır; en küçük kareler, hataların karesini aldığı için aykırı değerlere duyarlıdır. Sorun da bu: önce (9, 9) ölçüm hatası mı, gerçek mi, bakmak gerekir.

ŞEKİL 3.4 · ETİKETSİZ VERİYİ GRUPLA

Kendin dene. 1) B'nin yeni merkezi: $x = (7.5 + 6.5 + 7.8 + 6.8 + 8) / 5 = 7.32$, $y = (3.2 + 2.5 + 3.8 + 4 + 2.8) / 5 = 3.26$. Kayma $\sqrt{(0.32^2 + 0.26^2)} \approx 0.41$. 2) (4.5, 5.5): A'ya $\sqrt{(2^2 + 1.5^2)} = 2.5$, B'ye

$\sqrt{(2.5^2 + 2.5^2)} \approx 3.54$. En yakın merkezi A. Ama uzaklık 2.5, eşik 2.4'ün üstünde; kurala göre aykırı sayılır, A'nın öteki üyelerinden (en çok 1.22) iki kat uzak. 3) Merkeze uzaklık ve bir eşik: burada 2.4, yani A'nın en uzak sıradan üyesinin (1.22) yaklaşık iki katı. Eşiği de merkezleri de veri değil, biz seçtik; gösterimde aykırı nokta önceden belirlenmiştir, standart k-ortalamlar onu A'ya katardı. Kümeleme etiketsizdir; ama “ne kadar uzak, aykırıdır?” sorusunun cevabı insanın kararıdır.

ŞEKİL 3.5 • KAYIP VADİSİNDE İNİŞ

Kendin dene. 1) Eğim $0.36 \cdot (2.06 - 5) = -1.06$. Yeni $x = 2.06 + 0.18 \cdot 1.06 = 2.25$. Kayıp $0.18 \cdot (2.25 - 5)^2 + 0.1 = 1.46$. 2) $\eta = 6$: 1. adım $x = 0.6 + 6 \cdot 1.584 \approx 10.10$; eğim $0.36 \cdot 5.10 = 1.84$; 2. adım $x = 10.10 - 6 \cdot 1.837 \approx -0.92$. Dibe uzaklık $4.4 \rightarrow 5.1 \rightarrow 5.9$: büyüyor. Top uzaklaşıyor, ıraksama. (Çarpan $1 - 2.16 = -1.16$, mutlak değeri 1'den büyük.) 3) Evet: $\eta = 1 / 0.36 \approx 2.78$. O zaman $x - (x - 5) = 5$ olur, tek adımda dip. Bu, vadi tam parabol olduğu için mümkün; gerçek kayıp yüzeylerinde eğrilik her yerde farklıdır ve tek adımda dibe indiren bir oran yoktur.

ŞEKİL 3.6 • AYNI VERİ, ÜÇ MODEL

Kendin dene. 1) Eksik uyum: $3.1 - 1.8 = 1.3$. Temsili eğri: $3.0 - 1.6 + 0.15 \cdot \sin(6) \approx 1.36$. Kırık çizgi: $x = 9$ 'da biter, $x = 10$ için hiçbir şey söyleyemez; son parçayı uzatırsan 2.6 der, yani son iki noktanın rastgele yükselişini geleceğe taşır. 2) $x = 2$: kırık çizgi (1, 3.2) ile (3, 3.0) arasında 3.1 der, gerçek 2.4, hata 0.7; düz doğruyu kalan yedi noktaya en küçük kareler yöntemiyle yeniden uydurursan $y \approx 3.19 - 0.15x$ olur ve 2.88 der, hata 0.48. $x = 8$: kırık çizgi 2.15 der, hata 0.75; aynı doğru 1.95 der, hata 0.55. Yine ezberci kaybeder. 3) Modelin eğitim noktalarını hatırladığını kanıtlar; yeni bir noktada iyi olacağını kanıtlamaz. Saklama sınavı bunu gösterdi: eğitim hatası sıfırken doğrulama hatası düz doğrununkinden büyük.

Bölüm 4 cevapları

ŞEKİL 4.1 · NÖRONU ÇALIŞTIR

Kendin dene. 1) Toplam $0.7 + (-0.5) + 0.9 - 0.3 = 0.80$; sigmoid 0.690. 0.5'in üstünde, nöron ateşler (ReLU ile 0.800, yine ateşler). 2) Toplam $0.9 \times 1 - 0.3 = 0.60$; ReLU çıktısı 0.600 (sigmoid 0.646). Ateşler. 3) Toplam $-0.5 \times 1 - 0.3 = -0.80$; sigmoid 0.310. 0.5'i geçmez; karar kuralına göre nöron sessizdir, ama çıktısı sıfır değil, 0.310. x_2 tek başına yalnızca fren yapar.

ŞEKİL 4.2 · CANLI SİNİR AĞI (İLERİ BESLEME)

Kendin dene. 1) Bütün girdiler 0 olunca her gizli nöronun toplamı 0, sigmoid(0) = 0.50; dördü de yarı parlaklıkta. Ç1: $(0.7 - 0.5 + 0.6 + 0.4) \times 0.50 = 0.60$, sigmoid 0.65. Ç2: $(-0.4 + 0.6 + 0.5 - 0.6) \times 0.50 = 0.05$, sigmoid 0.51. Kazanan yine Ç1. 2) G1: $0.6 - 0.4 + 0.8 = 1.00$; sigmoid 0.73. 3) Hayır. Sekiz düzenin hepsinde Ç1 kazanır; en yakın yarış [0, 1, 0] durumunda 0.61'e 0.59. Örnekler: [1, 1, 0] için Ç1 0.61, Ç2 0.56; [0, 0, 1] için Ç1 0.71, Ç2 0.48; [1, 1, 1] için Ç1 0.68, Ç2 0.53. Ağırlıklar eğitilmediği için ağ hep aynı kapıyı gösteriyor.

ŞEKİL 4.3 · HATADAN ÖĞREN (GERÇEK EĞİTİM)

Kendin dene. 1) Çıktı toplamı $0.8080 \cdot 0.5957 + 0.0279 \cdot 0.5994 + 0.5427 = 0.4813 + 0.0167 + 0.5427 = 1.0407$; $\hat{y} = \text{sigmoid}(1.0407) = 0.7390$. Tabloyla aynı. 2) $b_2 = 0 - 1 \cdot (-0.0691) = 0.0691$; $\eta = 2$ ile 0.1383'tü. Adım yarıya iner: aynı gradyan, daha küçük hareket, daha yavaş iniş. 3) Hayır; en fazla "bu örnekte eğitim hatası küçüldü" diyebilirsin. 0.061, tek bir girdi için ölçülmüş eğitim hatasıdır; ağırlık hiç görmediği örneklerde ne yapacağını söylemez. Öğrenip öğrenmediği ancak yeni örneklerle sınıranır.

ŞEKİL 4.4 · EVRİŞİM: FİLTREYİ KAYDIR

Kendin dene. 1) -3. Pencere 5-7. satırlar, 2-4. sütunlar: sağ sütun (görüntünün 4. sütunu) üç satırda da 1, çekirdeğin sağ sütunu -1; üç

kez -1. 2) 0. Pencere 3–5. satırlar, 3–5. sütunlar: üst satır $[0, 1, 0] \times [1, 1, 1] = 1$; orta satır çekirdekten sıfır; alt satır $[0, 1, 0] \times [-1, -1, -1] = -1$; toplam 0. 3) Harita boyutu $(7 - 5 + 1) = 3$; $3 \times 3 = 9$ hücre.

ŞEKİL 4.5 · HAFIZALI İŞLEME (GERÇEK YİNELEME)

Kendin dene. 1) $W_h \cdot h_0$ terimi; $h_0 = 0$ olduğu için sıfırdır. İlk adımda gizli durum yalnız $W_x \cdot x_1$ 'den, yani W_x 'in ilk sütunundan gelir: $h_1 = \tanh([0.9, -0.6, 0.2, -0.8])$. 2) Üçüncü adımda “o” işlenir; $W_x \cdot x_t$ artık 4. sütundur: $[-0.7, 0.4, 0.6, 0.9]$. $W_h \cdot h_2 = [-0.14, 0.53, -0.12, -0.06]$ eklenince toplam $[-0.84, 0.93, 0.48, 0.84]$; $h_3 = [-0.68, 0.73, 0.45, 0.69]$. Tablodaki $[0.16, 0.32, 0.53, -0.58]$ 'den bambaşka: aynı kelimeler, farklı sıra, farklı hafıza. 3) Evet; sırayı umursamayan model (kelime torbası) iki cümle için aynı temsili üretir, çünkü kelimeler birebir aynı (köpek, adamı, ısırdı), yalnız sıra farklı. RNN her kelimeyi önceki hafızayla işlediği için farklı sırada farklı gizli durumlar üretir; iki cümle onun için ayrı şeylerdir. (“Adam köpeği ısırdı” farklı bir çift olurdu: hâl ekleri değiştiği için kelime torbası bile onu ayırt eder.)

ŞEKİL 4.6 · ÜRETİCİ İLE AYIRT EDİCİ

Kendin dene. 1) Tur 5; sahte olasılığı yüzde 40, ilk kez yüzde 50'nin altına iner. 2) Kural: $p(r) = 95 - 11 \times r$. $r = 8$ için $95 - 88 = 7$. Bu sayı şeridin kuralından gelir; ölçülmüş bir olasılık değildir. 3) Ayırt edici sahteyi gerçekten ayıramıyor; yazı tura atıyor. İdeal, iyi eğitilmiş bir ayırt edici varsayımıyla bu, oyunun dengesiyle uyumludur (teknik dilde $D(x) = 1/2$): üretici gerçek dağılımı yakalamıştır. Ama tek başına yüzde 50 bunu kanıtlamaz; ayırt edici öğrenememiş de olabilir.

Bölüm 5 cevapları

ŞEKİL 5.1 · CÜMLENİ TOKEN'LARA BÖL

Kendin dene. 1) 6 token: Yapay · zekâ · öğre · ##niyo · ##r · . (“öğreniyor” dokuz harf olduğu için üçe bölünür; nokta ayrı sayılır). 2) 5 parça: Bilg · ##isay · ##arla · ##rımı · ##zla (19 karakter, dörder karakterlik beş parça; sonuncusu üç karakter kalır). 3) Üçüncü

cümlede $11 / 4 = 2.75$ token/kelime; birinci cümlede $7 / 4 = 1.75$. Aynı kelime sayısı, yüzde 57 daha çok token: uzun kelimeler farkı yaratıyor.

ŞEKİL 5.2 · ANLAM HARİTASI

Kendin dene. 1) Farklar 66 ve 54; kareleri 4356 ve 2916; toplam 7272; karekök ≈ 85.3 . Tablodaki en büyük aile içi uzaklık 52.8 (köpek–kuş, ekmek–peynir de 42.8); 85.3 bunun çok üstünde; “kuş” ile “peynir” farklı mahallelerde. 2) Açık uçlu; ölçüt, üç hayvanın yakınına koymak. Örnek: (70, 75). Bu noktadan kedi (58, 62) 17.7, köpek (84, 50) 28.7, kuş (52, 92) 24.8 uzakta; en yakın iki komşu kedi ve kuş olur. 3) “ekmek” için üçüncü en yakın kelime “prens” (54.4), krallık ailesinden. Yiyecek ve krallık mahalleleri haritada birbirine değiyor; aileler ayrı ama aralarında keskin bir duvar yok.

ŞEKİL 5.3 · HANGİ KELİME HANGİSİNE BAKIYOR?

Kendin dene. 1) Evet, beş satırın hepsi 1.00: Kedi $0.50+0.30+0.05+0.10+0.05$; kaçtı $0.50+0.30+0.10+0.05+0.05$; çünkü $0.20+0.40+0.20+0.10+0.10$; o $0.55+0.10+0.05+0.20+0.10$; korkmuştu $0.30+0.10+0.05+0.40+0.15$. 2) En koyu hücre “köpek”e kayardı; “korkmuştu” fiili öznesini arar ve artık özne bir zamir değil, doğrudan “köpek”. “Kedi”ye giden pay da düşerdi, çünkü korkan artık kedi değil. 3) “çünkü” sütunu her satırda 0.05–0.20 arasında; bu temsili tabloda hiçbir kelime ona yaslanmıyor. Bağlaç iki olayı birbirine bağlar ama kendi başına kim, ne, nerede sorularına cevap taşımaz; tablo bu sezgiye göre kurulmuş. Kesin bir hüküm çıkmaz: ağırlıklar elle seçilmiştir ve gerçek bir modelde tek bir dikkat tablosundaki düşük ağırlık, kelimenin anlam yükü hakkında tek başına hüküm vermez; başka bir baş aynı bağlaca yüksek ağırlık verebilir.

ŞEKİL 5.4 · KELİME KELİME ÜRET

Kendin dene. 1) İkinci adımda birikimli toplamalar 0.34 (hızlı) ve 0.62 (güçlü); $U = 0.50$ ilk eşiği geçip ikincinin altında kalır, seçim “güçlü” olur. Üçüncü adım değişmez ($U = 0.12 \rightarrow$ gelişiyor); cümle “Yapay zekâ artık güçlü gelişiyor.” 2) Açgözlü: $0.42 \times 0.38 \times 0.50 \approx 0.080$

(yüzde 8). Örneklemme cümlesi: $0.28 \times 0.20 \times 0.50 = 0.028$ (yüzde 3). Ağgözlü cümle yaklaşık üç kat daha olası; örneklemme daha az olası yolları da açıyor, en olası cümleyi de seçebilir. 3) İkinci adım: en olası aday yalnızca yüzde 38 (birinci adımda 42, üçüncüde 50). Pay ne kadar düşükse olasılık öteki adaylara o kadar yayılmış, model o kadar kararsız.

ŞEKİL 5.5 · ÜÇ AŞAMADA BİR ASİSTAN

Kendin dene. 1) Açık uçlu; ölçüt, birincinin sürdürmesi, ikincinin cevaplayıp durması. Örnek: birinci aşama “Su deniz seviyesinde 100 °C’de kaynar ve bu sıcaklık yükseklikle düşer. Kaynama noktası...” gibi durmak bilmeyen bir ansiklopedi cümlesi yazar. İkinci aşama soruyu cevaplar ve durur: “Su deniz seviyesinde 100 °C’de kaynar.” 2) “Doğru ama kaba” tercih edilmeli; “dürüst” hedefi doğruluğu, “yardımcı” hedefi de işe yarar bilgiyi önde tutar. Kibarlık üçüncü aşamanın ayrı bir kazanımıdır: ideal cevap hem doğru hem kibar olandır, ama ikisi çatışırsa doğruluk önce gelir. Yanlış ama kibar cevabı tercih eden bir ödül sinyali, modeli halüsinasyona teşvik eder. 3) “Örnek çıktı” satırı: üç sütunda da bilgi aynı (Ankara), değişen yalnız cevabın uzunluğu ve tonu. Bu üç örnek aynı bilginin farklı yanıt biçimlerini gösteriyor. Ama üç hazır cümleden ince ayarın bilgiyi değiştiremeyeceği sonucu çıkmaz; ince ayar bilgiyi ve görev başarımını da değiştirebilir, örnek bunu göstermek için kurulmamıştır.

ŞEKİL 5.6 · GÜRÜLTÜDEN GÖRSELE

Kendin dene. 1) Rastgele açılan 29 pikselin turuncu olma payı $40 / 64 = 0.625$; beklenen $29 \times 0.625 \approx 18$ turuncu piksel (şekildeki sabit sıralamada 17 çıkar). 2) $100 / 16 = 6.25$, her adımda yüzde altı çeyrek ilerleme; şerit iki kat uzar, her kare bir öncekinden daha az farklı olur. 3) İleri süreç (görsele gürültü ekleme) şeritte sağdan sola okunur: temiz kalpten karıncalı ekrana. Ters süreç, modelin öğrendiği temizleme, soldan sağa okunur.

ŞEKİL 5.7 · BAĞLAM PENCERESİ

Kendin dene. 1) $12 - 5 = 7$ kelime unutulur. Pencerede kalanlar: “tutar ve eskiyi zamanla unuttur”. Cümleinin öznesi ve nesnesi tamamen gitmiş; model yalnızca yüklem tarafını görüyor. 2) 19 token: Yapay · zekâ · mode · ##ller · ##i · metni · sını · ##rlı · bir · penc · ##ered · ##e · tutar · ve · eskiyi · zama · ##nla · unuttur · . Pencere 8 token sayıyor olsaydı beşinci kelimede (“sınırlı”, 8. token) dolar, altıncı kelime (“bir”) girince “Yapay” dışarı düşerdi. Kelime saymaya göre üç kelime erken dolar; yaklaşık 1.6 kat. 3) Açık uçlu. Hata vermek en dürüst seçenek ama işi yapmaz; genelde özetleme: kırpma belgenin bir kısmını (çoğu zaman sonunu ya da başını) tamamen atar, özetleme her bölümden bir iz taşır. Ancak sorun belgenin belli bir parçasıyla ilgiliyse yalnız o parçayı kesip vermek daha isabetli olabilir.

Bölüm 6 cevapları**ŞEKİL 6.1 · BİR İSTEM İNŞA ET**

Kendin dene. 1) İki parça: gösterge = $40 + 2 \cdot 15 = \%70$, düzey orta. Temsili cevap orta düzey metni: “Deniz kenarında bir destinasyon öneriyorum: sabah plaj, öğleden sonra kısa bir kasaba turu, akşam balık restoranı. Bütçeye uygun bir pansiyon seçebilirsiniz.” Kural parçanın hangisi olduğuna bakmaz, sayısına bakar. 2) En az üç parça (%85). İki parça %70’te kalır; yüksek eşiği %85. 3) Açık uçlu. Örnek: Rol: “Sen bir müşteri ilişkileri uzmanısın.” Bağlam: “Müşterinin siparişi iki gün gecikti, özür dileyip yeni tarihi bildiriyorum.” Örnek: “Sayın ..., ... için özür dileriz; yeni teslim tarihi” Format: “Üç kısa paragraf; ilkinde özür, ikincisinde yeni tarih, üçüncüsünde iletişim bilgisi.”

ŞEKİL 6.2 · KAYNAĞA DAYALI CEVAP

Kendin dene. 1) “civarındadır”, “şirketinize göre değişebilir”, “sanırım”, “muhtemelen”, “emin değilim”. Kaynaklı cevaplarda böyle bir kelime yok; onun yerine madde numarası (§4, §7, §2) var. Ama

kesin ton tek başına doğruluk ölçütü değildir. İyi bir kaynaklı cevabın ölçütleri: atıf verilen parça iddiayı gerçekten destekliyor mu, kaynak doğru mu ve kaynak eksik ya da belirsizse cevap bunu söylüyor mu? Kaynak verilmesi belirsizliği ortadan kaldırmaz; gerektiğinde “belgede bu ayrıntı yok” demek doğru cevaptır. 2) “Belgelerde bu konuda bir madde bulamadım” demeli ve tahmin etmemeli. Getirme boş dönünce iyi bir sistem bunu açıkça söyler; boşluğu ezberle doldurmaz. 3) “İK Politikası §4’e göre yıllık izin 5 yıldan sonra 26 güne çıkar; altı yıllık bir çalışan için 26 gün.”

ŞEKİL 6.3 · BİR AJANI İZLE

Kendin dene. 1) Adım 1: düşünce “Önce toplam: 4×200 .”, araç hesap_makinesi(" $4 * 200$ "), gözlem 800. Adım 2: düşünce “Kişi başı: $800 \div 5$.”, araç hesap_makinesi(" $800 / 5$ "), gözlem 160. Adım 3: araç yok; son cevap “Toplam 800 TL; 5 kişiye bölününce kişi başı 160 TL.” 2) Ajan 450’yi doğru kabul eder; adım 2 hesap_makinesi(" $450 / 6$ ") → 75 olur ve son cevap “Toplam 450 TL, kişi başı 75 TL” çıkar. Ajan aracın çıktısını sorgulamaz; bu yüzden araçlar güvenilir olmalı ve gözlemler ayrıca doğrulanmalı. 3) En az bir çağrı daha: hesap_makinesi(" $90 + 30$ ") → 120. Son cevap: pizza toplamı 540 TL, kişi başı 120 TL ($90 \text{ pizza} + 30 \text{ içecek}$). İçecekli genel toplam da istenirse bir çağrı daha gerekir: hesap_makinesi(" $6 * 120$ ") → 720.

ŞEKİL 6.4 · BİR YZ UYGULAMASININ PARÇALARI

Kendin dene. 1) “Dün söylediğim tarih” → bellek. “Bugün dolar kaç?” → araçlar (güncel kur için arama ya da API). “Şirketin iade politikası” → bilgi tabanı (şirket belgesi, RAG). 2) Kutular değişmez; yalnız “model çağırısı” etiketinin arkasındaki model değişir. Beş kutu aynı kalır; değişen şey orkestrasyondaki ayarlardır: istem, yedek yöntem geçiş (fallback), değerlendirme. 3) Garson → arayüz; müdür → orkestrasyon; şef → model çağırısı; kiler → bilgi tabanı; tezgâh aletleri → araçlar; müdavim defteri → bellek.

ŞEKİL 6.5 · ALANLARI KEŞFET

Kendin dene. 1) Açık uçlu; Adım adım’daki ikinci tablo bir örnek eşleme, başkaları da savunulabilir. İki ya da daha çok beceriye

girenler: “Hasta notlarını özetleme ve kodlama” (özetleme + sınıflandırma), “Belge/sözleşme analizi ve risk skorlama” (getirme + tahmin), “İlaç keşfinde aday molekül tarama” (tahmin + tanıma), “Simülasyon ve hipotez üretimi” (tahmin + üretme), “Sesli asistanlar ve özetleme” (tanıma + üretme + özetleme; ajan ancak araç kullanıp eylem yapıyorsa), “Öneri sistemleri” (tahmin; getirme değil). “Büyük veri kümelerinde örüntü keşfi” beşliye tam oturmaz: etiketsiz veride yapı arama. 2) Örnek seçim: Sağlık: tıbbi görüntüde anormallik tespiti (gözden kaçan bulgu hayata mal olur). Finans: dolandırıcılık tespiti (kaçan işlem para, yanlış alarm müşteri kaybettirir). Üretim: kestirimci bakım (kaçan arıza bandı durdurur, iş güvenliğini tehdit eder). Bilim: protein yapısı tahmini (yanlış yapı yıllarca yanlış deneye yol açar). Sanat: üslup aktarımı ve restorasyon (özgün eser geri dönüşsüz bozulur). Günlük: sesli asistanlar ve özetleme (yanlış özet yanlış karara götürür). 3) Açık uçlu. Örnek: gelen müşteri e-postalarını aciliyetine göre sınıflama → beceri tanıma, alan günlük ya da finans; gereken kutular: arayüz, orkestrasyon, bilgi tabanı (geçmiş yanıtlar), bellek (müşteri geçmişi).

Bölüm 7 cevapları

ŞEKİL 7.1 • ÖNYARGI SİMÜLASYONU

Kendin dene. 1) $A = 50 + 0.4 \cdot 75 = 80$, $B = 50 - 0.4 \cdot 75 = 20$; parite farkı 60 puan. 2) Yüzde 9’da. Yüzde 8’de oranlar 53/47 (fark 6, hâlâ “dengeli”); yüzde 9’da 54/46 olur, fark 8 ve gösterim etiketi çarpığa çevirir. 3) Hayır. Ölçeğin ucunda ($e = 100$) A yüzde 90’da, B yüzde 10’da kalır. Yüzde 95 ve yüzde 5 sınırlarına ulaşmak için e’nin 112.5 olması gerekirdi; ölçek 100’de bitiyor. Sınırlar gösterimin kuralında emniyet kemeri olarak duruyor, hiç devreye girmiyor.

ŞEKİL 7.2 • BEYAZ KUTU: KARARI AÇIKLA

Kendin dene. 1) Evet, değişir. Toplam $-8 + 10 = +2$; sıfırdan büyük olduğu için kredi onaylanır. Tek etkendeki 10 puanlık düzelme kararı çevirir, çünkü borç tek başına değil, -12 ile birlikte reddetti-yordu;

bu soru katkının doğrudan toplama eklendiği toplamsal bir oyuncak model varsayar. Gerçek bir modelde bir girdiyi değiştirmek bütün katkıları yeniden hesaplatır; SHAP katkıları sonucun nasıl değişeceğini tek başına garanti etmez. Katkılar temsili; gerçek bir modelde taban değer ve ölçek ayrıca belirtilir. 2) -90 ya da daha düşük (büyüklüğü en az 90 puan). Artı etkenler $+40 + 28 + 22 = +90$ ediyor; eksi etken -90 olunca toplam 0 olur, “sıfırdan büyük” koşulu sağlanmaz ve karar redde döner. 3) Yaklaşık yüzde 70. Çubuk uzunluğu en büyük mutlak katkıya oranlanır: $32 / 46 \approx 0.70$.

ŞEKİL 7.3 · GERÇEK Mİ, YAPAY MI?

Kendini sına. Her kartta iki soru ayrı puanlanır: şüphe işaretini gördün mü, bağımsız bir doğrulama kanalı yazdın mı? Üretim yöntemi (gerçek kayıt mı, yapay mı) ile olayın doğruluğu iki ayrı eksendir; ipucu seni incelemeye götürür, kökene tek başına hükmetmez. 1) Video → Şüpheli: doğrula. İşaret: dudak hareketleri ile ses uyuşmuyor; bu bir deepfake izi olabilir, ama kötü sıkıştırma ya da dublaj da aynı görüntüyü verir. Kanal: konuşmanın tam kaydı, kişinin ya da kurumun resmî hesabı, en az bir haber ajansı. Cümlelerin söylenip söylenmediği, videonun nasıl üretildiğinden ayrı doğrulanır. 2) Telefon → Şüpheli: doğrula; işlem yapma. İşaret: aciliyet baskısı ile olağandışı istek bir arada; ses benzerliği kanıt değil, robotik tonlama da kesin kanıt değil. Kanal: aramayı kapat, patronu kendi bildiğin numaradan geri ara, transferi ikinci bir kişiye onaylat. Sesin gerçek mi klon mu olduğunu bilmesen de karar aynı: doğrulamadan para gitmez. 3) Haber → Gerçek görünüyor; olay doğrulanmış. İşaret: birden çok bağımsız kaynak ve izlenebilir köken olayın doğruluğunu destekler. Bu, metni insanın mı yapay zekânın mı yazdığını söylemez; yapay yazılmış doğru bir haber de olabilir. Kanal: kaynakların kendi sayfaları, varsa birincil belge. 4) Fotoğraf → Şüpheli: doğrula. İşaret: altı parmak ve anlamsız yazı, üretken modellerin bilinen izleri; ama fotoğraf düzenleme ya da gerçek bir anomali de olabilir. Kanal: tersine görsel arama ile ilk yayımlayanı bul; varsa içerik kimlik bilgisi (C2PA) kaydına bak. Kayıt yokluğu sahtelik kanıtı değildir. Belirlenemez ne zaman doğru cevap: ipucu yok, kaynak yok, doğrulayacak kanal yok. O zaman hüküm verme;

paylaşma. **Kendin dene.** 1) Örnek iki adım: aramayı kapat ve patronu kendi bildiğin numaradan geri ara; transferi ikinci bir kişiye ya da yazılı bir kanala (kurum e-postası, yüz yüze) doğrulatmadan yapma. Aciliyet baskısının kendisi bir uyarı işaretidir. 2) İyi bir cevap üç soruya da somut karşılık verir: içeriği ilk kim yayımladı, hangi hesaptan/siteden geldi, en az bir bağımsız kaynak aynı şeyi söylüyor mu. Cevapsız kalan soru, içeriği paylaşmadan önce kapatman gereken boşluktur. 3) Hayır. Dört karttaki izler bugünün modellerinin zayıflıklarıdır; üretim geliştikçe altı parmak ve bozuk dudak senkronu kaybolur. İz yokluğu gerçeklik kanıtı değildir; kaynak doğrulama yerine geçmez.

ŞEKİL 7.4 · RİSKİ SINIFLANDIR

Kendini sına. Kademeler, 27 Temmuz 2026 tarihli konsolide AB YZ Yasası metnine göre (erişim 1 Ekim 2026); dört basamak öğretici bir özetir, her örnek amaca, aktöre ve ilgili maddeye bağlıdır. 1) Sosyal davranış puanıyla ilgisiz alanlarda orantısız yaptırım uygulayan devlet sistemi → Yasak: Madde 5(1)(c), kişileri sosyal davranışlarına ya da kişisel özelliklerine göre puanlayan ve bu puanla (i) verinin toplandığı bağlamla ilgisiz alanlarda ya da (ii) davranışın ağırlığıyla orantısız veya gerekçesiz olumsuz muameleye yol açan sistemleri yasaklar; yasak yalnız kamu otoritelerine özgü değildir. Senaryo bu iki koşulu da taşıdığı için yasak kademededir. Her puanlama otomatik olarak yasak değildir; koşullar aranır. 2) İşe alımda adayları otomatik eleyen sistem → Yüksek: Ek III 4(a), işe alım ve seçmede aday eleyen sistemleri sayar; sıkı uyum ve insan gözetimi gerekir. 3) Müşteriyle konuşan sohbet botu → Sınırlı: Madde 50(1), kişinin bir YZ ile etkileştiğini bilmesini ister. Bu “sadece şeffaflık” demek değildir; kişisel veri ve tüketici hukuku ayrıca geçerlidir. 4) E-postada spam filtresi → Minimal: verilen bilgilerle Ek III'teki bir yüksek-risk kategorisi gösterilemiyor; yasa bu kullanıma özel yükümlülük koymuyor. 5) Kredi başvurusu değerlendiren model → Yüksek: Ek III 5(b), gerçek kişilerin kredi değerliliğini değerlendiren ya da kredi puanı belirleyen sistemleri kapsar; finansal dolandırıcılık tespiti bu bentten istisnadır. İşe alımla aynı basamak. 6) Oyun içindeki rakip yapay zekâ → Minimal: 4 ile aynı gerekçe; bu özel yüksek-risk

kategorisi gösterilemiyor. **Kendin dene.** 1) Harita uygulaması ve klavye önerisi → minimal; verilen bilgilerle bir yüksek-risk kategorisi gösterilemiyor. Bankanın dolandırıcılık uyarısı: finansal dolandırıcılık tespiti, kredi değerliliğine ilişkin Ek III 5(b) sınıfından istisnadır; hesabı otomatik dondurmak ciddi hak etkileri yaratabilir, ama bu tek başına sistemi yüksek-risk sınıfına sokmaz. Sistemin amacı, diğer uygulanabilir hükümler ve veri koruma yükümlülükleri ayrıca değerlendirilir. Kitabın “kişi hakkında karar veriyor mu?” sorusu öğretici bir merdiven; hukuki sınıflandırmayla aynı şey değil. 2) Evet. Telefon kilidini açan yüz tanıma, yalnız iddia edilen kimliği doğrular; Ek III 1(a)’daki uzaktan kimlik tespitinden ayrıdır ve bu özel yüksek-risk kategorisi gösterilemiyor. Kamuya açık alanda, kolluk amacıyla, gerçek zamanlı uzaktan biyometrik kimlik tespiti ise Madde 5(1)(h) ile yasaktır; sınırlı istisnaları (hedefli arama, ağır suç, izin usulü) ve koşulları vardır. Kolluk dışı kullanım bu yasağın kapsamında değildir ama serbest de değildir: GDPR Madde 9 ve Ek III 1(a) devreye girer. Teknoloji aynı, amaç ve hüküm farklı. 3) Kitabın ölçütüne göre kişi hakkında karar verenler: 1, 2, 5; böyle bir karar vermeyenler: 3, 4, 6. İkinci küme hakları hiç etkilemez demek değildir: sohbet botu yanıltabilir ya da kişisel veri işleyebilir, spam filtresi önemli bir e-postayı gözden kaçırabilir; bu kullanımlar şeffaflık, kişisel veri ve tüketici kurallarına yine tabidir. Karar verenlerden biri (1) yasak, ikisi (2 ve 5) yüksek basamakta; yüksek basamakta iki kart var. Karar vermeyenlerden 3 sınırlı, 4 ve 6 minimal. Bu sayım kitabın öğretici ölçütüyle yapıldı; yasadaki sınıf madde ve eke göre belirlenir.

ŞEKİL 7.5 • HEDEF İLE NİYET

Kendin dene. 1) Örnek: “Odadaki bütün çöpü çöp kutusuna koy; halı altı ve dolap içi dahil hiçbir yerde çöp kalmasın.” Yeni açıklar, talimatın her maddesini sağlayıp niyeti bozanlar: çöp olmayan eşyayı da çöp sayıp kutuya atmak; bütün çöpü kutuya koyup dolu kutuyu odanın ortasında ya da koridorda bırakmak (her madde sağlandı, oda yine kullanışsız); geri dönüşüm ve tehlikeli atık ayrımı yapmadan hepsini tek kutuya atmak (yazılmamış kural çiğnendi). Çöpü pencereden atmak bu listeye girmez: kutuya koymadığı için talimatı doğrudan ihlal eder, oyunlamaz. Her yeniden yazım bir açığı

kapatır, bir yenisini bırakır. Ders bu. 2) Zararsız kısa yol: her öğrencinin zayıf olduğu konulara ek alıştırma önermek. Zararlı kısa yol: düşük notlu öğrencileri sınava sokmamak ya da doğrudan sınav sorularını ezberletmek; ortalama yükselir, öğrenme yükselmez. 3) Örnek cümle: “Sistem verdiği ölçütü en üst düzeye çıkarır, ölçütün temsil etmesi gereken niyeti değil.”

Bölüm 8 cevapları

ŞEKİL 8.1 · İNSAN MI, MAKİNE Mİ?

Kendin dene. 1) Arkadaşın genelde kalıplı üsluba ve kişisel ayrıntı yokluğuna bakar; aşağıdaki ipuçlarının aynısı. Ayırt edebilmesi, ipuçlarının sağlam olduğunu değil, senin örneklerinin kolay olduğunu gösterir. 2) Muhtemelen hayır. Bugünkü sohbet modelleri istenirse duraksayabilir, yuvarlayabilir, hatta hata yapabilir; “kusursuz aritmetik” ipucu eskiyor. 3) Evet, savunulabilir. Turing’in kendi makalesinde de makine, aritmetik soruya kasıtlı yanlış cevap verir; çünkü insanı taklit etmek, insanın kusurlarını da taklit etmektir.

Kendini sına. Dört yazışma kurgusal; aşağıdaki “rol”, yazarın kurguda o cevabı kime yazdığınıdır, kesin bir tanı değil. İpuçları inceleme gerekçesi; tek bir anı ya da doğru bir hesap, yazarın insan ya da makine olduğunu kanıtlamaz. 1 → Kurguda makine: Aşırı kibar, kalıplı üslup ve “bir yapay zekâ olarak” ifadesi makineyi düşündürür; ama bu kalıbı bir insan da taklit edebilir. 2 → Kurguda insan: Kişisel ayrıntı, duygu, hafif şikâyet ve doğal samimiyet insanı düşündürür; ama bir model kurgusal ayrıntı da üretebilir. 3 → Kurguda makine: Kusursuz ve tereddütsüz aritmetik makineyi düşündürür; ama bir muhasebeci de 408’i duraksamadan söyler, kâğıtta cevap süresi zaten görünmez. 4 → Kurguda insan: Belirsiz ama yaşanmış görünen ayrıntılar ve gündelik dil insanı düşündürür; aynı dil sohbet verisiyle eğitilen her modelde de vardır.

ŞEKİL 8.2 • ÇİNCE ODA'DASIN

Kendin dene. 1) Örnek satır: “你几岁？” → “Ben bir yaşındayım.” Searle’e göre hiçbir şey değişmez; satırı yazan sen anlıyorsun, oda hâlâ eşleştiriyor. Sistem yanıtı savunucusu ise “oda artık daha zengin bir sistem” der. İki cevap da tutarlı; fark, anlamayı nerede aradığında. 2) Bu, “robot yanıtı”nın küçük bir örneğidir: cevap artık dış dünyaya bağlı (grounding). Kimine göre anlamaya doğru bir adım, kimine göre yalnızca daha karmaşık bir kural. Gerekçen, “anlama” için dünyayla bağı şart görüp görmediğine bağlı. 3) Searle’ün ezberleme cevabı, sistemi tek kişiye indirger ve “yine anlamıyorum” der. Karşı çıkanlar, ezberleyen kişinin artık iki ayrı sistemi taşıdığını söyler: Türkçe konuşan sen ve Çince “konuşan” alt sistem. İkna edici bulup bulmaman, bir kafanın içinde iki anlayan olabileceğine inanıp inanmamana bağlı.

ŞEKİL 8.3 • YETENEK BASAMAKLARI

Kendin dene. 1) Evet, üçü de ilk satıra düşer. Sohbet modeli birçok görevde çalışabilir ve bağlamdaki örneklerle uyum gösterebilir; ağırlıkları normal kullanımda genellikle sabittir, ama sabit ağırlık bağlam içi uyumun yokluğu demek değildir (Brown vd., 2020). Bu yetenekler tek başına insan düzeyinde güvenilir genel öğrenme ve aktarımı kanıtlamaz; AGI sınırı için uzlaşmış bir test de yoktur. Şekil 8.3’teki tablo da bunu söyler: “Bugünkü sistemler buradadır.” 2) İyi bir ölçüt yeni bir alanda aktarımı ölçer, ör. “hiç görmediği bir işi bir insan kadar hızlı öğrenir.” Turing testinden farkı: davranışın akıcılığına değil, yeni alana uyum yeteneğine bakar. 3) Örnekler: satrançta insanı geçen program, tek bir oyunda insanüstü ama masadan kalkıp çeviri yapamayan bir sistem.

ŞEKİL 8.4 • ÜÇ ZEKÂ EĞRİSİ

Kendin dene. 1) $10 \cdot (t/10)^3 = 5 \rightarrow (t/10)^3 = 0.5 \rightarrow t/10 \approx 0.794 \rightarrow t \approx 7.9$. Eğri, zamanın yaklaşık yüzde 80’i geçtikten sonra ancak yarıya gelir; patlama son beşte birde. 2) Hayır. $e^{-(t/2.2)}$ hiç sıfır olmaz; eğri 10’a yaklaşır ama değmez ($t = 10$ ’da 9.9). Tavan bir sınırdır, varış noktası değil. 3) Hiçbiri. Yavaşlayan egride bile sistemler bugünden çok daha

güçlü bir düzeye ulaşır; belirsiz eğride ise sıçramalar öngörülemez. Hangi eğri doğru olursa olsun, güvenli ve hizalı kılmaya yatırım boşa gitmez.

ŞEKİL 8.5 · SORUMLULUK KİMDE?

Kendin dene. 1) Örnek: sağlık uygulaması yanlış doz önerir, doktor kontrol etmeden onaylar. Pay, yazılım kusuru ile denetim ihmali arasında bölünür; hukukta bu tür paylaşım olağandır ve oran ülkeye, kusura ve sözleşmeye göre değişir. 2) İki duruş da savunulabilir: birincisi bugünkü sistemler için pratik, ikincisi belirsizlik altında ihtiyatlı. 3) Somut başlangıçlar: hangi ürünü kullandığın, verini kime verdiğin, işyerinde ya da okulda hangi kuralın konmasını istediğin, oy verirken bu konudaki tutuma bakıp bakmadığın.

Kendini sına. Aşağıdaki taraf, her senaryoda ilk incelenecek aktör; kesin bir hukuk hükmü değil. Diğer tarafların payı da araştırılır; sonuç ülkeye, kusura, sözleşmeye ve role bağlıdır. 1 → (a) Üretici / geliştirici: yazılım kusuru önce onların kapısını çalar (ürün sorumluluğu). İşleten kurumun güncelleme ve denetim yükümlülüğü ile sürücünün rolü de incelenir. 2 → (b) İşleten kurum: aracı denetlemeden kullanmak önce onların sorumluluğudur (özen yükümlülüğü). Araç yanıltıcı sunulduysa üreticinin, kurum içi karar zinciri varsa çalışanların payı da sorulur. 3 → (c) Son kullanıcı: aldatma ve zarar amacı ona aittir (ceza hukuku). Aracın koruma önlemleri yoksa üreticinin, yayın kanalı yaymışsa platformun sorumluluğu da ayrıca değerlendirilir. Sahte içerik üretmek tek başına kötü niyet kanıtı değildir; bu senaryoyu ayıran, aldatma ve zarar amacı. (d) YZ'nin kendisi: üç senaryoda da yaygın görüş değil; sorumluluk bugün insanlara ve kurumlara atfedilir.

Sözlük

Bölüm numarası (ör. 3.6), terimin kitapta tanıtıldığı bölümü ve o bölüm içindeki kesimi gösterir.

AB YZ Yasası (EU AI Act) · Bölüm 7.5 Avrupa Birliği'nin yapay zekâ kullanımlarını riske göre dört kademeye ayıran düzenlemesi: kabul edilemez (yasak), yüksek, sınırlı ve minimal. Risk arttıkça yükümlülükler de artar.

Açıklanabilir yapay zekâ (explainable AI, XAI) · Bölüm 7.3 Bir modelin çıktısını insanın anlayabileceği gerekçelere bağlama çabası. SHAP, LIME gibi sonradan açıklama yöntemleri belirli bir çıktıyı, seçilen bir taban değere göre özellik katkılarıyla açıklar (SHAP, 2017: yerel doğruluk, eksiklik, tutarlılık). Kararı denetlenabilir ve itiraz edilebilir kılar; modelin tamamını şeffaf yapmaz, nedensellik kanıtı da değildir. Bkz. Kara kutu.

Ağırlık (weight) · Bölüm 4.2 Yapay nöronun her girdiye verdiği önem değeri. Eğitim, bu değerleri kaybı azaltacak yönde ayarlama işidir. Bkz. Parametre.

Ajan (agent) · Bölüm 6.4 Dil modelini araçlarla (hesap, arama, API) donatıp “düşün, aracı kullan, sonucu gözlemle, tekrar dene” döngüsüyle (ReAct) çalıştıran sistem. Konuşmakla kalmaz, işe girişir.

Aktivasyon fonksiyonu (activation function) · Bölüm 4.2 Nöronun ağırlıklı toplamını çıktıya çeviren doğrusal olmayan işlev: sigmoid, tanh, ReLU. Çıktının bir eşiği aşıp aşmadığı (“ateşledi”) ayrı bir karar kuralıdır; aktivasyon eşik demek değildir. Onsuz üst üste konan katmanlar tek bir doğrusal işleve çöker.

Algoritma (algorithm) · Bölüm 1.3 Sonlu, kesin tanımlı adımlar dizisi; bir kek tarifi gibi. Sözcük, 9. yüzyıl matematikçisi el-Harezmi'nin adından gelir.

Anomali tespiti (anomaly detection) · Bölüm 3.5 Çoğunluğun dağılımından belirgin biçimde sapan örnekleri yakalama. Bankaların

şüpheli işlemi fark etmesi büyük ölçüde buna dayanır.

Aşırı uyum (overfitting) · Bölüm 3.7 Modelin eğitim örneklerini gürültüsüyle birlikte ezberleyip görülmemiş veride başarımını yitirmesi. Ters, örüntüyü yakalayamayacak kadar basit kalan eksik uyumdur (underfitting). İyi model ikisinin ortasında durur.

Bağlam penceresi (context window) · Bölüm 5.8 Dil modelinin aynı anda tutabildiği en fazla token sayısı; küçük bir not defteri. Defter dolunca ne olacağını kullanılan sistem belirler: hata verir, metni kırpar ya da özetler. Pencereyi büyütmek pahalıdır.

Belirtim oyunlama, ödül oyunlama (specification gaming, reward hacking) · Bölüm 7.6 Sistemin verilen ölçütü en üst düzeye çıkarırken asıl amacı kaçırmaması: “çöp görünmesin” deyince çöpü halının altına süpürmek. Bkz. Hizalama.

Bilgi edinme darboğazı (knowledge acquisition bottleneck) · Bölüm 2.6 Klasik YZ’nin duvarı: dünya hakkındaki bütün kuralları elle yazmak ölçeklenmez. Kırılganlıkla (öngörülmeven durumda çökme) birlikte veriden öğrenmeye geçişi hızlandırdı.

Bilgi temsili (knowledge representation) · Bölüm 2.2 Sembolik YZ’de bilgiyi açık semboller ve aralarındaki bağlarla (“Tekir bir kedidir”) kodlama. Semantik ağlar, çerçeveler ve mantık önermeleri bunun araçlarıdır.

Büyük dil modeli (large language model, LLM) · Bölüm 5.1 Devasa metinle eğitilmiş, Transformer tabanlı dil modeli. Bu kitapta incelenen otonöregresif üretici modeller her adımda bir sonraki token için skorlar (logit) üretir; softmax bu skorları bir olasılık dağılımına çevirir, seçilen token diziye eklenir. Her dil modeli otonöregresif değildir. Bugünkü sohbet asistanlarının temeli.

Çıkarım (inference) · Bölüm 2.2 Bilgi temsili üzerinde kurallar uygulayarak açıkça söylenmemiş bilgiye ulaşma: “Tekir kedidir, kedi memelidir” bağlarından “Tekir memelidir” sonucunu türetmek. Makine öğrenmesinde aynı sözcük, eğitilmiş bir modeli yeni girdiler üzerinde çalıştırmayı da anlatır (eğitim ile çıkarım aşamaları).

Çince Oda (Chinese Room) · Bölüm 8.3 Searle’ün düşünce deneyi: kural kitabıyla kusursuz Çince cevaplar üreten ama tek kelime Çince anlamayan kişi. Sembol işlemenin anlamaya yetmediğini savunur; karşı görüşler de güçlüdür.

Dar yapay zekâ (narrow AI) · Bölüm 1.6 Belirli görevlerde çalışan, alanlar arasında aktarılabilir genel yeteneği olmayan sistem. “Dar mı?” sorusunun ölçütü kaç iş yaptığı değil, insan düzeyinde alanlar arası genel öğrenme ve aktarım gösterip göstermediğidir; istemdeki örneklerle yeni bir işe bir ölçüde uyum gösteren (bağlam içi uyum) bir model de dar sınıfta kalabilir. Bugünkü bütün YZ sistemleri, çok görevli sohbet modelleri dahil, bu sınıftadır; dar olmak tek bir işe sıkışmak demek değildir, çok iş yapabilmek de tek başına genel olmak demek değildir. Gündelik dilde “zayıf YZ” de denir; Searle’ün zayıf/güçlü ayrımı ise ayrı, felsefi bir sorudur. Bkz. Güçlü yapay zekâ.

Deepfake (sentetik medya) · Bölüm 7.4 Sentetik medya, üretken modellerle (GAN, difüzyon, ses klonlama) üretilen içeriğin genel adıdır; deepfake bunun gerçek bir kişiyi taklit eden türüdür: hiç yaşanmamış konuşma, çekilmemiş fotoğraf. Tespit bir silahlanma yarışıdır ve kesin bir “gerçek/sahte” hükmü vermez: bir ipucu incelemeye götürür, kararı bağımsız doğrulama verir. İçeriğin yapay üretilmiş olması ile anlattığı olayın doğru olması ayrı sorulardır; en sağlam savunma kaynak doğrulama ve şüpheciliktir.

Denetimli öğrenme (supervised learning) · Bölüm 3.3 Doğru cevabı verilmiş örneklerden, yani özellik-etiket çiftlerinden, girdiden çıktıya eşlemeyi öğrenme. Sınıflandırma ve regresyon bu türdendir.

Denetimsiz öğrenme (unsupervised learning) · Bölüm 3.3 Hiç cevap verilmeden, yalnızca girdilerden yapı çıkarma. Kümeleme ve boyut indirgeme örnekleridir.

Depolanmış-program ilkesi (stored-program principle) · Bölüm 1.5 Programın da veri gibi bellekte tutulması. Makineyi yeniden kablolamak yerine yeni talimat yüklersin; cebindeki telefon dahil bugünkü hemen her bilgisayar bu düzeni kullanır.

Derin öğrenme (deep learning) · Bölüm 4.1 Çok sayıda gizli katmanın üst üste konduğu sinir ağlarıyla öğrenme. Derinlik, ham veriden giderek soyut temsiller (kenar, şekil, nesne) çıkarmayı sağlar.

Difüzyon modeli (diffusion model) · Bölüm 5.7 Saf gürültüden başlayıp her adımda biraz temizleyerek görsel üreten model; buğulu camı yavaş yavaş silmek gibi. Gürültü ekleme sürecini tersine çevirmeyi öğrenir.

Dikkat (attention) · Bölüm 5.4 Her token'ın, anlamı için dizideki hangi token'lara bakacağını öğrenmesi; “o” zamirinin kime baktığı böyle görünür, ama dikkat ağırlığı tek başına göndergenin çözüldüğünü kanıtlamaz. Öz-dikkat (self-attention) bunu sorgu, anahtar ve değer vektörleriyle hesaplar. Transformer'ın temeli.

Düzenliler ve dağınıklar (neats and scruffies) · Bölüm 2.6 YZ'deki yöntemsel gerilim: her adımın temiz matematikle kanıtlanmasını isteyenler (neats) ile “çalışıyorsa iyidir, teorisini sonra buluruz” diyenler (scruffies). Bugünün YZ'si ikisinin karışımıdır.

Etiket (label) · Bölüm 3.2 Bir eğitim örneğinin doğru cevabı: “Spam” ya da “Normal”. Etiket bir kategoriye sınıflandırma, ölçülen bir nicelikse (fiyat, sıcaklık gibi sürekli bir sayı) regresyon görevi doğar. Kategoriler sayıyla kodlanabilir (ör. 0 = Normal, 1 = Spam); bu onları regresyon hedefi yapmaz. Bkz. Özellik.

Evrişimli sinir ağı (convolutional neural network, CNN) · Bölüm 4.5 Küçük bir filtreyi (çekirdek) görüntü üzerinde gezdirip yerel desenleri (kenar, köşe) arayan ve bulduklarını özellik haritasına işleyen ağ. Aynı filtre her yerde kullanılır (ağırlık paylaşımı); girdi kayınca özellik haritası da kayar (ötelemeye eşdeğişkenlik). Az parametreyle öğrenir.

Genel yapay zekâ (artificial general intelligence, AGI) · Bölüm 1.6 İnsan gibi her alanda öğrenip uyum sağlayabilen, yetenekleri alanlar arasında aktarılabilen varsayımsal sistem; henüz yapılmadı. Çok iş yapabilmek tek başına genel olmak değildir; genel olmak da bilinçli olmak demek değildir. Bkz. Güçlü yapay zekâ.

Genişlik-öncelikli arama (breadth-first search, BFS) · Bölüm 2.4 Hiçbir yön bilgisi kullanmadan her yeri katman katman tarayan bilgisiz arama. Tüm kenarların maliyeti eşitse en az adımlı, dolayısıyla en düşük maliyetli yolu garanti eder; ama çok düğüm açar ve farklı kenar maliyetlerinde başka yöntemler gerekir. Bkz. Sezgisel.

Geri yayılım (backpropagation) · Bölüm 4.4 Hatanın çıkıştan girişe geri geri yürüyüp her bağlantıya “payını düzelt” demesi; zincir kuralıyla gradyan hesabı. Derin ağların eğitilebilmesinin anahtarı.

Gömü (embedding) · Bölüm 5.3 Bir token’ın vektör gösterimi (kısaca gömü): anlamını taşıyan yoğun bir sayı vektörü, kocaman bir şehirdeki adres. Anlamca benzer kelimeler yakın düşer; “kedi” ile “köpek” kapı komşusudur.

Gradyan inişi (gradient descent) · Bölüm 3.6 Her adımda eğimi yoklayıp kaybı azaltacak yönde küçük bir adım atarak parametreleri güncelleme. Sisli vadide dibe iniş gibi. Bkz. Öğrenme oranı.

Güçlü yapay zekâ (strong AI) · Bölüm 1.6 Searle’ün terimi: makinenin gerçekten anlayıp anlamadığı, bir zihne sahip olup olmadığı iddiası. Felsefi bir sorudur; genel yapay zekâ (AGI) ile karıştırılmamalı. Zekâ, bilinç, öz farkındalık ve öznel deneyim yaşayabilme kapasitesi (sentience: haz ve acı duyabilme) birbirinin yerine kullanılmaz.

Halüsinasyon (hallucination) · Bölüm 5.8 Modelin kulağa doğru gelen ama yanlış bilgiyi bozuntuya vermeden üretmesi. İşi doğruyu bilmek değil, olası devamı üretmektir; akıcı ya da yüksek olasılıklı bir cevap doğru olduğunun garantisi değildir. Kaynaklarla temellendirme (RAG) ve doğrulama bunu azaltır.

Hesaplamacılık (computationalism) · Bölüm 1.3 Zihnin bir bilgi-işleme sistemi, düşünmenin de sembol manipülasyonu biçiminde bir hesaplama olduğu görüşü. Kökleri Hobbes’a uzanır; yapay zekânın temelindeki fikir.

Hizalama (alignment) · Bölüm 7.6 Bir sistemin davranışını insan niyet ve değerleriyle uyumlu kılma problemi. Söylediğini yapıp

kastettiğini kaçıran makine bu sorunun örneğidir. Hem teknik hem normatif bir sorudur.

İkili gösterim (binary representation) · Bölüm 1.3 Her bilgiyi 0 ve 1'lerle (bit) kodlama; makinenin basit alfabesi. Sekiz bit bir bayt eder ve 0 ile 255 arası her sayıyı tutar.

İleri besleme (feedforward) · Bölüm 4.3 Sinyalin giriş katmanından gizli katmanlar üzerinden çıkış katmanına akması; ağın “tahmin et” adımı. Geri yayılım ise “hatadan ders al” adımıdır.

İnce ayar (fine-tuning) · Bölüm 5.6 Ön eğitilmiş bir modeli daha küçük, hedefli bir veriyle yeniden eğiterek belirli bir davranış ya da görev kazandırma; genel kavram. Talimat–cevap çiftleriyle yapılan denetimli ince ayar (SFT) bunun bir türüdür: soruya cevap vermeyi, yönergeyi izlemeyi öğretir. Bilgiyi ve görev başarımını da değiştirebilir.

İstem mühendisliği (prompt engineering) · Bölüm 6.2 Modeli yeniden eğitmeden, istemi (prompt) iyi kurarak davranışını yönlendirme pratiği: açık rol, yeterli bağlam, net format, gerekirse örnek (few-shot).

Kara kutu (black box) · Bölüm 7.3 Kararını veren ama gerekçesini anlatamayan model. Kredi, işe alım gibi kararlarda “neden?” diye sorabilmek bir hak meselesidir. Kapak bir karar için aralanabilir; modelin tamamı yine kapalı kalabilir. Bkz. Açıklanabilir yapay zekâ.

Kayıp (loss) · Bölüm 3.6 Modelin ne kadar yanıldığının ölçüsü; ne kadar büyükse vadide o kadar yukarıdasın. Eğitim, kaybı en aza indirecek parametreleri bulmaktır.

Kaynaklarla temellendirme (grounding) · Bölüm 6.3 Modelin cevabını isteme eklenen kaynak metinlere (belge, veri, arama sonucu) dayandırması; böylece cevap bu kaynaklara kadar izlenebilir. RAG bunun yaygın bir yoludur. Temellendirme doğruluğu artırır ama garanti etmez; kaynak yanlış ya da eksikse cevap da yanlış olabilir. Bkz. RAG.

Koruyucu kontroller (guardrails) · Bölüm 6.5 Bir YZ uygulamasında istenmeyen girdi ve çıktıları süzen, araç izinlerini sınırlayan ve riskli eylemleri onaya bağlayan kurallar ve denetimler. Bkz. Orkestrasyon.

Kümeleme (clustering) · Bölüm 3.5 Etiketsiz veriyi benzerliğe göre kendiliğinden grupta; kutudaki düğmeleri ayırmak gibi. k-ortalamlar (k-means) noktaları en yakın küme merkezine atar, merkezleri günceller.

KVKK / GDPR (Kişisel Verilerin Korunması Kanunu / General Data Protection Regulation) · Bölüm 7.5 Kişisel veri rejimleri: rıza, amaç sınırlaması, veri minimizasyonu ve otomatik kararlara itiraz hakkı. AB YZ Yasası'nı tamamlar.

Makine öğrenmesi (machine learning) · Bölüm 3.1 Kural yazmak yerine bol örnek gösterip örüntüyü makinenin kendisinin yakalamasını sağlayan yaklaşım. Model, kayıp ve optimizasyon döngüsüne dayanır.

Markov zinciri (Markov chain) · Bölüm 2.5 Mevcut durum bilindiğinde bir sonraki durumun daha eski geçmişe bağlı olmadığı olasılıksal süreç (Markov özelliği: mevcut duruma koşullu bağımsızlık). Sonlu, indirgenemez ve periyodik olmayan bir zincirin dağılımı, başlangıç ne olursa olsun, tek bir kararlı dağılıma yakınsar; kararlı dağılımın var olması tek başına yakınsama demek değildir.

Moore yasası (Moore's law) · Bölüm 1.7 Çipteki transistör sayısının düzenli aralıklarla ikiye katlandığı gözlemi: 1965 öngörüsünde yaklaşık her yıl, 1975 revizyonunda yaklaşık iki yılda bir. Doğa yasası değil, ampirik bir eğilim; transistör yapıları atomik ölçeğe yaklaştıkça yavaşlıyor. Daha çok transistör, her iş yükünde aynı oranda hız demek değildir.

Orkestrasyon (orchestration) · Bölüm 6.5 YZ uygulamasında istem oluşturma, yönlendirme, araç ve RAG çağrılarını, yedek yöntem geçişi (fallback) ve koruyucu kontrolleri (guardrails) koordine eden yöneten katman; restoranın müdürü. Model çoğu zaman değiştirilebilir bir parçadır.

Öğrenme oranı (learning rate) · Bölüm 3.6 Gradyan inişinde her adımın boyu. Çok küçükse yakınsama yavaşlar; çok büyükse top vadinin dibini ıskalayıp karşı yamaca fırlar.

Ön eğitim (pretraining) · Bölüm 5.6 Devasa metin derlemi üzerinde, bir sonraki token'ı tahmin hedefiyle (öz-denetimli) dilin ve dünyanın istatistiğini öğrenme aşaması. Model “ne bildiğini” burada kazanır.

Önyargı, algoritmik yanlılık (bias) · Bölüm 7.2 Toplumsal anlamda yanlılık (bias): modelin verideki tarihsel önyargıyı, eksik temsili, etiketleme hatasını ya da vekil değişkenleri öğrenip pekiştirmesi. Kaynak yalnız veri değildir; ölçüm ve modelleme tercihleri, kurumsal süreçler ve kullanım bağlamı da yanlılık üretir. Model niyet taşımaz. Nörondaki sabit terim için bkz. Sabit terim; istatistiksel yanlılık için bkz. Yanlılık-varyans dengesi.

Özellik (feature) · Bölüm 3.2 Bir örneği tarif eden ölçülebilir ipucu: e-postada “link var mı”, “bedava geçiyor mu”. Model, özelliklerden etikete giden eşlemeyi öğrenir. Bkz. Etiket.

Parametre (parameter) · Bölüm 3.1 Modelin eğitimle öğrendiği sayısal değerler: ağırlıklar ve sabit terimler (bias). Model, girdileri bu değerler aracılığıyla çıktılarına eşler.

Pekiştirmeli öğrenme (reinforcement learning) · Bölüm 3.3 Bir ajanın ortamda deneyerek, ödül ya da ceza toplayarak, ödülü en üst düzeye çıkaran bir politika öğrenmesi. Öğretmen ders anlatmaz; makine oyuna girer.

RAG (retrieval-augmented generation, bilgiyle desteklenmiş üretim) · Bölüm 6.3 Cevaptan önce ilgili kaynak parçalarını bulup isteme ekleyerek modeli kaynaklarla temellendirme (grounding). Erişim vektör benzerliğiyle, anahtar sözcükle ya da ikisinin karışımıyla yapılabilir; vektör veri tabanı yaygın bir seçenektir, zorunlu değildir. Açık kitap sınavı gibi: halüsinasyon azalır, kaynak gösterilebilir; cevabın kaynağa dayandığı yine doğrulanmalıdır.

Regresyon (regression) · Bölüm 3.4 “Ne kadar?” sorusuna sayı cevabı veren denetimli görev: evin fiyatı kaç lira? En basit hâli,

noktalara en küçük kareler doğrusu uydurmaktır.

RLHF (reinforcement learning from human feedback, insan geri bildirimiyile hizalama) · Bölüm 5.6 İnsan tercihlerinden türetilen sinyale göre dil modelini güncelleme. PPO tabanlı RLHF ayrı bir ödül modeli eğitir; DPO ise tercih çiftlerinden doğrudan optimize eder. Asistanın yardımcı, dürüst ve güvenli olmayı, yani görgüyü öğrendiği aşama; her model aynı aşamalardan geçmez.

Sabit terim (bias) · Bölüm 4.2 Nöronun ağırlıklı toplamına eklenen, toplamı kaydıran sabit; kapı bekçisinin kendi huyu. Eğitimle öğrenilir; “ateşledi” eşiği ise ayrı bir karar kuralıdır. Toplumsal yanlılık anlamı için bkz. Önyargı.

Sembolik yapay zekâ (symbolic AI, GOFAl) · Bölüm 2.1 Zekâyı açıkça yazılmış semboller ve kurallar üzerinde mantıksal işlem olarak ele alan klasik yaklaşım; 1950’lerden 1980’lere baskındı. Mantık, arama ve uzman sistemler alet çantasıydı.

Sezgisel (heuristic) · Bölüm 2.4 Aramada “hangi yön daha umut verici?” diye tahmin yürüten kısayol. Açgözlü sezgisel arama hız kazandırır ama en iyi çözümü kaçırabilir; A*, kabul edilebilir bir sezgiyle garantiyi geri alır.

Sıcaklık (temperature) · Bölüm 5.5 Üretimde softmax(z/T) dağılımını keskinleştiren ya da yumuşatan ayar. Düşük T dağılımı sivriltilir, yüksek T düzleştirir; T sıfırdan büyük olduğu sürece örnekleme rastlantısaldır. Her adımda en olasıyı seçmek (argmax, açgözlü çözümleme) ayrı bir kuraldır, düşük sıcaklık değildir.

Sınıflandırma (classification) · Bölüm 3.4 “Hangisi?” sorusuna kategori cevabı veren denetimli görev: spam mı, değil mi? Sınıfları ayıran bir karar sınırı öğrenir; sınır doğrusal olmak zorunda değildir.

Sorumluluk (accountability) · Bölüm 8.6 Bir YZ zarara yol açtığında hesap verme yükümlülüğü. Bugün ezici biçimde insanlara ve kurumlara (geliştirici, işleten, kullanıcı) atfedilir; “YZ’nin kendisine” değil.

Süper zekâ (superintelligence) · Bölüm 8.4 Her bilişsel alanda insanı kat kat aşan varsayımsal YZ; dar YZ ve AGI'den sonraki basamak. Şimdilik spekülâtif.

Tekillik (singularity) · Bölüm 8.5 Kendini geliştirebilen bir YZ'nin (özyinelemeli özgeleşim) zekâ patlamasıyla öngörülemez bir noktaya varacağı hipotezi. Ne kesin ne imkânsız; ciddiye alınması gereken belirsiz bir senaryo.

Token · Bölüm 5.2 Dil modelinin metni böldüğü küçük lego parçası: bazen bir kelime, bazen kelimedenden kopmuş bir parça, bazen bir virgül. Bağlam penceresi ve maliyet token'la ölçülür.

Topluluk öğrenmesi (ensemble learning) · Bölüm 3.7 Birçok modelin tahminini birleştirerek (oylama, bagging, boosting) tek modelden daha iyi ve kararlı sonuç alma. Rastgele orman ve gradyan artırma en bilinen örnekleridir.

Transformer · Bölüm 5.1 Dikkat mekanizmasına dayanan mimari (2017, "Attention Is All You Need"). Eğitimde dizinin konumlarını birlikte (paralel) işler; otoregresif üretim ise token'ları sırayla ekler ve nedensel maskeyle her konum kendisini ve önceki konumları görür; sonrakiler maskelenir. Bugünün büyük dil modellerinin temeli.

Turing makinesi (Turing machine) · Bölüm 1.4 Bandı okuyan, yazan ve iki yana kayan tek bir kutucuktan oluşan soyut makine. Kurallarına göre belirli bir işi yapar (Şekil 1.3'teki makine yalnızca 1 ekler); evrensel Turing makinesi ise doğru program ve yeterli bant verildiğinde algoritmaya hesaplanabilen her işlemi yürütebilir. Hesaplanamayan problemler de vardır. Hesaplanabilirliğin biçimsel temeli (1936).

Turing testi (Turing test) · Bölüm 8.2 Turing'in taklit oyunu (1950): makine yazışmada insandan ayırt edilemiyorsa yeterli sayılır. Davranışçı bir ölçüttür; akıcı taklit, anlama ya da bilinç kanıtı değildir.

Uzman sistem (expert system) · Bölüm 2.3 Bir alandaki uzman bilgisini "EĞER şu doğruysa O ZAMAN şunu yap" kurallarına döken

sistem: bilgi tabanı artı çıkarım motoru. Motor, olgularla eşleşen kuralları tetikler; ileri zincirleme olgulardan sonuca, geri zincirleme hedeften kanıta gider; bir sistem bunlardan birini ya da ikisini kullanabilir. 1980’lerde yaygındı.

Üretken çekişmeli ağ (generative adversarial network, GAN) · Bölüm 4.7 Kalpazan (üretici) ile dedektifi (ayırt edici) aynı odaya kilitleyen ikili ağ. Biri sahte üretir, öteki yakalamaya çalışır; yarış sürdükçe üretilenler gerçeğe yaklaşır.

Üretken yapay zekâ (generative AI) · Bölüm 5.1 Tanımakla kalmayıp yazı, görsel, kod üreten modeller; verinin kendisini üreten dağılımı öğrenir. Transformer ve difüzyon bu çağın motorlarıdır.

Üstel büyüme (exponential growth) · Bölüm 1.7 Her adımda sabit bir çarpanla büyüme; belirli aralıklarla ikiye katlanma bunun özel bir örneğidir. Satranç tahtasındaki piring gibi masum başlar, birkaç katlamada kontrolden çıkar. Modern YZ’yi taşıyan işlem gücü böyle birikti.

Yanlılık-varyans dengesi (bias–variance tradeoff) · Bölüm 3.7 Eksik uyum ile aşırı uyum arasındaki denge; buradaki yanlılık istatistiksel anlamdadır (modelin sistematik hatası), toplumsal önyargı değil. İyi model ikisini dengeleyip görülmemiş veriye genelleşendir; örüntüyü öğrenir.

Yapay nöron (artificial neuron) · Bölüm 4.2 Girdileri ağırlıklarıyla tartıp toplayan, sabit terim (bias) ekleyen ve sonucu aktivasyondan geçiren basit birim; kapı bekçisi gibi. Beynin kopyası değil, kaba bir matematiksel benzetme.

Yapay sinir ağı (artificial neural network) · Bölüm 4.1 Yapay nöronların katman katman dizilmesiyle kurulan öğrenme makinesi; özünde doğrusal olmayan dönüşümler yığını. Biyolojik nöronlardan yalnızca gevşek biçimde esinlenir.

Yapay zekâ (artificial intelligence, YZ) · Bölüm 1.1 İnsan zekâsı gerektiren görevleri yerine getirebilen sistemler; alanın pragmatik tanımı. Hesaplama kuramı ile zihin felsefesinin kesişiminde doğdu.

Yedek yöntemge geçiş (fallback) · Bölüm 6.5 Bir adım başarısız olduğunda (araç hata verir, kaynak bulunamaz, model emin değildir) sistemin önceden belirlenmiş daha güvenli bir yola geçmesi: başka bir araç, kısa bir “bilmiyorum” cevabı ya da işi bir insana devretmek. Bkz. Orkestrasyon.

Yinelemeli sinir ağı (recurrent neural network, RNN) · Bölüm 4.6 Diziyi kelime kelime okuyup her adımda bir hafıza (gizli durum) taşıyan ağ: $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$. Aynı hücre ve aynı ağırlıklar her adımda yeniden kullanılır; “yinelemeli” (recurrent) adı buradan gelir. Ağaç yapılı “özyinelemeli” (recursive) ağlardan ayrılır. Uzun bağımlılıklarda kaybolan gradyanla zorlanır; LSTM bunu hafifletir. Yerini büyük ölçüde Transformer aldı.

Kaynakça ve İleri Okuma

Künye biçimi: dergi makalelerinde DOI, bildirilerde arXiv ya da kalıcı adres; mevzuatta madde ve sürüm tarihi. NeurIPS, *Advances in Neural Information Processing Systems* bildirileridir; 2018 öncesi bildiriler NIPS adıyla yayımlandı, burada tek ad kullanıldı. Bağlantılara erişim: 1 Ekim 2026.

Metinde adı geçen çalışmalar

- Gardner, H. (1983). *Frames of Mind: The Theory of Multiple Intelligences*. Basic Books. (Bölüm 1)
- Turing, A. M. (1936). "On Computable Numbers, with an Application to the Entscheidungsproblem." *Proceedings of the London Mathematical Society*, s2-42(1), 230–265. <https://doi.org/10.1112/plms/s2-42.1.230> Cilt 1937 tarihlidir; makale 1936'da dergiye ulaşmış ve Derneğe sunulmuştur. Bu kitap, yaygın kullanıma uyarak 1936 yılını yazar. (Bölüm 1)
- Turing, A. M. (1950). "Computing Machinery and Intelligence." *Mind*, 59(236), 433–460. <https://doi.org/10.1093/mind/LIX.236.433> (Bölüm 8)
- von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania. Yeniden basım: *IEEE Annals of the History of Computing*, 15(4), 27–75 (1993). <https://doi.org/10.1109/85.238389> (Bölüm 1)
- Moore, G. E. (1965). "Cramming More Components onto Integrated Circuits." *Electronics*, 38(8), 114–117. Yeniden basım: *IEEE Solid-State Circuits Society Newsletter*, 11(3), 33–35 (2006). <https://doi.org/10.1109/N-SSC.2006.4785860> (Bölüm 1)
- Rumelhart, D. E., Hinton, G. E. ve Williams, R. J. (1986). "Learning Representations by Back-propagating Errors." *Nature*, 323, 533–536. <https://doi.org/10.1038/323533a0> (Bölüm 4)

- Hochreiter, S. ve Schmidhuber, J. (1997). "Long Short-Term Memory." *Neural Computation*, 9(8), 1735–1780.
<https://doi.org/10.1162/neco.1997.9.8.1735> (Bölüm 4)
- Krizhevsky, A., Sutskever, I. ve Hinton, G. E. (2012). "ImageNet Classification with Deep Convolutional Neural Networks." *NeurIPS* 25. <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks> (Bölüm 4)
- Goodfellow, I. vd. (2014). "Generative Adversarial Nets." *NeurIPS* 27. arXiv:1406.2661. <https://doi.org/10.48550/arXiv.1406.2661> (Bölüm 4)
- Sennrich, R., Haddow, B. ve Birch, A. (2016). "Neural Machine Translation of Rare Words with Subword Units." *ACL*.
<https://doi.org/10.18653/v1/P16-1162> (BPE; Bölüm 5)
- Vaswani, A. vd. (2017). "Attention Is All You Need." *NeurIPS* 30. arXiv:1706.03762. <https://doi.org/10.48550/arXiv.1706.03762> (Bölüm 5)
- Ho, J., Jain, A. ve Abbeel, P. (2020). "Denoising Diffusion Probabilistic Models." *NeurIPS* 33. arXiv:2006.11239. <https://doi.org/10.48550/arXiv.2006.11239> (Bölüm 5)
- Ouyang, L. vd. (2022). "Training Language Models to Follow Instructions with Human Feedback." *NeurIPS* 35. arXiv:2203.02155. <https://doi.org/10.48550/arXiv.2203.02155> (RLHF; Bölüm 5)
- Rafailov, R. vd. (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." *NeurIPS* 36. arXiv:2305.18290. <https://doi.org/10.48550/arXiv.2305.18290> (DPO; Bölüm 5)
- Lewis, P. vd. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS* 33. arXiv:2005.11401. <https://doi.org/10.48550/arXiv.2005.11401> (RAG; Bölüm 6)
- Yao, S. vd. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR*. arXiv:2210.03629. <https://doi.org/10.48550/arXiv.2210.03629> (Bölüm 6)

- Lundberg, S. M. ve Lee, S.-I. (2017). "A Unified Approach to Interpreting Model Predictions." *NeurIPS 30*. arXiv:1705.07874. <https://doi.org/10.48550/arXiv.1705.07874> (SHAP; Bölüm 7)
- Schwartz, R. vd. (2022). *Towards a Standard for Identifying and Managing Bias in Artificial Intelligence*. NIST Special Publication 1270. <https://doi.org/10.6028/NIST.SP.1270> (Yanlılık kaynakları; Bölüm 7)
- Avrupa Parlamentosu ve Konseyi (2024). *Yapay Zekâ Tüzüğü (AI Act)*, (AB) 2024/1689 sayılı Tüzük; (AB) 2026/1744 sayılı Tüzükle (AB Resmî Gazetesi, 24 Temmuz 2026; yürürlük 27 Temmuz 2026) değişik, 27 Temmuz 2026 tarihli konsolide sürüm; Madde 5, 6, 50, 111, 113 ve Ek III. Konsolide sürüm: <http://data.europa.eu/eli/reg/2024/1689/2026-07-27> · Değiştiren tüzük: <http://data.europa.eu/eli/reg/2026/1744/oj> · İlk yayımlanan metin: <http://data.europa.eu/eli/reg/2024/1689/oj> (Bölüm 7)
- Avrupa Parlamentosu ve Konseyi (2016). *Genel Veri Koruma Tüzüğü (GDPR)*, (AB) 2016/679 sayılı Tüzük; Madde 6, 9 ve 22. <http://data.europa.eu/eli/reg/2016/679/oj> (Bölüm 7)
- 6698 sayılı Kişisel Verilerin Korunması Kanunu (2016). *Resmî Gazete* 29677, 7 Nisan 2016; m.5, m.6, m.9 ve m.11. <https://www.mevzuat.gov.tr/mevzuat?MevzuatNo=6698&MevzuatTur=1&MevzuatTertip=5> (Bölüm 7)
- Searle, J. R. (1980). "Minds, Brains, and Programs." *Behavioral and Brain Sciences*, 3(3), 417–457. <https://doi.org/10.1017/S0140525X00005756> (Çince Oda; Bölüm 8)
- Good, I. J. (1965). "Speculations Concerning the First Ultraintelligent Machine." *Advances in Computers*, 6, 31–88. [https://doi.org/10.1016/S0065-2458\(08\)60418-0](https://doi.org/10.1016/S0065-2458(08)60418-0) (Bölüm 8)
- Brown, T. B. vd. (2020). "Language Models are Few-Shot Learners." *NeurIPS 33*. arXiv:2005.14165. <https://doi.org/10.48550/arXiv.2005.14165> (Bağlam içi öğrenme; Bölüm 8)
- Jones, C. R. ve Bergen, B. K. (2025). "Large Language Models Pass the Turing Test." arXiv:2503.23674.

<https://doi.org/10.48550/arXiv.2503.23674> (Turing testi deneyi; Bölüm 8)

Bölümlerin dayandığı genel kaynaklar

- Vinge, V. (1993). "The Coming Technological Singularity." *VISION-21 Symposium*, NASA Conference Publication 10129, 11–22.
<https://ntrs.nasa.gov/citations/19940022856> (Bölüm 8)
- Russell, S. ve Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. (Bölüm 2, 3)
- Nilsson, N. J. (2010). *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press. (Bölüm 1, 2)
- Mitchell, M. (2019). *Artificial Intelligence: A Guide for Thinking Humans*. Farrar, Straus and Giroux. (Bölüm 7, 8)
- Goodfellow, I., Bengio, Y. ve Courville, A. (2016). *Deep Learning*. MIT Press. (Bölüm 3, 4)
- Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press. (Bölüm 8)
- Christian, B. (2020). *The Alignment Problem*. W. W. Norton. (Bölüm 7)

Canlı Demolar

Kitaptaki her şekil, dijital sürümde elle oynanan bir demodur. Aşağıdaki bağlantılar (ve şekillerin altındaki QR kodlar) yalnız o demoyu açar; giriş gerekmez. Kitabın tamamı dijital sürümde: <https://book.onuronder.com>

Şekil	Demo	Bağlantı
1.1	Çoklu zekâyı keşfet	book.onuronder.com/d/1dbc74d595
1.2	İkili kodu çöz	book.onuronder.com/d/7c96c14a25
1.3	Çalışan bir Turing makinesi (+1)	book.onuronder.com/d/ebaea4754b
1.4	Getir – Yürüt – Yaz döngüsü	book.onuronder.com/d/093bb7d970
1.5	Bugün var mı, yoksa bilim kurgu mu?	book.onuronder.com/d/30661431ad
1.6	Üstel büyümeyi hisset	book.onuronder.com/d/d7182b2b78
2.1	Bilgi zinciriyle çıkarım	book.onuronder.com/d/4ea6179a6f
2.2	Küçük bir uzman sistem	book.onuronder.com/d/045489d51c
2.3	Yol bulma: sezgisiz ile sezgili	book.onuronder.com/d/b8544b96e9
2.4	Hava durumu Markov zinciri	book.onuronder.com/d/8c423f9c01
2.5	Hangi yaklaşım?	book.onuronder.com/d/330f95d0cf
3.1	Özellikleri ve etiketi gör	book.onuronder.com/d/1946fb9748
3.2	Hangi öğrenme türü?	book.onuronder.com/d/3afa1bb6c0
3.3	İki temel görev	book.onuronder.com/d/3324149b70
3.4	Etiketsiz veriyi grupla	book.onuronder.com/d/76fce219a4
3.5	Kayıp vadisinde iniş	book.onuronder.com/d/0a8c400a67
3.6	Aynı veri, üç model	book.onuronder.com/d/7f55ca4f28
4.1	Nöronu çalıştır	book.onuronder.com/d/1cb90a39a8
4.2	Canlı sinir ağı (ileri besleme)	book.onuronder.com/d/63f05dccf2
4.3	Hatadan öğren (gerçek eğitim)	book.onuronder.com/d/37fa54d6c3

Şekil	Demo	Bağlantı
4.4	Evrişim: filtreyi kaydır	book.onuronder.com/d/3ba8a6fcb2
4.5	Hafızalı işleme (gerçek yineleme)	book.onuronder.com/d/a6130db86e
4.6	Üretici ile ayırt edici	book.onuronder.com/d/8ae feaa8e9
5.1	Cümle ni token'lara böl	book.onuronder.com/d/301fb38607
5.2	Anlam haritası	book.onuronder.com/d/09ebf3e5df
5.3	Hangi kelime hangisine bakıyor?	book.onuronder.com/d/ce48215398
5.4	Kelime kelime üret	book.onuronder.com/d/8eb50398ea
5.5	Üç aşamada bir asistan	book.onuronder.com/d/ef1d09e9b4
5.6	Gürültüden görsele	book.onuronder.com/d/86f3a41e00
5.7	Bağlam penceresi	book.onuronder.com/d/59df65a586
6.1	Bir istem inşa et	book.onuronder.com/d/a4cb8276ea
6.2	Kaynağa dayalı cevap	book.onuronder.com/d/8405aeacde
6.3	Bir ajanı izle	book.onuronder.com/d/cf387db4c4
6.4	Bir YZ uygulamasının parçaları	book.onuronder.com/d/d956416424
6.5	Alanları keşfet	book.onuronder.com/d/37c06e819b
7.1	Önyargı simülasyonu	book.onuronder.com/d/af69b26c43
7.2	Beyaz kutu: kararı açıkla	book.onuronder.com/d/64c2b6e579
7.3	Gerçek mi, yapay mı?	book.onuronder.com/d/df72f49bed
7.4	Riski sınıflandır	book.onuronder.com/d/3508162a0b
7.5	Hedef ile niyet	book.onuronder.com/d/1efe20e51c
8.1	İnsan mı, makine mi?	book.onuronder.com/d/3e9862eecc
8.2	Çince Oda'dasın	book.onuronder.com/d/965a20fb79
8.3	Yetenek basamakları	book.onuronder.com/d/023e49955d
8.4	Üç zekâ eğrisi	book.onuronder.com/d/bddd5bc368
8.5	Sorumluluk kimde?	book.onuronder.com/d/4f50232ae1

Dizin

Sayılar sayfa numarasını gösterir.

AB YZ Yasası 168, 181, 188
 Açıklanabilir yapay zekâ 168, 175
 Ağırlık (sinir ağı) 83, 89, 96, 116, 137
 Ajan 140, 151, 158, 161, 165
 Aktivasyon fonksiyonu 85, 105
 AlexNet 82, 97
 Algoritma 14, 18, 35, 36, 72
 Anomali tespiti 68
 Aşırı uyum 75, 79
 Babbage, Charles 14, 21
 Bağlam penceresi 109, 112, 119, 132, 137, 150
 Basamak değeri (ikili gösterim) 19
 Belirtim oynama, ödül oynama 127, 187
 Bilgi edinme darboğazı 38, 54
 Bilgi temsili 41
 BPE / WordPiece 112
 Büyük dil modeli 109, 136
 Çıkarım 41, 45
 Çince Oda 31, 192, 194, 198, 209
 Dar yapay zekâ 31, 192, 201, 209
 Deepfake 168, 179, 183, 188
 Denetimli öğrenme 64
 Denetimsiz öğrenme 64
 Depolanmış-program ilkesi 14, 28, 35, 36
 Derin öğrenme 82, 97
 Difüzyon modeli 128, 137
 Dikkat 108, 116, 135, 136, 175
 DPO 126
 Düzenliler ve dağınıklar 52, 56
 ENIAC 21, 25

Etiket 59, 71, 79, 108
Evrişimli sinir ağı (CNN) 96, 106
Gardner, Howard 15, 35
Genel yapay zekâ (AGI) 28, 36, 192, 199, 209
Genişlik-öncelikli arama 48
Geri yayılım 89, 105
Gömü 101, 108, 113, 135, 136, 150
GPT 195
Gradyan inişi 58, 72, 79, 89, 106
Güçlü yapay zekâ 14, 31, 36, 198
Halüsinasyon 109, 132, 137, 146
Hesaplamacılık 20
Hinton, Geoffrey 93
Hizalama 124, 135, 137, 168, 187, 189
Hochreiter, Sepp 100
ImageNet 82
İkili gösterim 14, 21
İleri besleme 86, 106
İnce ayar 109, 124, 136
İstem mühendisliği 140, 145
Kara kutu 168, 172, 188
Kayıp 58, 72, 90, 127
Kaynaklarla temellendirme (grounding) 135, 140, 150
Koruyucu kontroller (guardrails) 158, 162
Kümeleme 64, 68, 79
KVKK / GDPR 168, 181
Makine öğrenmesi 58, 80
Markov zinciri 49
Moore, Gordon 31
Moore yasası 34, 36
Orkestrasyon 140, 154, 155, 164
Öğrenme oranı 74, 90
Ön eğitim 109, 127, 136
Önyargı (algoritmik yanlılık) 135, 168, 169, 188

Özellik 61, 79, 94, 175
 Parametre 75, 96, 100
 Pekiştirmeli öğrenme 64
 RAG 135, 140, 150, 158, 164
 ReAct 140, 154
 Regresyon 62, 64, 65, 75, 79
 RLHF 109, 124, 136, 186
 Rumelhart, David 93
 Sabit terim (bias) 82, 83, 87, 90, 105
 Searle, John 31, 196
 Sembolik yapay zekâ 38
 Sezgisel 45, 55
 SHAP 175
 Sıcaklık 123, 137
 Sınıflandırma 62, 64, 65, 79, 163
 Sorumluluk 192, 208, 209
 Süper zekâ 192, 199, 209
 Tekillik 192, 203, 209
 Token 108, 109, 113, 119, 120, 127, 132, 136, 145
 Topluluk öğrenmesi 78
 Transformer 100, 105, 108, 119
 Turing, Alan 21, 192, 193
 Turing makinesi 14, 22, 35
 Turing testi 192, 193, 198, 209
 Uzman sistem 42, 56
 Üretken çekişmeli ağ 101, 106, 108, 128, 179
 Üretken yapay zekâ 108, 176
 Üstel büyüme 34
 von Neumann, John 28, 35
 Yanlılık (istatistiksel) 76
 Yanlılık-varyans dengesi 77
 Yapay nöron 82, 85, 105
 Yapay sinir ağı 14, 82, 131
 Yedek yöntem geçiş (fallback) 158

Yinelemeli sinir ağı (RNN) 97, 106